

Complete IT Certification Practice Exams - All Courses in One PDF

Total Courses: 36 | Total Questions: 1080 | 10 Beginner + 10 Intermediate + 10 Advanced per course | Generated 2026

Course File: 01_AWS_Cloud.md

Course: AWS Cloud

1. Beginner Level (10 Questions)

****Q1.** A startup wants to deploy a web application without managing servers, but needs to run custom Docker containers. Which AWS service should they choose?******

- A. Amazon EC2
- B. AWS Lambda
- C. AWS Fargate with Amazon ECS
- D. Amazon Lightsail

****Q2.** You store 10 TB of financial reports that must be retained for 7 years for compliance, accessed less than once a year, with retrieval in 12 hours acceptable. Which S3 storage class is MOST cost-effective?******

- A. S3 Standard
- B. S3 Glacier Flexible Retrieval
- C. S3 Glacier Deep Archive
- D. S3 Intelligent-Tiering

****Q3.** A company wants to isolate its production VPC traffic from the internet while still allowing EC2 instances to download security updates. What should they implement?******

- A. Internet Gateway with restrictive Security Group
- B. NAT Gateway in a public subnet and route private subnet traffic to it
- C. VPN Gateway
- D. Place all instances in public subnets and disable Elastic IPs

****Q4.** An IAM user needs permission to start/stop EC2 instances tagged with Env=Dev but not production. What is the best approach?******

- A. Give AmazonEC2FullAccess managed policy
- B. Create a custom IAM policy with ec2:StartInstances, ec2:StopInstances and Condition aws:ResourceTag/Env = Dev
- C. Put Dev and Prod in separate AWS accounts and use SCPs
- D. Use IAM role with AdministratorAccess and rely on MFA

****Q5.** Your RDS MySQL instance is experiencing high read traffic. You need to scale reads without affecting the primary. What should you do?******

- A. Increase the instance class to db.r6g.4xlarge
- B. Enable Multi-AZ
- C. Create RDS Read Replicas
- D. Move data to DynamoDB

****Q6.** You have an Application Load Balancer (ALB) in front of 3 EC2 instances. A user reports that their shopping cart empties on every request. What is the likely fix?******

- A. Enable Sticky Sessions (session affinity) on the ALB target group, or better, externalize session to ElastiCache/DynamoDB
- B. Enable Cross-Zone Load Balancing
- C. Switch ALB to NLB
- D. Attach a WAF to the ALB

****Q7.** Which AWS service provides a managed, highly available DNS service with routing policies like latency-based and geolocation routing?******

- A. AWS Direct Connect
- B. Amazon Route 53
- C. AWS Global Accelerator

D. Amazon CloudFront

****Q8. You accidentally deleted an S3 bucket containing critical data, with no versioning enabled. How could you have prevented permanent loss?****

A. Enable S3 Versioning and MFA Delete, and use S3 Object Lock for compliance

B. Enable S3 Transfer Acceleration

C. Enable S3 Access Points

D. Use S3 Replication alone

****Q9. A developer wants to run code in response to an S3 object upload without provisioning servers and only paying for execution time. Which service fits?****

A. Amazon EC2 Auto Scaling

B. AWS Lambda triggered by S3 Event Notification

C. Amazon ECS on EC2

D. AWS Batch

****Q10. Your organization needs to centrally manage billing and apply guardrails to prevent use of certain regions across 20 AWS accounts. Which service combination should you use?****

A. AWS Organizations with Service Control Policies (SCPs) and consolidated billing

B. IAM Groups with resource policies

C. AWS Control Tower alone without Organizations

D. AWS Config alone

Beginner Answer Key & Explanations

****Q1 Answer: C - AWS Fargate with Amazon ECS****

Fargate is serverless compute for containers that eliminates EC2 management while allowing custom Docker images. Lambda does not natively support long-running containers with arbitrary ports without extra constraints.

****Q2 Answer: C - S3 Glacier Deep Archive****

Deep Archive is the lowest-cost class for long-term retention (7-10 years) when retrieval within 12 hours is acceptable. Flexible Retrieval is more expensive but faster, while Standard would cost ~10x more.

****Q3 Answer: B - NAT Gateway in a public subnet and route private subnet traffic to it****

A NAT Gateway allows outbound internet access for private subnets while blocking inbound initiation. An Internet Gateway alone would make subnets public, violating isolation.

****Q4 Answer: B - Create a custom IAM policy with Condition on ResourceTag****

ABAC using aws:ResourceTag allows fine-grained control to limit actions to resources with specific tags. Giving FullAccess violates least privilege.

****Q5 Answer: C - Create RDS Read Replicas****

Read Replicas offload read queries using asynchronous replication from the primary DB. Multi-AZ is for high availability/failover, not read scaling.

****Q6 Answer: A - Enable Sticky Sessions or externalize session****

ALB by default distributes requests round-robin, breaking in-memory sessions. Sticky sessions fix it short-term, but externalizing session state to ElastiCache is the resilient best practice.

****Q7 Answer: B - Amazon Route 53****

Route 53 is AWS's managed DNS with advanced routing policies including latency-based, geolocation, failover, and weighted routing. CloudFront is CDN, Global Accelerator is anycast network acceleration.

****Q8 Answer: A - Enable S3 Versioning and MFA Delete, and use S3 Object Lock****

Versioning retains previous versions after deletion, MFA Delete requires MFA for permanent deletion, and Object Lock prevents deletion via governance. Replication alone would replicate the delete.

****Q9 Answer: B - AWS Lambda triggered by S3 Event Notification****

Lambda is event-driven, serverless, and integrates natively with S3 events to process objects. EC2 and ECS require server management and are not billed per 100ms execution.

****Q10 Answer: A - AWS Organizations with Service Control Policies (SCPs) and consolidated billing****

Organizations provides consolidated billing and OU hierarchies, while SCPs enforce permission guardrails like denying regions across accounts. IAM Groups only work within a single account.

2. Intermediate Level (10 Questions)

****Q11. You have an S3 bucket receiving 5,000 PUTs/sec with key prefix logs/year=2024/month=11/day=15/hour=09/... where all keys share same prefix. You see 503 SlowDown errors. What architectural change in 2024-2026 best solves this? (Note: S3 now has no prefix partition limits since 2018, but contention still occurs)****

- A. Add random hex prefix to keys
- B. The bottleneck is no longer prefix partitioning but request rate to a single partition per prefix is still mitigated by using S3 Express One Zone with directory buckets for high TPS per prefix
- C. Enable Transfer Acceleration
- D. Switch to multipart upload

****Q12. An EC2 Auto Scaling Group behind ALB sees uneven CPU, new instances fail health checks during deployment. You are using rolling updates with ELB health check type but instance-level 2-minute app warmup. What should you configure?****

- A. Increase ALB health check interval to 5 minutes
- B. Use Lifecycle Hook + warmup pool and increase ASG health check grace period to 180s
- C. Disable ELB health checks
- D. Use all-at-once deployment

****Q13. You need to connect a VPC to S3 without traversing the public internet and without using a NAT Gateway, with policy limiting bucket access. Which option?****

- A. Internet Gateway
- B. NAT Gateway
- C. Gateway VPC Endpoint for S3 with endpoint policy
- D. Interface Endpoint for S3 using PrivateLink

****Q14. A Lambda function in VPC needs to access DynamoDB and Secrets Manager. It times out when trying to reach DynamoDB. What is MOST likely and optimal fix?****

- A. Attach Internet Gateway to Lambda VPC
- B. Create Gateway VPC Endpoint for DynamoDB and Interface Endpoint for Secrets Manager, ensure Lambda subnet route tables and SG allow it
- C. Put Lambda in public subnet
- D. Increase Lambda timeout to 15 minutes

****Q15. You are designing cross-region active-active serverless API. You have API Gateway + Lambda + DynamoDB Global Tables in us-east-1 and eu-west-1. How should you route users to lowest latency and handle failover automatically?****

- A. Use Route 53 latency-based routing with health checks to API Gateway regional endpoints
- B. Use ALB with cross-region target groups
- C. Use CloudFront with origin failover and latency metrics
- D. Hardcode region in client

****Q16. Your EKS cluster's CoreDNS pods are crashing with OOMKilled. Node CPU is high due to many short-lived Jobs spawning. You use Cluster Autoscaler. What is the best combination fix?****

- A. Increase CoreDNS memory limits, use Fargate for CoreDNS, and implement Karpenter with consolidation + use proper PodDisruptionBudgets and resource requests for Jobs
- B. Delete CoreDNS
- C. Switch to ECS
- D. Overprovision nodes by 200%

****Q17. You need to enforce that all EBS volumes are encrypted and that all S3 buckets block public access across entire organization. Where do you enforce preventatively BEFORE creation?****

- A. AWS Config Rules only

B. SCPs with conditions `ec2:Encrypted` and `s3:x-amz-acl` + AWS Config for detection, plus enable EBS encryption by default at account level

C. IAM policy alone

D. CloudTrail

****Q18.** An RDS Postgres instance on gp3 runs out of IOPS during nightly batch. You see WriteLatency spikes. Storage is 1000 GiB with 3000 IOPS provisioned. Batch needs 12000 IOPS for 1 hour. Most cost-effective 2024-2025 solution?

A. Change to io2 with 12000 IOPS permanently

B. Modify gp3 to provision 12000 IOPS and 500 MiB/s throughput before batch, then scale down after (gp3 allows independent IOPS/throughput scaling)

C. Add Read Replica

D. Migrate to Aurora Serverless v1

****Q19.** You have CloudFront distribution fronting S3 + ALB origins. Users request `/api/*` should go to ALB with caching disabled, `/static/*` to S3 with caching. How to implement securely with Origin Access?

A. One behavior only

B. Two cache behaviors: `/api/*` with Cache Policy `CachingDisabled` and Origin Request Policy `AllViewer`, ALB origin; `/static/*` with `CachingOptimized`, S3 origin using Origin Access Control (OAC)

C. Use two CloudFront distributions

D. Use S3 static website hosting with public bucket

****Q20.** A company needs to capture all VPC flow logs, DNS query logs (Route 53 Resolver), and CloudTrail events in central security account with near-real-time detection. Architecture?

A. Enable VPC Flow Logs to S3 in each account; manual review

B. Use AWS Organizations + enable organization trail for CloudTrail to central S3, centralize VPC Flow Logs to Kinesis Data Firehose -> S3 + Athena, Route53 Resolver query logging to S3/OpenSearch, and Security Lake or OpenSearch + GuardDuty

C. Use CloudWatch Logs only in each account

D. Disable logging to save cost

Intermediate Answer Key & Explanations

****Q11 Answer:** B - Use S3 Express One Zone directory buckets for high TPS per prefix

While classic S3 bucket prefix limits were removed, extremely high per-prefix TPS (like 5k PUTs/sec sequential) can still cause throttling in general-purpose buckets. S3 Express One Zone directory buckets provide 10x higher performance and single-digit ms latency for such workloads.

****Q12 Answer:** B - Lifecycle Hook + warmup pool and increase ASG health check grace period

ELB health checks start immediately, so short grace period causes new instances to be terminated before app starts. A warmup pool and 180s+ grace period allow JIT compilation and cache warmup to succeed.

****Q13 Answer:** C - Gateway VPC Endpoint for S3 with endpoint policy

Gateway endpoints for S3/DynamoDB are free, route via prefix list, and support endpoint policies to restrict bucket access. Interface endpoints for S3 exist but incur hourly cost and are for niche PrivateLink needs.

****Q14 Answer:** B - Create Gateway Endpoint for DynamoDB and Interface Endpoint for Secrets Manager

Lambda in VPC loses internet by default; it needs VPC endpoints to reach AWS services privately. Gateway for DynamoDB (free) and Interface Endpoint (PrivateLink) for Secrets Manager avoids NAT cost and timeouts.

****Q15 Answer:** A - Route 53 latency-based routing with health checks

Latency-based routing directs users to lowest latency region with health checks for automatic failover at DNS layer. Global Accelerator is also valid, but Route 53 + regional API Gateway custom domains is native and widely used.

****Q16 Answer:** A - Increase CoreDNS limits, use Karpenter, set resource requests

CoreDNS OOM due to unreserved burstable nodes and Job spikes is common; fixing requests/limits, using Karpenter for faster consolidation/scaling, and PDBs ensures CoreDNS stability. Overprovisioning alone is wasteful.

****Q17 Answer:** B - SCPs + EBS default encryption + Config

SCPs can Deny ec2:CreateVolume when ec2:Encrypted:false and Deny S3 Put* with public ACLs, preventing non-compliant creation at org level. AWS Config detects drift, and account-level EBS encryption by default ensures compliance.

****Q18 Answer: B - Modify gp3 to provision IOPS/throughput temporarily then scale down****

gp3 decouples IOPS and throughput from storage size (unlike gp2), allowing you to provision 12k IOPS for batch window and revert. This is far cheaper than permanent io2 provisioning for a 1-hour workload.

****Q19 Answer: B - Two cache behaviors with CachingDisabled for /api and OAC for S3****

Separate behaviors allow path-based routing with different cache policies. OAC is the 2023+ replacement for OAI for secure S3 origin access, and disabling caching for API avoids serving stale dynamic responses.

****Q20 Answer: B - Org Trail + centralized flow logs to Firehose + Resolver logs + Security Lake****

A multi-account security baseline uses Organization CloudTrail, centralized VPC Flow Logs via Kinesis Firehose to central account (or S3 managed by Security Lake), and Resolver query logs aggregated for detection. Security Lake normalizes to OCSF.

3. Advanced Level (10 Questions)

****Q21. You are designing a multi-tenant SaaS control plane on EKS that uses IAM Roles for Service Accounts (IRSA) for tenant isolation. A tenant's pod is able to assume another tenant's IAM role. What is the most likely misconfiguration in trust policy?****

- A. Trust policy uses StringLike: aud: sts.amazonaws.com but missing StringEquals: sub = system:serviceaccount:tenant-a:sa
- B. Using EKS Pod Identity instead of IRSA
- C. Using instance profile
- D. Using OIDC provider URL incorrectly

****Q22. You have DynamoDB table with 100k WCU, using on-demand bursts. Application does batch hot partition write to same partition key for 10 minutes (e.g., tenantId=GLOBAL). Throttling persists even with DAX. What advanced pattern fixes this without increasing WCU?****

- A. Increase WCU to 200k
- B. Use SQS buffer + randomized suffix (write-sharding) with aggregation via Streams + Lambda: write to GLOBAL#rand(1-20) and aggregate reads
- C. Use Strongly consistent reads
- D. Switch to RDS

****Q23. Your organization uses AWS Direct Connect + Transit Gateway + Direct Connect Gateway. You have 50 VPCs and on-premises ASN 65000. You advertise 200 on-prem routes over Direct Connect to AWS. You hit route quota on Transit Gateway route table. Optimal fix?****

- A. Increase TGW route quota via ticket to 10000 (still not enough scalable long-term)
- B. Summarize on-prem routes to aggregate CIDRs before advertising via BGP, use route filtering, and leverage Transit Gateway Connect or segments
- C. Create more Transit Gateways
- D. Move to VPN only

****Q24. A Step Functions Express workflow orchestrates API calls for financial transaction requiring exactly-once semantics and idempotency across retries. Step Functions retries a Lambda that called third-party payment API, causing duplicate charge. What design ensures idempotency?****

- A. Rely on Express workflow exactly-once
- B. Lambda generates idempotency key (transactionId) stored in DynamoDB with conditional write (attribute_not_exists) and passes to payment API; check before calling external API; use DynamoDB Transactions
- C. Increase retry backoff
- D. Switch to Standard workflow alone

****Q25. You run Aurora PostgreSQL Global Database with primary in us-east-1, secondary in eu-west-1 with <1 sec lag. You need RTO <1 min, RPO <5 sec for regional failure, and to prevent split-brain writes. What configuration?****

- A. Use managed failover with Aurora Global Database Failover, enable Global Database Write Forwarding disabled for secondary, use Route53 health checks + cluster endpoint, ensure application uses idempotent writes
- B. Promote secondary manually after crash

C. Use snapshot copy

D. Use DMS replication

****Q26.** An account's S3 data lake has 500M objects. Athena queries scanning entire bucket are expensive. You need to provide sub-second dashboard queries for recent 30 days, with row-level security per team, while still keeping historical Parquet. Which stack in 2025?******

A. Keep Athena only

B. Use S3 Tables (Apache Iceberg) with compaction + S3 Metadata + Lake Formation row filters + Athena v3, and put hot data into CloudWatch Metrics

C. Use S3 Tables with automatic compaction, Athena + Lake Formation row-level security, and Amazon QuickSight SPICE or Redshift Serverless or Athena with partition projection + result cache, plus materialized view via Glue ETL job aggregating last 30 days

D. Move all to DynamoDB

****Q27.** You use AWS KMS multi-Region keys for encryption across regions. After rotating primary key in us-east-1, data encrypted in eu-west-1 can't be decrypted. What did you miss?******

A. Rotation of multi-Region primary does not automatically rotate replicas; you must manually rotate replica keys or use primary to create new primary key material and sync; also need to ensure alias points and application fetched new key ID

B. KMS rotation breaks old ciphertext

C. Multi-Region keys don't support rotation

D. Need to re-encrypt all data immediately

****Q28.** Your AWS Lambda function with 10GB RAM, 6 vCPU, Graviton2, processing video, cold start 8 seconds. Image is 5GB container. You need P99 latency <1 sec with sporadic 1000 concurrent invocations. Cost-optimized architecture?******

A. Increase to 12GB

B. Use Provisioned Concurrency + SnapStart? (SnapStart only Java) -> Use Lambda Provisioned Concurrency with optimized 2GB image (multi-stage) using Lambda's new faster image pull + Tiered caching, plus Lambda Function URL with response streaming, and consider AWS Fargate or Compute Savings

C. Correct answer: Use Lambda Provisioned Concurrency for baseline + optimize container multi-stage to <1GB, use Lambda Arm64, enable Lambda SnapStart not applicable for container, so use Provisioned Concurrency + ElastiCache, also consider moving to Fargate for sustained workloads

D. Use EC2 only

****Q29.** You must achieve 99.99% avail SLA for API across two regions, with no DNS propagation dependency for failover (needs <30 sec). Traffic must be anycasted. Which AWS setup?******

A. Route 53 failover routing

B. AWS Global Accelerator with endpoint groups in two regions (ALB + health checks), client affinity, instant failover at edge

C. CloudFront alone

D. Transit Gateway peering

****Q30.** A CI/CD pipeline builds container and deploys to EKS. Security requires: no long-lived AWS credentials in GitHub, scanned images with no critical CVEs, signed images verified in cluster, and least-privilege IAM for deploy. Modern 2025 solution?******

A. Store AWS_ACCESS_KEY in GitHub Secrets

B. Use GitHub OIDC federation with IAM role (no keys), ECR with enhanced scanning (Inspector), AWS Signer + Notation, and Kyverno/Policy Controller for signature verification, deploy via ArgoCD with IRSA Pod Identity

C. Use Jenkins with static credentials

D. Deploy directly from developer laptop

Advanced Answer Key & Explanations

****Q21 Answer:** A - Trust policy missing sub condition******

IRSA relies on OIDC token's sub claim to bind SA to role. If trust policy only checks aud, any SA in cluster can assume role. Fix requires StringEquals: sub = system:serviceaccount:<ns>:<sa> and aud: sts.amazonaws.com.

****Q22 Answer:** B - Write-sharding with randomized suffix + Streams aggregation******

Hot partition on single PK exceeds 1k WCU per partition limit (and 3k RCU). Sharding write across 20 logical partitions spreads throughput. Reads aggregate via DynamoDB Streams and eventual consistency, avoiding throttling without over-provisioning.

****Q23 Answer: B - Summarize routes to aggregate CIDRs, use filtering****

Transit Gateway route tables default quota 10k routes; advertising 200 more-specific /24s is inefficient. Best practice is summarize on-prem to supernets (e.g., /16), filter at DX GW, and segment via TGW routing domains/peering for scale.

****Q24 Answer: B - Lambda idempotency via DynamoDB conditional write****

Step Functions Express is at-least-once for external calls. Exactly-once must be implemented in business logic using idempotency key with DynamoDB conditional write and storing payment gateway response to prevent duplicate charges on retry.

****Q25 Answer: A - Managed failover with write forwarding disabled****

Aurora Global DB managed failover achieves <1 min RTO with data lag typically <1 sec for RPO. Write forwarding must be off for secondary to avoid split-brain, and app must be aware of cluster endpoints and idempotent recovery after DNS.

****Q26 Answer: C - S3 Tables + compaction + Lake Formation + materialized view****

S3 Tables (Iceberg) provides ACID compaction and time travel, reducing small files pain at 500M objects. Combined with Lake Formation row filters for multi-tenancy and SPICE/Redshift Serverless for sub-second BI is the modern lakehouse pattern.

****Q27 Answer: A - Replica keys don't auto-rotate****

Multi-Region primary rotation only rotates primary key material; replicas must be rotated independently via ReplicateKey update or manual rotation. Applications caching key ID will also fail if not refreshed to alias.

****Q28 Answer: C - Provisioned Concurrency + optimized image, or move to Fargate****

Large container image worsens cold start; Provisioned Concurrency keeps execution environments warm for P99 <1s. Real fix is multi-stage build to shrink to <500MB and evaluate Fargate for sustained video workloads rather than Lambda.

****Q29 Answer: B - AWS Global Accelerator with endpoint groups****

Global Accelerator uses AWS global edge anycast IPs, independent of DNS TTL/caching, providing <30s failover via health checks at edge. Route 53 failover depends on DNS TTL (60-300s) and client caching.

****Q30 Answer: B - GitHub OIDC + ECR Inspector + Signer + Policy Controller + ArgoCD with IRSA****

This is the 2025 secure software supply chain: OIDC federation eliminates long-lived keys, Inspector scans CVEs, Signer + Notation signs OCI artifacts, admission controller verifies signature in cluster, and ArgoCD with short-lived IRSA reduces blast radius.

Course File: 02_Azure.md

Course: Microsoft Azure

1. Beginner Level (10 Questions)

****Q1: What is Microsoft Azure primarily classified as?****

- A) A cloud computing platform offering IaaS, PaaS, and SaaS services
- B) A virtualization software for on-premises servers only
- C) A database management system
- D) An operating system for mobile devices

****Q2: In Azure Resource Manager (ARM), what is the primary purpose of a Resource Group?****

- A) To group resources that share the same lifecycle, permissions, and policies for deployment and management
- B) To provide physical isolation of servers in a datacenter
- C) To automatically backup resources
- D) To define network firewall rules

****Q3: You deploy a VM in Azure. Your company requires 99.99% SLA for VM uptime within a single region without using Availability Sets. Which feature should you use?****

- A) Deploy VMs across multiple Availability Zones within the same region
- B) Deploy VMs in a single Availability Set with 2 fault domains
- C) Deploy VMs in two different Azure regions
- D) Enable Azure Backup on a single VM

****Q4: Which Azure storage service is best suited for storing unstructured object data like images, videos, and backups accessible via REST API?****

- A) Azure Blob Storage
- B) Azure File Storage
- C) Azure Queue Storage
- D) Azure Disk Storage

****Q5: A developer wants to run small pieces of code in response to events (e.g., a file uploaded to Blob) without managing any servers or paying for idle time. Which service should they use?****

- A) Azure Functions (Consumption Plan)
- B) Azure Virtual Machines
- C) Azure Kubernetes Service (AKS)
- D) Azure App Service Environment v3

****Q6: Your organization has 100 developers. You need to manage identities and provide SSO to thousands of SaaS apps like M365 and Salesforce. What is the Azure service for this?****

- A) Microsoft Entra ID (formerly Azure Active Directory)
- B) Microsoft Entra Domain Services
- C) Azure Key Vault
- D) Azure Role-Based Access Control (RBAC) alone

****Q7: What does the concept of a Virtual Network (VNet) in Azure provide?****

- A) An isolated, private network in the cloud to securely run VMs and services with control over IP ranges, subnets, and routing
- B) A public internet gateway with unlimited bandwidth
- C) A content delivery network for caching
- D) A DNS service for internet domains only

****Q8: Which pricing model allows you to pay only for what you use with no upfront commitment, ideal for dev/test?****

- A) Pay-As-You-Go
- B) Reserved Instances (1 or 3 years)
- C) Spot VMs only
- D) Enterprise Dev/Test license

****Q9: You need to host a simple web app with built-in CI/CD, scaling, and custom domains without managing OS patches. Which PaaS option is most appropriate?****

- A) Azure App Service
- B) Azure Virtual Machine Scale Sets
- C) Azure Bare Metal Infrastructure
- D) Azure ExpressRoute

****Q10: What is the main difference between Azure Availability Set and Availability Zone?****

- A) Availability Sets protect within a datacenter (fault/update domains), while Availability Zones are physically separate datacenters in a region
- B) Availability Sets are across regions, Zones are within a datacenter
- C) They are identical, just different names
- D) Availability Sets are for storage, Zones are for compute

Beginner Answer Key & Explanations

****Q1 Answer: A - A cloud computing platform offering IaaS, PaaS, and SaaS services****

Azure is Microsoft's public cloud platform providing over 200+ services across infrastructure, platform, and software as a service. It is not just virtualization or a single product.

****Q2 Answer: A - To group resources that share the same lifecycle, permissions, and policies for deployment and management****

Resource Groups are logical containers for resources that should be managed together, deployed, updated, or deleted as a unit. They also scope RBAC and Azure Policy.

****Q3 Answer: A - Deploy VMs across multiple Availability Zones within the same region****

Availability Zones offer 99.99% VM SLA when using 2+ VMs in 2+ zones, as each zone is independent power, cooling, and networking. Availability Sets only provide 99.95% SLA and are legacy within a single datacenter.

****Q4 Answer: A - Azure Blob Storage****

Blob Storage is optimized for massive amounts of unstructured data via REST, with tiers Hot/Cool/Cold/Archive. File Storage provides SMB file shares, not REST object access.

****Q5 Answer: A - Azure Functions (Consumption Plan)****

Functions is serverless compute where you run event-driven code and pay per execution (100ms increments) with automatic scale to zero. VMs and AKS require server/cluster management.

****Q6 Answer: A - Microsoft Entra ID (formerly Azure Active Directory)****

Microsoft Entra ID is the cloud identity and access management service providing SSO, MFA, and Conditional Access for users. Key Vault is for secrets, not user identity.

****Q7 Answer: A - An isolated, private network in the cloud to securely run VMs and services with control over IP ranges, subnets, and routing****

VNet is the fundamental building block for private networking in Azure, analogous to your on-prem network. It enables you to segment, filter traffic, and connect to on-prem via VPN/ExpressRoute.

****Q8 Answer: A - Pay-As-You-Go****

Pay-As-You-Go has no upfront cost and per-second billing, perfect for unpredictable or short-term workloads. Reserved Instances give discount but require 1-3 year commitment.

****Q9 Answer: A - Azure App Service****

App Service is a fully managed PaaS for web apps, APIs, and mobile backends handling OS patching, load balancing, and autoscale. VM Scale Sets is IaaS requiring OS management.

****Q10 Answer: A - Availability Sets protect within a datacenter (fault/update domains), while Availability Zones are physically separate datacenters in a region****

An Availability Zone is a unique physical location with independent infrastructure, providing higher resiliency than Availability Sets. Availability Sets distribute VMs across isolated hardware within one datacenter.

2. Intermediate Level (10 Questions)

****Q11: You have a Hub-Spoke topology with Hub VNet 10.0.0.0/16 peered to Spoke1 and Spoke2. VMs in Spoke1 cannot reach VMs in Spoke2. What is required?****

- A) Enable "Allow forwarded traffic" and "Allow gateway transit" plus a Network Virtual Appliance or Azure Route Server in Hub, or peer spokes directly
- B) VNet peering is transitive by default, so just wait 10 minutes
- C) Create NSG allowing all traffic
- D) Enable Azure Bastion

****Q12: An app running on Azure VM needs to access Azure Key Vault without storing any credentials in code or config. What is the secure, recommended approach?****

- A) Enable System-Assigned Managed Identity on the VM and grant it Key Vault Secrets User role via RBAC
- B) Store Client ID/Secret in appsettings.json
- C) Use a Shared Access Signature (SAS) token hardcoded
- D) Open Key Vault firewall to 0.0.0.0/0

****Q13: You need to enforce that all storage accounts in subscription "Prod" must use Private Endpoints and deny public network access. How do you implement this at scale?****

- A) Create Azure Policy with Deny effect auditing publicNetworkAccess and networkAcls.defaultAction, assign at subscription scope with remediation
- B) Manually configure each storage account firewall
- C) Use Azure Blueprints to copy settings
- D) Send an email to owners

****Q14: Your team uses Bicep for IaC. You have a module that should deploy only if environment is "prod". How do you implement this?****

- A) Use if (environment == 'prod') condition on the module/resource in Bicep
- B) Use a separate subscription
- C) Bicep does not support conditions, use ARM JSON copy loops
- D) Delete the resource manually after deployment

****Q15: You have an Azure Load Balancer (Standard, Layer 4) and an Application Gateway. When should you choose Application Gateway over Load Balancer?****

- A) When you need Layer 7 routing, SSL termination, WAF, and path-based routing for HTTP/S
- B) When you need ultra-low latency TCP/UDP load balancing
- C) When you need to load balance across regions
- D) When you need to balance non-HTTP traffic on port 22

****Q16: An AKS cluster needs to pull images from ACR securely without Kubernetes Secrets. The most secure, managed way is:****

- A) Attach ACR to AKS with az aks update --attach-acr which creates role assignment for kubelet Managed Identity with AcrPull role
- B) Create a service principal and store password in YAML
- C) Make ACR public anonymously
- D) Use imagePullSecrets with admin user enabled

****Q17: You need point-in-time restore for an Azure SQL Database up to 35 days and long-term retention for 10 years. How?****

- A) PITR is built-in (7-35 days configurable) and configure Long-Term Retention (LTR) policy on the server for weekly/monthly/yearly backups to Blob

- B) Enable Azure Backup vault only
- C) Use geo-restore only
- D) Export BACPAC files daily

****Q18: What is the difference between Azure RBAC roles "Contributor" and "User Access Administrator"?****

- A) Contributor can manage resources but not manage access; User Access Administrator can manage role assignments but not resources
- B) They are identical
- C) Contributor can only read
- D) User Access Administrator can delete subscriptions

****Q19: You deployed App Service with 2 deployment slots: Production and Staging. You want zero-downtime deployment with warming up before swapping. Best practice?****

- A) Deploy to Staging, validate, enable auto-swap or manually swap with preview, using WEBSITE_SWAP_WARMUP_PING_PATH
- B) Deploy directly to Production and restart
- C) Stop Production slot before deploy
- D) Use FTP only

****Q20: Your on-prem network 192.168.0.0/16 overlaps with Azure VNet 192.168.0.0/16 and you need hybrid connectivity via VPN. What must you do?****

- A) Re-address the Azure VNet to non-overlapping range (e.g., 10.0.0.0/16) before establishing Site-to-Site VPN, as overlapping ranges cannot route
- B) Enable NAT on VPN gateway to keep overlapping ranges
- C) Use ExpressRoute to ignore IP conflicts
- D) Just enable VNet peering

Intermediate Answer Key & Explanations

****Q11 Answer: A - Enable "Allow forwarded traffic" and "Allow gateway transit" plus a Network Virtual Appliance or Azure Route Server in Hub, or peer spokes directly****

VNet peering is non-transitive: spoke-to-spoke communication doesn't flow through hub by default. You need transitive routing via NVA/firewall in hub and user-defined routes, or create direct peerings between spokes.

****Q12 Answer: A - Enable System-Assigned Managed Identity on the VM and grant it Key Vault Secrets User role via RBAC****
Managed Identity provides an Entra ID identity for the Azure resource, eliminating credential storage. Azure AD authenticates the VM automatically to Key Vault using RBAC or access policies.

****Q13 Answer: A - Create Azure Policy with Deny effect auditing publicNetworkAccess and networkAcls.defaultAction, assign at subscription scope with remediation****

Azure Policy enforces governance at scale with Deny/AuditIfNotExists effects, and can auto-remediate non-compliant resources. Manual configuration doesn't scale and doesn't prevent drift.

****Q14 Answer: A - Use if (environment == 'prod') condition on the module/resource in Bicep****

Bicep supports conditional deployment via if keyword at resource, module, and loop level, which translates to ARM condition. This enables environment-aware infrastructure as code.

****Q15 Answer: A - When you need Layer 7 routing, SSL termination, WAF, and path-based routing for HTTP/S****

Application Gateway is L7 HTTP/S load balancer with WAF, cookie affinity, and URL routing. Azure Load Balancer is L4 TCP/UDP with high performance and low latency.

****Q16 Answer: A - Attach ACR to AKS with az aks update --attach-acr which creates role assignment for kubelet Managed Identity with AcrPull role****

This command assigns AcrPull role to the kubelet's managed identity, avoiding secrets. Using admin user or service principal secrets is legacy and insecure.

****Q17 Answer: A - PITR is built-in (7-35 days configurable) and configure Long-Term Retention (LTR) policy on the server for weekly/monthly/yearly backups to Blob****

Azure SQL automatically handles PITR backups to RA-GRS storage with configurable retention. LTR uses separate backup blobs with up to 10-year retention for compliance, distinct from short-term PITR.

****Q18 Answer: A - Contributor can manage resources but not manage access; User Access Administrator can manage role assignments but not resources****

Contributor is data-plane/resource management without IAM rights, following least privilege. User Access Administrator can grant roles (control-plane IAM) to implement separation of duties.

****Q19 Answer: A - Deploy to Staging, validate, enable auto-swap or manually swap with preview, using WEBSITE_SWAP_WARMUP_PING_PATH****

Deployment slots enable blue-green deployments with warm-up to avoid cold start. Swap with preview ensures traffic routing only after app initialization ping succeeds, providing true zero-downtime.

****Q20 Answer: A - Re-address the Azure VNet to non-overlapping range (e.g., 10.0.0.0/16) before establishing Site-to-Site VPN, as overlapping ranges cannot route****

Overlapping IP ranges break routing as VPN gateway cannot disambiguate destinations. You must plan non-overlapping RFC1918 space; Azure Virtual Network NAT can help for limited scenarios but re-addressing is standard.

3. Advanced Level (10 Questions)

****Q21: You are designing Azure Landing Zone for 500 subscriptions. App teams require Private Endpoints for PaaS services but DNS resolution fails for privatelink.blob.core.windows.net. What is the enterprise-scale solution?****

A) Deploy Azure Private DNS Zones in connectivity subscription hub, link to all spokes, with conditional forwarding and Azure Policy to ensure auto-registration of A records from Private Endpoints

B) Use hosts file entries on each VM

C) Use public DNS for private IPs

D) Disable Private Endpoints

****Q22: A global web app is deployed in West US, East US, and West Europe. You need geo-routing to nearest healthy region with TLS termination at edge, WAF, and private origin with Private Link. Which solution?****

A) Azure Front Door Premium with custom domains, WAF policy, origin groups with health probes, and Private Link to App Service / Load Balancer origins

B) Azure Traffic Manager with Performance routing

C) Azure Application Gateway in one region

D) Azure Load Balancer Global Tier

****Q23: In AKS, you need pod-level network isolation and to assign VNet IPs directly to pods for NSG filtering (no overlay). You also have 5000 pods per cluster. Which CNI and caveats apply?****

A) Azure CNI (not overlay) gives VNet IPs to pods enabling NSG support, but requires large subnet planning (/16) for IP exhaustion; use Azure CNI Overlay or Cilium for better scale

B) Kubenet gives VNet IPs by default with infinite scale

C) Azure CNI Overlay cannot use NSG

D) Use Flannel only

****Q24: You need to prevent exfiltration of data via azcopy to external storage accounts while allowing access to approved storage accounts only. Enterprise control?****

A) Implement Azure Storage Resource Instance Rules + Service Endpoint Policies + Azure Firewall / Private Link with Entra ID Conditional Access for workload identities and Trusted Access

B) Just block internet with NSG

C) Use SAS tokens only

D) Enable public access but monitor logs

****Q25: A company runs 1000 VMs using Spot VMs for batch jobs. Jobs are evicted frequently causing failures. What resilient architecture improves completion while keeping 60-70% cost saving?****

A) Use Spot with Eviction Policy Delete, mix with Standard VMs via Virtual Machine Scale Sets flexible orchestration with priority mix (e.g., 80% Spot, 20% Standard), handle eviction notices via Scheduled Events and checkpointing

B) Use only Spot with no handling

C) Use Availability Zones to prevent eviction

D) Use Reserved Instances only

****Q26:** You have ExpressRoute Circuit (10 Gbps) and need transitive routing between 50 spokes, deep packet inspection, and inspection of East-West traffic with <2ms added latency. Which hub design?

A) Azure Virtual WAN Secured Hub with Azure Firewall Manager or Hub with Gateway Load Balancer + third-party NVA farm + Azure Route Server for BGP

B) Basic Hub with VNet peering only

C) Use User-Defined Routes without firewall

D) Use ExpressRoute directly to each spoke

****Q27:** For Azure SQL Hyperscale with 100TB database, you need near-instant backup (no impact) and read scale-out to 30 replicas for analytics without copying data. How does Hyperscale enable this?

A) Hyperscale uses page servers with log-based snapshots and separate compute replicas sharing same storage layer, enabling instant backups via snapshots and up to 30 Named Replicas

B) Use BACPAC export

C) Hyperscale doesn't support PITR

D) Use transactional replication only

****Q28:** You are implementing Zero Trust for Entra ID. Requirement: Require compliant device OR Entra hybrid joined device for accessing Azure Portal, block legacy auth, require phishing-resistant MFA for admins, and risk-based access. What Conditional Access configuration?

A) Multiple policies: 1) Block legacy auth client apps, 2) Require compliant/Hybrid Joined for all users on Azure Management, 3) Require phishing-resistant MFA (FIDO2/Windows Hello) for high-risk + admin roles with Authentication Strengths + Entra ID Protection risk policy

B) Single policy allowing all

C) Disable MFA for convenience

D) Use only Identity Protection without CA

****Q29:** You need to audit and remediate 10,000 existing storage accounts that have allowBlobPublicAccess=true. You want to set it to false and have continuous compliance without deployment failures. Implementation?

A) Azure Policy with Modify effect using managed identity remediation task to set allowBlobPublicAccess=false, with roleDefinitionIds for Storage Account Contributor, deployed via remediation task and DeployIfNotExists

B) Azure Policy Deny only

C) PowerShell script run once

D) Manually edit portal

****Q30:** Your organization uses Bicep + Azure DevOps. A PR triggers what-if but deployments fail intermittently with "Another deployment in progress". You have 20 pipelines deploying to same Resource Group concurrently. Fix?

A) Implement deployment concurrency control using deployment locks / pipeline dependsOn, serialize deployments to same RG, use deployment stacking or distinct deployment names with incremental mode, and check Microsoft.Resources/deployments locks

B) Increase agent count

C) Use Complete mode always

D) Delete resource group before each deployment

Advanced Answer Key & Explanations

****Q21 Answer:** A - Deploy Azure Private DNS Zones in connectivity subscription hub, link to all spokes, with conditional forwarding and Azure Policy to ensure auto-registration of A records from Private Endpoints

Private Endpoints need Private DNS A record in privatelink zone to resolve to private IP. Enterprise pattern centralizes zones in hub and links VNets to avoid split-brain DNS, enforced by Policy that deploys Private DNS Zone Group.

****Q22 Answer:** A - Azure Front Door Premium with custom domains, WAF policy, origin groups with health probes, and Private Link to App Service / Load Balancer origins

Front Door is Microsoft's global L7 reverse proxy with anycast edge, TLS termination, and Private Link integration to prevent origin public exposure. Traffic Manager is DNS-only, no WAF or TLS termination, and fails for Private Link.

****Q23 Answer: A - Azure CNI (not overlay) gives VNet IPs to pods enabling NSG support, but requires large subnet planning (/16) for IP exhaustion; use Azure CNI Overlay or Cilium for better scale****

Traditional Azure CNI allocates VNet IP per pod, allowing direct NSG filtering but consuming subnet IPs quickly. Overlay (with Cilium) uses overlay network for pods, scaling to thousands without subnet exhaustion while still supporting Network Policy.

****Q24 Answer: A - Implement Azure Storage Resource Instance Rules + Service Endpoint Policies + Azure Firewall / Private Link with Entra ID Conditional Access for workload identities and Trusted Access****

Defense in depth: Service Endpoint policies restrict to allowed storage accounts, Resource Instance Rules allow only specific resources via RBAC, and Private Link removes public access. NSG alone cannot filter storage account FQDN.

****Q25 Answer: A - Use Spot with Eviction Policy Delete, mix with Standard VMs via Virtual Machine Scale Sets flexible orchestration with priority mix (e.g., 80% Spot, 20% Standard), handle eviction notices via Scheduled Events and checkpointing****

Flexible VMSS allows automated replenishment and priority mix balancing cost vs availability. Handling 30-sec eviction notice via Azure Metadata Service (Scheduled Events) with checkpointing ensures batch progress isn't lost.

****Q26 Answer: A - Azure Virtual WAN Secured Hub with Azure Firewall Manager or Hub with Gateway Load Balancer + third-party NVA farm + Azure Route Server for BGP****

Virtual WAN Secured Hub provides managed transitive routing, and Gateway Load Balancer chains NVA for East-West inspection at scale. Basic peering without routing or firewall doesn't provide inspection or transitive control.

****Q27 Answer: A - Hyperscale uses page servers with log-based snapshots and separate compute replicas sharing same storage layer, enabling instant backups via snapshots and up to 30 Named Replicas****

Hyperscale architecture separates compute and storage into page servers with log service, giving near-instant snapshot backups regardless of size and highly scalable read replicas without data copy.

****Q28 Answer: A - Multiple policies: 1) Block legacy auth client apps, 2) Require compliant/Hybrid Joined for all users on Azure Management, 3) Require phishing-resistant MFA (FIDO2/Windows Hello) for high-risk + admin roles with Authentication Strengths + Entra ID Protection risk policy****

Zero Trust requires layered CA policies targeting conditions incrementally, not monolithic policy. Authentication Strengths enforce phishing-resistant methods, and risk policies integrate with Entra ID Protection user/sign-in risk.

****Q29 Answer: A - Azure Policy with Modify effect using managed identity remediation task to set allowBlobPublicAccess=false, with roleDefinitionIds for Storage Account Contributor, deployed via remediation task and DeployIfNotExists****

Modify effect remediates existing non-compliant resources via managed identity. Deny only blocks future deployments, not remediate existing. Portal manual edit doesn't scale to 10k accounts and lacks continuous enforcement.

****Q30 Answer: A - Implement deployment concurrency control using deployment locks / pipeline dependsOn, serialize deployments to same RG, use deployment stacking or distinct deployment names with incremental mode, and check Microsoft.Resources/deployments locks****

ARM allows only one deployment per RG at a time (per deployment name lock). Concurrent deployments cause 409 Conflict. Serialization, using Deployment Stacks to manage lifecycle, and distinct names avoid collisions versus increasing agents.

Course File: 03_GCP.md

Course: Google Cloud Platform

1. Beginner Level (10 Questions)

****Q1.**** What is the primary purpose of a GCP Project?

- A) To store billing information only
- B) To be a fundamental organizing entity that holds resources, enables APIs, and isolates billing and permissions
- C) To manage physical data centers
- D) To define network routing rules globally

****Q2.**** Which of the following best describes a GCP Region and Zone?

- A) A Region is a single data center, and a Zone is a continent
- B) A Region is a specific geographical location composed of multiple Zones, where Zones are isolated deployment areas within a Region
- C) Regions and Zones are the same thing
- D) A Zone contains multiple Regions

****Q3.**** You want to store unstructured images for a website with high durability and low cost. Which storage class in Cloud Storage is the default cost-effective choice for frequently accessed data?

- A) Archive
- B) Nearline
- C) Standard
- D) Coldline

****Q4.**** Which IAM role structure follows Google's principle of least privilege?

- A) Always grant Owner at organization level
- B) Grant predefined roles with minimal permissions needed at the lowest resource level required
- C) Grant Editor role to all users for simplicity
- D) Use only custom roles for everything

****Q5.**** What is the difference between Google Compute Engine and Cloud Run?

- A) Compute Engine is for serverless containers, Cloud Run is for VMs
- B) Compute Engine provides configurable virtual machines (IaaS), while Cloud Run is a fully managed serverless platform for containerized applications
- C) There is no difference
- D) Compute Engine only runs Windows, Cloud Run only runs Linux

****Q6.**** Which service is GCP's fully managed data warehouse for analytics?

- A) Cloud SQL
- B) BigQuery
- C) Cloud Spanner
- D) Firestore

****Q7.**** What does VPC stand for and what does it provide?

- A) Virtual Private Cloud - an isolated private network for your GCP resources
- B) Virtual Public Cloud - a public internet gateway
- C) Verified Private Connection - a VPN service
- D) Virtual Partition Cluster - a Compute Engine cluster

****Q8.**** Which command-line tool is used to manage GCP resources from terminal?

- A) aws cli

- B) gcloud cli
- C) az cli
- D) kubectl only

****Q9.**** You deploy an application and want GCP to automatically manage patching, scaling, and availability of the underlying infrastructure for a MySQL database. Which service should you choose?

- A) Compute Engine with MySQL installed manually
- B) Cloud SQL
- C) Persistent Disk
- D) Cloud Storage

****Q10.**** What is Cloud Billing's resource hierarchy order from top to bottom?

- A) Project > Folder > Organization > Resource
- B) Organization > Folder > Project > Resource
- C) Folder > Organization > Project > Resource
- D) Resource > Project > Folder > Organization

Beginner Answer Key & Explanations

****Q1 Answer: B**** - A GCP Project is the base entity for organizing resources, enabling APIs, managing permissions, and billing isolation. Without a Project, no GCP resources can be created.

****Q2 Answer: B**** - Regions are geographic locations (e.g., us-central1) with 3+ Zones for high availability. Zones are isolated failure domains within a region, allowing fault-tolerant deployments.

****Q3 Answer: C**** - Standard Storage is ideal for hot, frequently accessed data like website assets. It offers high performance and 99.999999999% durability while other classes are for infrequently accessed data.

****Q4 Answer: B**** - Least privilege means granting only necessary permissions at the narrowest scope. Predefined roles are maintained by Google and should be preferred over primitive roles like Owner/Editor/Viewer.

****Q5 Answer: B**** - Compute Engine is IaaS requiring you to manage VMs, OS, and scaling. Cloud Run abstracts infrastructure, auto-scales to zero, and runs stateless containers on request.

****Q6 Answer: B**** - BigQuery is serverless, petabyte-scale data warehouse with SQL interface for analytics. Cloud SQL is OLTP, Firestore is NoSQL document DB, Spanner is globally distributed relational DB.

****Q7 Answer: A**** - A VPC is a global, private, isolated software-defined network. It lets you define subnets, firewalls, routes, and connects resources securely within GCP and to on-premises via VPN/Interconnect.

****Q8 Answer: B**** - gcloud is GCP's official CLI for creating, managing, and interacting with GCP resources. It also includes gsutil for storage and bq for BigQuery.

****Q9 Answer: B**** - Cloud SQL is fully managed relational DB (MySQL, PostgreSQL, SQL Server) handling backups, replication, patching, and scaling. Self-managing on GCE adds operational overhead.

****Q10 Answer: B**** - Organization is top node (your company), Folders group Projects (e.g., by department), Projects contain actual Resources (VMs, buckets). IAM policies inherit top-down.

2. Intermediate Level (10 Questions)

****Q11.**** Your team runs GKE Autopilot and needs to securely pull private images from Artifact Registry. What is the recommended least-privilege approach?

- A) Create a service account key and mount it as a Kubernetes secret with storage.objectViewer
- B) Use Workload Identity to bind the Kubernetes service account to a Google service account with artifactregistry.reader role
- C) Make the Artifact Registry repository public
- D) Give the GKE nodes the Owner role

****Q12.**** You have a Cloud Function (2nd gen) triggered by Pub/Sub that must process messages in order and guarantee exactly-once processing semantics. How do you address this?

- A) Pub/Sub guarantees exactly-once and ordering by default, no action needed

B) Enable Pub/Sub ordered delivery with ordering keys, and make your Cloud Function idempotent because Pub/Sub guarantees at-least-once delivery

C) Use Cloud Tasks instead of Pub/Sub

D) Set max concurrency to 1000 to ensure order

****Q13.**** An application in us-central1 needs <10ms latency to read/write a relational database that spans us-central1 and us-east1 for global consistency. Which database should you choose?

A) Cloud SQL with cross-region read replicas

B) Cloud Spanner

C) Bigtable

D) Firestore in Datastore mode

****Q14.**** You need to enforce that only images from your org's Artifact Registry with binary attestation can be deployed to GKE. Which feature should you use?

A) VPC Service Controls

B) Binary Authorization

C) Organization Policy Service with IAM Deny

D) Security Command Center

****Q15.**** Your Cloud Run service connects to Cloud SQL and experiences cold start latency due to connection establishment. What is the best practice fix in 2024-2026?

A) Set min instances to 0 and increase memory to 32Gi

B) Use Cloud SQL Auth Proxy sidecar is deprecated; instead use the built-in Cloud SQL connector via volume mount / --add-cloudsql-instances and implement connection pooling (e.g., HikariCP / pgxpool) with Unix socket

C) Make Cloud SQL public with 0.0.0.0/0 firewall

D) Migrate from Cloud Run to App Engine Standard

****Q16.**** You manage 200 projects. You want to prevent anyone from creating external IP addresses and from disabling Cloud Audit Logs. Where do you enforce this?

A) In each Project's IAM policy

B) Using Organization Policies (Org Policies) constraints: constraints/compute.vmExternalIpAccess = DENY and constraints/gcp.detailedAuditLoggingMode and disable audit log exemptions via constraints

C) Using VPC firewall rules

D) Using Cloud Armor

****Q17.**** You are designing a hub-and-spoke VPC architecture with Shared VPC and need centralized egress inspection. What is the modern recommended pattern?

A) Create 200 separate VPCs with peering mesh

B) Hub VPC hosts Shared VPC with centralized firewall/NVA, spokes are service projects; use Private Service Connect and Cloud NAT + custom route advertisements for egress inspection via hub

C) Use only one VPC for everything

D) Use legacy Shared VPC without VPC Service Controls

****Q18.**** A Dataflow streaming job reading from Pub/Sub shows high system lag and hot keys. What optimizations should you apply?

A) Increase number of zones and use default windowing

B) Re-key data with salted keys, enable Dataflow Shuffle, use appropriate windowing/triggers, and check for fusion optimization breaks

C) Switch to batch mode

D) Delete the job and recreate

****Q19.**** You need to export BigQuery audit logs and cost data to analyze who queried what and the slot consumption by team. What solution?

A) Enable INFORMATION_SCHEMA queries and export Audit Logs via Log Sink to BigQuery dataset, then join with INFORMATION_SCHEMA.JOBS and label billing export

B) Only use Cloud Logging console manually

C) Use Cloud Trace

D) Enable Dataflow

****Q20.**** Your organization uses Cloud Interconnect and you need to encrypt traffic in transit from on-prem to GCP with line-rate 100Gbps without managing your own VPN devices. Which option?

A) Use Cloud VPN over Interconnect with IPSec

B) Use MACsec for Cloud Interconnect (MACsec for Cloud Interconnect provides line-rate L2 encryption)

C) Use Cloud NAT

D) Use TLS only at application layer

Intermediate Answer Key & Explanations

****Q11 Answer: B**** - Workload Identity Federation for GKE avoids long-lived keys by federating K8s SA to Google SA. Granting artifactregistry.reader is minimal permission vs. storage legacy roles or Owner.

****Q12 Answer: B**** - Pub/Sub ordered delivery ensures order when using ordering keys, but still only offers at-least-once delivery. Your consumer must be idempotent and handle duplicates using message deduplication IDs.

****Q13 Answer: B**** - Cloud Spanner is horizontally scalable, strongly consistent, external-consistency relational DB for global workloads. Cloud SQL is regional with async replicas, Bigtable is wide-column NoSQL, Firestore is not ideal for relational low-latency multi-region OLTP.

****Q14 Answer: B**** - Binary Authorization enforces deploy-time attestations (from Cloud Build/Binauthz attestors) ensuring only verified images run. VPC SC is perimeter security, Org Policy prevents resource types but doesn't attest images.

****Q15 Answer: B**** - Since 2023, Cloud Run has native Cloud SQL integration (no sidecar proxy container needed). Combine with connection pooling and Unix sockets to reuse connections across warm instances, reducing cold start overhead significantly.

****Q16 Answer: B**** - Organization Policy Service enforces guardrails across org/folder/project hierarchy, e.g., constraints/compute.vmExternallpAccess and constraints/iam.disableAuditLoggingExemption. IAM manages who can do what, Org Policy manages what can be done.

****Q17 Answer: B**** - Hub-spoke with Shared VPC is current enterprise landing zone best practice. Centralized egress via hub with Cloud NAT/next-gen firewall VM/third-party NVA and Private Service Connect avoids peering sprawl and allows inspection and hierarchical firewall policies.

****Q18 Answer: B**** - Hot keys cause worker skew; salting distributes load. Dataflow Shuffle moves shuffle off workers. Windowing tuning and avoiding fusion breaks (e.g., unnecessary GroupByKey) reduces lag. System lag metric indicates backlog.

****Q19 Answer: A**** - Log Router sinks can export Admin Data Access logs to BigQuery. INFORMATION_SCHEMA.JOBS_BY_ORGANIZATION provides job stats, bytes, slots, labels. Joining with billing export labeled by team gives cost attribution per query/user.

****Q20 Answer: B**** - MACsec for Cloud Interconnect (GA 2023) provides Layer 2 encryption at line rate (10G/100G) without IPSec overhead. Cloud VPN over Interconnect adds encryption but caps at ~3Gbps per tunnel and adds complexity.

3. Advanced Level (10 Questions)

****Q21.**** You run a multi-tenant SaaS on GKE Enterprise with 5000 namespaces. You need per-tenant cost showback, network isolation with zero-trust, and policy enforcement preventing privileged pods. Which stack is optimal?

A) Use GKE cost allocation labels + billing export, Dataplane V2 with Cilium NetworkPolicies + Istio mTLS (or Anthos Service Mesh), Policy Controller (OPA Gatekeeper) with PSS baseline constraint templates

B) Use only kubectl top

C) Use Calico with HostNetwork true

D) Give all tenants cluster-admin

****Q22.**** A BigQuery query scans 10TB daily via BI Engine and costs spike. You need predictable cost and <1s dashboard latency. What design meets both in 2025?

A) Keep on-demand pricing and increase BI Engine reservation to 100GB

B) Switch to BigQuery Editions (Enterprise/Plus) with baseline + autoscaling slots, create materialized views and BI Engine reserved capacity, enable result caching and partition/cluster tables to reduce scan

C) Export to Cloud SQL

D) Disable caching

****Q23.**** Your organization requires CMEK for all storage including BigQuery, GKE secrets, and Pub/Sub, with auto-rotation and revocation capability. What is the complete implementation?

A) Use Google-managed keys only

B) Use Cloud KMS with CMEK: enable Cloud KMS API, create key ring + keys in desired region, grant encrypter/decrypter roles to service agents (bigquery, gke, pubsub), enforce org policy constraints/gcp.restrictCmekServices and constraints/gcp.restrictNonCmekServices, enable rotation schedule and use Cloud EKM if HSM needed, handle key destruction = data inaccessible

C) Use keys stored in Compute Engine metadata

D) Use customer-supplied encryption keys (CSEK) for everything

****Q24.**** You design event-driven architecture with Eventarc Advanced (GA 2024) for 100M Cloud Storage events/day across 200 buckets to trigger Cloud Run with filtering and dead-lettering. What configuration achieves scale?

A) Create 200 Cloud Functions triggers with no filters

B) Use Eventarc Advanced bus with custom pipeline: enable Eventarc Advanced API, create message bus, create pipeline with filters (bucket + event type prefix/suffix), set Pub/Sub as transport, configure Cloud Run as destination with retry policy + dead-letter Pub/Sub topic + IAM for eventarc.eventReceiver

C) Use Audit Logs only

D) Use Cloud Scheduler to poll buckets

****Q25.**** For a mission-critical workload requiring RPO=0 and RTO < 5min across regions us-central1 and us-east4, with stateful data in Compute Engine disks, which DR architecture?

A) Snapshot daily and manual restore

B) Use Compute Engine synchronous replication is impossible; deploy active-active with Regional Persistent Disks in each region (actually zonal), plus use asynchronous PD replication (Cross-region PD async replication GA 2024) or use database layer (Spanner) for RPO=0, and use Global Load Balancer + Managed Instance Groups + health checks + automated failover via DNS/GLB; for true RPO=0 on block storage, use application-level sync like MySQL group replication / use Filestore Enterprise multi-share or NetApp Volumes

C) Use only one zone

D) Use Local SSD

****Q26.**** You need to implement confident security posture across 500 projects: inventory assets, detect misconfigurations like public buckets, IAM anomalies, and remediate automatically. Which services together?

A) Only Cloud Logging

B) Security Command Center Premium + Asset Inventory + Event Threat Detection + Policy Analyzer + Security Health Analytics + custom remediation via Cloud Functions/Eventarc + VPC Service Controls for perimeter + Forseti is deprecated, replaced by SCC

C) Use third-party only

D) Use Cloud Armor only

****Q27.**** You have a petabyte-scale data lake on Cloud Storage. Queries via Hive are slow. You want open-format serverless query with governance, columnar acceleration, and fine-grained access to BigLake. What is the solution?

A) Use Dataproc Hive only

B) Use BigLake tables (BigQuery + BigLake) over Cloud Storage with open formats (Parquet, Iceberg via BigLake Iceberg GA), enable BigQuery connections, use Dataplex for governance/tagging, use columnar caching and access via RLS and column masking, accelerate via BigQuery acceleration + BI Engine

C) Move all data to Cloud SQL

D) Use Firestore

****Q28.**** Explain the nuance of Private Google Access vs Private Service Connect vs Private Services Access (PSA) vs Serverless VPC Access.

A) They are all identical

B) Private Google Access: VM in private subnet accesses Google APIs via private IP; PSA: allocates IP range in your VPC for managed services like Cloud SQL/Redis via VPC peering; Private Service Connect: consumer endpoint to producer services (service-oriented, no peering, supports consumer control); Serverless VPC Access Connector: legacy way for serverless (Cloud Run/Functions/App Engine) to connect to VPC, now largely replaced by direct VPC egress

C) PSA is same as PSC

D) Private Google Access is for on-prem

****Q29.**** Your Terraform deployment via Cloud Build fails with 403 on service account impersonation when deploying to prod folder protected by VPC Service Controls and Org Policy. What is systematic fix?

A) Grant Owner everywhere

B) Check: 1) Cloud Build SA has iam.serviceAccountTokenCreator on deploy SA, 2) deploy SA has VPC SC ingress/egress rules allowing Cloud Build project access, 3) Org Policy exemption for service account impersonation, 4) access level for Cloud Build worker pool (private pool inside perimeter), 5) Use workload identity federation if external

C) Disable VPC SC

D) Use gcloud from laptop

****Q30.**** You need to optimize Cloud Run + BigQuery + Cloud Storage cost by 40% while meeting SRE SLOs of 99.9% availability. Which SRE-driven FinOps strategy?

A) Randomly delete resources

B) Implement SLO-based rightsizing: use Cloud Monitoring SLOs, set error budget burn rate alerts, rightsizing: Cloud Run min instances = 0 for non-critical + CPU always allocated = false + concurrency tuning via load test; BigQuery: slot autoscaling baseline tuned via INFORMATION_SCHEMA + materialized views + clustering; Storage: Autoclass + lifecycle + Object Versioning off; use Recommender + Active Assist + carbon footprint; implement hierarchical budget alerts and label enforcement via Org Policy constraints/requireLabels

C) Set everything to max

D) Use only on-demand pricing

Advanced Answer Key & Explanations

****Q21 Answer: A**** - At scale, cost allocation requires labels on namespaces and billing export joined with GKE cost allocation. Dataplane V2 Cilium provides eBPF-based network policy and better performance; Anthos Service Mesh gives mTLS zero-trust. Policy Controller enforces Pod Security Standards preventing privileged escalation.

****Q22 Answer: B**** - BigQuery Editions (now called BigQuery Reservations with baseline/autoscaling) provides predictable slot cost. Materialized views pre-aggregate, partitioning/clustering prunes 10TB scan, BI Engine accelerates sub-second dashboards, caching avoids re-scan.

****Q23 Answer: B**** - CMEK implementation requires granting Google service agents encrypter/decrypter roles on KMS keys. Org policies enforce CMEK mandatory. Rotation is native in KMS, revocation via IAM or key destruction makes data crypto-shredded. CSEK is manual and not applicable to all services.

****Q24 Answer: B**** - Eventarc Advanced (2024) separates bus/pipeline model for fan-in scale (100M events). filtering at pipeline reduces Cloud Run invocations, Pub/Sub transport buffers, DLQ handling prevents event loss. Classic Eventarc standard (1000s buckets) doesn't scale.

****Q25 Answer: B**** - Compute Engine PD async replication now supports cross-region replication but async = RPO > 0. True RPO=0 requires application-level synchronous replication (e.g., Spanner, or DB cluster replication). RTO < 5min requires GLB + MIG autohealing + automated failover; block storage alone cannot achieve RPO=0 cross-region.

****Q26 Answer: A**** - SCC Premium is centralized risk management: Asset Inventory gives resource snapshot, SHA finds misconfigurations (public bucket, open firewall), ETD detects threats via logs, Policy Analyzer finds who can access what, remediation via Eventarc triggers. VPC SC adds data exfiltration protection.

****Q27 Answer: B**** - BigLake decouples storage and compute with governance over GCS open formats including Apache Iceberg (GA 2024). Dataplex provides auto-discover, policy tags, quality. BigQuery acceleration and caching gives fast query without moving data, plus fine-grained RLS/masking.

****Q28 Answer: B**** - This is classic confusion: PGA is for private subnet VMs -> Google APIs; PSA is peering-based private connectivity for managed services allocating your IP range; PSC is more modern service-attachment model without peering exhaustion; Serverless VPC Access connector is legacy L2 connector now superseded by direct VPC egress for Cloud Run/Functions.

****Q29 Answer: B**** - 403 impersonation + VPC SC is common. Systematic fix: token creator role, VPC SC perimeter bridging or ingress rule for Cloud Build prod perimeter, private worker pool inside perimeter to avoid internet egress, Org Policy exceptions for SA creation. This avoids Owner permission anti-pattern.

****Q30 Answer: B**** - FinOps with SRE: error budgets allow cost optimization when SLOs healthy. Rightsizing: Cloud Run concurrency/CPU allocation tuning saves 30%+, Autoclass moves objects to cheaper classes automatically, slot baseline tuning prevents over-provisioning, label enforcement ensures chargeback. Active Assist Recommender automates idle resource detection.

Course File: 04_Docker.md

Course: Docker

1. Beginner Level (10 Questions)

****Q1. What does the docker pull nginx:alpine command actually do?****

- A. Builds a new nginx image from a Dockerfile named alpine
- B. Downloads the nginx image with the alpine tag from Docker Hub (or configured registry) to local storage
- C. Starts an nginx container using Alpine Linux as host OS
- D. Creates a volume named alpine and mounts it to nginx

****Q2. Which Dockerfile instruction is used to specify the default command that runs when a container starts, but can be overridden by docker run arguments?****

- A. ENTRYPOINT
- B. RUN
- C. CMD
- D. START

****Q3. You run docker ps. What do you see?****

- A. All containers including stopped ones
- B. Only running containers
- C. All images on the host
- D. All Docker networks and volumes

****Q4. What is the purpose of .dockerignore file?****

- A. To list containers that should be ignored by docker ps
- B. To exclude files and directories from the build context sent to the Docker daemon
- C. To prevent images from being pushed to Docker Hub
- D. To ignore vulnerabilities during docker scout scans

****Q5. How do you map host port 8080 to container port 80 when running a container?****

- A. docker run -p 8080:80 nginx
- B. docker run --link 8080:80 nginx
- C. docker run -v 8080:80 nginx
- D. docker run --expose 8080:80 nginx

****Q6. What does docker build -t myapp:1.0 . do?****

- A. Runs a container from myapp:1.0 image
- B. Builds an image from Dockerfile in current directory and tags it as myapp:1.0
- C. Pulls myapp:1.0 from registry and tests it
- D. Exports the current directory as a tarball named myapp:1.0

****Q7. Which command removes a stopped container named web?****

- A. docker rmi web
- B. docker rm web
- C. docker delete web
- D. docker image prune web

****Q8. What happens to data written inside a container's writable layer when the container is removed with docker rm?****

- A. It is automatically persisted in a volume
- B. It is committed to the underlying image

C. It is permanently deleted unless stored in a volume or bind mount

D. It is moved to /var/lib/docker/backup

****Q9. What is the difference between COPY and ADD in Dockerfile for most use cases?****

A. No difference, they are aliases

B. COPY only copies local files, ADD can also handle remote URLs and auto-extract tar archives

C. ADD only works for binaries, COPY only for text files

D. COPY requires root privileges, ADD does not

****Q10. Which command shows the logs of a running container named api?****

A. docker logs api

B. docker inspect api

C. docker history api

D. docker events api

Beginner Answer Key & Explanations

****Q1. Answer: B**** - docker pull fetches image layers from a registry to local cache without creating or starting a container. The :alpine is a tag variant built on Alpine Linux for smaller size.

****Q2. Answer: C**** - CMD provides default executable/args that are replaced if user supplies args to docker run. ENTRYPOINT is harder to override and is meant for fixed main command.

****Q3. Answer: B**** - docker ps lists only running containers by default. Use docker ps -a to see all containers including exited ones.

****Q4. Answer: B**** - .dockerignore reduces build context size and avoids sending secrets/node_modules/.git to daemon, speeding up builds. It works like .gitignore for the build.

****Q5. Answer: A**** - -p host:container publishes container port to host. 8080:80 means localhost:8080 forwards to container's port 80.

****Q6. Answer: B**** - -t tags the built image, . sets build context to current dir where Dockerfile is located. Daemon executes Dockerfile steps to create layers.

****Q7. Answer: B**** - docker rm removes containers, docker rmi removes images. You must stop container first unless using -f.

****Q8. Answer: C**** - Container writable layer is ephemeral and tied to container lifecycle. For persistence, you must use volumes (docker volume) or bind mounts.

****Q9. Answer: B**** - Docker best practices recommend using COPY for straightforward file copying. ADD's URL and tar auto-extraction can cause unexpected behavior.

****Q10. Answer: A**** - docker logs streams stdout/stderr of main process (PID 1). Useful for debugging even after container exited, unless using different logging driver.

2. Intermediate Level (10 Questions)

****Q11. A multi-stage Dockerfile has FROM golang:1.22 AS builder ... FROM gcr.io/distroless/static-debian12. What is the primary benefit?****

A. It allows running two containers at once

B. It reduces final image size and attack surface by discarding build tools

C. It is required to compile Go applications

D. It automatically creates a Docker Compose setup

****Q12. You have a container writing heavily to /data. You want data to persist and be efficiently managed by Docker, surviving docker rm. Which should you use?****

A. Keep data in container writable layer and use docker commit

B. Use a bind mount -v /tmp/data:/data

C. Use a named volume docker volume create appdata and mount with -v appdata:/data

D. Use COPY /data in Dockerfile

****Q13. What does docker run --init do?****

- A. Initializes a new Docker swarm
- B. Injects a tiny init process (tini) as PID 1 to handle signals and reap zombie processes
- C. Clears all caches before starting container
- D. Runs the container in privileged mode

****Q14. In Docker Compose, what does depends_on with condition service_healthy actually guarantee?****

- A. It waits until dependency container's application is ready to serve traffic based on its healthcheck
- B. It guarantees dependency data is migrated
- C. It builds dependency image before main service
- D. It ensures dependency container was started at some point, not necessarily healthy

****Q15. How do you limit a container to use at most 1GB RAM and 1 CPU?****

- A. docker run --memory=1g --cpus=1 myapp
- B. docker run -m 1g --cpu 1 myapp is invalid syntax
- C. docker run --ram 1GB --cpu-count 1 myapp
- D. You can only limit via Kubernetes, not Docker

****Q16. Your Dockerfile RUN apt-get update && apt-get install -y curl creates large image. Which optimization is best for 2024+ best practices?****

- A. Split into two RUN instructions for better caching
- B. Use RUN apt-get update && apt-get install -y --no-install-recommends curl && rm -rf /var/lib/apt/lists/*
- C. Add apt-get upgrade -y to keep image fresh
- D. Use ADD instead of RUN

****Q17. What is the difference between bridge and host network mode?****

- A. Bridge creates private network with NAT/port mapping, host removes network isolation and uses host's network stack directly
- B. Host mode is more secure than bridge
- C. Bridge mode only works on Linux, host only on Windows
- D. No difference since Docker 25

****Q18. You run docker exec -it mycontainer /bin/sh. What happens if the container's main process was started with USER nobody and sh doesn't exist?****

- A. It always works because exec runs as root by default and sh exists in all images
- B. It runs as root by default (unless --user specified) but will fail if /bin/sh binary not present in image (e.g., distroless, scratch)
- C. It automatically installs sh
- D. It restarts container with root user

****Q19. What does docker build --no-cache and BuildKit's --mount=type=cache achieve?****

- A. Both disable caching entirely
- B. --no-cache disables layer cache for full rebuild, while --mount=type=cache provides persistent cache mount for package managers across builds without bloating layers
- C. They are identical
- D. --mount=type=cache is used to cache container logs

****Q20. In Docker Compose v2, you have two services needing to share the same network alias and communicate via http://backend:8000. How do they reach each other by default?****

- A. They cannot, must use host network
- B. Compose creates default network where services are DNS-discoverable by service name automatically
- C. Must manually create external: true network
- D. Only possible with --link

Intermediate Answer Key & Explanations

- **Q11. Answer: B**** - Multi-stage builds let you use heavy builder image then COPY only artifacts to minimal runtime like distroless/alpine/scratch. This shrinks image from ~1GB to ~20MB and removes shell and package managers.
- **Q12. Answer: C**** - Named volumes are Docker-managed, portable, work on all OSes, and support volume drivers/backups. Bind mounts tie to host path and have host permission issues, while writable layer is ephemeral.
- **Q13. Answer: B**** - Without proper init, PID 1 doesn't reap zombies and may not forward SIGTERM, causing graceful shutdown issues in Node/JVM apps. --init uses tini, now built-in, critical for production.
- **Q14. Answer: A**** - Since Compose spec, depends_on: condition: service_healthy waits for healthcheck to be passing. Without condition, it only waits for container start, leading to race conditions.
- **Q15. Answer: A**** - --memory=1g (or -m) sets cgroup memory limit, --cpus=1 limits CPU time across cores. Docker uses cgroups v2 on modern hosts; always set limits to prevent noisy neighbor.
- **Q16. Answer: B**** - Combining update+install+caching cleanup in single RUN avoids stale layer and reduces size. --no-install-recommends and removing apt lists is official best practice since 2023 Docker docs.
- **Q17. Answer: A**** - Bridge (default) gives container its own veth + iptables NAT. Host mode (--network host) sacrifices isolation for performance/lower latency, but loses port mapping and is Linux-only.
- **Q18. Answer: B**** - docker exec defaults to root user inside container namespace, but filesystem is container's. Minimal images lack shell; you need docker debug or include busybox for troubleshooting.
- **Q19. Answer: B**** - --no-cache forces all layers to rebuild, useful but slow. BuildKit cache mounts (RUN --mount=type=cache,target=/root/.cache/pip pip install) persist outside layers, speeding rebuilds without increasing final size.
- **Q20. Answer: B**** - Compose automatically creates a custom bridge network for project, with embedded DNS server. Service names resolve to container IPs, no links needed. Custom network names improve isolation vs default bridge.

3. Advanced Level (10 Questions)

- **Q21. You've enabled Docker Content Trust and BuildKit. Your CI builds image myreg.io/app:prod with cosign signature. What is the most secure way to ensure deployment pulls only signed image in production?****
- A. DOCKER_CONTENT_TRUST=1 docker pull myreg.io/app:prod
- B. Rely on tag mutability and pull by tag
- C. Use digest pinning myreg.io/app@sha256:<digest> plus signature verification with cosign verification policy / admission controller (e.g., OPA, Kyverno, Sigstore policy controller)
- D. Use docker pull --verify=false
- **Q22. Your container app OOMKilled intermittently despite --memory=2g. docker stats shows MEM usage 1.2g/2g. What is likely root cause?****
- A. Docker stats is wrong
- B. Kernel memory, tmpfs mounts, or memory used by processes inside container counted differently; also Go/Java memory limits not aware of cgroup unless configured, causing heap to exceed limit
- C. Need to increase CPUs
- D. This cannot happen
- **Q23. How does BuildKit's --mount=type=secret differ from ARG or COPY .env for handling private NPM_TOKEN / SSH keys during build?****
- A. No difference
- B. ARG and COPY leave secrets in image layers/history, visible via docker history. --mount=type=secret mounts secret only during RUN step, never persisted to layer, and not cached
- C. Secret mount is slower
- D. ARG is more secure
- **Q24. For a Docker Swarm / Compose production stack, you need config reload without restart and secrets rotation for DB password. How should you handle it?****
- A. Use ENV variables and docker restart

B. Use Docker configs/secrets (mounted as /run/secrets/, in-memory tmpfs) + support for file-based watch, or with Swarm use docker service update --secret-rm/add with rotation, app watches file change via inotify

C. Bake password into image

D. Use bind mount of host /etc/passwd

****Q25. You see docker system df shows huge build cache and you run docker builder prune -a. A container using overlay2 driver suddenly fails to start with no space left. Why?****

A. builder prune deletes volumes

B. overlay2 driver shares space with /var/lib/docker/buildkit; but more likely disk pressure from volumes/log files filling /var partition causing overlay to fail; pruning build cache alone doesn't free volume/logs, need docker system prune --volumes and log rotation with json-file max-size

C. This never happens

D. You must reboot

****Q26. In rootless Docker (2024 default for many distros), which limitation is true?****

A. Rootless can use privileged containers and modify host network without restriction exactly like rootful

B. Rootless cannot bind to privileged ports <1024 by default unless sysctl set, AppArmor may restrict, overlay over fuse-overlayfs performance penalty, and --network host doesn't give real host network due to user namespace

C. Rootless is faster

D. Rootless supports cgroups v1 only

****Q27. You need to debug a distroless image running in Kubernetes that has no shell. What is valid 2024+ approach?****

A. docker exec with bash

B. Use kubectl debug -it pod --image=busybox --target=app --share-processes (ephemeral debug container) and docker:24-dind context, or cdebug/docker debug with docker build --platform debug / Nixery debug variants, attaching via process namespace

C. Rebuild with ubuntu base in production

D. Impossible

****Q28. CVE-2024-21626 (runc) is exploited via WORKDIR inside container being /proc/self/fd/7. How do you mitigate before patching Docker?****

A. Nothing

B. Ensure base images don't contain /proc symlink tricks, don't run untrusted Dockerfiles with attacker-controlled WORKDIR, enforce policy with image scanning, and update runc >=1.1.12 / Docker >=25.0.2 immediately; use user namespaces and drop capability

C. Use docker pull only

D. Disable overlay2

****Q29. You are optimizing CI/CD: building 10 microservices each with identical npm ci layer. Build takes 10m. Best BuildKit optimization?****

A. Keep separate builds

B. Use shared BuildKit cache backend: --cache-from type=registry,ref=myreg/cache:service --cache-to type=registry,ref=myreg/cache:service,mode=max plus type=gha for GitHub Actions, and common layer for dependencies via COPY package*.json before source, with mode=max cross-repo mounts

C. Use docker commit

D. Disable BuildKit

****Q30. When using docker run --read-only --tmpfs /tmp --tmpfs /run -p 3000:3000 myapp, app writes to /app/logs which fails. Correct secure fix?****

A. Remove --read-only

B. Keep read-only root filesystem, mount writable tmpfs or volume only where needed: --tmpfs /app/logs:uid=1000,mode=755 or -v app-logs:/app/logs, keeping rest immutable to prevent binary tampering and comply with CIS benchmarks

C. Run privileged

D. chmod 777 in Dockerfile

Advanced Answer Key & Explanations

****Q21. Answer: C**** - Tags are mutable. Production should pin by digest and verify Sigstore/cosign signature via policy controller. DCT is deprecated in favor of Notation/Cosign + OPA/Kyverno admission.

****Q22. Answer: B**** - OOM can be from cgroup memory includes page cache, kernel mem, or Java default heap sizing 1/4 host RAM not container. Configure -XX:MaxRAMPercentage or GOMEMLIMIT for Go to respect cgroup limits.

****Q23. Answer: B**** - docker history and layer export reveals ARG and ENV. Secret mounts (RUN --mount=type=secret,id=npm_token npm ci) are BuildKit-only, tmpfs-mounted during build, never committed, safe for CI.

****Q24. Answer: B**** - Docker secrets are tmpfs mounted at /run/secrets, 400 permissions, not in env ps. Apps should watch file change or receive SIGHUP. Swarm supports rolling update with secret rotation without full downtime.

****Q25. Answer: B**** - docker system df counts build cache, images, volumes. builder prune doesn't touch volumes or container logs. Disk full on /var/lib/docker affects overlay2 copy-up causing ENOSPC. Implement log rotation and monitor volumes.

****Q26. Answer: B**** - Rootless runs daemon in user namespace, cannot use privileged ports without net.ipv4.ip_unprivileged_port_start, host network is actually namespaced slirp. Supports cgroups v2 with systemd, good for security but with networking/perf nuances.

****Q27. Answer: B**** - Modern debugging uses ephemeral containers sharing PID namespace. kubectl debug, Docker 25+ docker debug (beta), or ko/cdebug allow inspecting distroless without shipping shell to prod, preserving minimal attack surface.

****Q28. Answer: B**** - CVE-2024-21626 allows container escape via leaked file descriptor due to WORKDIR inside proc. Mitigation is patch runc ASAP and enforce policy via image linter (e.g., forbidding WORKDIR starting with /proc), plus user namespace remapping.

****Q29. Answer: B**** - BuildKit cache backends (registry, gha, s3) with mode=max cache all layers not just final, shared across parallel builds. Combined with proper layer ordering (deps first) cuts 10m to ~1-2m.

****Q30. Answer: B**** - --read-only sets rootfs immutable per CIS Docker Benchmark 4.1. It prevents runtime modification/exploits. Provide writable tmpfs/volumes only for required paths, preserving immutability and least privilege, not full privileged or 777.

Course File: 05_Kubernetes.md

Course: Kubernetes

1. Beginner Level (10 Questions)

****Q1: Which component is responsible for maintaining the desired state of the cluster by managing Pods via ReplicaSets?****

- A) kube-proxy
- B) kubelet
- C) Deployment Controller
- D) etcd

****Q2: What is the smallest deployable unit in Kubernetes?****

- A) Container
- B) Pod
- C) ReplicaSet
- D) Deployment

****Q3: Which command creates a Deployment named nginx using image nginx:1.25?****

- A) `kubectl run nginx --image=nginx:1.25 --restart=Never`
- B) `kubectl create deployment nginx --image=nginx:1.25`
- C) `kubectl expose nginx --image=nginx:1.25`
- D) `kubectl apply -f nginx:1.25`

****Q4: In Kubernetes, what does a ClusterIP Service do by default?****

- A) Exposes the Service on each Node's IP at a static port
- B) Exposes the Service externally using a cloud load balancer
- C) Exposes the Service on an internal IP reachable only within the cluster
- D) Maps incoming requests based on HTTP host and path

****Q5: Where does kubelet run, and what is its primary role?****

- A) On control plane nodes, scheduling Pods to worker nodes
- B) On each worker node, ensuring containers in PodSpecs are running and healthy
- C) Inside etcd, storing cluster state
- D) As a DaemonSet, providing network proxying

****Q6: What happens when you run `kubectl apply -f deployment.yaml` if the Deployment already exists?****

- A) It fails with "already exists" error
- B) It deletes and recreates the Deployment
- C) It performs a declarative three-way merge patch to update to desired state
- D) It only scales the Deployment

****Q7: Which resource is used to persist data beyond Pod lifecycle and must be bound to a PersistentVolumeClaim?****

- A) ConfigMap
- B) PersistentVolume (PV)
- C) EmptyDir
- D) Secret

****Q8: What does the restartPolicy: Never mean in a Pod spec?****

- A) The kubelet will never restart containers in that Pod if they fail
- B) The Deployment will never create new Pods
- C) The Pod is evicted and never rescheduled

D) The livenessProbe is disabled

****Q9: Which namespace holds control plane components like kube-system components in a standard cluster?****

A) default

B) kube-system

C) kube-public

D) kube-node-lease

****Q10: How do you get detailed information about a failing Pod including events?****

A) kubectl logs pod-name

B) kubectl get pods --watch

C) kubectl describe pod pod-name

D) kubectl explain pod

Beginner Answer Key & Explanations

****Q1 Answer: C) Deployment Controller**** - The Deployment controller, running in kube-controller-manager, reconciles desired vs current state via ReplicaSets. It handles rolling updates and rollbacks.

****Q2 Answer: B) Pod**** - A Pod wraps one or more containers with shared network and storage. You never deploy containers directly, only via Pods.

****Q3 Answer: B) kubectl create deployment nginx --image=nginx:1.25**** - create deployment is the imperative way to create a Deployment. Option A creates a Pod, not a Deployment.

****Q4 Answer: C) Exposes the Service on an internal IP reachable only within the cluster**** - ClusterIP is the default Service type, allocating a virtual IP for internal service discovery via kube-proxy and CoreDNS.

****Q5 Answer: B) On each worker node, ensuring containers in PodSpecs are running and healthy**** - kubelet is the node agent that watches API server PodSpecs bound to its node and manages the CRI to run them.

****Q6 Answer: C) It performs a declarative three-way merge patch to update to desired state**** - apply uses last-applied-configuration annotation to patch, enabling declarative management and gitops workflows.

****Q7 Answer: B) PersistentVolume (PV)**** - PVs are cluster storage provisioned by admins, and PVCs claim them. This decouples Pod from storage implementation.

****Q8 Answer: A) The kubelet will never restart containers in that Pod if they fail**** - This policy is primarily used for Jobs. Always and OnFailure are the other options; Deployments require Always.

****Q9 Answer: B) kube-system**** - This namespace contains CoreDNS, kube-proxy, and control plane Pods in some distributions. User workloads should use default or custom namespaces.

****Q10 Answer: C) kubectl describe pod pod-name**** - Describe shows conditions, container statuses, and crucial Events like FailedScheduling or ImagePullBackOff. Logs only show stdout.

2. Intermediate Level (10 Questions)

****Q11: A Deployment has 3 replicas. One Node fails. What happens to Pods on that Node after 5 minutes (default toleration)?****

A) They remain pending forever

B) They are immediately rescheduled elsewhere

C) After tolerationSeconds of 300s for not-ready/unreachable taints, they are evicted and rescheduled

D) The Deployment scales down to 2 replicas automatically

****Q12: You have a StatefulSet with volumeClaimTemplates. What is true when you delete the StatefulSet with kubectl delete sts mydb?****

A) Pods and PVCs are deleted automatically

B) Pods are deleted but PVCs are retained by default to prevent data loss

C) PVCs are deleted but PVs remain

D) Nothing is deleted due to finalizers

****Q13: Which probe would you use to prevent a Pod from receiving traffic via Service during startup, but not restart it?****

- A) livenessProbe
- B) readinessProbe
- C) startupProbe
- D) Both livenessProbe and readinessProbe with same config

****Q14: A Pod needs to access API server with specific RBAC permissions. What should you configure?****

- A) Create a ServiceAccount, bind it to Role/ClusterRole via RoleBinding, and set serviceAccountName in Pod
- B) Add user credentials to KUBECONFIG in Pod
- C) Use kubelet's credentials via hostPath mount
- D) Annotate Pod with rbac.authorization.kubernetes.io/auto-update

****Q15: What is the effect of resources.requests vs resources.limits for CPU?****

- A) Requests used for scheduling, limits enforced via CFS quota throttling
- B) Both are used only for scheduling
- C) Limits used for scheduling, requests used for eviction order
- D) Requests are hard limits, limits are soft advisory

****Q16: You run two containers in same Pod. How do they communicate via localhost, and what is shared?****

- A) They share network namespace (same IP, can use localhost) and can share volumes, but have isolated filesystem otherwise
- B) They have different IPs and cannot use localhost
- C) They share everything including PID namespace by default
- D) They can only communicate via Service

****Q17: An Ingress resource alone will not work without what?****

- A) An Ingress Controller (e.g., ingress-nginx, traefik) that implements Ingress logic
- B) A LoadBalancer Service for every backend
- C) A NetworkPolicy allowing traffic
- D) An ExternalName Service

****Q18: What does podDisruptionBudget with minAvailable: 2 guarantee for a Deployment with 3 replicas during voluntary disruptions?****

- A) At least 2 Pods must remain available during drain/upgrade operations
- B) Exactly 2 Pods are terminated during disruption
- C) It prevents any eviction ever
- D) It guarantees 2 Pods on different Nodes via anti-affinity

****Q19: How does a HorizontalPodAutoscaler (HPA v2) decide to scale?****

- A) It scales based on CPU only via metrics-server
- B) It queries metrics APIs (resource, custom, external) and calculates $\text{desiredReplicas} = \text{ceil}[\text{currentReplicas} * (\text{currentMetric}/\text{desiredMetric})]$
- C) It directly increases container resources via in-place resize
- D) It watches queue length via etcd

****Q20: Explain the difference between ConfigMap and Secret in etcd storage and usage.****

- A) Both are identical, only naming difference
- B) Secrets are base64-encoded and can be encrypted at rest via EncryptionConfiguration, intended for sensitive data, while ConfigMaps are plaintext for non-confidential config
- C) ConfigMaps are encrypted, Secrets are not
- D) Secrets can only be mounted as env vars, ConfigMaps only as volumes

Intermediate Answer Key & Explanations

****Q11 Answer: C)** After tolerationSeconds of 300s for not-ready/unreachable taints, they are evicted and rescheduled** - Default tolerations allow 5 minutes before eviction. After eviction, ReplicaSet creates new Pods elsewhere if resources allow.

****Q12 Answer: B)** Pods are deleted but PVCs are retained by default to prevent data loss** - StatefulSet's PVC retention is intentional for stateful data safety. You must manually delete PVCs or use spec.persistentVolumeClaimRetentionPolicy (K8s 1.27+).

****Q13 Answer: B)** readinessProbe** - Readiness failing removes Pod from Endpoints, so no Service traffic. Liveness failure causes restart. StartupProbe gates liveness/readiness during slow start.

****Q14 Answer: A)** Create a ServiceAccount, bind it to Role/ClusterRole via RoleBinding, and set serviceAccountName in Pod** - This follows least privilege. Pods get projected token via TokenRequest API mounted at /var/run/secrets/kubernetes.io/serviceaccount.

****Q15 Answer: A)** Requests used for scheduling, limits enforced via CFS quota throttling** - Scheduler sums requests. Kubelet sets cpu.shares from requests and cpu.cfs_quota_us from limits. Memory limits trigger OOMKill if exceeded.

****Q16 Answer: A)** They share network namespace (same IP, can use localhost) and can share volumes, but have isolated filesystem otherwise** - Pod is the network boundary. Containers can communicate via localhost:port and share EmptyDir/Volumes explicitly configured.

****Q17 Answer: A)** An Ingress Controller (e.g., ingress-nginx, traefik) that implements Ingress logic** - Ingress is just API spec; controller watches it and configures reverse proxy. Without controller, resource does nothing.

****Q18 Answer: A)** At least 2 Pods must remain available during drain/upgrade operations** - PDB is respected by kubelet drain and voluntary eviction API. It does not affect involuntary failures like Node crash.

****Q19 Answer: B)** It queries metrics APIs (resource, custom, external) and calculates desiredReplicas = ceil[currentReplicas * (currentMetric/desiredMetric)]** - HPA control loop fetches metrics every 15s by default and stabilizes scaling with windows to avoid flapping.

****Q20 Answer: B)** Secrets are base64-encoded and can be encrypted at rest via EncryptionConfiguration, intended for sensitive data, while ConfigMaps are plaintext for non-confidential config** - Secrets have 1MB limit, enable encryption at rest, and have slightly more protection in logs and etcd. Neither is secure without RBAC and encryption.

3. Advanced Level (10 Questions)

****Q21: You see CrashLoopBackOff with OOMKilled (exit 137). Memory request=256Mi, limit=512Mi, node has 4Gi free. Application heap uses 700Mi. What is correct fix?****

- A) Increase Node memory
- B) Increase memory limit to >700Mi (e.g., 1Gi) and set request appropriately, plus fix heap/mem leak; consider VPA
- C) Decrease memory request to 128Mi to allow better binpacking
- D) Add CPU limit to prevent OOM

****Q22: A cluster uses Cilium CNI with kube-proxy replacement. You apply NetworkPolicy with policyTypes Ingress but Pods still communicate. Possible reason?****

- A) Cilium ignores Kubernetes NetworkPolicies
- B) Cilium requires CiliumNetworkPolicy or needs NetworkPolicy enforcement enabled and endpoint selected by label correctly; also check if policy allows DNS egress which may block resolution
- C) NetworkPolicy only works with Calico
- D) ClusterIP Services bypass NetworkPolicy by design

****Q23: In an EKS/GKE cluster, you need to mount S3/GCS bucket as volume for read-heavy ML inference. Best production approach in 2024-2026?****

- A) Use hostPath to mount bucket on Node
- B) Use CSI driver: Mountpoint for Amazon S3 CSI driver or GCS Fuse CSI driver with appropriate cache and permission settings
- C) Use PersistentVolume with NFS and copy data from S3
- D) Use ConfigMap to store bucket data

****Q24: You are implementing zero-downtime deployment for a web app with 10s startup reading config. Users report 502s during rolling update with maxUnavailable: 0, maxSurge: 1. Most robust fix?****

- A) Just increase replicas

B) Configure readinessProbe + startupProbe to delay traffic, add minReadySeconds, terminationGracePeriodSeconds > 0, and handle SIGTERM with preStop hook for connection draining

C) Use Recreate strategy

D) Set readinessProbe same as livenessProbe and remove Services

****Q25: etcd shows high DB size and etcdserver: mvcc: database space exceeded. What happened and how to fix without data loss?****

A) Compact revision and defrag, increase quota-backend-bytes (default 2GB -> 8GB max), check for runaway Lease/Events creating spam (e.g., misbehaving controller spamming events)

B) Delete etcd pod and restore from scratch

C) Delete all ConfigMaps

D) Scale down all Deployments to free space

****Q26: You need to enforce that all Pods in namespace prod must have resources.limits, seccompProfile type RuntimeDefault, and disallow privileged. Which technology replaces deprecated PodSecurityPolicy (removed in 1.25)?****

A) OPA Gatekeeper / Kyverno via ValidatingAdmissionPolicy (1.28+ ValidatingAdmissionPolicy) and Pod Security Standards enforced via namespace labels

B) Use NetworkPolicy

C) PodSecurityPolicy still works in 1.29

D) Use RBAC alone

****Q27: A Job with 100 parallel Pods doing batch processing suddenly gets many FailedScheduling due to insufficient CPU. Cluster has Cluster Autoscaler. What advanced pattern optimizes cost and time?****

A) Disable autoscaler

B) Use Karpenter with consolidation, Pod resource rightsizing, set Job parallelism with work queue (e.g., Kueue + JobSet), overprovisioning pause pods, and node affinity to mix spot/on-demand

C) Manually create Nodes with kubectl

D) Use DaemonSet instead of Job

****Q28: Debugging: kubectl exec -it pod -- /bin/sh hangs. API server logs show timeout. Pod is Running, kubelet is healthy. What is likely root cause chain?****

A) Corrupt etcd

B) CNI overlay failure, kubelet <-> API server network issue, or container runtime exec not responding; check kubelet logs, CRI socket, CNI plugin health, and whether kubectl port-forward also fails due to network path

C) ConfigMap too large

D) ServiceAccount missing

****Q29: For multi-tenant cluster with strict isolation, you need per-tenant network encryption, node isolation, and API audit. Complete solution?****

A) Install Cilium with WireGuard encryption + mutual TLS via CiliumNetworkPolicy L7, use Kata Containers/gVisor runtimeClass for node isolation, taint nodes per tenant, and enable API Server Audit logging with structured webhooks

B) Use only ClusterIP Services

C) Use hostNetwork: true for all Pods

D) Rely on default Docker bridge

****Q30: You need to upgrade Kubernetes from 1.28 to 1.29 in production with zero control plane downtime. Correct kubeadm procedure?****

A) Upgrade all Nodes at once with kubeadm upgrade apply

B) Upgrade control plane one by one: kubeadm upgrade plan, kubeadm upgrade apply v1.29.x on first control plane, then kubeadm upgrade node on others, then drain & upgrade kubelet/kubectl on worker nodes one by one respecting skew policy (kubelet may be up to 2 minor older than apiserver)

C) Delete cluster and create new one

D) Upgrade kubelet before apiserver

Advanced Answer Key & Explanations

****Q21 Answer:** B) Increase memory limit to >700Mi (e.g., 1Gi) and set request appropriately, plus fix heap/mem leak; consider VPA** - OOMKill occurs when container exceeds cgroup memory limit enforced by kernel. Node free memory is irrelevant. Setting limit too low kills process; need to profile and set request=usage, limit=peak+bump.

****Q22 Answer:** B) Cilium requires CiliumNetworkPolicy or needs NetworkPolicy enforcement enabled and endpoint selected by label correctly; also check if policy allows DNS egress which may block resolution** - Cilium supports standard NetworkPolicy but enforcement can be disabled. Common pitfall: Ingress policy denies DNS (port 53) breaking name resolution, and label selector mismatch.

****Q23 Answer:** B) Use CSI driver: Mountpoint for Amazon S3 CSI driver or GCS Fuse CSI driver with appropriate cache and permission settings** - CSI drivers provide native bucket mounts with caching, mount options, and lifecycle management. Mountpoint S3 is AWS GA since 2024 and optimizes read throughput for ML.

****Q24 Answer:** B) Configure readinessProbe + startupProbe to delay traffic, add minReadySeconds, terminationGracePeriodSeconds > 0, and handle SIGTERM with preStop hook for connection draining** - 502s during rollout happen due to race between endpoint removal and connection close. Proper lifecycle hooks + probes + draining solves; maxSurge 0 ensures capacity.

****Q25 Answer:** A) Compact revision and defrag, increase quota-backend-bytes (default 2GB -> 8GB max), check for runaway Lease/Events creating spam (e.g., misbehaving controller spamming events)** - etcd keeps MVCC history; without compaction DB grows. Defrag reclaims space. Quota prevents runaway growth. Find source of spam via etcd metrics.

****Q26 Answer:** A) OPA Gatekeeper / Kyverno via ValidatingAdmissionPolicy (1.28+ ValidatingAdmissionPolicy) and Pod Security Standards enforced via namespace labels** - PSP removed. Replacement is PSS (privileged/baseline/restricted via labels pod-security.kubernetes.io/enforce) + policy engines for custom rules.

****Q27 Answer:** B) Use Karpenter with consolidation, Pod resource rightsizing, set Job parallelism with work queue (e.g., Kueue + JobSet), overprovisioning pause pods, and node affinity to mix spot/on-demand** - This pattern handles bursty batch efficiently. Karpenter scales faster than CAS, Kueue manages quota, JobSet groups jobs, pause pods reduce scale-up latency.

****Q28 Answer:** B) CNI overlay failure, kubelet <-> API server network issue, or container runtime exec not responding; check kubelet logs, CRI socket, CNI plugin health, and whether kubectrl port-forward also fails due to network path** - Exec uses SPDY/websocket via apiserver->kubelet->CRI. Hang indicates network path broken or runtime deadlock. CNI outage breaks pod network but not host network.

****Q29 Answer:** A) Install Cilium with WireGuard encryption + mutual TLS via CiliumNetworkPolicy L7, use Kata Containers/gVisor runtimeClass for node isolation, taint nodes per tenant, and enable API Server Audit logging with structured webhooks** - Full multi-tenant security requires network encryption, runtime sandboxing, scheduling isolation, and audit. This stack provides all layers.

****Q30 Answer:** B) Upgrade control plane one by one: kubeadm upgrade plan, kubeadm upgrade apply v1.29.x on first control plane, then kubeadm upgrade node on others, then drain & upgrade kubelet/kubectrl on worker nodes one by one respecting skew policy (kubelet may be up to 2 minor older than apiserver)** - This is the supported kubeadm HA upgrade flow ensuring rolling upgrade and version skew compliance. Apiserver must be upgraded before kubelet.

Course File: 06_Terraform.md

Course: Terraform

1. Beginner Level (10 Questions)

****Q1. What is the primary purpose of terraform init?****

- A. It applies the execution plan to create infrastructure
- B. It initializes a working directory by downloading provider plugins, modules, and initializing the backend
- C. It validates the syntax of Terraform configuration files
- D. It destroys all managed infrastructure

****Q2. Which file extension is used for Terraform configuration files, and what is the default state file name?****

- A. .tf and terraform.tfstate
- B. .terraform and state.json
- C. .hcl and terraform.state
- D. .tf.json only and terraform.tfstate.d

****Q3. What does terraform plan do?****

- A. It immediately creates resources without showing changes
- B. It generates an execution plan showing what actions Terraform will take to achieve the desired state, without making changes
- C. It imports existing infrastructure into state
- D. It formats Terraform code according to canonical style

****Q4. In HCL, how do you define an input variable with a default value and type?****

- A. variable "instance_type" { default = "t3.micro" type = string }
- B. variable "instance_type" { type = string default = "t3.micro" description = "EC2 instance type" }
- C. input "instance_type" = "t3.micro"
- D. var.instance_type = { value = "t3.micro" }

****Q5. What is the purpose of terraform.tfvars or *.auto.tfvars?****

- A. To store provider credentials securely encrypted
- B. To define backend configuration
- C. To assign values to input variables automatically without CLI flags
- D. To define output values

****Q6. Consider: resource "aws_instance" "web" { ami = "ami-12345" instance_type = "t3.micro" }. How do you reference its public IP in another resource?****

- A. aws_instance.web.public_ip
- B. \${aws_instance.web.public_ip}
- C. aws_instance.web.public_ip.value
- D. The syntax is invalid, public_ip cannot be referenced

****Q7. Which command is used to change formatting of Terraform files to canonical style?****

- A. terraform validate
- B. terraform fmt
- C. terraform lint
- D. terraform format

****Q8. What does an output block do in Terraform?****

- A. It prints debug logs during apply
- B. It defines values that are exposed to the user after apply, and can be consumed by other modules or workspaces

C. It writes data directly to the backend

D. It creates a file output on disk

****Q9. What happens if you manually delete a resource in the cloud console that is managed by Terraform, then run terraform plan?****

A. Nothing, Terraform ignores manual changes

B. Terraform detects drift and proposes to recreate the resource to match desired state

C. Terraform automatically deletes it from state

D. Terraform will error and require terraform refresh only

****Q10. Which block is used to configure a Terraform provider version constraint?****

A. provider "aws" { version = "~> 5.0" }

B. terraform { required_providers { aws = { source = "hashicorp/aws" version = "~> 5.0" } } }

C. versions { aws = "~> 5.0" }

D. dependency "aws" { version = "~> 5.0" }

Beginner Answer Key & Explanations

****Q1. Answer: B**** - terraform init initializes the working directory, downloading providers, modules, and configuring the backend. It must be run before plan/apply in a new directory.

****Q2. Answer: A**** - Terraform configs use .tf (or .tf.json) extension, and default local state file is terraform.tfstate. This is fundamental working knowledge.

****Q3. Answer: B**** - terraform plan is a dry-run that shows creates, updates, deletes to align real infrastructure with desired configuration. It never modifies infrastructure.

****Q4. Answer: B**** - Correct syntax requires type = string and default = "t3.micro" inside a variable block. Terraform 0.12+ requires type constraints without interpolation.

****Q5. Answer: C**** - .tfvars files provide variable values; terraform.tfvars and *.auto.tfvars are loaded automatically. They simplify variable injection without -var flags.

****Q6. Answer: A**** - Terraform reference syntax is resource_type.name.attribute, e.g., aws_instance.web.public_ip. Legacy interpolation syntax B is still valid but unnecessary since Terraform 0.12.

****Q7. Answer: B**** - terraform fmt rewrites configs to canonical HCL style (spacing, alignment). It's used in CI to enforce consistent formatting.

****Q8. Answer: B**** - Outputs expose values after apply, useful for parent modules, CLI display, and remote state references. They are essential for module composition.

****Q9. Answer: B**** - Terraform compares desired config vs state vs real infra; drift detection in plan will show the resource needs to be recreated. This is core to declarative IaC.

****Q10. Answer: B**** - Since Terraform 0.13+, version constraints belong in required_providers inside terraform block with source address. Option A was deprecated in 0.13.

2. Intermediate Level (10 Questions)

****Q11. What is the purpose of terraform state mv vs terraform import?****

A. Both do the same thing

B. state mv moves/renames resources within state or between states without changing real infrastructure; import brings existing real infrastructure under Terraform management

C. state mv imports from cloud; import moves inside state

D. state mv deletes state entries; import creates infrastructure

****Q12. You have count = 3 on aws_instance.web. How do you reference the second instance's ID, and what is a key limitation of count?****

A. aws_instance.web[1].id; count creates a list and removing an item in the middle causes re-indexing and unwanted recreation

B. aws_instance.web.1.id; count has no limitations

C. `aws_instance.web["1"].id`; count cannot be used with instances

D. `element(aws_instance.web.*.id, 1)` only

****Q13. What does `for_each` require as input, and how does it improve over `count`?****

A. It requires a list; it is identical to `count`

B. It requires a map or set of strings, and creates a map of resources keyed by stable identifiers, avoiding reordering issues when items are removed

C. It requires an integer; it automatically handles dependencies

D. It requires a boolean; it enables parallel execution

****Q14. Consider this backend config: `terraform { backend "s3" { bucket = "my-tf-state" key = "prod/terraform.tfstate" region = "us-east-1" dynamodb_table = "tf-lock" encrypt = true } }`. What is the role of `dynamodb_table`?****

A. It stores the state file

B. It provides state locking and consistency checking to prevent concurrent writes

C. It stores provider plugins

D. It is deprecated and unused

****Q15. What is the difference between `depends_on` and implicit dependencies?***

A. There is no difference

B. Implicit dependencies are created via attribute references (Terraform infers order); `depends_on` is explicit for hidden dependencies where no attribute is referenced, e.g., IAM role must exist before instance that assumes it without direct reference

C. `depends_on` is required for all dependencies

D. Implicit dependencies only work for data sources

****Q16. In Terraform modules, how do you pass data from a child module to root?***

A. Using `export` block in child module

B. Child module defines output blocks, root accesses via `module.<name>.<output_name>`

C. Using global variables

D. Modules cannot return data

****Q17. What does `terraform taint` (legacy) or `-replace` target do, and when would you use it?***

A. It deletes the resource from state permanently

B. It marks a resource for recreation on next apply, useful when resource is degraded but Terraform doesn't detect drift (e.g., corrupted VM)

C. It locks the resource to prevent changes

D. It imports the resource

****Q18. What is the behavior of `lifecycle { create_before_destroy = true }`?***

A. It prevents any destruction

B. Terraform creates the replacement resource first, then destroys the old one, critical for zero-downtime resources like launch templates or certificates requiring name uniqueness

C. It deletes first then creates

D. It ignores changes to the resource

****Q19. How does terraform workspace work and when should you avoid it?***

A. Workspaces create completely separate configs

B. Workspaces are named isolated state files within same backend/config, useful for simple env separation but should be avoided for strong isolation (use separate directories/backends and variable files) due to shared code and risk of misapplied changes

C. Workspaces delete other environments

D. Workspaces are same as modules

****Q20. What does `dynamic` block do in Terraform?***

A. Dynamically creates providers

- B. Generates nested blocks dynamically inside a resource/data source based on a collection, replacing repetitive code; e.g., dynamic ingress blocks for security groups from a list of ports
- C. Allows dynamic backend configuration
- D. Enables runtime evaluation of count

Intermediate Answer Key & Explanations

- **Q11. Answer: B**** - state mv is refactoring tool for state structure without touching real infra; import maps existing real infra to a resource address. Confusing them leads to accidental deletions or duplication.
- **Q12. Answer: A**** - With count, resources are indexed 0..n-1; referenced as resource[1]. Removing index 0 shifts all others, causing recreate. This is why for_each is preferred.
- **Q13. Answer: B**** - for_each requires map or set(string) with stable keys, e.g., for_each = toset(["a","b"]). It maps keys to instances, so removal doesn't affect others, solving count's weakness.
- **Q14. Answer: B**** - DynamoDB table provides state locking for S3 backend. Before S3 native conditional writes (2024), DynamoDB prevented concurrent applies from corrupting state. Now optional but still common.
- **Q15. Answer: B**** - Implicit via aws_instance.example.id creates DAG edge automatically. Explicit depends_on = [aws_iam_role.role] needed when dependency isn't captured by reference, like provisioners or side-effects.
- **Q16. Answer: B**** - Child outputs become attributes on module.<NAME> object. Root cannot access child resources directly, only via declared outputs, enforcing encapsulation.
- **Q17. Answer: B**** - terraform apply -replace="aws_instance.web[0]" forces recreate next apply. Useful for immutable infra pattern or when instance is unhealthy but Terraform sees no diff.
- **Q18. Answer: B**** - create_before_destroy inverts default order to avoid downtime and handle name conflicts. Essential for aws_launch_configuration, ACM certificates, etc., often needs allow_overlapping considerations.
- **Q19. Answer: B**** - Workspaces share backend config and code but separate state via key prefix. For prod/non-prod strong isolation, separate root modules/backends + code reuse via modules is safer and more explicit than workspaces.
- **Q20. Answer: B**** - dynamic with for_each + content generates repeatable nested blocks. It keeps DRY for security groups, lifecycle rules, etc., where Terraform doesn't support count/for_each directly on nested blocks.

3. Advanced Level (10 Questions)

- **Q21. You are using Terraform Cloud remote execution with import blocks (Terraform 1.5+). Which statement is accurate about declarative import?****
- A. import { to = aws_instance.web id = "i-12345" } only works with terraform import CLI
 - B. import { to = aws_instance.web id = "i-12345" } defines import in config; on next plan/apply Terraform merges existing resource into state and then shows diff vs config, enabling codified, reviewable imports and chaining with generation via -generate-config-out
 - C. Import blocks automatically generate correct resource configuration
 - D. Import blocks delete existing state
- **Q22. How do you handle provider version upgrades that introduce breaking state schema changes for resources with moved blocks?****
- A. Delete the state
 - B. Use moved { from = aws_instance.old_name to = aws_instance.new_name } block to refactor without destroying; Terraform treats it as moved in state during plan. For provider schema upgrades, run terraform apply -refresh-only first, check changelog for state migration, and pin providers with ~> constraints
 - C. Moved blocks are not needed
 - D. Always taint and recreate
- **Q23. You have a module with for_each = var.env_map where var.env_map contains sensitive values. terraform plan leaks sensitive values in output. How do you properly mitigate?****
- A. Use sensitive = true on output and variable, but note for_each keys/values exposure: keys are never sensitive, so don't put secrets in map keys; values marked sensitive are redacted, but for_each itself cannot be sensitive - restructure to use non-sensitive keys and handle sensitive data inside resources

B. Set `TF_CLI_CONFIG_FILE`

C. There is no mitigation

D. Use count instead

****Q24. Consider performance at scale (5000 resources). Which patterns optimize plan time?****

A. Always use `-refresh=false`

B. Use `-parallelism=n` tuning (default 10), minimize data sources with dependencies, use moved blocks carefully, leverage `-target` only for emergencies, enable provider-level `skip_*` flags where available, use `terraform_data` instead of `null_resource`, and consider breaking into smaller root modules with dependencies via remote state or Stacks

C. Increase instance size

D. Use count = 5000 on one resource

****Q25. Explain check blocks (Terraform 1.5+) vs precondition/postcondition (Terraform 1.2+). When would you use each?****

A. They are identical

B. check blocks are independent data-source style validations that don't block apply but show warnings/errors for assertions on infra health (e.g., endpoint responds); precondition/postcondition are lifecycle validations that block apply if violated (e.g., postcondition on `aws_instance` ensuring `instance_type` meets policy, or data source precondition ensuring VPC exists)

C. Checks replace providers

D. Preconditions only work for outputs

****Q26. You manage AWS with Terraform Stacks (TF Stacks, GA 2024-2025) and want to handle multi-region deployment with shared components. How should you structure it?****

A. Use one big root module with provider alias per region

B. Define components in `.tfcomponent.hcl`, use component blocks with `for_each` per region, stack variables, and deployment orchestration with deployment blocks; providers configured per component, dependencies via component outputs, enabling orchestrated multi-region/component lifecycle

C. Stacks are same as workspaces

D. You must use Terragrunt

****Q27. What is the implication of using `ignore_changes` with a map attribute, and how does it interact with `jsonencode` drift?****

A. It has no effect

B. `lifecycle { ignore_changes = [tags, ami] }` ignores all future changes to those attributes (useful for externally-managed tags or ASG AMI); but if you ignore `tags_all` or entire map, replacement detection breaks; for `jsonencode` fields (e.g., `aws_iam_policy`), Terraform compares normalized JSON - ignoring it prevents drift fixes but necessary if policy managed outside Terraform

C. `ignore_changes` deletes resources

D. It only works for id

****Q28. You have a provider that hits API rate limits during refresh. How do you design around it in Terraform 1.7+?****

A. There is no solution

B. Strategies: use `terraform_data` with `triggers_replace` + provisioners replaced by API calls, set provider retry config (e.g., `max_retries` for AWS), implement `-parallelism=5` or lower, use removed `{}` block for resources no longer needed without refresh, use ephemeral resources (ephemeral block in TF 1.10+) for transient creds, and split state or use `-refresh-only` + `cached plan`

C. Always use count to reduce API calls

D. Delete provider

****Q29. How do Terraform test framework (.tftest.hcl, GA 1.6+) tests work vs legacy Terratest?****

A. They are identical

B. Native tests: define run blocks with `command` = plan or apply, variables, and assert conditions on outputs/resources; executes in isolated state, supports mocking providers with `mock_provider "aws" {}`, unlike Terratest which uses Go wrapper executing real CLI and needs real creds; native tests are faster for unit-style validation

C. Native tests cannot assert

D. They require Python

****Q30.** You need to perform a zero-downtime migration from `aws_s3_bucket` legacy arguments (`acl`, `versioning {} inline`) to new standalone resources (`aws_s3_bucket_acl`, `aws_s3_bucket_versioning`). Describe the safe path.**

A. Just delete and recreate bucket

B. Safe path: Import legacy bucket then upgrade AWS provider to v4+, then remove inline `acl/versioning/server_side_encryption_configuration` from `aws_s3_bucket` config (now deprecated), create new standalone resources referencing bucket ID, add lifecycle { } if needed, run `terraform plan` - plan should show no bucket recreation, only creation of new versioning/acl resources adopting existing settings; if state mismatch, `terraform state mv` is not needed; key is provider upgrade preserves state schema

C. Use `terraform taint`

D. Buckets cannot be migrated

Advanced Answer Key & Explanations

****Q21.** Answer: B** - Import blocks made imports declarative and reviewable in VCS since TF 1.5. Combined with `terraform plan -generate-config-out` they generate stub configs. This is crucial for brownfield adoption at scale.

****Q22.** Answer: B** - moved blocks enable refactoring addresses without destroy. For provider schema upgrades, check for state upgrade path and test with `terraform plan`. Provider pinning prevents unexpected major upgrades causing state incompatibility.

****Q23.** Answer: A** - `for_each` keys cannot be sensitive (they appear in resource addresses and plan). If secrets are needed as map values, structure map with non-sensitive keys. Outputs/variables with `sensitive = true` redact UI but still store in state - consider remote state encryption.

****Q24.** Answer: B** - Large states cause slow plans due to refresh parallelism. Tuning `-parallelism`, reducing data sources, splitting into micro-stacks (or TF Stacks), and using provider performance features drastically improves time. `-target` and `-refresh=false` are dangerous long-term.

****Q25.** Answer: B** - check blocks decoupled health checks from lifecycle (non-blocking, continue on error). `precondition/postcondition` are blocking assertions evaluated during `plan/apply` for contract enforcement, essential for policy-as-code without Sentinel/OPA.

****Q26.** Answer: B** - Terraform Stacks (`.tfstack.hcl` / `.tfcomponent.hcl`) provide orchestration layer for components across deployments. Unlike workspaces, they manage dependencies, order, and multi-region matrix deployments natively in HCP Terraform.

****Q27.** Answer: B** - `ignore_changes` is powerful for externally mutated fields but hides drift. For JSON fields, Terraform normalizes ordering, so use `jsonencode()` consistently. Ignoring all tags with provider `default_tags` interaction requires ignoring tags not `tags_all`.

****Q28.** Answer: B** - Provider retry + parallelism tuning are primary defenses against rate limits. Ephemeral resources (TF 1.10+) avoid persisting sensitive short-lived data in state. removed block gracefully deletes without requiring refresh of already-removed infra.

****Q29.** Answer: B** - Native TF test framework (1.6+) runs `terraform test`, supports mocks and assertions without external tooling, enabling fast unit testing. Terratest remains for integration/e2e testing with real infra, but native tests are preferred for module contracts.

****Q30.** Answer: B** - AWS provider v4 split S3 bucket inline args into standalone resources to fix perpetual diffs. Safe migration requires provider upgrade plus config refactoring without bucket recreation. Plan should show only new resources being added; if it shows bucket recreation, inline args not fully removed.

Course File: 07_Ansible.md

Course: Ansible

1. Beginner Level (10 Questions)

****Q1: What is the primary purpose of an Ansible Inventory?****

- A) To store encrypted secrets for playbooks
- B) To define the hosts and groups on which Ansible operates
- C) To define reusable roles and collections
- D) To configure Ansible Tower/AWX job templates

****Q2: Which file is checked FIRST for ansible configuration when running ansible-playbook?****

- A) ~/.ansible.cfg
- B) /etc/ansible/ansible.cfg
- C) ANSIBLE_CONFIG environment variable
- D) ./ansible.cfg in current directory

****Q3: What does an Ansible ad-hoc command do compared to a playbook?****

- A) It runs a single task/module on hosts without writing a YAML file
- B) It installs Ansible collections permanently
- C) It encrypts inventory files
- D) It only works with localhost

****Q4: In Ansible, what is idempotency?****

- A) The ability to run a task multiple times with the same result and no unintended changes on subsequent runs
- B) The ability to run playbooks in parallel across 1000s of hosts
- C) The ability to rollback automatically on failure
- D) The ability to encrypt variables at rest

****Q5: What is the default connection method Ansible uses for Linux hosts?****

- A) WinRM
- B) Paramiko
- C) SSH
- D) Local exec

****Q6: You run ansible all -m ping. What does the ping module actually test?****

- A) ICMP network connectivity like the Linux ping command
- B) If the host can reach internet
- C) If Ansible can log in and execute Python code on the target
- D) If the Ansible controller is reachable

****Q7: What is the correct YAML syntax for a basic playbook targeting web servers?****

- A) - hosts: web servers / tasks: - ping
- name: Test connectivity

hosts: web servers

tasks:

- name: Ping hosts

ansible.builtin.ping:

- C) hosts=web servers task=ping
- D) [web servers] ping

****Q8: Where are Ansible facts gathered from, by default?****

- A) From group_vars/all.yml
- B) From the Ansible controller's environment variables
- C) From the target host at the start of a play via the setup module
- D) From Ansible Galaxy

****Q9: Which command installs a role from Ansible Galaxy?****

- A) ansible-galaxy role install <role_name>
- B) ansible install role <role_name>
- C) ansible-playbook --install-role <role_name>
- D) pip install ansible-role-<role_name>

****Q10: What does become: yes do in a play or task?****

- A) Enables privilege escalation (e.g., sudo to root)
- B) Becomes the inventory file
- C) Enables async mode
- D) Forces Ansible to become idempotent

Beginner Answer Key & Explanations

****Q1 Answer: B**** - Inventory is the source of truth for managed nodes, defining hostnames/IPs and group hierarchies. Without an inventory, Ansible doesn't know where to run tasks.

****Q2 Answer: C**** - Precedence order is: ANSIBLE_CONFIG env var > ./ansible.cfg (cwd) > ~/.ansible.cfg > /etc/ansible/ansible.cfg. Since ansible-core 2.12, this is strictly enforced.

****Q3 Answer: A**** - Ad-hoc commands use ansible -m <module> -a args pattern for quick one-off operations. Playbooks are for repeatable, multi-task automation in YAML.

****Q4 Answer: A**** - Most modules are designed to be idempotent, checking state before acting. This makes automation safe to rerun and critical for configuration management.

****Q5 Answer: C**** - The default connection plugin is ssh (now ansible.builtin.ssh). WinRM is used for Windows, and paramiko is a fallback option.

****Q6 Answer: C**** - The ping module does NOT use ICMP; it logs in over SSH/WinRM and runs a small Python snippet returning pong. Failure indicates authentication, Python, or connection issues.

****Q7 Answer: B**** - A playbook is a list of plays with name, hosts, and tasks keys. Option B follows ansible-lint and ansible.builtin FQCN best practices for 2024+.

****Q8 Answer: C**** - By default gather_facts: true runs ansible.builtin.setup at play start, collecting OS, IP, memory facts into ansible_facts. Disable for faster execution on known targets.

****Q9 Answer: A**** - ansible-galaxy role install and ansible-galaxy collection install are the legacy/standard commands. In 2024+ with ansible-core 2.14+, you can also use ansible-galaxy install -r requirements.yml.

****Q10 Answer: A**** - become abstracts sudo/su/runas/doas via become_method. It is the modern replacement for the old sudo: yes which was removed years ago.

2. Intermediate Level (10 Questions)

****Q11: What is the correct variable precedence for group_vars vs host_vars vs play vars?****

- A) host_vars > group_vars/all > play vars: > extra vars -e
- B) extra vars -e > host_vars > group_vars > role defaults
- C) role defaults > group_vars > host_vars > extra vars
- D) Inventory vars never override playbook vars

****Q12: How do you safely handle a task that may fail but should not stop the play?****

- command: /bin/false

ignore_errors: yes

- B) failed_when: false
- C) any_errors_fatal: false
- D) Both A and B are valid depending on intent, but B still marks as changed vs failed

****Q13: What does a Handler do in Ansible?****

- A) Catches Python exceptions from modules
- B) A task that runs only when notified via notify and only once at the end of the play (by default)
- C) Handles inventory connections
- D) Manages Ansible Vault passwords

****Q14: You need to loop over a list of users and create them. What is the modern (2024+) preferred loop syntax?****

- A) with_items: "{{ users }}"
- B) loop: "{{ users }}" with loop_control if needed
- C) for_each: users
- D) with_list:

****Q15: What is Ansible Vault used for?****

- A) To speed up playbook execution by caching facts
- B) To encrypt sensitive data like passwords and keys at rest in YAML files
- C) To store SSH keys for the controller
- D) To lock inventory files from editing

****Q16: Consider this task:****

- template:

src: nginx.conf.j2

dest: /etc/nginx/nginx.conf

notify: Reload nginx

When will Reload nginx handler run if the task runs on 3 hosts but only 1 host changed?

- A) On all 3 hosts at end of play
- B) Only on the 1 host where file changed, once at end of play
- C) Immediately after the task on changed host
- D) Never, unless --force-handlers is used

****Q17: What's the difference between ansible.builtin.include_tasks and ansible.builtin.import_tasks?****

- A) No difference, they are aliases
- B) import_tasks is static/pre-processed at playbook parse time, include_tasks is dynamic at runtime and can be looped/conditional
- C) include_tasks is static, import_tasks is dynamic
- D) import_tasks was removed in ansible-core 2.17

****Q18: How do you install collections in an air-gapped or CI/CD reproducible way (Ansible 2.16+)?****

- A) ansible-galaxy collection install -r requirements.yml checked into repo with pinned versions
- B) Always use ansible-galaxy collection install community.general on controller without requirements
- C) Copy collections to /usr/share/ansible
- D) Use pip install for collections

****Q19: In a role, what is the execution order of defaults/main.yml, vars/main.yml, tasks/main.yml, and meta/main.yml?****

- A) meta dependencies run first, then defaults (lowest precedence), vars override many, and tasks use the final merged vars
- B) tasks > defaults > vars > meta
- C) All run simultaneously
- D) vars is lowest, defaults is highest

****Q20: What does ansible-inventory --graph and ansible-inventory -i inv --host <hostname> help with?****

- A) First shows group hierarchy visually, second shows composed variables for a specific host after all group_vars/host_vars/plugins resolution
- B) Both install inventory plugins
- C) First encrypts inventory, second decrypts
- D) First is deprecated in favor of ansible --graph

Intermediate Answer Key & Explanations

****Q11 Answer: B**** - Extra vars (-e) always win (precedence 21). Host_vars beats group_vars, which beats play vars, which beats role defaults (lowest). This is crucial for environment overrides like dev vs prod.

****Q12 Answer: D**** - ignore_errors: yes ignores failure but shows error, failed_when: false never marks failed. Use failed_when with conditions for semantic failure detection, e.g., failed_when: rc not in [0,2].

****Q13 Answer: B**** - Handlers are idempotent action coalescers, notified only on changed status. Use listen: topic to notify multiple handlers, and meta: flush_handlers to run mid-play.

****Q14 Answer: B**** - loop replaced with *_ lookups since 2.5. with_items still works but is not recommended. loop supports loop_control: {label, pause, extended} and filters better.

****Q15 Answer: B**** - Vault encrypts YAML/files via AES256: ansible-vault encrypt_string, encrypt, view. In 2024+, use multiple vault-ids with vault-id labels for team workflows.

****Q16 Answer: B**** - Handlers run once per host if that host triggered notify via a changed task, batched at end of play. Use flush_handlers or --force-handlers changes behavior on failure.

****Q17 Answer: B**** - Imports are static: cannot be looped, condition applies to all imported tasks, tags are inherited differently. Includes are dynamic, support loops, and are evaluated at runtime. Critical for large playbooks.

****Q18 Answer: A**** - Pinning in requirements.yml with name: community.general version: 9.4.0 + ansible-galaxy install -r requirements.yml ensures reproducible builds. Execution Environments also bundle them.

****Q19 Answer: A**** - Role dependency resolution: meta/main.yml dependencies execute before parent role. defaults has lowest precedence (meant to be overridden), vars/main.yml has high precedence (hardcoded role intent).

****Q20 Answer: A**** - Inventory variable composition is complex with constructed, group_vars, host_vars plugins. These two commands are the primary debugging tools before you actually run a playbook.

3. Advanced Level (10 Questions)

****Q21: You are on ansible-core 2.17+ where ansible-test dropped Python 3.8. In an Execution Environment (EE), what is the role of execution-environment.yml and ansible-builder?****

- A) It defines the base image, Python dependencies, collections, system packages, and bindeps to build a container image containing ansible and its dependencies for consistent controller execution
- B) It is the same as requirements.yml
- C) It defines the target host's OS
- D) It replaces inventory

****Q22: What is the difference between strategy: free vs strategy: linear vs strategy: host_pinned?****

- A) No difference
- B) linear (default): all hosts execute task N before any host starts task N+1; free: each host runs through tasks as fast as possible without batch lockstep; host_pinned: like free but sticky to same worker in AWX/serial batches for fact cache optimization
- C) free runs tasks in random order
- D) host_pinned was removed in 2023

****Q23: You need to patch 200 hosts with zero downtime, 20 at a time, and abort if >25% fail in a batch. Correct pattern?****

- hosts: web

serial: 20

max_fail_percentage: 25

tasks: [...]

B) serial: 20% max_fail_percentage: 25%

C) throttle: 20

D) forks: 20

****Q24: How do you correctly filter/sanitize untrusted input from ansible_facts in a role consumed by others (avoid injection via Jinja2 templating)?****

A) Use {{ user_input }} directly

B) Use {{ user_input | string }} and avoid eval style via | ansible.builtin.quote for shell or rely on module argument escaping, not string concatenation in shell/command; enable jinja2_native and avoid unsafe promotion

C) Ansible automatically sanitizes all facts

D) Disable facts gathering

****Q25: You write a custom filter plugin filter_plugins/custom.py. In ansible-core 2.17, after adding ANSIBLE_JINJA2_NATIVE_WARNING=False, you see performance degradation when returning dicts. Why?****

A) Filter plugins should return native Python types and in 2.14+ jinja2_native=True is default in config, causing type preservation but templating overhead; returning complex types is fine, but you must ensure FilterModule.filters dict mapping is correct and avoid re-templating large dicts via | to_json unnecessarily

B) Filter plugins cannot return dicts

C) Python version is too old

D) You must use query not filter

****Q26: Consider using ansible.builtin.reboot module in a play with serial: 10. Reboots hang after kernel updates on RHEL 9. What is the robust best practice fix in 2024-2026?****

A) Use reboot_timeout: 600, connect_timeout: 20, test_command: whoami, and post_reboot_delay: 30 plus reboot -t now verification

B) Just increase forks

C) Use command: reboot

D) Disable gather_facts

****Q27: How does ansible-core handle delegate_to and run_once interaction with hostvars and delegate_facts?****

A) delegate_to: otherhost runs task on other host but hostvars[inventory_hostname] still refers to original host; delegate_facts: true assigns facts to delegated host not original; run_once: true runs on first host in batch but applies to all hosts unless delegate_to and loop are used carefully

B) delegate_to changes inventory_hostname

C) run_once runs on all hosts

D) delegate_facts is ignored

****Q28: You are migrating from with_items: "{{ some_registered_var.results }}" to loop. After register: result on a looped task, what structure does result.results have and how do you correctly loop over it in next task?****

A) result.results is list of dicts each with item, ansible_facts, changed, failed; iterate with loop: "{{ result.results }}" and access item.item for original item

B) result.results is a dict

C) result no longer contains results

D) Use with_items: result

****Q29: In Molecule 6.x (2024+) for testing Ansible roles, what driver and provisioner combo is recommended for minimal CI without Docker?****

A) driver: podman + provisioner: ansible is common, or driver: delegated with ephemeral VMs; Molecule 6 dropped docker driver as built-in, requiring molecule-plugins and recommends Podman for rootless; Use molecule test --destroy=never is deprecated

B) Use driver: docker only

C) Molecule is deprecated

D) Use driver: ec2 only

****Q30: You need to implement dynamic inventory from AWS with filtering, composition, and caching (amazon.aws collection 8.x). What's correct?****

- A) Use aws_ec2 inventory plugin YAML file demo.aws_ec2.yml with plugin: amazon.aws.aws_ec2, filters: {tag:Env: prod}, keyed_groups:, compose:, cache: true, cache_plugin: jsonfile, and run with ansible-inventory -i demo.aws_ec2.yml --graph
- B) Use static inventory only
- C) Use ec2.py script only (deprecated)
- D) Inventory plugins cannot cache

Advanced Answer Key & Explanations

****Q21 Answer: A**** - Execution Environments provide immutable controller images replacing pip venvs. ansible-builder build uses execution-environment.yml with dependencies: {python: requirements.txt, system: bindeps.txt, galaxy: requirements.yml}. Required for Automation Platform 2.4+ (2024).

****Q22 Answer: B**** - linear ensures ordered steps for rolling deployments needing synchronization. free maximizes speed at cost of inter-host ordering. host_pinned (from 2.12) improves performance for large inventories by pinning hosts to workers, crucial for AWX/AAP.

****Q23 Answer: A**** - serial allows rolling/batch updates (number or percentage). Combined with max_fail_percentage (allowed failure percent in batch before aborting whole play) it's the standard SRE pattern for safe patching. throttle limits concurrent tasks within a batch, forks is global parallelism.

****Q24 Answer: B**** - Ansible's command/shell with string interpolation is injection-prone. Always use args list via argv, or quote filter for shell. Module args are escaped, but shell: "echo {{ user_var }}" is unsafe. jinja2_native changes typing behavior important for security.

****Q25 Answer: A**** - Since ansible-core 2.14, jinja2_native = True default preserves native types reducing string round-trips but requires filter plugins to return proper Python types. Performance issues usually come from unnecessary serialization or bad plugin caching, not dict returns themselves.

****Q26 Answer: A**** - RHEL 9/cloud boots can take >300s after kernel updates. reboot_timeout is total wait, connect_timeout is per attempt, test_command validates readiness (default whoami may fail before cloud-init), use uptime | awk '{print \$3}' alternative checks. Never use raw reboot command.

****Q27 Answer: A**** - This triad is a frequent bug source. inventory_hostname never changes with delegation. delegate_facts controls fact assignment target. run_once executes on batch's first host context, so hostvars and when must be explicit to avoid applying first-host vars everywhere.

****Q28 Answer: A**** - Registered loop results store 1 entry per loop iteration in .results. Next task needs nested item.item access. Common pitfall: forgetting results is list, not dict, and using item directly instead of item.item or item.ansible_facts.

****Q29 Answer: A**** - Molecule 6.0 (2024) refactored drivers out to molecule-plugins. Docker driver requires external plugin, Podman is preferred rootless for GitHub Actions. delegated driver points at Terraform-provisioned infrastructure for realistic tests.

****Q30 Answer: A**** - Current amazon.aws.aws_ec2 plugin (replacing aws_ec2 short name) supports filters, keyed_groups: [{prefix: env, key: tags.Env}], compose: {ansible_host: public_ip_address}, and caching to avoid API rate limits. The legacy ec2.py script was removed in Dec 2023.

Course File: 08_Jenkins.md

Course: Jenkins

1. Beginner Level (10 Questions)

****Q1: What is Jenkins primarily used for in software development?****

- A) Container orchestration
- B) Continuous Integration and Continuous Delivery (CI/CD) automation
- C) Database management
- D) Machine learning model training

****Q2: What is the default language for Jenkins pipelines defined via Jenkinsfile?****

- A) YAML
- B) Groovy-based DSL
- C) Python
- D) Shell Script

****Q3: In a Declarative Pipeline, which block is mandatory and defines the execution environment?****

- A) tools
- B) stages
- C) agent
- D) environment

****Q4: Which directory convention does Jenkins use to store job configurations and builds by default?****

- A) JENKINS_HOME
- B) /etc/jenkins
- C) /var/lib/jenkins_config
- D) ~/.jenkins

****Q5: What is the purpose of the post section in a Declarative Jenkinsfile?****

- A) To publish artifacts to Nexus
- B) To define actions that run after pipeline execution completes (e.g., always, success, failure)
- C) To trigger post-commit hooks in Git
- D) To send HTTP POST requests

****Q6: Which plugin is required to run Docker containers as build agents in Jenkins?****

- A) Docker Pipeline plugin
- B) Kubernetes plugin
- C) Docker plugin / Docker Agent templates
- D) Containerd plugin

****Q7: What does the checkout scm step do in a Jenkinsfile?****

- A) Checks out the source code defined in the job's SCM configuration
- B) Validates SCM credentials
- C) Creates a new SCM repository
- D) Merges branches automatically

****Q8: Which of the following is TRUE about Freestyle vs Pipeline jobs?****

- A) Freestyle jobs support Jenkinsfile
- B) Pipeline jobs are legacy and deprecated since 2024
- C) Freestyle jobs are configured via UI forms, while Pipeline jobs are defined as code and versioned

D) Pipeline jobs cannot use plugins

****Q9: How do you manually trigger a parameterized build with a STRING parameter named VERSION in Jenkins UI?****

A) Click "Build Now"

B) Click "Build with Parameters" and enter VERSION value

C) Run `jenkins-cli build VERSION`

D) Edit `config.xml` directly

****Q10: What is Jenkins Blue Ocean?****

A) A security hardening tool for Jenkins

B) A modern, visual user interface for creating and visualizing pipelines

C) A cloud hosting service for Jenkins

D) A backup and restore plugin

Beginner Answer Key & Explanations

****Q1 Answer: B****

Jenkins is an open-source automation server focused on CI/CD. It automates building, testing, and deploying code, enabling rapid feedback loops.

****Q2 Answer: B****

Jenkins pipelines use a Groovy-based Domain Specific Language (DSL) in Jenkinsfile. Declarative and Scripted syntaxes are both built on top of Groovy.

****Q3 Answer: C****

The `agent` directive is required in Declarative Pipeline and specifies where the pipeline or stage executes (any, none, label, docker, kubernetes). Without it, the pipeline has no executor.

****Q4 Answer: A****

`JENKINS_HOME` (default `/var/jenkins_home` or `~/.jenkins`) contains all jobs, builds, plugins, and configs. Backing up this directory is standard for Jenkins backup.

****Q5 Answer: B****

The `post` block runs conditionally after pipeline completion, e.g., `always { junit ... }`, `failure { slackSend ... }`. It is ideal for cleanup, notifications, and test reporting.

****Q6 Answer: C****

The Docker plugin allows defining Docker Agent templates, while Docker Pipeline plugin provides docker DSL. For ephemeral agents via Docker, Docker plugin/cloud config is the core requirement.

****Q7 Answer: B****

Actually Answer: A - `checkout scm` clones the repository using the SCM configuration from the Multibranch or Pipeline job definition. It ensures Jenkinsfile and sources use same branch/commit.

****Q8 Answer: C****

Freestyle is UI-driven and older, hard to version. Pipeline-as-code via Jenkinsfile enables code review, versioning, durability, and complex workflows — the standard since 2020+.

****Q9 Answer: B****

Parameterized jobs expose "Build with Parameters" in UI, allowing inputs before run. "Build Now" uses defaults without prompting.

****Q10 Answer: B****

Blue Ocean provides a pipeline-centric UX with visual editor and stage visualization. While maintenance mode since 2023, it remains widely recognized for pipeline visualization.

2. Intermediate Level (10 Questions)

****Q11: In Declarative Pipeline, what is the difference between `agent any` at top-level vs `agent none` with per-stage agents?****

A) No difference

B) agent any allocates one executor for entire pipeline; agent none defers allocation allowing each stage to use different agents and saving executors when idle

C) agent none is invalid syntax

D) agent any only works on master

****Q12: You need credentials with ID aws-prod in a pipeline without exposing them in logs. Which is the secure pattern?****

A) `env.AWS_KEY = credentials('aws-prod')`

B) `withCredentials([string(credentialsId: 'aws-prod', variable: 'AWS_KEY')]) { sh 'echo $AWS_KEY' }`

C) `withCredentials([usernamePassword...])` always masks values, but you must use double-quoted Groovy interpolation carefully and avoid echo in plain sh

D) Store in Jenkinsfile as plaintext

****Q13: How do you achieve parallel execution of tests in Declarative Pipeline in 2024+ syntax?****

A) Using parallel directive inside a stage with nested stages

B) Using parallel { } step directly — only available in Scripted

C) Parallel is not supported in Declarative

D) Using throttle plugin only

****Q14: What does options { skipDefaultCheckout() } do, and when would you use it?****

A) Disables automatic checkout scm at start of Declarative Pipeline to manually control checkout timing for efficiency or custom workspace handling

B) Skips all future checkouts

C) Deletes workspace

D) Disables SCM polling

****Q15: How does Jenkins handle shared logic across pipelines in enterprise setups?****

A) Copy-paste Jenkinsfiles

B) Shared Libraries loaded via `@Library('my-lib')` _ with vars/ and src/ structure

C) Symlinking Jenkinsfiles

D) Shared Libraries only work for Freestyle jobs

****Q16: Consider: when { branch 'main'; changeset 'src/**' }. What does this do?****

A) Runs stage only if building main branch AND changes occurred in src/**

B) Runs stage if either condition true

C) Invalid syntax — when cannot have multiple conditions

D) Runs only if files were deleted in src/**

****Q17: You have flaky integration tests. Which Declarative option best handles automatic retry?****

A) options { retry(3) } at stage level

B) stage { options { retry(3) } } or retry(3) { } wrapped in script step

C) Use loop plugin

D) post { retry }

****Q18: How do you archive artifacts and fingerprint them for traceability between builds?****

A) `archiveArtifacts artifacts: 'target/*.jar', fingerprint: true, onlyIfSuccessful: true`

B) `archive 'target/*.jar'`

C) `stash '*.jar'`

D) `publishArtifacts`

****Q19: In Jenkins distributed builds, what is the difference between Master/Agent (formerly Master/Slave) with JNLP vs SSH agents launched in Kubernetes?****

A) No difference

B) JNLP agents have controller initiate connection; SSH agents have agent initiate; Kubernetes plugin dynamically provisions ephemeral pods as agents using JNLP in pod template

C) Kubernetes agents cannot run pipelines

D) SSH launcher is deprecated in 2025

****Q20: How do webhooks trigger Jenkins Multibranch Pipeline efficiently vs SCM polling?****

A) Polling is faster than webhooks

B) Webhooks (e.g., GitHub webhook to /github-webhook/) trigger immediate indexing and build for changed branches/PRs, reducing load; polling scans repo periodically wasting resources

C) Webhooks only work for GitLab

D) Multibranch cannot use webhooks

Intermediate Answer Key & Explanations

****Q11 Answer: B****

agent any locks an executor for whole run, inefficient if pipeline waits. agent none + per-stage agent { label '...' } allocates only when needed, enabling different labels/containers per stage.

****Q12 Answer: C****

withCredentials binds credentials to env vars and Jenkins masks them in logs (with credential masking). However, using Groovy double quotes "\$TOKEN" can cause interpolation before masking; best to use single quotes in sh and let shell expand, e.g., sh 'printenv | grep ...' should avoid printing.

****Q13 Answer: A****

Declarative Pipeline since 2019 supports stage('Test') { parallel { stage('Unit') { steps {...} } stage('Integration') {...} } }. This is the standards-compliant way; Scripted uses parallel mapOfBranches.

****Q14 Answer: A****

By default Declarative checks out SCM automatically. skipDefaultCheckout() disables it so you can e.g., checkout in specific stage, do sparse checkout, or avoid double checkout when using agent docker that would otherwise checkout twice.

****Q15 Answer: B****

Shared Libraries (@Library) define reusable steps in vars/*.groovy and classes in src/. Loaded implicitly via global config or explicitly via annotation, they enable DRY, governance, and versioning (e.g., @Library('my-lib@1.2')).

****Q16 Answer: A****

Multiple conditions inside same when are ANDed by default; use anyOf, allOf, not for logic. Here stage runs only on main branch when changeset includes files under src/, useful for monorepo optimization.

****Q17 Answer: B****

Stage-level options { retry(n) } is valid for retrying whole stage. Alternatively retry(n) { sh '...' } works in scripted context. Post does not support retry logic directly.

****Q18 Answer: A****

archiveArtifacts stores files on controller for download; fingerprint: true tracks which build produced which artifact for dependency tracing. stash/unstash is for inter-stage transfer within same build, not long-term.

****Q19 Answer: B****

Traditional JNLP: agent connects to controller (firewall friendly). SSH: controller connects to agent. Kubernetes plugin launches pod with jnlp container that connects back, creating ephemeral, scalable, isolated builds — standard in 2024-2026 cloud-native setups.

****Q20 Answer: B****

Webhook-driven triggers push model: Git provider POSTs to Jenkins on push/PR, instant. SCM polling (e.g., * * * * *) pulls periodically, high load and delay. For Multibranch, GitHub Branch Source plugin handles webhooks optimally.

3. Advanced Level (10 Questions)

****Q21: You use Jenkinsfile with agent { kubernetes { yaml "" apiVersion: v1 kind: Pod spec: containers: - name: golang image: golang:1.22 command: [cat] tty: true "" } }. The container keeps exiting. Root cause and fix?****

A) Need args for image

B) Command must keep container alive; [cat] is correct but you must disable default tty override — issue is missing yamlFile parameter

C) Kubernetes plugin requires containers to stay running; command: [cat] + tty: true is actually correct pattern, but failure likely due to resourceQuota or missing jnlp container — ensure pod template includes implicit jnlp sidecar or specify defaultContainer

D) Use sleep infinity

****Q22: How do you prevent Script Security sandbox violations when using Shared Library src/ classes that call Jenkins pipeline steps?****

A) Disable sandbox

B) Steps are not accessible in src/ — need to pass Script context (this or steps) to class, or make class implement Serializable and use vars/ wrapper; or use @NonCPS appropriately

C) Annotate all methods with @Trusted

D) Shared libraries bypass sandbox entirely

****Q23: Jenkins controller shows high GC pressure and java.lang.OutOfMemoryError after migrating to Jenkins 2.479+ (Java 21, Jakarta). Most likely mitigation?****

A) Jenkinsfile has memory leak

B) Since Jenkins 2.479+ requires Java 17/21 and Jakarta EE 9, plugins must be updated; old plugins cause metaspace leaks; increase -Xmx and -XX:MaxMetaspaceSize, update Pipeline: Groovy plugin to 2.114+ and remove deprecated Unsafe usages; also use jenkins.model.Jenkins.crumbIssuer optimization

C) Downgrade to Java 8

D) Increase executors on controller to 0 is already required — OOM unrelated

****Q24: You need to implement a Jenkins Pipeline that builds Docker images without Docker-in-Docker (DinD) security risk, in Kubernetes. Best 2024-2026 practice?****

A) Mount docker.sock

B) Use Kaniko, Buildah, or docker buildx with podman rootless, or AWS CodeBuild style; e.g., container gcr.io/kaniko-project/executor:debug with args: --dockerfile=--context=--destination=

C) Always use DinD, it's secure now

D) Use docker { } DSL with privileged true

****Q25: What is the purpose and risk of org.jenkinsci.plugins.scriptsecurity.sandbox.groovy.SecureGroovyScript approval in "In-process Script Approval"?****

A) Approves entire pipeline

B) Admin must approve Groovy methods/signatures that escape sandbox; over-approving method java.lang.Runtime exec creates RCE; should minimize approvals and prefer Shared Library trusted code

C) Automatically approves all plugins

D) No risk

****Q26: You have a large monorepo with 50 microservices. You want Jenkins to build only affected services using changeset AND to avoid rebuilding if Jenkinsfile didn't change. Optimal strategy?****

A) Use when { changeset pattern comparator } per service stage + not { changeset Jenkinsfile}? Better: use Generic Webhook Trigger parsing commit diff, or use pipeline logic with git diff --name-only in script block and set env flags; combine with Shared Library that computes build matrix dynamically and use parallel dynamic stages created in script block

B) Build all always

C) Separate repo per service only solution

D) Use Multibranch for each service folder impossible

****Q27: How does Jenkins achieve Configuration as Code (JCasC) and ephemeral controllers in production (2024-2026)?****

A) JCasC YAML + Job DSL + Helm chart with EFS; controller bootstraps from CASC_JENKINS_CONFIG env, zero manual UI config, secrets via Vault plugin or Kubernetes secrets; jobs recreated via Seed job

B) Manual UI only

C) JCasC deprecated in 2025

D) Groovy init scripts only

****Q28: Explain CPS (Continuation Passing Style) transformation issue causing NotSerializableException or org.jenkinsci.plugins.workflow.cps.CpsCallableInvocation error in pipeline.****

A) Jenkins pipeline must checkpoint; all variables in pipeline scope must be Serializable; local non-serializable objects inside script should be scoped with def inside method or annotated @NonCPS; avoid iterating non-serializable iterators across sh steps

B) Bug in Jenkins

C) Means you need more memory

D) Use serializable true

****Q29: You need to implement strict compliance: audit trail, RBAC per folder, build provenance (SLSA) and prevent sh 'curl | bash'. Architecture?***

A) Impossible in Jenkins

B) Use Role-based Authorization Strategy + Folders Plus, Audit Trail plugin, Pipeline: Groovy library restrictions (reject arbitrary sh?), Warnings NG, use OPA/Conftest in Shared Library validator to block untrusted steps, generate SLSA provenance via slsa-framework/slsa-github-generator style but for Jenkins: attest-provenance with Sigstore Cosign plugin and archive SBOM

C) Only use Freestyle

D) Use Jenkins Enterprise only

****Q30: A Declarative Pipeline uses post { always { junit 'build/test-results/**/*.*.xml' } } but flaky test upload fails silently causing build UNSTABLE misclassification. How to make robust and collect Prometheus metrics for pipeline duration per stage?***

A) Use junit testResults:..., allowEmptyResults: true, skipPublishingChecks: false and wrap in script { try { } catch { unstable() } }; for metrics use OpenTelemetry plugin + Prometheus plugin exposing jenkins_pipeline_stage_duration_seconds and dashboards; also use metrics step for custom latency

B) Ignore

C) post cannot use junit

D) Use HTML publisher only

Advanced Answer Key & Explanations

****Q21 Answer: C****

Kubernetes plugin auto injects jnlp container unless yaml overrides spec completely; if you provide full yaml, you must still ensure JNLP connectivity and resource limits. Pattern command: [cat] tty: true keeps container alive, but pod failure often due to OOMKilled or missing serviceAccount. Debug via kubectl get pod -o yaml and Jenkins podTemplate logs.

****Q22 Answer: B****

src/ code runs as pure Groovy classes, not CPS-transformed pipeline DSL, so sh, echo etc. are not directly available. Pass script context: new MyClass(this).doThing() where this is pipeline script, then steps.sh. Also classes must implement Serializable for checkpointing. vars/ are implicitly trusted and CPS-transformed.

****Q23 Answer: B****

Jenkins 2.479.1 switched to Spring Security 6 + Jakarta Servlet 6 + Java 17 minimum (Java 21 recommended). Outdated plugins using javax.* cause ClassNotFoundException and memory leaks. Fix: upgrade all plugins, use latest workflow-cps, increase heap to 4-8GB for large controllers, enable G1GC -XX:+UseG1GC.

****Q24 Answer: B****

Mounting docker.sock or privileged DinD gives root on node — security anti-pattern. Kaniko builds in userspace without daemon, Buildah rootless, or docker buildx with remote builder. In Kubernetes, use Kaniko container template with volume for Docker config secret, push without privileges.

****Q25 Answer: B****

Sandbox protects controller from malicious pipeline. Admin page Script Approvals lists pending signatures. Approving dangerous signatures like System.exit, Runtime.exec, File methods opens remote code execution for any writer. Best practice: keep approvals minimal, use Shared Libraries marked as trusted which bypass sandbox but are code-reviewed.

****Q26 Answer: A****

Declarative when { changeset } works for simple cases but limited to git comparison against previous build. For monorepo scale, compute affected modules via git diff origin/main...HEAD --name-only in script, build dynamic map of parallel stages. Use changeset as optimization but combine with Shared Library build-matrix generator. Consider Bazel/gradle detection for true incremental.

****Q27 Answer: A****

Modern Jenkins on Kubernetes uses Helm + JCasC YAML (jenkins: systemMessage: credentials: etc.) mounted via ConfigMap, seeds jobs via Job DSL seed job that scans repo for Jenkinsfiles. Controllers ephemeral: stateful part only JENKINS_HOME/jobs optionally backed by PVC or externalized via S3 artifacts + CloudBees backup, secrets from Vault/K8s Secrets.

****Q28 Answer: A****

Jenkins pipeline serializes execution state after each step (CPS). Non-serializable objects (e.g., Matcher, JsonSlurper result without def, DB connections) held across steps cause failure. Fix: confine them to method with @NonCPS or local scope, or make variable non-persistent, e.g., wrap in script { def data = readJSON ... }. Cannot use pipeline steps inside @NonCPS.

****Q29 Answer: B****

Enterprise compliance via: RBAC Folder-based Authorization, Audit Trail + JobConfigHistory plugins for audit, Script Security + Approved Shared Library only, Policy enforcement in Shared Library global wrapper (e.g., reject sh containing curl .. | sh), plus supply-chain security: generate SBOM with cyclonedx plugin, attest via Sigstore Cosign and SLSA provenance for artifact traceability.

****Q30 Answer: A****

allowEmptyResults prevents pipeline failure if no XML found, but you should catchError or unstable conditionally. For observability, OpenTelemetry plugin (since 2023) emits traces/metrics to OTLP/Prometheus, providing per-stage duration, queue time. Prometheus plugin exposes endpoint /prometheus. Create Grafana dashboard for MTTR and flake tracking.

Course File: 09_Linux.md

Course: Linux

1. Beginner Level (10 Questions)

****Q1: Which command displays the current working directory's absolute path?****

- B) pwd
- C) cwd
- D) curdir

****Q2: In the Linux Filesystem Hierarchy Standard (FHS), where are essential user binaries (like ls, cp) typically located in modern systemd-based distros (2024+)?****

- A) /bin -> /usr/bin
- B) /sbin
- C) /opt/bin
- D) /usr/local/bin

****Q3: What does chmod 754 myfile.sh mean in terms of permissions?****

- A) Owner: rwx, Group: r-x, Others: r--
- B) Owner: rwx, Group: r-x, Others: r--
- C) Owner: rwx, Group: r--, Others: r-x (Note: 754 = rwxr-xr--)
- D) Owner: rwx, Group: r-x, Others: r-- Wait - let's be precise: 7=rwx, 5=r-x, 4=r--
- A) Owner: rwx, Group: r-x, Others: r--
- B) Owner: rwx, Group: r--, Others: --x
- C) Owner: rwxr-xr-- (Owner rwx, Group r-x, Others r--)
- D) Owner: r--, Group: rwx, Others: r-x

****Q4: Which command correctly appends the line "hello" to file.txt without overwriting it?****

- A) echo "hello" > file.txt
- B) echo "hello" >> file.txt
- C) echo "hello" >| file.txt
- D) cat "hello" >> file.txt

****Q5: How do you list all files, including hidden files (dotfiles), in long format in a directory?****

- A) ls -l
- B) ls -la or ls -l -a
- C) ls --hidden
- D) ls -h

****Q6: What is the purpose of the grep -r "error" /var/log/ command?****

- A) Removes all files containing "error" from /var/log/
- B) Recursively searches for the string "error" in all files under /var/log/
- C) Replaces "error" with empty string in log files
- D) Shows only file names with read errors in /var/log/

****Q7: Which key combination and command is used to get help/manual for the ls command?****

- A) help ls
- B) man ls
- C) info ls -h
- D) ls --manual

****Q8:** You try to delete a file but get "Permission denied". You check with `ls -l` and see `-r--r--r-- 1 root root`. As a regular user, what is fundamentally preventing deletion?******

- A) The file's read-only permissions prevent deletion; you need to `chmod` it
- B) Deletion depends on the parent directory's write and execute permissions, not the file's permissions
- C) Only root can ever delete files owned by root
- D) The file has the immutable attribute set

****Q9:** What does the command `sudo apt update && sudo apt upgrade -y` do on Ubuntu/Debian (2024)?******

- A) Updates all installed packages to latest versions after refreshing package index
- B) Upgrades the distribution to the next Ubuntu release
- C) Updates only the apt tool itself
- D) Cleans apt cache and reinstalls all packages

****Q10:** How can you view the contents of a file named `large.log` page by page, with forward/backward scrolling and search?******

- A) `cat large.log`
- B) `less large.log`
- C) `more large.log`
- D) `open large.log`

Beginner Answer Key & Explanations

****Q1 Answer:** B) `pwd` - print working directory******

Explanation: `pwd` is a POSIX-standard builtin that prints the absolute canonical path of the current directory. It is essential for scripting and navigation to confirm location.

****Q2 Answer:** A) `/bin -> /usr/bin`******

Explanation: Since the `usr-merge` (completed in Debian 12, Ubuntu 20.04+, Fedora, RHEL 9), `/bin` is a symlink to `/usr/bin` for compatibility. All essential binaries actually live in `/usr/bin`.

****Q3 Answer:** C) Owner `rx` (7), Group `r-x` (5), Others `r--` (4) -> `rxr-xr--`******

Explanation: The octal 754 translates as $7=4+2+1=rx$, $5=4+1=r-x$, $4=r--$. This is a common permission for a script that owner can edit/execute while others can read/execute or only read.

****Q4 Answer:** B) `echo "hello" >> file.txt`******

Explanation: `>>` is the append redirection operator while `>` truncates and overwrites. Appending is critical for logging and building files safely without data loss.

****Q5 Answer:** B) `ls -la` or `ls -l -a`******

Explanation: `-a` shows all files including dotfiles, and `-l` uses long listing format with permissions, owner, size, and timestamp. Combined as `-la` it is the standard way to audit a directory.

****Q6 Answer:** B) Recursively searches for the string "error" in all files under `/var/log`******

Explanation: `-r` makes `grep` recursive, searching subdirectories. This command is a fundamental log troubleshooting pattern for sysadmins diagnosing system issues.

****Q7 Answer:** B) `man ls`******

Explanation: `man` displays the manual page from the system's man database, the canonical documentation for commands, system calls, and config files. It's the primary help system since UNIX V7.

****Q8 Answer:** B) Deletion depends on the parent directory's write and execute permissions, not the file's permissions******

Explanation: In Linux, unlinking a file modifies the directory, so you need `wx` on the directory. The file's own permissions control reading/writing content, not deletion, unless sticky bit or immutable flag applies.

****Q9 Answer:** A) Updates all installed packages to latest versions after refreshing package index******

Explanation: `apt update` refreshes package lists from repositories without installing anything, while `apt upgrade` installs newer versions of installed packages. The `&&` chains them safely and `-y` auto-confirms.

****Q10 Answer:** B) `less large.log`******

Explanation: less is a pager that loads files efficiently on demand, supports backward scrolling, forward search with /, and doesn't load the entire file into memory like cat. It's standard on all 2024+ distros.

2. Intermediate Level (10 Questions)

****Q11: What is the difference between systemctl restart nginx and systemctl reload nginx?****

- A) Both do the same thing
- B) restart stops and starts the process (new PID, drops connections), reload re-reads config without terminating process if supported
- C) reload restarts the service, restart only reloads config
- D) restart is deprecated in systemd 255+

****Q12: You run ps aux | grep python and see [python] <defunct>. What does this indicate and how do you fix it properly?****

- A) The process is running normally; defunct means optimized
- B) It's a zombie process - its parent hasn't called wait() to read exit status. Fix by fixing/killing parent, not the defunct itself
- C) It's an orphan process; adopt it with disown
- D) Kernel bug; you must reboot

****Q13: Explain the meaning of 2>&1 in bash command ./script.sh > output.log 2>&1****

- A) Redirects FD 2 (stderr) to the same location as FD 1 (stdout), so both go to output.log
- B) Redirects stdout to stderr
- C) Duplicates process ID 2 to process ID 1
- D) Merges stdout with stdin

****Q14: What does the SUID bit do, and what is a security risk of chmod u+s /usr/bin/myapp?****

- A) It makes file undeletable; no risk
- B) When set on executable, it runs with file owner privileges, not executor. Risk: if myapp is vulnerable, privilege escalation to owner (often root)
- C) It makes file hidden; risk is obscurity
- D) It forces file into swap; risk is memory exhaustion

****Q15: You need to find files larger than 500MB under /home modified in last 7 days. Which command is correct?****

- A) find /home -size +500M -mtime -7 -type f
- B) find /home -size 500MB --modified 7d
- C) ls -lh /home | grep 500M
- D) du -sh /home/* | grep 500M

****Q16: Difference between hard link and symbolic (soft) link when original file is deleted?****

- A) Both become invalid
- B) Hard link still contains data as long as one link count >0; symlink becomes dangling/broken
- C) Hard link is deleted automatically, symlink preserves data
- D) No difference on ext4/xfs in 2024 kernels

****Q17: In iptables vs nftables on a RHEL 9 / Ubuntu 22.04+ system, which statement is true for 2024-2026?****

- A) iptables is still the kernel backend; nftables is user wrapper
- B) nftables is the default backend; iptables command is often a compatibility shim via iptables-nft using nftables kernel API
- C) nftables is deprecated
- D) They cannot coexist

****Q18: What does journalctl -u sshd --since "1 hour ago" -p err do?****

- A) Edits SSH config for errors in last hour
- B) Queries systemd journal for sshd unit logs in last hour with priority err and higher
- C) Deletes sshd logs older than 1 hour
- D) Restarts sshd if errors in last hour

****Q19: How do you persistently add a static route on a system using NetworkManager (nmcli) in RHEL 9?****

- A) route add -net 10.0.0.0/24 gw 192.168.1.1
- B) nmcli connection modify eth0 +ipv4.routes "10.0.0.0/24 192.168.1.1" and nmcli con up eth0
- C) Edit /etc/gateway.conf
- D) ip route add alone is persistent across reboots

****Q20: In bash, what is the difference between source script.sh (or . script.sh) and ./script.sh?****

- A) No difference
- B) source runs in current shell environment (variables/functions persist), ./script.sh runs in a new subshell
- C) ./script.sh is faster and sources env
- D) source is only for Python scripts

Intermediate Answer Key & Explanations

****Q11 Answer: B) restart stops and starts the process (new PID, drops connections), reload re-reads config without terminating process if supported****

Explanation: Restart causes downtime and breaks active connections as PID changes. Reload sends SIGHUP or equivalent for zero-downtime config reload, critical for production web servers.

****Q12 Answer: B) It's a zombie process - its parent hasn't called wait() to read exit status. Fix by fixing/killing parent, not the defunct itself****

Explanation: Defunct entries remain in process table until parent reaps them. Killing the zombie with kill -9 does nothing; you must kill the parent so init/systemd re-parents and reaps it.

****Q13 Answer: A) Redirects FD 2 (stderr) to the same location as FD 1 (stdout), so both go to output.log****

Explanation: File descriptors 0=stdin,1=stdout,2=stderr. Order matters: > output.log 2>&1 must have stdout redirect first so stderr copies its destination. Reversing order behaves differently.

****Q14 Answer: B) When set on executable, it runs with file owner privileges, not executor. Risk: if myapp is vulnerable, privilege escalation to owner (often root)****

Explanation: SUID allows controlled privilege elevation like passwd or sudo. If set on arbitrary binaries or writable interpreters, any local user can get root. Auditing SUID binaries with find / -perm -4000 is a core hardening step.

****Q15 Answer: A) find /home -size +500M -mtime -7 -type f****

Explanation: find predicates: +500M means greater than 500 MiB, -mtime -7 means modified less than 7*24h ago, -type f regular files. It's efficient because it filters at filesystem level.

****Q16 Answer: B) Hard link still contains data as long as one link count >0; symlink becomes dangling/broken****

Explanation: Hard link shares same inode and data blocks; data persists while link count >=1. Symlink stores path string, so if target is deleted it breaks. Hard links cannot cross filesystems.

****Q17 Answer: B) nftables is the default backend; iptables command is often a compatibility shim via iptables-nft using nftables kernel API****

Explanation: Since Debian 10, RHEL 8+, Ubuntu 20.10+, kernel packet filtering uses nf_tables. The iptables binary uses xtables2nft translation layer, seen via iptables -V and update-alternatives.

****Q18 Answer: B) Queries systemd journal for sshd unit logs in last hour with priority err and higher****

Explanation: journalctl is the systemd log query tool replacing /var/log/messages. Filtering by unit -u, time --since, and priority -p err (0 emerg to 3 err) is essential for modern troubleshooting.

****Q19 Answer: B) nmcli connection modify eth0 +ipv4.routes "10.0.0.0/24 192.168.1.1" and nmcli con up eth0****

Explanation: ip route add is ephemeral until reboot. NetworkManager stores persistent config in /etc/NetworkManager/system-connections/. Using nmcli with +ipv4.routes ensures survival after reboot and integration with NM dispatcher.

****Q20 Answer: B) source runs in current shell environment (variables/functions persist), ./script.sh runs in a new subshell****

Explanation: Sourcing is required for environment setup scripts like venv activate, /etc/profile changes. Executing creates child process that cannot modify parent shell's environment variables.

3. Advanced Level (10 Questions)

****Q21:** Explain the Linux OOM Killer behavior with `oom_score_adj`. A container with cgroup v2 and high memory usage is getting killed despite `oom_score_adj = -1000` on its main process. Why?******

- A) `oom_score_adj` is ignored in cgroup v2
- B) cgroup-level OOM kills at cgroup level bypass per-process `oom_score_adj`; `memory.max` enforcement kills entire cgroup; `oom_score_adj` only affects system-wide OOM selection, not cgroup limit enforcement
- C) `-1000` means kill first
- D) OOM killer is disabled in containers

****Q22:** You debug production latency spikes. `perf top` shows high overhead in `native_queued_spin_lock_slowpath`. `cat /proc/softirqs` and `/proc/interrupts` look normal. Which is most likely root cause and next tool in 2024+?******

- A) Disk IO; use `iostat`
- B) Severe lock contention on kernel spinlocks / `qspinlock`, often due to many vCPUs contending on same lock (e.g., mm, scheduler). Next: `perf c2c`, `bpftool` or `lock_stat`, check `schedstat`, reduce vCPU or sharding
- C) Network bug; use `tcpdump`
- D) Normal behavior, ignore

****Q23:** Difference between PID namespace and cgroup in container isolation (Docker/Podman/K8s) and why you need both?******

- A) They are same
- B) PID namespace provides visibility/isolation of PIDs and prevents seeing/killing host processes; cgroup provides resource limiting/accounting (CPU, memory, io). Need both for security + resource fairness
- C) PID namespace limits memory; cgroup limits PIDs
- D) cgroup is deprecated in favor of PID namespace in kernel 6.x

****Q24:** Your pod shows `nsenter -t <PID> -n ip` has no interface, but `nsenter -t <PID> -m` shows files. What state? Also how would eBPF help debug CNI issues vs sidecar?******

- A) Normal pod
- B) Network namespace is isolated/empty due to CNI failure to attach veth; mount namespace still has containerFS. Debug with eBPF via `bpftool`, cilium monitor, tc programs to trace netlink CNI calls and packet drops without restarting pod
- C) Container is privileged
- D) Kernel panic

****Q25:** Systemd user says `systemd-run --scope --user -p MemoryMax=1G stress --vm 1 --vm-bytes 2G` fails to limit. On Ubuntu 22.04+ with cgroup v2, what is required to enable resource control for non-root user?******

- A) Nothing; it should work
- B) Must enable delegation via `systemd Delegate=yes` and enable controllers in `/sys/fs/cgroup/user.slice/.../cgroup.subtree_control`, ensure user has cgroup controller delegation; by default user slices may not have memory controller enabled without `systemctl set-property` or `logind EnableControllers`
- C) Use cgroup v1 only
- D) Run as root only

****Q26:** Secure Boot + kexec + kernel lockdown: On a kernel in lockdown=confidentiality mode (2024+), what fails and why?******

- A) Nothing fails
- B) kexec of unsigned kernel, `/dev/mem` access, BPF that reads kernel memory, and unsigned module loads are blocked because lockdown prevents bypassing Secure Boot-verified integrity boundary by user still with root
- C) Only SSH fails
- D) eBPF is always blocked

****Q27:** You tune a high-throughput NUMA server (2 sockets, NVMe, 100G NIC) Linux 6.6+. `ethtool -G, sysctl net.core.busy_poll, irqbalance` disabled with manual affinity, but PPS still low and `cat /proc/net/softnet_stat` shows many drops in column 2 and `netstat -s` shows RPS? Actually column 2 is `time_squeezed`. What combination fixes it?******

- A) Increase ring buffer only
- B) Enable RPS/RFS/XPS tuning, adjust `netdev_budget` and `netdev_max_backlog`, use XDP or AF_XDP, pin application to same NUMA node as NIC/IRQs, check `ksoftirqd` saturation and use `busy_poll` or thread-ized NAPI

C) Disable NUMA

D) Reboot

****Q28:** A process opens file then unlinks it while still holding FD. What happens to disk space and how do you recover content on ext4/xfs (2024 kernel) without restarting?******

A) Space freed immediately

B) Space not freed until FD closed; inode stays allocated with link count 0. Recover via `/proc/<pid>/fd/<fd>` (readlink or cp), or lsof +L1 shows, and `/proc` path provides access to inode data

C) Data is lost forever

D) Kernel bug

****Q29:** Compare OverlayFS (used in Docker/Container) performance vs native - why might find with many stat() be slow and what workaround in 2024+?******

A) OverlayFS always fast

B) Overlay requires lookup in all lower layers for each file (metacopy + redirect) causing dentry chain walk and possible metacopy check; use volatile mount for scratch, enable metacopy=on and index=on, or use fscache, or avoid excessive stat by using st_dev check and caching, or use overlayfs-fast or bind mount for build dirs

C) Disable SELinux

D) No workaround

****Q30:** You need to trace why execve() for a binary intermittently fails with EACCES despite rwx and no SELinux/AppArmor denial. Auditd shows nothing. Which advanced Linux feature should you check? (Hint: 2024+ container/sandbox hardening)******

A) chmod

B) Check Landlock, seccomp-bpf filters, NoNewPrivs, BPF LSM, and Integrity (IMA/EVM) or mount noexec, or fs.protected_regular and capability checks (FCaps). Use bpftool prog, grep Seccomp `/proc/<pid>/status`, cat `/proc/<pid>/attr/current`, check landlock via audit or strace of landlock rules, and dmesg for AppArmor implicit deny

C) Reinstall OS

D) Disk failure

Advanced Answer Key & Explanations

****Q21 Answer:** B) cgroup-level OOM kills at cgroup level bypass per-process oom_score_adj; memory.max enforcement kills entire cgroup; oom_score_adj only affects system-wide OOM selection, not cgroup limit enforcement******

Explanation: In cgroup v2, hitting memory.max triggers cgroup allocator OOM, not system-wide OOM Killer. The cgroup controller kills processes inside regardless of oom_score_adj=-1000 which only protects against global OOM. Fix is increasing memory.max/memory.high or tuning memory.oom.group.

****Q22 Answer:** B) Severe lock contention on kernel spinlocks / qspinlock, often due to many vCPUs contending on same lock (e.g., mm, scheduler). Next: perf c2c, bpfftrace or lock_stat, check schedstat, reduce vCPU or sharding******

Explanation: native_queued_spin_lock_slowpath in perf top is classic sign of highly contended spinlocks, especially on overcommitted VMs or large SMP. Tools like bpfftrace lock tracing, perf lock, or `/proc/lock_stat` (if enabled) pinpoint holders.

****Q23 Answer:** B) PID namespace provides visibility/isolation of PIDs and prevents seeing/killing host processes; cgroup provides resource limiting/accounting (CPU, memory, io). Need both for security + resource fairness******

Explanation: Namespaces (PID, NET, MNT, UTS, IPC, USER, Cgroup, Time) give illusion of isolated system. Cgroups don't isolate visibility, only limit/account resources. Containers need both plus seccomp/SELinux/caps for full isolation.

****Q24 Answer:** B) Network namespace is isolated/empty due to CNI failure to attach veth; mount namespace still has containerFS. Debug with eBPF via bpftool, cilium monitor, tc programs to trace netlink CNI calls and packet drops without restarting pod******

Explanation: Pod sandbox creation failure leaves netns empty. Using nsenter -n isolates network view. eBPF-based CNI (Cilium, Calico eBPF) allows introspection with zero downtime eBPF programs on tc hook to trace why veth attach or routing failed.

****Q25 Answer:** B) Must enable delegation via systemd Delegate=yes and enable controllers in `/sys/fs/cgroup/user.slice/.../cgroup.subtree_control`, ensure user has cgroup controller delegation; by default user slices may not have memory controller enabled without systemctl set-property or logind EnableControllers******

Explanation: Cgroup v2 strict delegation model: parent must enable controllers for child via cgroup.subtree_control. For user slice, systemd only delegates cpu by default; memory/io need explicit enable via systemd property Delegate=cpu cpuset io

memory pids and +memory echo into subtree_control.

****Q26 Answer: B)** kexec of unsigned kernel, /dev/mem access, BPF that reads kernel memory, and unsigned module loads are blocked because lockdown prevents bypassing Secure Boot-verified integrity boundary by user still with root**

Explanation: Kernel lockdown in confidentiality/integrity mode, auto-enabled when Secure Boot is on since kernel 5.4+, restricts even root from violating trust model. It blocks kexec unsigned kernels, debugfs, /dev/mem/kcore, and BPF helpers that leak kernel addresses.

****Q27 Answer: B)** Enable RPS/RFS/XPS tuning, adjust netdev_budget and netdev_max_backlog, use XDP or AF_XDP, pin application to same NUMA node as NIC/IRQs, check ksoftirqd saturation and use busy_poll or thread-ized NAPI**

Explanation: Column 2 in /proc/net/softnet_stat is time_squeezed, meaning ksoftirqd couldn't process all budget in time, due to CPU saturation on IRQ core. Proper fix is distribute processing via RPS, align affinity, increase net.core.netdev_budget, or use XDP for kernel-bypass, all critical for 100G tuning in 2024+ kernels.

****Q28 Answer: B)** Space not freed until FD closed; inode stays allocated with link count 0. Recover via /proc/<pid>/fd/<fd> (readlink or cp), or lsof +L1 shows, and /proc path provides access to inode data**

Explanation: Unlink removes directory entry but inode remains while open FD holds refcount. df still shows space used. /proc/<pid>/fd entries point to deleted inode ((deleted) in ls -l), allowing copy recovery until process closes FD or exits.

****Q29 Answer: B)** Overlay requires lookup in all lower layers for each file (metacopy + redirect) causing dentry chain walk and possible metacopy check; use volatile mount for scratch, enable metacopy=on and index=on, or use fscache, or avoid excessive stat by using st_dev check and caching, or use overlayfs-fast or bind mount for build dirs**

Explanation: OverlayFS whiteouts and metacopy mean stat must merge upper and lower. On deep layers (common in Docker with many RUN), lookup cost O(n_layers). Modern optimizations: metacopy reduces copy_up of data, volatile disables sync, and using cache=bind for builds improves drastically.

****Q30 Answer: B)** Check Landlock, seccomp-bpf filters, NoNewPrivs, BPF LSM, and Integrity (IMA/EVM) or mount noexec, or fs.protected_regular and capability checks (FCaps). Use bpftool prog, grep Seccomp /proc/<pid>/status, cat /proc/<pid>/attr/current, check landlock via audit or strace of landlock rules, and dmesg for AppArmor implicit deny**

Explanation: Modern sandbox (Flatpak, Chrome, systemd ProtectSystem, containerd) uses Landlock unprivileged LSM and seccomp-bpf that can deny execve without audit log. Seccomp shows in /proc/pid/status, Landlock in kernel 5.13+, and BPF LSM allows custom policies invisible to traditional auditd. Need specialized tooling to diagnose.

Course File: 10_Networking.md

Course: Networking

1. Beginner Level (10 Questions)

****Q1. In the OSI model, which layer is responsible for logical addressing (IP) and routing?****

- A) Layer 2 - Data Link
- B) Layer 3 - Network
- C) Layer 4 - Transport
- D) Layer 7 - Application

****Q2. What does a /24 subnet mask (255.255.255.0) indicate?****

- A) 24 usable host addresses
- B) 254 usable host addresses in the network
- C) 24 networks in the subnet
- D) The network supports IPv6

****Q3. Which protocol automatically assigns IP addresses to devices on a LAN?****

- A) DNS
- B) DHCP
- C) ARP
- D) NAT

****Q4. You run ping 8.8.8.8 and get replies, but ping google.com fails. What is the most likely issue?****

- A) Default gateway misconfigured
- B) DNS resolution failure
- C) Physical cable disconnected
- D) DHCP server down

****Q5. What is the primary purpose of NAT (Network Address Translation) in a home router?****

- A) To encrypt Wi-Fi traffic
- B) To allow multiple private IP devices to share a single public IP to access the internet
- C) To block malware
- D) To increase internet speed

****Q6. Which device operates at Layer 2 and forwards frames based on MAC addresses?****

- A) Router
- B) Hub
- C) Managed Switch
- D) Modem

****Q7. TCP vs UDP: Which statement is TRUE?****

- A) UDP provides guaranteed delivery and ordering, TCP does not
- B) TCP is connection-oriented with retransmissions; UDP is connectionless with low overhead
- C) Both use three-way handshake
- D) DNS always uses TCP only

****Q8. What is the function of ARP?****

- A) To translate domain names to IP addresses
- B) To map an IP address to a MAC address on a local network
- C) To assign IP addresses dynamically

D) To route packets between VLANs

****Q9. Which Wi-Fi standard from the Wi-Fi 6/6E family operates in the 6 GHz band to reduce congestion?****

A) 802.11n

B) 802.11ac (Wi-Fi 5)

C) 802.11ax (Wi-Fi 6E)

D) 802.11b

****Q10. What does 1000BASE-T mean for an Ethernet cable?****

A) 1000 Mbps over fiber optic

B) 100 Mbps over twisted pair

C) 1000 Mbps (1 Gbps) over twisted pair copper

D) 1000 Mbps over wireless

Beginner Answer Key & Explanations

****Q1 Answer: B) Layer 3 - Network****

The Network layer handles logical addressing (IPv4/IPv6) and routing across networks. Routers operate here to forward packets based on destination IP.

****Q2 Answer: B) 254 usable host addresses in the network****

A /24 leaves 8 host bits (32-24), giving 256 total addresses minus network and broadcast = 254 usable hosts. This is the most common subnet for small LANs.

****Q3 Answer: B) DHCP****

DHCP (Dynamic Host Configuration Protocol) automatically leases IP, subnet mask, gateway, and DNS. Without it, devices need static configuration.

****Q4 Answer: B) DNS resolution failure****

Since ICMP to a public IP works, L1-L3 connectivity is fine. Failure by domain name points to DNS not resolving names to IPs.

****Q5 Answer: B) To allow multiple private IP devices to share a single public IP to access the internet****

NAT translates private RFC1918 addresses (e.g., 192.168.1.x) to the router's public IP. This conserves IPv4 addresses and provides basic isolation.

****Q6 Answer: C) Managed Switch****

Switches build MAC address tables and forward frames only to the destination port, unlike hubs which flood. Routers operate at L3 with IP.

****Q7 Answer: B) TCP is connection-oriented with retransmissions; UDP is connectionless with low overhead****

TCP provides reliability, ordering, and flow control (used for HTTP, SMTP). UDP is fast with no guarantees, ideal for DNS, VoIP, and gaming.

****Q8 Answer: B) To map an IP address to a MAC address on a local network****

ARP broadcasts "who has 10.0.0.1?" and caches the MAC reply. Without ARP, Layer 3 packets cannot be encapsulated into Layer 2 frames.

****Q9 Answer: C) 802.11ax (Wi-Fi 6E)****

Wi-Fi 6E extends Wi-Fi 6 (802.11ax) into the 6 GHz band, adding up to 1200 MHz of spectrum and 59 non-overlapping channels to avoid 2.4/5 GHz congestion.

****Q10 Answer: C) 1000 Mbps (1 Gbps) over twisted pair copper****

1000 = speed in Mbps, BASE = baseband, T = twisted-pair copper. It's the standard for Gigabit Ethernet on Cat5e/Cat6.

2. Intermediate Level (10 Questions)

****Q11. A network engineer configures two VLANs (VLAN10 and VLAN20) on a single L2 switch, but devices in different VLANs cannot communicate. What is required for inter-VLAN communication?****

A) Enable STP

- B) A Layer 3 device (router or L3 switch) with SVIs or router-on-a-stick
- C) Increase MTU to 9000
- D) Configure LACP

****Q12. In OSPF, what is the purpose of Area 0 (backbone area)?****

- A) To provide NAT translation
- B) All other areas must connect to Area 0 to prevent routing loops and ensure inter-area reachability
- C) To filter BGP routes
- D) To elect the root bridge

****Q13. You see high retransmissions and poor 2.4 GHz Wi-Fi throughput in a dense apartment building. Analyzer shows overlapping channels. Best fix?****

- A) Set all APs to channel 6 at max power
- B) Use 20 MHz channels 1, 6, 11 with proper power tuning and enable WPA3, plus migrate capable clients to 5 GHz / 6 GHz
- C) Disable encryption
- D) Use 40 MHz width on 2.4 GHz

****Q14. TCP 3-way handshake is SYN -> SYN-ACK -> ACK. What security attack exploits this by sending many SYNs with spoofed IPs and never completing?****

- A) ARP poisoning
- B) SYN flood (DoS)
- C) DNS cache poisoning
- D) DHCP starvation

****Q15. An enterprise uses 10.0.0.0/8 internally. When a user accesses the internet, what happens to the source IP in the packet at the edge firewall?****

- A) It remains 10.x.x.x on the internet
- B) It is translated via NAT/PAT to a public IP
- C) It is encrypted via IPsec
- D) It is dropped by default

****Q16. You configure spanning-tree mode rapid-pvst. What does RSTP improve over original STP (802.1D)?****

- A) Increases bandwidth
- B) Reduces convergence from ~30-50s to <5s using proposal/agreement and alternate/backup port roles
- C) Adds VLAN tagging
- D) Enables 100 Gbps links

****Q17. What is the key difference between TCP BBR and CUBIC congestion control (relevant for 2024-2026 cloud networks)?****

- A) BBR is loss-based, CUBIC is delay-based
- B) CUBIC uses packet loss as congestion signal; BBR models bottleneck bandwidth and RTT to avoid bufferbloat and improves throughput on lossy/high-RTT paths
- C) BBR only works for UDP
- D) No difference

****Q18. A branch office connects to HQ via MPLS, but you want encrypted overlay with dynamic path selection over internet broadband as backup. Which modern WAN technology achieves this?****

- A) Frame Relay
- B) SD-WAN with IPsec tunnels and application-aware routing
- C) Hub with repeaters
- D) ISDN

****Q19. IPv6 address 2001:db8::1 is being pinged. What protocol replaces ARP in IPv6 for neighbor discovery?****

- A) ARP still
- B) NDP using ICMPv6 (Neighbor Solicitation/Advertisement) multicast to ff02::1:ff00:0/104

C) DHCPv6 only

D) RARP

****Q20. You configure LACP between two switches with 2x 10G links. What is the result?****

A) Only one link active, other blocked by STP

B) Both links aggregated into a single logical EtherChannel with ~20 Gbps and redundancy / load balancing via hashing

C) Creates a routing loop

D) Requires BGP

Intermediate Answer Key & Explanations

****Q11 Answer: B) A Layer 3 device (router or L3 switch) with SVIs or router-on-a-stick****

VLANs isolate broadcast domains at L2. Inter-VLAN routing requires an L3 interface with an IP in each VLAN to route between them.

****Q12 Answer: B) All other areas must connect to Area 0 to prevent routing loops and ensure inter-area reachability****

OSPF's hierarchical design mandates Area 0 as transit. Inter-area routes must go through Area 0, preventing loops via LSAs.

****Q13 Answer: B) Use 20 MHz channels 1, 6, 11 with proper power tuning and enable WPA3, plus migrate capable clients to 5 GHz / 6 GHz****

Channels 1, 6, 11 are the only non-overlapping 2.4 GHz channels. 20 MHz reduces interference, power tuning reduces CCI, and 5/6 GHz offload is standard 2024+ best practice.

****Q14 Answer: B) SYN flood (DoS)****

The attacker exhausts the target's TCP backlog with half-open connections. Mitigations include SYN cookies, rate limiting, and modern firewall defenses.

****Q15 Answer: B) It is translated via NAT/PAT to a public IP****

RFC1918 private addresses are not routable on the public internet. Edge NAT/PAT translates source IP/port to a public routable IP.

****Q16 Answer: B) Reduces convergence from ~30-50s to <5s using proposal/agreement and alternate/backup port roles****

RSTP (802.1w) defines P/A handshake, faster topology change propagation, and port roles that pre-alternate loop-free paths.

****Q17 Answer: B) CUBIC uses packet loss as congestion signal; BBR models bottleneck bandwidth and RTT to avoid bufferbloat and improves throughput on lossy/high-RTT paths****

CUBIC fills buffers until loss. BBRv2/v3 (deployed widely by Google/CLOUDFLARE in 2024-2026) measures BtlBw and RTprop, yielding lower latency and better performance on modern WANs.

****Q18 Answer: B) SD-WAN with IPsec tunnels and application-aware routing****

SD-WAN builds encrypted overlays (IPsec/DTLS) over any transport (MPLS, broadband, LTE), measures SLA, and dynamically steers traffic per application policy.

****Q19 Answer: B) NDP using ICMPv6 (Neighbor Solicitation/Advertisement) multicast to ff02::1:ff00:0/104****

IPv6 has no broadcast ARP. NDP uses ICMPv6 types 135/136 to solicited-node multicast, plus DAD and router discovery.

****Q20 Answer: B) Both links aggregated into a single logical EtherChannel with ~20 Gbps and redundancy / load balancing via hashing****

LACP (802.1AX) negotiates bundling. Traffic is load-shared via src-dst IP/MAC hash, and failure of one link causes sub-second failover, not STP blocking.

3. Advanced Level (10 Questions)

****Q21. In BGP, you receive the same prefix /24 from two eBGP peers. Path1: AS_PATH 64500 64501, MED 100, LocalPref default 100. Path2: AS_PATH 64500 64502 64503, MED 50. All else equal, which path is preferred by BGP best-path selection and why?****

A) Path2 because MED is lower

B) Path1 because shorter AS_PATH is evaluated before MED

C) Path2 because longer AS_PATH is better

D) Equal, load-balanced by default

****Q22.** An enterprise data center deploys EVPN-VXLAN fabric (RFC 8365). Leaf1 learns a MAC/IP behind it. How is this advertised?**

A) Via OSPF LSA Type 7

B) As EVPN Route Type 2 (MAC/IP advertisement) in MP-BGP EVPN address family, encapsulating L2 over VXLAN with VNI

C) Via ARP flood only

D) Via STP BPDU

****Q23.** You troubleshoot TCP throughput on a 10 Gbps link with 80ms RTT. Bandwidth-Delay Product is ~100 MB. Clients use 64KB default receive window with no window scaling. What symptom and fix?**

A) Link will be fully utilized; no fix

B) Throughput capped at ~6.5 Mbps per flow (64KB / RTT); enable TCP window scaling (RFC 1323) and increase buffers, enable jumbo MTU, tune BBR

C) Need to enable half-duplex

D) Reduce MTU to 500

****Q24.** In 2024-2026, DNS-over-HTTPS (DoH) and DNS-over-TLS (DoT) vs traditional DNS (port 53). Which statement is operationally accurate for enterprise inspection?**

A) DoH uses port 853, DoT uses port 443

B) DoH encapsulates DNS in HTTPS on port 443, making firewall-based DNS filtering and visibility harder without TLS inspection or endpoint policy; DoT uses port 853

C) DoH is unencrypted

D) DoT is deprecated by IETF

****Q25.** You deploy Anycast for a global DNS service. Same IP /24 is advertised from 20 PoPs globally via BGP. What happens?**

A) BGP will fail due to loop

B) Clients are routed to the topologically nearest PoP per BGP best-path (lowest AS_PATH, MED, IGP cost), providing low latency and DDoS absorption

C) All traffic goes to one PoP

D) Anycast requires NAT

****Q26.** A Kubernetes cluster uses Cilium eBPF with VXLAN overlay turned OFF (native routing) and kube-proxy replacement. Compared to iptables-based kube-proxy, what is the networking benefit?**

A) No difference

B) eBPF provides O(1) service load balancing via eBPF maps, reduced conntrack overhead, support for direct server return (DSR), and network policy enforcement without linear iptables traversal

C) Requires all pods to use hostNetwork

D) Only supports UDP

****Q27.** Analyze this Wireshark: [TCP Retransmission] storm, low window, duplicate ACKs, and high RTT variance after enabling new IDS inline. What is most likely cause and proper remediation?**

A) Normal behavior

B) IDS performing full TLS inspection causing bufferbloat, TCP segmentation offload mismatch and asymmetric routing; remediate by bypassing trusted flows, enabling TSO/GRO consistency, fixing MTU/MSS clamping for IPsec/VXLAN (e.g., MSS 1380), and ensuring symmetric path

C) Upgrade cable to Cat6

D) Disable TCP checksums

****Q28.** In MPLS L3VPN, CE1 pings CE2 across provider. Packet flow involves two labels. What are they?**

A) Both are transport labels

B) Inner label = VPN label (identifies VRF/BGP next-hop) allocated by egress PE via MP-BGP; Outer label = transport/LDP/RSVP-TE/SR label to reach egress PE

C) Inner = HTTP label

D) MPLS does not use labels

****Q29. You enable ECN (Explicit Congestion Notification) and DCTCP in a data center with RoCEv2. Why is this critical for 2024-2026 AI/ML fabrics with 400GbE?****

- A) To increase packet loss
- B) ECN marks packets with CE when queue builds (before drop) allowing sender to reduce rate earlier; DCTCP reacts proportionally to marking fraction, preventing PFC storms and tail latency for RDMA over Converged Ethernet
- C) To disable congestion control
- D) Only for Wi-Fi

****Q30. Zero Trust Network Access (ZTNA 2.0) vs traditional VPN concentrator model in 2025. What architectural shift matters most?****

- A) ZTNA assigns full /16 to client; VPN does not
- B) VPN provides broad network access (L3 tunnel); ZTNA provides identity-aware, per-application least-privilege access with continuous trust verification, device posture, and no listening port exposure (brokered via cloud edge, e.g., SDP/ZTNA)
- C) Both are identical
- D) ZTNA uses only IPv6

Advanced Answer Key & Explanations

****Q21 Answer: B) Path1 because shorter AS_PATH is evaluated before MED****

BGP selection order: Weight (Cisco), LocalPref, locally originated, AS_PATH length (shortest wins), Origin, MED (compared only if same neighboring AS), etc. AS_PATH is evaluated before MED.

****Q22 Answer: B) As EVPN Route Type 2 (MAC/IP advertisement) in MP-BGP EVPN address family, encapsulating L2 over VXLAN with VNI****

EVPN Type 2 carries MAC + IP, VNI, and next-hop VTEP IP. Type 3 is inclusive multicast for BUM, Type 5 for IP prefix. This decouples L2 learning from data-plane flooding.

****Q23 Answer: B) Throughput capped at ~6.5 Mbps per flow (64KB / RTT); enable TCP window scaling (RFC 1323) and increase buffers, enable jumbo MTU, tune BBR****

Max throughput = window / RTT. $65535 / 0.08 = \sim 819\text{KB/s} \sim 6.5\text{Mbps}$, far below 10 Gbps. Requires window scaling (up to 1GB), larger socket buffers, and modern CC.

****Q24 Answer: B) DoH encapsulates DNS in HTTPS on port 443, making firewall-based DNS filtering and visibility harder without TLS inspection or endpoint policy; DoT uses port 853****

Traditional port-53 inspection fails for DoH. Enterprises must enforce managed DNS via endpoint config, DNS filtering policies, or TLS interception/EDR to retain visibility.

****Q25 Answer: B) Clients are routed to the topologically nearest PoP per BGP best-path (lowest AS_PATH, MED, IGP cost), providing low latency and DDoS absorption****

Anycast leverages BGP's decision process to route to nearest instance. If one PoP fails, its route is withdrawn and traffic reconverges elsewhere.

****Q26 Answer: B) eBPF provides O(1) service load balancing via eBPF maps, reduced conntrack overhead, support for direct server return (DSR), and network policy enforcement without linear iptables traversal****

iptables scales O(n) rules. Cilium eBPF replaces kube-proxy with maps, supports magneta-based load balancing, socket LB, and eBPF host-routing that bypasses overlay overhead when possible.

****Q27 Answer: B) IDS performing full TLS inspection causing bufferbloat, TCP segmentation offload mismatch and asymmetric routing; remediate by bypassing trusted flows, enabling TSO/GRO consistency, fixing MTU/MSS clamping for IPsec/VXLAN (e.g., MSS 1380), and ensuring symmetric path****

Inline IDS/NGFW can cause latency spikes, MSS/MTU blackhole after encapsulation, and stateful drop on asymmetry. Fix with proper MSS clamping, path MTU discovery, symmetric routing, and hardware offload alignment.

****Q28 Answer: B) Inner label = VPN label (identifies VRF/BGP next-hop) allocated by egress PE via MP-BGP; Outer label = transport/LDP/RSVP-TE/SR label to reach egress PE****

Egress PE advertises VPN label via MP-BGP. Ingress PE pushes VPN + transport labels; P routers switch on outer only (penultimate hop pop may pop outer).

****Q29 Answer: B) ECN marks packets with CE when queue builds (before drop) allowing sender to reduce rate earlier; DCTCP reacts proportionally to marking fraction, preventing PFC storms and tail latency for RDMA over Converged**

Ethernet**

At 400G, packet loss/PFC pause causes head-of-line blocking. ECN+DCTCP provides early feedback, maintaining low queues and high throughput crucial for distributed training/collectives.

Q30 Answer: B) VPN provides broad network access (L3 tunnel); ZTNA provides identity-aware, per-application least-privilege access with continuous trust verification, device posture, and no listening port exposure (brokered via cloud edge, e.g., SDP/ZTNA)

ZTNA 2.0 implements never-trust, always-verify, micro-segmented app access vs network-wide VPN. It removes inbound exposure and continuously validates user/device/context per session.

Course File: 11_Bash.md

Course: Bash & Shell Scripting

1. Beginner Level (10 Questions)

****Q1.**** What does the shebang `#!/usr/bin/env bash` do when you execute `./script.sh`?

- A) It is a comment ignored by the shell
- B) It tells the kernel to locate bash via PATH and use it as the interpreter
- C) It sets the SHELL environment variable to bash
- D) It enables bash strict mode automatically

****Q2.**** What is the difference between `$*` and `$@` inside double quotes?

- A) Both are identical in all contexts
- B) `"$"` expands to a single word with positional params joined by first char of IFS, `"$@"` expands to separate words for each positional param
- C) `"$@"` joins with spaces, `"$"` splits into separate args
- D) `$*` includes `$0`, `$@` does not

****Q3.**** Given `file="my document.txt"`, which correctly handles spaces?

- A) `cat $file`
- B) `cat "$file"`
- C) `cat ${file}`
- D) `cat <$file>`

****Q4.**** What does `chmod +x script.sh` do?

- A) Adds execute permission for owner, group, others based on umask
- B) Sets SUID bit on the script
- C) Makes the file executable only for root
- D) Converts DOS line endings to Unix

****Q5.**** Which command will show the exit status of the last executed command?

- A) `echo $?`
- B) `echo $!`
- C) `echo $$`
- D) `echo $#`

****Q6.**** What does `2>&1` do in `ls /nonexistent >out.log 2>&1`?

- A) Redirects stdout to stderr
- B) Redirects stderr (fd 2) to the same destination as stdout (fd 1)
- C) Duplicates stdout to two files
- D) Suppresses both stdout and stderr

****Q7.**** How do you define a local variable inside a bash function that doesn't leak to global scope?

- A) `var=value`
- B) `local var=value` or `declare var=value` inside function
- C) `export var=value`
- D) `let var=value`

****Q8.**** What does `set -e` do?

- A) Enables echoing of commands before execution
- B) Causes the shell to exit immediately if a simple command fails (non-zero status), with exceptions for conditional contexts

- C) Disables globbing
 - D) Exports all variables automatically
- **Q9.**** Which is the correct way to test if file data.csv exists and is a regular file?
- A) [-d data.csv]
 - B) [-f data.csv]
 - C) [-x data.csv]
 - D) [-e data.csv] is enough to prove it's regular file
- **Q10.**** What does `#!/bin/bash` script do with `echo $(( 5 + 3 * 2 ))`?
- A) Prints 16
 - B) Prints 11
 - C) Prints 5 + 3 * 2
 - D) Syntax error

Beginner Answer Key & Explanations

- **Q1. Answer: B**** – The kernel reads the shebang line on `execve`. Using `/usr/bin/env` searches `PATH` for `bash`, making it portable across systems where `bash` isn't in `/bin`.
- **Q2. Answer: B**** – This is a classic distinction. `"$@"` preserves argument boundaries, critical for scripts handling filenames with spaces, while `"$"` concatenates into one string.
- **Q3. Answer: B**** – Double quoting prevents word splitting and globbing. Without quotes, `my document.txt` becomes two arguments to `cat`.
- **Q4. Answer: A**** – `+x` adds execute bits. Final permissions depend on `umask` and existing modes, and apply to `user/group/other` by default.
- **Q5. Answer: A**** – `?` is last exit code, `!` is PID of last background job, `$$` is current shell PID, `#` is number of positional args.
- **Q6. Answer: B**** – Order matters: `>out.log` first sets `fd1` to file, then `2>&1` makes `fd2` duplicate `fd1`'s current target, so both go to `out.log`.
- **Q7. Answer: B**** – `local` is `bash` keyword to scope variables to function. Without it, variables in functions are global by default in `bash`.
- **Q8. Answer: B**** – `set -e` / `set -o errexit` exits on failure. It doesn't trigger inside `if`, `while`, `until`, `&&||` lists, or when the failing command is part of a pipeline except last.
- **Q9. Answer: B**** – `-f` tests regular file existence. `-d` is directory, `-x` executable, `-e` exists but could be any type (`dir`, `symlink`, `fifo`).
- **Q10. Answer: B**** – `$(( ))` does arithmetic expansion with standard precedence. Multiplication precedes addition, so `3*2=6+5=11`.

2. Intermediate Level (10 Questions)

- **Q11.**** What is the output of: `x=5; echo ${x:-10}; echo ${x:=10}; echo $x` ?
- A) 5, 5, 5
 - B) 5, 10, 10
 - C) 5, 5, 10
 - D) 10, 10, 5
- **Q12.**** Consider `set -o pipefail`. What does `false | true; echo $?` output?
- A) 0 (without pipefail) and 0 (with pipefail)
 - B) 0 without pipefail, 1 with pipefail
 - C) 1 without pipefail, 1 with pipefail
 - D) 1 without pipefail, 0 with pipefail
- **Q13.**** What does `[[ $str =~ ^[0-9]{3}-[0-9]{4}$ ]]` do differently from `[ "$str" = pattern ]`?
- A) They are identical

- B) `[[ =~ ]]` does POSIX ERE regex matching and populates `BASH_REMATCH`; `[ = ]` does string equality/glob, not regex
- C) `[ ]` does regex, `[[ ]]` does glob
- D) `=~` only works with `#!/bin/sh`

****Q14.**** What is the purpose of `IFS= read -r line` when reading a file line by line?

- A) Disables backslash escape interpretation (`-r`) and prevents leading/trailing whitespace trimming by setting `IFS` empty
- B) Makes `read` faster
- C) Splits line by colon
- D) Reads binary files

****Q15.**** In bash, what does `trap 'rm -f /tmp/tmp.$$' EXIT` achieve?

- A) Deletes temp file when shell receives `SIGKILL`
- B) Registers a handler to remove temp file on shell exit (normal, or due to exit command/error with `set -e`, excluding `SIGKILL` which cannot be trapped)
- C) Deletes file immediately
- D) Creates file on exit

****Q16.**** What is the difference between `source script.sh` and `./script.sh`?

- A) No difference
- B) `source` runs script in current shell process (variables/functions persist), `./script.sh` spawns a subshell/new process
- C) `./script.sh` sources, `source` executes in subshell
- D) `source` only works for python scripts

****Q17.**** What will `echo "${array[@]}"` vs `"${array[*]}"` output when inside quotes for `array=(a "b c" d)`?

- A) Both identical
- B) `"${array[@]}"` -> 3 words: a, b c, d; `"${array[*]}"` -> 1 word: a b c d joined by first IFS char
- C) `"${array[@]}"` -> 1 word, `"${array[*]}"` -> 3 words
- D) Both produce 4 words

****Q18.**** What does `exec 3> output.log; echo "hi" >&3` do?

- A) Opens fd 3 for writing to `output.log`, then redirects `echo` to fd 3
- B) Syntax error
- C) Opens fd 3 for reading
- D) Redirects `stderr` to `output.log`

****Q19.**** Analyze: `find /var/log -type f -name "*.log" -mtime +7 -print0 | xargs -0 rm -f`. Why use `-print0` and `-0`?

- A) For faster execution
- B) To safely handle filenames containing spaces, newlines, or special characters by using NUL delimiter instead of newline/space
- C) Required for `rm -f`
- D) It compresses the list

****Q20.**** What does `set -u` (or `set -o nounset`) cause?

- A) Treats unset variables as error and exits/non-zero (except for special cases like `$*`, `$@`)
- B) Unsets all exported variables
- C) Makes all variables uppercase
- D) Enables debugging

Intermediate Answer Key & Explanations

****Q11. Answer: A**** – `${x:-10}` uses default without assignment, so `x` stays 5. `${x:=10}` would assign only if `x` unset/null, but here `x=5`, so still 5. Hence 5,5,5.

****Q12. Answer: B**** – Normally pipeline exit is last command (`true=0`). With `pipefail`, pipeline returns last non-zero, or 0 if all succeed, so it returns 1 from false.

****Q13.** Answer: B** – `[[ =~ ]]` is bash's regex operator using ERE. It sets `BASH_REMATCH` array for capture groups. `[` is POSIX test, no regex support.

****Q14.** Answer: A** – `-r` stops backslash escapes being removed. `IFS=` prevents word-splitting trimming. This is the robust idiom for `while IFS= read -r line; do ... loops`.

****Q15.** Answer: B** – `trap` on `EXIT` ensures cleanup runs on shell termination regardless of success/failure. `SIGKILL` (9) cannot be trapped or ignored.

****Q16.** Answer: B** – Sourcing executes in current shell environment, crucial for loading env vars, functions. Direct execution forks subprocess, isolating side effects.

****Q17.** Answer: B** – This mirrors the `$@` vs `$*` rule but for arrays. `[@]` preserves elements, `[*]` joins. Quoting is essential to preserve spaces in `"b c"`.

****Q18.** Answer: A** – `exec` without a command manipulates file descriptors of current shell. `3> file` opens fd 3 to file, `>&3` redirects stdout to fd 3.

****Q19.** Answer: B** – `NUL` is the only byte not allowed in Linux filenames. Using `NUL` delimiter avoids breakage on spaces/newlines, a production-critical pattern for safe batching.

****Q20.** Answer: A** – `set -u` helps catch typos: referencing unset variable errors out. You must handle optional vars with `${var-}` or `${var:-default}` idioms.

3. Advanced Level (10 Questions)

****Q21.**** What is the output and caveat of: `count=0; cat file.txt | while read -r line; do ((count++)); done; echo $count` ?

- A) Prints correct line count
- B) Prints 0 because while after pipe runs in subshell (in bash without `lastpipe`), so count modification is lost
- C) Syntax error
- D) Prints 1

****Q22.**** In bash 5.1+, what does `declare -n ref=$1; ref="new"` do inside a function?

- A) Creates a copy
- B) Creates a nameref (like pointer/reference) where assignment to `ref` actually modifies variable whose name is in `$1`, enabling pass-by-reference
- C) Creates a numeric variable
- D) Dereferences symlink

****Q23.**** Consider `set -E; trap 'echo "ERR at $LINENO"' ERR; myfunc() { false; }; myfunc`. When does `ERR` trap inherit?

- A) `ERR` trap never inherited
- B) With `set -E` (`errtrace`), `ERR` trap is inherited into functions, subshells, command substitutions, allowing global error reporting
- C) Only with `set -T`
- D) Automatically without `set -E`

****Q24.**** What is the behavior of `[[ -v myarray[key] ]]` vs `[[ -v myarray[@] ]]` in Bash 5.2?

- A) Both same
- B) First tests if specific associative array element with key 'key' is set (even if empty), second tests something else; `-v` checks variable existence without triggering `nounset` and works for array elements
- C) `-v` doesn't work for arrays
- D) They test values, not setness

****Q25.**** Analyze: `exec {fd}>>(logger -t myscrip); echo "log msg" >&${fd}`. What is happening?

- A) Uses dynamic fd allocation `{fd}` and process substitution `>(...)` to open a pipe to `logger` command and redirects logs to `syslog`
- B) Creates a file named `logger`
- C) Syntax error in `bash < 5`
- D) Closes `fd`

****Q26.**** What does `shopt -s extglob` enable, and what does `rm -f !(*.sh|*.md)` do?

- A) Enables extended globbing (ksh-style), and removes all files except those ending .sh or .md
- B) Enables case-insensitive globbing
- C) Disables globbing
- D) Expands ! to history

****Q27.**** How do you safely handle concurrent writes to a file from multiple instances of a bash script?

- A) Use >> file and hope atomic
- B) Use flock with file descriptor locking: (flock -x 200; echo \$\$ >> file;) 200>file.lock or flock /tmp/lock -c command
- C) Use chmod to lock
- D) Use sleep random

****Q28.**** What is the difference between process substitution <(cmd) and command substitution \$(cmd) regarding binary data and large output?

- A) No difference
- B) <(cmd) creates a named pipe or /dev/fd file providing streaming output to another command without waiting for full output or subshell variable capture; \$(cmd) captures all stdout into memory as string, stripping trailing newlines and unsuitable for binary/null bytes
- C) \$(cmd) streams, <() captures
- D) Both strip binary

****Q29.**** Given: getopts ":f:v" opt; case \$opt in ... What does leading : in optstring do and how to detect invalid option?

- A) Leading colon enables silent error mode: invalid options set opt to ? normally but with :, sets to ? and OPTARG contains invalid char? Actually leading : causes getopts to set opt to ? for invalid? No: with :, invalid option sets opt to ?? Correction: with leading :, invalid option sets opt to ?? Let's recall: Without :, error printed. With leading :, errors suppressed: invalid option sets opt to ?? Actually with :, invalid sets opt to ? and OPTARG to option char? Wait: POSIX: with leading :, invalid option sets opt to ?? Actually: invalid option -> ? without :, ? with OPTARG? With : -> ? becomes ?? Let's be precise: with : optstring, invalid option sets opt to ? and OPTARG holds option char, missing arg sets opt to ..
- B) Leading : suppresses getopts default error messages; invalid option sets opt to ? with OPTARG being invalid char, missing argument sets opt to : with OPTARG being option requiring arg, allowing custom handling
- C) Leading : makes all options require args
- D) It makes getopts case-insensitive

****Q30.**** Write: How would you implement a bash function retry that retries a command up to 3 times with exponential backoff (1s,2s,4s) and returns last exit code if all fail?

- A) retry() { for i in 1 2 3; do \$@ && return 0; sleep \$i; done; return 1; }
- B) retry() { local tries=0 max=3 delay=1; local ec=0; while ((tries < max)); do "\$@" && return 0; ec=\$?; ((tries++)); ((tries < max)) && sleep \$delay && delay=\$((delay*2)); done; return \$ec; } with proper "\$@" handling and exponential backoff
- C) retry() { until \$@; do sleep 1; done } infinite loop
- D) retry() { eval \$@ ; }

Advanced Answer Key & Explanations

****Q21. Answer: B**** – In bash, each pipe segment runs in subshell. Modifications to count inside while are in child process. Fix via shopt -s lastpipe (only in non-interactive without job control) or process substitution: while ... done < <(cat file).

****Q22. Answer: B**** – declare -n is nameref introduced bash 4.3. It allows true pass-by-reference without eval. Must be careful about circular references. Bash 5.0+ improves resolution.

****Q23. Answer: B**** – set -E enables errtrace, causing ERR traps to be inherited. This is critical for library error handling, otherwise function failures wouldn't trigger global ERR trap. set -T is similar for DEBUG/RETURN traps.

****Q24. Answer: B**** – [[-v var]] (bash 4.2+) tests if variable/array element is set, even empty string. This avoids unbound variable errors under set -u unlike [-z \${var+x}] tricks, and supports associative array key checks safely.

****Q25. Answer: A**** – This advanced pattern combines automatic fd allocation via {fd}> (bash 4.1+) which assigns free fd >10 to var fd, and process substitution feeding to logger. It enables logging without blocking and allows later exec {fd}>&- to close.

****Q26. Answer: A**** – extglob enables !(pattern), @(pattern), *(pattern), +(pattern), ?(pattern). !(*.sh|*.md) negates match: all files NOT matching .sh or .md. Essential for safe cleanup scripts.

****Q27. Answer: B**** – flock uses kernel advisory locks. Using fd-based locking (flock -n 200) inside subshell is modern robust pattern for critical sections. Append alone is atomic for small writes < PIPE_BUF but not for complex operations.

****Q28. Answer: B**** – Process substitution is streaming (FIFO /dev/fd/63) suitable for large data, avoids command substitution's memory capture and NUL truncation issues. Use <() when feeding command expecting file argument, e.g., diff <(sort f1) <(sort f2).

****Q29. Answer: B**** – The silent mode is best practice for CLI tools. Without it, getopt prints to stderr "illegal option". With :, you control error messages and can distinguish missing arg (:) vs invalid option (?) via case handling.

****Q30. Answer: B**** – Option A doesn't double delay (uses linear) and doesn't preserve "\$@" quoting correctly for some cases? Actually \$@ vs "\$@" matters. Option B correctly uses "\$@" to preserve args, tracks exit code, implements exponential backoff delay*=2, and returns last error. Option C is infinite retry without backoff logic.

Course File: 12_PowerShell.md

Course: PowerShell

1. Beginner Level (10 Questions)

****Q1:**** Which command correctly retrieves help for the Get-Process cmdlet including examples?

- A) Get-Help Get-Process -Full
- B) Get-Help Get-Process -Examples
- C) Help Get-Process -samples
- D) Show-Help Get-Process

****Q2:**** In PowerShell 7, what is the default execution policy on Windows client?

- A) Unrestricted
- B) AllSigned
- C) Restricted
- D) RemoteSigned

****Q3:**** What does the automatic variable \$_ represent inside a ForEach-Object script block?

- A) The total number of objects in the pipeline
- B) The current object being processed
- C) The previous pipeline object
- D) The path of the current script

****Q4:**** Which operator is used for case-insensitive comparison to check if a string contains a substring?

- A) -contains
- B) -like
- C) -match
- D) -in

****Q5:**** How do you create a new variable that is only visible within the current function scope?

- A) \$global:myVar = 10
- B) \$local:myVar = 10
- C) \$myVar = 10 (inside function, default is local)
- D) Set-Variable -Name myVar -Scope Private

****Q6:**** Which cmdlet is used to export objects to a CSV file without type information header in PowerShell 7.4+?

- A) Export-Csv -Path data.csv -NoTypeInfoInformation is still required but default changed
- B) Export-Csv -Path data.csv (NoTypeInfoInformation is now default in PS 7.4+)
- C) ConvertTo-Csv -NoTypeInfoInformation | Out-File data.csv
- D) Out-Csv -Path data.csv

****Q7:**** What does Get-ExecutionPolicy -List show?

- A) Only the current user policy
- B) Execution policies for all scopes: MachinePolicy, UserPolicy, Process, CurrentUser, LocalMachine
- C) All signed scripts on the system
- D) The history of execution policy changes

****Q8:**** Which statement about PowerShell aliases is TRUE?

- A) Aliases are persistent across sessions automatically
- B) gal is an alias for Get-Alias and gci for Get-ChildItem
- C) Aliases cannot be used in scripts

- D) Is is not a valid alias in PowerShell

****Q9:**** How do you stop a running service named Spooler using PowerShell?

- A) Stop-Service -Name Spooler
- B) Kill-Service Spooler
- C) Set-Service -Name Spooler -Status Stopped
- D) Get-Service Spooler | Stop

****Q10:**** What is the purpose of \$null on the left side like \$null -eq \$myVar?

- A) It is a syntax error
- B) It prevents accidental assignment and handles cases where \$myVar is an array, avoiding unwanted enumeration
- C) It makes the comparison case-sensitive
- D) It improves performance by 10x

Beginner Answer Key & Explanations

****Q1 Answer: B) Get-Help Get-Process -Examples****

The -Examples parameter shows only practical examples. -Full shows detailed info including parameters and examples, but -Examples is the direct way to request examples.

****Q2 Answer: C) Restricted****

On Windows client, PowerShell's default execution policy is Restricted, blocking script execution. On Windows Server it's RemoteSigned by default, and on non-Windows platforms it's Unrestricted.

****Q3 Answer: B) The current object being processed****

\$_ and its alias \$PSItem represent the current pipeline object inside script blocks for ForEach-Object and Where-Object. This is fundamental to pipeline processing in PowerShell.

****Q4 Answer: B) -like****

While -match uses regex, -like uses wildcard patterns and is case-insensitive by default. For substring check with wildcards you use "hello world" -like "*world*", whereas -contains checks collection membership.

****Q5 Answer: C) \$myVar = 10 (inside function, default is local)****

Variables assigned inside a function default to local scope automatically. While \$local: is explicit, simply assigning \$myVar inside function creates a local copy without affecting parent scope.

****Q6 Answer: B) Export-Csv -Path data.csv (NoTypeInfoInformation is now default in PS 7.4+)****

Starting with PowerShell 6.0, -NoTypeInfoInformation became default, so the #TYPE header is no longer emitted by default. In 7.4+ this behavior remains, making explicit -NoTypeInfoInformation redundant but still accepted for compatibility.

****Q7 Answer: B) Execution policies for all scopes: MachinePolicy, UserPolicy, Process, CurrentUser, LocalMachine****

Get-ExecutionPolicy -List enumerates all policy scopes in precedence order. This helps diagnose which Group Policy or local setting is actually determining effective execution policy.

****Q8 Answer: B) gal is an alias for Get-Alias and gci for Get-ChildItem****

PowerShell includes many familiar aliases for discoverability: gal -> Get-Alias, gci -> Get-ChildItem, ls -> Get-ChildItem, dir -> Get-ChildItem. They are not persistent unless exported via profile.

****Q9 Answer: A) Stop-Service -Name Spooler****

Stop-Service is the correct cmdlet to stop a service. Option D is invalid syntax; the pipeline form would be Get-Service Spooler | Stop-Service. Stop verb is approved and discoverable via Get-Verb.

****Q10 Answer: B) It prevents accidental assignment and handles cases where \$myVar is an array, avoiding unwanted enumeration****

Putting \$null on left avoids \$myVar = \$null typo bugs and ensures correct boolean logic when \$myVar is an array. If array is on left side, comparison enumerates and may return an array of true/false instead of single boolean.

2. Intermediate Level (10 Questions)

****Q11:**** Given: \$a = @(1,2,3); \$b = \$a; \$b += 4. What is \$a.Count and \$b.Count after execution?

- A) a=3, b=4
- B) a=4, b=4 (reference type)
- C) a=3, b=3
- D) a=4, b=3

****Q12:**** What is the difference between -eq and -ceq in PowerShell?

- A) -eq is case-sensitive, -ceq is case-insensitive
- B) -eq is case-insensitive by default, -ceq is case-sensitive
- C) -ceq compares objects by reference
- D) -eq only works for numbers

****Q13:**** Consider this advanced function snippet:

powershell

```
function Get-Data {
```

```
[CmdletBinding()]
```

```
param(
```

```
[Parameter(ValueFromPipeline=$true, Mandatory=$true)]
```

```
[string]$Name
```

```
process { Write-Output "Processing $Name" }
```

If called as Get-Data -Name @"a","b") without pipeline, how many times does process block execute?

- A) 2 times
- B) 1 time (array passed as single parameter, process runs once, but \$_ behavior differs)
- C) 0 times because Mandatory fails
- D) 3 times (begin+process+end)

****Q14:**** Which splatting syntax is correct for passing hashtable \$params to Get-ChildItem?

- A) Get-ChildItem \$params
- B) Get-ChildItem @params
- C) Get-ChildItem -params @params
- D) Get-ChildItem --@params

****Q15:**** How does PowerShell remoting over SSH differ from WinRM in PowerShell 7?

- A) SSH remoting requires WinRM running
- B) SSH remoting uses New-PSSession -HostName and -SSHTransport, works cross-platform without WinRM, ideal for Linux/macOS
- C) SSH remoting is only for file copy
- D) WinRM is faster for Linux

****Q16:**** What will this pipeline output?

powershell

```
1..5 | Where-Object { $_ -gt 2 } | Measure-Object -Sum | Select-Object -ExpandProperty Sum
```

- A) 12 (3+4+5)
- B) 9
- C) Object with Sum property
- D) Error because Where-Object cannot be piped to Measure-Object

****Q17:**** Which cmdlet should you use to securely store a credential for unattended scripts in 2024-2026 best practice?

- A) Store plain text password in script: \$pass = "P@ssw0rd"
- B) Export-Clixml with PSCredential object encrypted via DPAPI (current user only) or use SecretManagement module vault
- C) ConvertTo-SecureString -AsPlainText -Force hardcoded in script
- D) Get-Credential | Export-Csv

****Q18:**** What is the correct way to handle terminating and non-terminating errors to stop execution?

- A) Get-Item NoFile -ErrorAction Stop wrapped in try/catch
- B) Get-Item NoFile -ErrorAction Continue; catch { }
- C) \$ErrorActionPreference = "SilentlyContinue"; try { }
- D) trap { continue }

****Q19:**** What does \$PSScriptRoot contain when used inside a module's .psm1 file?

- A) The current working directory
- B) The directory where the .psm1 module file resides
- C) The PowerShell installation folder
- D) Always empty

****Q20:**** How do you create a custom PowerShell object with properties Name and Age in PowerShell 7?

- A) New-Object PSObject -Property @{Name='John';Age=30}
- B) [PSCustomObject]@{Name='John';Age=30}
- C) Both A and B work, but B preserves property order and is faster
- D) Create-Object -Type PSCustom

Intermediate Answer Key & Explanations

****Q11 Answer:** A) a=3, b=4**

Arrays with += create a new array instance; \$b is reassigned to new array, \$a still references original 3-element array. For Arrays, assignment copies reference but += operator clones plus adds, breaking the reference link unlike List<T> .Add() behavior.

****Q12 Answer:** B) -eq is case-insensitive by default, -ceq is case-sensitive**

PowerShell comparison operators are case-insensitive by default (-eq, -like, -match). Prefixing with 'c' enforces case-sensitivity (-ceq, -clike), and 'i' explicitly enforces case-insensitivity (-ieq), which is default behavior.

****Q13 Answer:** B) 1 time (array passed as single parameter, process runs once, but \$_ behavior differs)**

When array is passed as parameter argument (not via pipeline), process block runs once with \$Name bound to entire array, and PowerShell coerces array to string as "a b" if used as string. Pipeline input @"a","b" | Get-Data would invoke process twice, highlighting parameter binding vs pipeline binding difference.

****Q14 Answer:** B) Get-ChildItem @params**

Splatting uses @ instead of \$ to expand hashtable or array into parameters: \$params = @{Path='C:\'; Recurse=\$true}; Get-ChildItem @params. Using \$ would pass the hashtable as single positional argument, causing binding error.

****Q15 Answer:** B) SSH remoting uses New-PSSession -HostName and -SSHTransport, works cross-platform without WinRM, ideal for Linux/macOS**

PowerShell 7 introduced SSH transport for remoting via -HostName, -UserName, -SSHTransport without WinRM dependencies. This enables cross-platform remoting to Linux/macOS where OpenSSH is configured, using Enter-PSSession -HostName.

****Q16 Answer:** A) 12 (3+4+5)**

Where-Object filters to 3,4,5, Measure-Object calculates Sum=12, and Select-Object -ExpandProperty returns raw value. This demonstrates pipeline composition: filtering -> measuring -> expanding property to get scalar.

****Q17 Answer:** B) Export-Clixml with PSCredential object encrypted via DPAPI (current user only) or use SecretManagement module vault**

Best practice for 2024-2026 is Microsoft.PowerShell.SecretManagement module with SecretStore or Azure KeyVault, or DPAPI-encrypted CliXml only for current user/machine. Hardcoding passwords violates credential hygiene and security baselines like ASD, CIS.

****Q18 Answer:** A) Get-Item NoFile -ErrorAction Stop wrapped in try/catch**

Non-terminating errors don't trigger catch by default; -ErrorAction Stop converts them to terminating errors capturable by try/catch. Setting \$ErrorActionPreference to Stop globally achieves same but less targeted; try/catch only catches terminating errors.

****Q19 Answer:** B) The directory where the .psm1 module file resides**

\$PSScriptRoot is automatic variable containing the directory of the currently executing script or module file. This is crucial for relative path resolution inside modules, unlike Get-Location which returns working directory that may differ.

****Q20 Answer: C) Both A and B work, but B preserves property order and is faster****

[PSCustomObject] type accelerator with hashtable literal is idiomatic PS3+ syntax, preserves insertion order and is highly optimized. New-Object works but doesn't guarantee property order on older PS and is slower due to cmdlet binding overhead.

3. Advanced Level (10 Questions)

****Q21:**** In a PowerShell class definition, why does this fail: `class MyClass { [string]$path = $PSScriptRoot }` and what is the fix for PowerShell 7.4?

- A) Classes cannot have properties; use function instead
- B) \$PSScriptRoot not accessible at property initialization time; initialize in constructor: `MyClass(){ $this.path = $PSScriptRoot }` or use static initialization workaround
- C) Need to use `$global:PSScriptRoot`
- D) String type cannot hold path

****Q22:**** What is the execution order of Begin/Process/End blocks with pipeline input of 3 objects in an advanced function?

- A) Begin(1x) -> Process(3x) -> End(1x)
- B) Begin(3x) -> Process(3x) -> End(3x)
- C) Process(3x) only
- D) Begin(1x) -> Process(1x) -> End(1x) receiving all 3 as array

****Q23:**** Which statement about PowerShell runspaces vs jobs is correct for high-throughput parallelism in 2024-2026?

- A) Start-Job creates a new process and is fastest for 1000+ tasks
- B) ForEach-Object -Parallel -ThrottleLimit uses runspaces with isolated runspace per iteration, far more memory-efficient than Start-Job; ThreadJob module is lighter than PSJob
- C) Runspaces cannot share variables
- D) Jobs and runspaces are identical

****Q24:**** How do you correctly implement a DSC resource with PowerShell 7 vs legacy MOF approach in modern 2024 pattern?

- A) DSC is removed in PS7, cannot use
- B) Use class-based DSC resource with `[DscResource()]` attribute and Tests/Set/Get methods, or prefer DSC v3 / PSDSCResources / Guest Configuration; MOF compiled via Configuration keyword still supported in Windows PowerShell 5.1 but not actively developed for PS7.2+
- C) Only script resources allowed
- D) DSC only works via Azure

****Q25:**** What does this code produce regarding closure?

powershell

```
$funcs = @()
```

```
for ($i=0; $i -lt 3; $i++) {
```

```
    $funcs += { $i }.GetNewClosure()
```

```
    $funcs[0].Invoke(); $funcs[1].Invoke()
```

- A) 3 and 3 (no closure capture)
- B) 0 and 1 (with GetNewClosure, captures current iteration value due to closure snapshot)
- C) Error
- D) \$null and \$null

****Q26:**** Consider performance: Which is fastest to filter 1 million lines from a 100MB log file for "ERROR" in PowerShell 7.4?

- A) `Get-Content log.txt | Where-Object { $_ -match "ERROR" }`
- B) `Select-String -Path log.txt -Pattern "ERROR" -SimpleMatch`
- C) `[System.IO.File]::ReadLines("log.txt") | Where-Object { $_.Contains("ERROR") }` with .NET method

- D) `Get-Content log.txt -ReadCount 1000 | ForEach-Object { $_ -match "ERROR" }`

Explanation nuance: B and C are fastest, but B uses optimized file reading; A is slowest due to object pipeline overhead

****Q27:**** What is the correct way to handle PowerShell pipeline to avoid memory exhaustion with large datasets using `System.IO.Pipeline` or streaming pattern?

- A) Always collect via `$data = Get-HugeData` then process

- B) Use streaming: implement function that uses `process {}` block and emits one-by-one, or use `IAsyncEnumerable` pattern via `Write-Output -NoEnumerate`, and avoid `+=` array building; use `List<T>` for accumulation

- C) Increase RAM only

- D) Use Out-Null to free memory

****Q28:**** In PowerShell 7.4, what is the behavior of `??` and `??=` operators?

- A) `??` is null-coalescing operator, `??=` assigns if left is null; e.g. `$x ?? "default"` and `$x ??= "default"`

- B) They are regex operators

- C) They compare types

- D) They don't exist in PS7

****Q29:**** How do you properly dispose and handle async .NET Tasks in PowerShell 7.4+ to avoid deadlocks?

- A) `$task.Result` synchronously always safe

- B) Use `Await` keyword like C#: `await $task`

- C) Use `$task.GetAwaiter().GetResult()` may still deadlock in STA; preferred pattern is `$task | Wait-Task -?` actually: use `while (-not $task.AsyncWaitHandle.WaitOne(200)) {}` or leverage `ForEach-Object -Parallel` or `Task.Wait()` with timeout, or use `Start-ThreadJob` with `Receive-Job -Wait -AutoRemoveJob`; PowerShell 7.4 adds Task support via `.GetAwaiter().GetResult()` but safe pattern is `[System.Threading.Tasks.Task]::WaitAll()`

- D) Never use .NET tasks in PowerShell

****Q30:**** Examine this advanced parameter set definition:

powershell

```
function Test-Param {
```

```
[CmdletBinding(DefaultParameterSetName='ByName')]
```

```
param(
```

```
[Parameter(Mandatory, ParameterSetName='ByName', Position=0)]
```

```
[string]$Name,
```

```
[Parameter(Mandatory, ParameterSetName='ById', Position=0)]
```

```
[int]$Id,
```

```
[Parameter(ParameterSetName='ByName')]
```

```
[Parameter(ParameterSetName='ById')]
```

```
[switch]$Force
```

Which call is INVALID?

- A) `Test-Param -Name "Foo" -Force`

- B) `Test-Param -Id 5`

- C) `Test-Param -Name "Foo" -Id 5`

- D) `Test-Param "Foo"`

Advanced Answer Key & Explanations

****Q21 Answer:** B) `$PSScriptRoot` not accessible at property initialization time; initialize in constructor: `MyClass(){ $this.path = $PSScriptRoot }` or use static initialization workaround**

PowerShell classes execute property initializers in a context where automatic variables like `$PSScriptRoot` are not available. The fix is assignment inside constructor where runspace context is fully initialized.

****Q22 Answer:** A) `Begin(1x) -> Process(3x) -> End(1x)**`

Begin executes once at start, Process executes once per pipeline object (3 times), End executes once at completion. This is fundamental to writing pipeline-aware advanced functions that accumulate state in Begin/End and handle per-item logic in Process.

****Q23 Answer: B) ForEach-Object -Parallel -ThrottleLimit uses runspaces with isolated runspace per iteration, far more memory-efficient than Start-Job; ThreadJob module is lighter than PSJob****

Start-Job spawns new process (heavy), ThreadJob uses threads (lighter), and ForEach-Object -Parallel (PS7+) uses runspace pool with ThrottleLimit default 5, offering best balance. Runspaces can share via \$using: scope modifier to capture outside variables.

****Q24 Answer: B) Use class-based DSC resource with [DscResource()] attribute and Tests/Set/Get methods, or prefer DSC v3 / PSDSCResources / Guest Configuration; MOF compiled via Configuration keyword still supported in Windows PowerShell 5.1 but not actively developed for PS7.2+****

Microsoft moved DSC focus to DSC v3 (separate DSC executable, YAML/JSON) and class-based resources. Legacy Configuration compilation is Windows PowerShell 5.1 only; PS 7.1+ removed DSC engine but module PSDscResources bridges functionality. Modern practice uses Guest Configuration or DSCv3.

****Q25 Answer: B) 0 and 1 (with GetNewClosure, captures current iteration value due to closure snapshot)****

Without GetNewClosure, all scriptblocks share same \$i variable reference returning final value 3. GetNewClosure() creates dynamic module capturing current lexical values, preserving iteration state, which is critical for deferred execution in loops.

****Q26 Answer: B) Select-String -Path log.txt -Pattern "ERROR" -SimpleMatch****

Select-String uses fast internal file streaming and avoids assigning each line to PowerShell object with extra properties. Option C with [System.IO.File]::ReadLines and .Contains() is also very fast; Get-Content | Where-Object is slowest due to heavy pipeline object wrapping and overhead of regex matching per line.

****Q27 Answer: B) Use streaming: implement function that uses process {} block and emits one-by-one, or use IAsyncEnumerable pattern via Write-Output -NoEnumerate, and avoid += array building; use List<T> for accumulation****

Best practice is streaming architecture: produce/consume one object at a time to keep memory constant $O(1)$. Array concatenation with += is $O(N^2)$ because it copies entire array each iteration; use [System.Collections.Generic.List[string]] and .Add() for efficient accumulation.

****Q28 Answer: A) ?? is null-coalescing operator, ??= assigns if left is null; e.g. \$x ?? "default" and \$x ??= "default"****

Introduced in PowerShell 7.0, null-coalescing operators simplify null handling: \$value = \$maybeNull ?? "fallback". The assignment variant ??= only assigns if left operand is \$null, optimizing default initialization pattern without overwriting existing values.

****Q29 Answer: C) Use \$task.GetAwaiter().GetResult() may still deadlock in STA; preferred pattern is \$task | Wait-Task -? actually: use while (-not \$task.AsyncWaitHandle.WaitOne(200)) {} or leverage ForEach-Object -Parallel or Task.Wait() with timeout, or use Start-ThreadJob with Receive-Job -Wait -AutoRemoveJob; PowerShell 7.4 adds Task support via .GetAwaiter().GetResult() but safe pattern is [System.Threading.Tasks.Task]::WaitAll()****

PowerShell doesn't have native await keyword like C#. Calling .Result can block thread. Safest approach is using GetAwaiter().GetResult() which unwraps exceptions, but avoid in STA mode or pipelines; wrap in ThreadJob or use WaitAll with timeout and handle AggregateException. PowerShell 7.4 improved Task handling but deadlock risk remains.

****Q30 Answer: C) Test-Param -Name "Foo" -Id 5****

Parameter sets are mutually exclusive; -Name belongs to ByName set, -Id to ById set, cannot be combined. Parameter binder throws "Parameter set cannot be resolved" when arguments match multiple sets ambiguously. -Force belongs to both sets, so it's allowed with either.

Course File: 13_VMware.md

VMware & Virtualisation - Comprehensive Exam

Total Questions: 30 | Beginner: 10, Intermediate: 10, Advanced: 10

Valid for: vSphere 8.0 U2/U3, VCF 5.1/5.2, vSAN 8.0 ESA/OSA, NSX, Broadcom Era 2024-2026

Passing Score: 70%

Section 1: Beginner (Questions 1-10)

Q1. What is the primary role of the ESXi hypervisor in a VMware environment?

- A) To provide network routing between VMs
- B) To install guest operating systems automatically
- C) To abstract physical hardware and run virtual machines directly on it
- D) To manage Active Directory authentication

Answer: C

Explanation: ESXi is a Type-1 bare-metal hypervisor that installs directly on physical hardware. It abstracts CPU, memory, storage, and networking to create and run multiple isolated VMs.

Q2. What does vCenter Server primarily provide?

- A) Physical switch management
- B) Centralized management for multiple ESXi hosts and VMs
- C) Backup for VMs only
- D) Guest OS patching

Answer: B

Explanation: vCenter Server is the centralized management plane for vSphere, managing clusters, hosts, VMs, and features like vMotion and DRS. Since vSphere 7.0+, it is only available as the vCenter Server Appliance (VCSA).

Q3. Which feature allows a running VM to be moved live from one ESXi host to another with zero downtime?

- A) Storage vMotion
- B) vMotion
- C) vSphere HA
- D) Fault Tolerance

Answer: B

Explanation: vMotion performs live migration of a running VM's memory and execution state between hosts. Storage vMotion moves only the VM's disk files, while vMotion moves compute without downtime if requirements are met.

Q4. In the Broadcom era (2024-2026), how is VMware vSphere primarily licensed?

- A) Perpetual per-CPU with no support
- B) Free open-source license
- C) Subscription-based per-core bundles: VCF and VMware vSphere Foundation (VVF)
- D) Per-VM monthly license only

Answer: C

****Explanation:**** After Broadcom's acquisition, perpetual licenses were discontinued in favor of subscription per-core bundles. The two main offerings are VMware Cloud Foundation (VCF) and VMware vSphere Foundation (VVF), with a 16-core minimum per CPU (72-core minimum per order in 2025 updates).

Q5. What is a datastore in vSphere?

- A) A database storing vCenter credentials
- B) A logical container that holds VM files, ISOs, and templates on storage
- C) A physical hard disk in the server
- D) A network share for backups only

****Answer: B****

****Explanation:**** A datastore is a storage abstraction layer presented to ESXi hosts from VMFS, NFS, or vSAN. It stores VM configuration, VMDKs, snapshots, ISOs, and templates as files.

Q6. What is the purpose of VMware Tools / open-vm-tools?

- A) To provide antivirus in the VM
- B) To improve VM performance, enable graceful shutdown, and provide drivers and ballooning
- C) To convert VMs to physical servers
- D) To license the guest OS

****Answer: B****

****Explanation:**** VMware Tools provides paravirtualized drivers (VMXNET3, PVSCSI), memory balloon driver, and host-guest communication. It enables accurate time sync, graceful OS shutdown/reboot, and improved performance metrics.

Q7. What is the difference between vSAN OSA and vSAN ESA in vSphere 8?

- A) OSA is faster always
- B) ESA (Express Storage Architecture) is the new architecture optimized for NVMe/TLC with single-tier, log-structured file system; OSA is Original Storage Architecture with cache+capacity tiers
- C) ESA only works with HDDs
- D) There is no difference

****Answer: B****

****Explanation:**** vSAN 8 introduced ESA as an optional architecture requiring all-NVMe and redesigned for high performance and efficiency. OSA retains the traditional two-tier disk group model with caching tier and is broader in hardware compatibility.

Q8. What does DRS (Distributed Resource Scheduler) do?

- A) Powers off all VMs at night
- B) Automatically load-balances VMs across hosts in a cluster based on resource usage and rules
- C) Backs up VMs daily
- D) Creates snapshots automatically

****Answer: B****

****Explanation:**** DRS monitors CPU/memory pressure and uses vMotion to balance workloads. Since vSphere 7.0+, it uses a workload-centric algorithm and integrates with vSphere Lifecycle Manager and DPM.

Q9. Which virtual switch type is standard in vSphere and managed per-host?

- A) vSphere Distributed Switch (VDS)
- B) vSphere Standard Switch (VSS)

C) NSX Logical Switch

D) Cisco Nexus 1000V

****Answer: B****

****Explanation:**** The Standard Switch (VSS) is configured individually on each ESXi host. The Distributed Switch (VDS) is centralized via vCenter, provides advanced features like NIOC, port mirroring, and is required for VCF.

Q10. What is a VM snapshot?

A) A full backup copy stored offsite

B) A point-in-time delta file preserving VM state and disk, allowing rollback

C) A clone that uses no extra space

D) A memory dump only

****Answer: B****

****Explanation:**** A snapshot creates delta VMDKs and optionally memory state, saving the VM's point-in-time. It is not a full backup and can degrade performance if left long-term; VMware recommends <72 hours.

Section 2: Intermediate (Questions 11-20)

Q11. In vSphere 8.0 U3, what changed for vMotion to improve live migration of VMs with large memory pages or AI/ML workloads?

A) vMotion was removed

B) Improved vMotion with faster page tracing and up to 6x faster for large VMs using Data Plane Development Kit (DPDK) improvements and vMotion stun time reduction

C) vMotion now requires VM power-off

D) Only cold migration is supported

****Answer: B****

****Explanation:**** vSphere 8.0 U3 introduced significant vMotion performance improvements for monster VMs (up to 24Tbps logical bandwidth). It reduces stun time via adaptive page tracing, critical for vGPU and large-memory SAP/HANA and AI workloads.

Q12. You have a VCF 5.2 deployment. How much vSAN capacity entitlement is included per VCF core license?

A) 0 TiB

B) 0.25 TiB per core (VVF level)

C) 1 TiB per core

D) Unlimited

****Answer: C****

****Explanation:**** VCF 5.2 entitles 1 TiB of vSAN storage per core license (OSA or ESA or vSAN Max when used as principal storage). VVF entitles 0.25 TiB per core; excess requires separate vSAN capacity add-on subscription.

Q13. What is vSphere Lifecycle Manager (vLCM) and its key advantage over Update Manager?

A) It only updates VMware Tools

B) vLCM provides desired-state image-based lifecycle for hosts, clusters, firmware, drivers combined; supports vSAN ESA and single image for entire cluster

C) It deletes VMs automatically

D) Same as Update Manager with different name

****Answer: B****

****Explanation:**** vLCM replaces legacy baselines with declarative cluster images (ESXi version, vendor add-on, firmware). In vSphere 8, it can remediate entire clusters with parallel remediation and integrates with vSAN ESA for non-disruptive updates.

Q14. An admin wants to enforce VM-to-Host affinity and anti-affinity for licensing or performance. Which feature should be used?

- A) VM Overrides inside DRS rules
- B) Storage DRS only
- C) HA admission control
- D) vMotion without rules

****Answer: A****

****Explanation:**** DRS VM-Host rules (Must/Should run on, Should Not) and VM-VM anti-affinity rules control placement. In 2024+, these are managed via the vSphere Client > Cluster > Configure > VM/Host rules and are respected during DRS balancing and maintenance mode.

Q15. What is vSphere Trust Authority and VM Encryption in vSphere 8?

- A) Encrypts only network traffic
- B) A key provider architecture where remote attestation clusters attest trusted hosts; enables VM encryption, vMotion encryption, and vSAN encryption without requiring TPM on all workload hosts
- C) Disables Secure Boot
- D) Only for vCenter Server

****Answer: B****

****Explanation:**** vTrust Authority separates key management from attestation using a dedicated trusted cluster. This enables encrypted vMotion by default in vSphere 8, TPM 2.0 attestation, and KMS-based VM encryption for compliance.

Q16. You need to configure stretched cluster for vSAN ESA in VCF 5.2. What is newly supported in vSAN 8.0 U3?

- A) Stretched clusters are not supported on ESA
- B) Stretched cluster support on ESA is now GA in VCF 5.2 with witness and up to 5ms RTT between sites
- C) Only 2 nodes supported
- D) Requires OSA only

****Answer: B****

****Explanation:**** vSAN 8.0 U3 added full stretched cluster support for ESA, previously OSA-only. In VCF 5.2, ESA stretched clusters support site failure tolerance with witness appliance and flexible topologies, enabling active-active data centers.

Q17. What is the purpose of vSphere Distributed Services Engine and DPUs (Data Processing Units) support added in vSphere 8?

- A) To run games faster
- B) Offload NSX, vSphere networking, and security services (NSX, vDefend) to DPU, freeing CPU cores and improving performance for ESXi host
- C) To disable NSX
- D) Only for storage

****Answer: B****

****Explanation:**** vSphere 8 U2+ supports DPUs (NVIDIA BlueField, AMD Pensando) via vSphere Distributed Services Engine. Infrastructure services offload from x86 CPUs, allowing workload consolidation and zero-trust host isolation.

Q18. How does vSphere HA admission control work with the new vSAN ESA clusters large scale?

- A) It is disabled
- B) It reserves resources to ensure N+1 or specific failure tolerance (e.g., 1 host, % cluster resources, or dedicated failover hosts) and calculates usable capacity excluding vSAN overhead
- C) It powers off random hosts
- D) It only uses slot policy since 2008

****Answer: B****

****Explanation:**** HA admission control in 8.0 supports Cluster Resource Percentage, Failover Hosts, and Slot Policy (legacy). For vSAN ESA/hyperconverged, it must consider storage policy overhead and host rebuild reserve; vLCM + HA orchestrated restart order.

Q19. In NSX 4.2 integrated with VCF 5.2, what is the VCF default networking for management and workload domains?

- A) Only VSS
- B) NSX-managed VDS with overlay (GENEVE) for workload segments and separate management VDS/VLAN for management
- C) No networking required
- D) Physical only

****Answer: B****

****Explanation:**** VCF 5.2 uses NSX 4.2.1 as mandatory component with NSX-managed VDS. Management traffic uses VLAN-backed port groups; workload segments use GENEVE overlay with T0/T1 gateways for north-south/east-west and micro-segmentation via Distributed Firewall.

Q20. How does Storage vMotion handle encrypted VMs with vSphere 8.0 U3?

- A) It decrypts them automatically
- B) Supports live migration with encryption preserved via KMS, with improved performance using fast file cloning APIs (VAAI/NAS) when storage supports it
- C) Cannot migrate encrypted VMs
- D) Requires power off

****Answer: B****

****Explanation:**** Encrypted VMs can be Storage vMotioned without decrypting if hosts share KMS trust. vSphere 8 U3 leverages VAAI Block Clone and vSAN native snapshots for rapid cloning, keeping I/O encryption intact.

Section 3: Advanced (Questions 21-30)

Q21. You are designing a VCF 5.2.1 private cloud with vSAN ESA, 50 hosts, 2TB RAM per host, and GPU-enabled clusters for AI. What advanced vSphere 8 features must be considered for AI/ML workloads?

- A) Disable DRS and use only standard switches
- B) vGPU with Dynamic DirectPath I/O, RDMA over Converged Ethernet v2 (RoCE v2) support, vMotion for vGPU-enabled VMs (with NVIDIA vGPU 17+), and large memory pages with NUMA optimizations
- C) Only run AI on physical servers
- D) AI workloads not supported in vSphere

****Answer: B****

****Explanation:**** vSphere 8 U2/U3 added support for vMotion of vGPU VMs (stun time <2s), Device Groups for DPU/GPU passthrough, and RDMA. For LLMs, use 2MB/1GB pages, VM latency sensitivity High, reservation, and NUMA alignment with vSphere's virtual NUMA (vNUMA) auto-sizing.

Q22. Explain the difference between vSAN ESA RAID-5/6 erasure coding vs OSA and impact on capacity and resync in 8.0 U3.

- A) Same as OSA
- B) ESA uses adaptive RAID-5/6 with dynamic stripe width, per-object optimized parity, and high-performance scalable snapshots; resync is log-structured and up to 2x faster, with 4x more efficient capacity vs OSA RAID-1 default
- C) ESA only supports RAID-1
- D) ESA requires 10 nodes minimum for RAID-6

****Answer: B****

****Explanation:**** ESA implements new log-structured file system (LFS) with adaptive erasure coding that chooses optimal stripe. It reduces write amplification, enables space-efficient snapshots (up to 100x snapshots per object) and faster resync due to journal-based resync, improving usable TiB per VCF core entitlement.

Q23. In a VCF 5.2 SDDC Manager lifecycle upgrade from VCF 5.1.1 to 5.2.1, what is the required upgrade sequence and pre-check?

- A) Direct upgrade of ESXi only
- B) Use VCF Upgrade Bundle: SDDC Manager -> NSX -> vCenter -> ESXi/vSAN (via vLCM image) after using VCF Async Patch Tool, VCF Validate, and vSAN HCL pre-checks; Management WLD first, then VI WLDs
- C) Reinstall everything manually
- D) NSX first then ignore SDDC Manager

****Answer: B****

****Explanation:**** VCF has strict BOM (Bill of Materials) order enforced by SDDC Manager; skipping breaks interoperability. Pre-checks include vSAN Skyline Health, NSX upgrade coordinator, VCF Upgrade Precheck, and offline depot for air-gapped. Management domain must be upgraded before VI workload domains.

Q24. How does vSphere 8.0 U3 handle TPM 2.0, Secure Boot, and virtual TPM (vTPM) for attestation with Broadcom hardware HSM?

- A) vTPM not supported anymore
- B) Attestation is integrated: ESXi requires Secure Boot + TPM 2.0 for trusted boot; vTPM 2.0 devices backed by native key provider or KMS for VM's measured boot, with vCenter as native key provider for small deployments since 7.0 U2
- C) TPM causes boot failure always
- D) No encryption

****Answer: B****

****Explanation:**** vSphere Trustchain uses host TPM for measured boot into ESXi, vCenter verifies attestation. vTPM provides virtual TPM inside VM for Windows 11, measured boot, BitLocker, without physical TPM per host. Native key provider simplifies vTPM without external KMS but not for full KMIP compliance.

Q25. You have a 4-node vSAN ESA cluster that loses a full disk group (NVMe failure). What is the failure handling and rebuild process difference in ESA vs OSA?

- A) Same as OSA, full disk group rebuild needed
- B) ESA is disk-abstracted: no disk groups; failure domain is disk level + node level, using adaptive resync with individual drive failure handling, partial rebuilds, and dynamic capacity rebalancing without traditional disk group concept
- C) ESA cluster fails completely

D) Requires full cluster reformat

****Answer: B****

****Explanation:**** ESA removes disk group construct; all disks form single storage pool per host. A single NVMe failure triggers object repair at extent level, not entire disk group evacuation. This lowers blast radius, reduces resync time, and keeps storage pool online, improving MTTR and reducing vSAN capacity overhead.

Q26. In advanced troubleshooting, an ESXi 8.0 host shows PURPLE DIAGONAL SCREEN (PSOD) due to RDMA driver. What log and tool should you analyze post-reboot, and how to prevent?

A) Ignore PSOD

B) Analyze /var/log/vmkernel.log, vobd.log, vmksummary, and core dump via vsish, esxcli system coredump, check vSAN ESA RDMA/DPDK compatibility via VCG/HCL; isolate driver with vLCM vendor add-on quarantine or disable RoCE if unsupported; file Broadcom SR with vm-support bundle

C) Reboot again without logs

D) Only check Windows Event Viewer

****Answer: B****

****Explanation:**** PSOD data includes backtrace and hardware info; core dump preserved in vmk dump partition. RDMA/RoCE requires specific firmware/driver combos certified on VCG for vSAN ESA; mismatched MLNX OFED drivers cause PF exception. Use vmkdump -l and ESXi host client > Monitor > Hardware.

Q27. Design a VCF 5.2 multi-tenancy architecture using NSX Projects, VCF Automation, and vSphere Namespaces for Kubernetes with GPU.

A) Multi-tenancy impossible in VCF

B) Use NSX Projects for isolated VRF/LF isolation per tenant, VCF Automation (Aria Automation) 8.18+ with Projects + Cloud Zones, vSphere IaaS control plane (Tanzu) Namespaces with VM Service, Supervisor with resource quotas, and NSX VPCs per tenant, plus GPU Operator via vGPU profiles in namespace

C) Only one tenant supported

D) Use separate vCenters only

****Answer: B****

****Explanation:**** VCF 5.2 supports enhanced multi-tenancy: NSX 4.2 Projects provide enterprise-grade tenant isolation without multiple NSX Managers. vSphere IaaS control plane enables self-service K8s namespaces and VM pools with quotas. VCF Automation orchestrates tenancy with Aria Operations for capacity metering and chargeback, critical for Broadcom portfolio private cloud.

Q28. You must automate day-2 operations for 1000 VMs across VCF with Ansible and PowerCLI in 2025+. What is best practice for API authentication and lifecycle after Broadcom entitlement changes?

A) Use root passwords in scripts

B) Use OAuth 2.0 / API tokens via vCenter SSO with vSphere Automation API, Aria Automation API tokens with rotating secrets in HashiCorp Vault, service accounts per VCF WLD, and subscribe to VCF license usage metering API to avoid over-entitlement; use PowerCLI 13.3+ with certificate auth

C) Hardcode admin credentials

D) No API automation possible

****Answer: B****

****Explanation:**** Broadcom tightened security: vSphere 8 supports token-based auth, vAPI token exchange, and short-lived certificates. Best practice is least-privilege service principals per workload domain, secret management, and monitoring of subscription core usage via VCF Operations/Usage Meter to stay within VVF/VCF subscription allocation and avoid compliance issues.

Q29. During VCF 5.2.1 to VCF 9.0 readiness assessment (released late 2024 tech preview), what is the key architectural shift to VCF 9.0 private cloud?

A) No change

B) VCF 9.0 converges VCF and VVF into single unified platform with VCF Operations (Aria) fleet management, integrated vSphere Kubernetes Service (VKS), vSAN unified data path for ESA + vSAN Max + VMFS, and single SSO with NSX 9 integration; migration uses VCF Import tool for existing vSphere/vSAN inventories

C) Removes vSAN entirely

D) VCF 9.0 is only SaaS

****Answer: B****

****Explanation:**** VCF 9.0 (announced VMware Explore 2024, GA roadmap 2025) is Broadcom's vision for private cloud: one SKU, 9.0+ consolidates management, adds VCF Import to bring brownfield vSphere 7/8 clusters into VCF Inventory Manager. It introduces fleet-level operations, consolidated VCF Ops console, and mandatory NSX for unified private cloud with AI services.

Q30. A customer reports 40% performance degradation after enabling vSAN ESA RAID-6 and enabling virtual machine encryption cluster-wide. What is root cause analysis and remediation?

A) Encryption always degrades 40% - normal

B) Root cause likely CPU contention from KMIP encryption offload + RAID-6 parity compute overhead + lack of AES-NI offload or DPU offload; Check: Enable AES-NI, vSAN data-at-rest encryption offload to DPU or CPU optimized, use RAID-5 where FTT=1 suffices, separate KMS latency (<5ms), and enable vSAN compression only if CPU headroom >30%; Use esxtop - encryption world counters and vSAN performance service

C) Disable vCenter

D) Only reboot solves it

****Answer: B****

****Explanation:**** ESA RAID-6 has 2x parity calc vs RAID-5; combining with VM encryption causes double encryption (vSAN + VM). ESA does not support erasure-coded encryption offload to controller, relies on CPU AES-NI and VMkernel Crypto. Remediate with data-at-rest only OR VM encryption per sensitivity, tune cluster to RAID-5 adaptive where applicable, ensure KMS RTT low, and use DPUs for encryption offload.

****End of Exam****

Generated: 2024-2026 aligned | Focus: Broadcom Licensing, VCF 5.2, vSphere 8.0 U3, vSAN ESA, NSX, Tanzu, DPU, vGPU AI

Course File: 14_CSharp.md

Exam 14: C# & .NET - Comprehensive Assessment

Total Questions: 30 | Duration: 90 minutes | Passing Score: 70%

Coverage: C# 12/13, .NET 8/9 (2024-2026), Modern .NET Ecosystem

Part 1: Beginner (Questions 1-10)

Q1. What is the difference between struct and class in C#?

- A) Struct is reference type, Class is value type
- B) Struct is value type stored on stack (or inline), Class is reference type stored on heap; struct cannot have parameterless destructor and supports value equality by default for record struct
- C) Struct cannot have methods, Class can
- D) There is no difference since .NET 8

Q2. What does string? mean in C# with nullable reference types enabled?

- A) It is a syntax error - string cannot be nullable
- B) It denotes a nullable reference type, meaning the string variable is allowed to be null and compiler will warn if dereferenced without check
- C) It means the string is non-nullable
- D) It automatically initializes string to empty

Q3. What will var x = 5; infer in C#?

- A) dynamic
- B) object
- C) int
- D) var is not allowed for locals

Q4. Which keyword prevents inheritance of a class?

- A) static
- B) sealed
- C) abstract
- D) readonly

Q5. What is the output of "hello".ToUpper()? Does it mutate original string?

- A) Mutates original to HELLO and returns HELLO
- B) Returns HELLO but original string is immutable, so a new string is created and original remains hello
- C) Returns hello
- D) Throws compilation error

Q6. Which loop is guaranteed to execute at least once?

- A) for
- B) while
- C) do-while

D) foreach

Q7. What does using var fs = new FileStream(...); do in C# 8+?

- A) Same as var, no disposal
- B) Declares a using declaration that disposes fs at end of scope automatically
- C) Requires explicit fs.Dispose() call
- D) Only works for async

Q8. What is the correct way to handle exceptions in C#?

- A) Use try-catch-finally; finally always executes regardless of exception except for Environment.FailFast or stack overflow
- B) Exceptions cannot be caught in C#
- C) Use if-else for all errors
- D) Use try...except like Python

Q9. What does int[] numbers = [1,2,3]; use in C# 12?

- A) Invalid syntax
- B) Collection expression - new C# 12 feature to initialize collections concisely
- C) Creates a List<int>
- D) Creates an array of size 0

Q10. Difference between == and Equals() for strings?

- A) == compares references always, Equals() compares values
- B) For strings, == is overloaded to compare values, similar to Equals(), but == does reference check for null while Equals may throw NRE if called on null
- C) They are identical in all cases
- D) == is not defined for strings

Part 2: Intermediate (Questions 11-20)

Q11. What is the difference between Task, Task<T>, and ValueTask<T>?

- A) No difference
- B) Task represents async void work, Task<T> returns value; ValueTask<T> is struct to reduce allocations for hot paths where result may be synchronously available, but should not be awaited multiple times
- C) ValueTask is always faster
- D) Task is obsolete in .NET 8

Q12. In .NET 8, what does IEnumerable<T> vs IQueryable<T> difference imply for EF Core?

- A) Both same
- B) IEnumerable executes LINQ in memory (LINQ-to-Objects), IQueryable builds expression tree translated to SQL and executed on database server, enabling server-side filtering
- C) IQueryable is for in-memory only
- D) IEnumerable is translated to SQL

Q13. What is record vs record struct in C# 10+?

- A) Same as class
- B) record is reference type with value-based equality and with-expressions; record struct is value type variant with same value equality semantics, mutable by default unless readonly record struct
- C) Record cannot have methods
- D) Record struct is reference type

Q14. Explain primary constructors introduced in C# 12.

- A) Feature removed in C# 12
- B) Classes and structs can declare parameters directly in type declaration: `class Person(string name)`, parameters available throughout type body as constructor parameters and auto-captured fields; replaces boilerplate for simple DI models
- C) Only for records
- D) Same as static constructor

Q15. What is the purpose of `Span<T>` and `ReadOnlySpan<T>`?

- A) To replace arrays always
- B) Stack-only ref structs providing contiguous memory view over arrays, strings, stackalloc without allocation; enable zero-allocation slicing and high-performance parsing
- C) They are heap allocated
- D) They are for async only

Q16. What happens if you use `async void`?

- A) Recommended for all async methods
- B) `async void` is fire-and-forget, exceptions cannot be caught by caller and crash process; should only be used for event handlers
- C) Same as `async Task`
- D) Compile error

Q17. How does pattern matching `if (obj is List<int> { Count: >0 } list)` work?

- A) Invalid syntax
- B) Property pattern + list pattern - checks if `obj` is `List<int>` with `Count > 0` and assigns to variable `list`; introduced as part of enhanced pattern matching C# 8+
- C) Only checks type
- D) Throws if null

Q18. What is dependency injection lifestyle Scoped vs Singleton vs Transient in ASP.NET Core?

- A) All same
- B) Singleton: one instance app lifetime; Scoped: one per request/ scope; Transient: new instance every injection; Scoped cannot be injected into Singleton (captive)
- C) Scoped same as Singleton
- D) Transient same as Scoped

Q19. What does `required` keyword in C# 11+ do?

- A) Marks parameter as required at runtime only
- B) `public required string Name { get; set; }` forces object initializer or constructor to set property, compiler error CS9035 if not set; works with `SetsRequiredMembersAttribute`

- C) Makes property nullable
- D) Only for methods

Q20. In .NET 8/9, what is AOT (Native AOT) publishing limitation?

- A) No limitations
- B) Native AOT compiles IL to native code ahead-of-time for startup/size; restricts dynamic code: no MakeGenericType with unknown types, limited reflection, requires Trim-compatible and RDG analyzed libraries
- C) AOT means JIT
- D) AOT only works for Framework

Part 3: Advanced (Questions 21-30)

Q21. How does ConfigureAwait(false) affect execution in .NET? Is it still needed in .NET 8+ ASP.NET Core?

- A) Always required
- B) ConfigureAwait(false) avoids capturing SynchronizationContext; in ASP.NET Core (.NET 5+) there is no SyncContext, so it is not needed for library code targeting only ASP.NET Core, but still valuable for UI apps and reusable libraries to avoid deadlocks and context switches
- C) It makes code synchronous
- D) It throws exception

Q22. What is IAsyncEnumerable<T> and await foreach vs yield return async behavior with cancellation?

- A) IAsyncEnumerable<T> is synchronous
- B) Allows async streaming: async IAsyncEnumerable<int> Get() { await Task.Delay(...); yield return 1; }; consumer uses await foreach (var x in Get().WithCancellation(token)); supports [EnumeratorCancellation] attribute to propagate token
- C) Cannot be cancelled
- D) Same as IEnumerable<Task<T>>

Q23. Explain .NET 8 FrozenDictionary / FrozenSet in System.Collections.Frozen.

- A) Just alias for Dictionary
- B) Immutable optimized collections for read-heavy workloads: after creation, lookup is faster than regular Dictionary via optimized layout (hash spread, avoids rehash); ideal for static lookup tables created once during startup
- C) They are mutable
- D) They are slower for reads

Q24. What is the role of Channel<T> vs BlockingCollection<T> for producer-consumer in .NET modern code?

- A) Channel is obsolete
- B) Channel<T> provides async-first bounded/unbounded producer-consumer with backpressure, ValueTask based, integrates with async/await; BlockingCollection<T> is blocking sync API, not optimal for async; Channel preferred in .NET 6+ for high-throughput pipelines
- C) Both sync only
- D) Channel cannot be bounded

Q25. How do ref, in, ref readonly, ref struct and ref field (C# 12) interact?

- A) All same as value passing
- B) ref passes by reference mutable; in readonly ref; ref readonly returns readonly ref; ref struct stack-only; C# 12 ref fields allow ref struct to hold ref: `ref struct S { public ref int Field; }` enabling Span-like types to wrap references safely
- C) ref field is allowed in class
- D) in allows mutation

Q26. What is the difference between lock, Monitor, SemaphoreSlim, ReaderWriterLockSlim, and Mutex in .NET concurrency?

- A) All identical
- B) lock syntactic sugar for Monitor (thread-affine, sync only); SemaphoreSlim for counting async-compatible waits; ReaderWriterLockSlim allows multiple readers / single writer; Mutex is cross-process kernel object; for async code use SemaphoreSlim or .NET 9 Lock object with improved API
- C) lock works async
- D) Mutex is fastest

Q27. In C# 13 (.NET 9), what is params enhancements and field keyword in properties?

- A) No changes
- B) C# 13 allows params `Span<T>`, params `IEnumerable<T>`, params `ReadOnlySpan<T>` and collections beyond array; field keyword in semi-auto properties lets you access compiler-generated backing field: `public string Name { get; set { field = value.Trim(); OnChanged(); } }`
- C) params only works for int
- D) field is for methods

Q28. How does Garbage Collector (GC) Server vs Workstation and new .NET 8+ Dynamic Adaptation work?

- A) No modes
- B) Workstation GC optimized for low latency UI with smaller heaps, Server GC uses per-processor heaps and threads for throughput; .NET 8 introduced DATAS (Dynamic Adaptation To Application Sizes) to dynamically adjust heap size based on memory pressure, improving containerized scenarios
- C) Server GC only for Framework
- D) DATAS disables GC

Q29. Explain source generators vs interceptors (C# 12) vs inline arrays.

- A) Same feature
- B) Source Generators run at compile time to generate extra code based on syntax analysis (e.g. System.Text.Json, Regex Generator); Interceptors (C# 12, experimental in .NET 8) allow replacing method calls with generated code (used for minimal API AOT); Inline arrays `[InlineArray(10)] struct Buffer { int _element; }` provide fixed-size struct arrays without heap
- C) Source generators run at runtime
- D) Interceptors are stable public API in .NET 9 for production

Q30. How do you implement zero-allocation string building and UTF8 string literals in .NET 8/9?

- A) Use + concatenation always
- B) Use `DefaultInterpolatedStringHandler` via interpolated string handler pattern, `StringBuilder` with `Span<char>` pooling, `stackalloc char[256]`, and new C# 11+ UTF8 literals producing `ReadOnlySpan<byte>` via `u8` suffix; .NET 8+ has

SearchValues<T> and CompositeFormat for optimized formatting

C) UTF8 literals not exist

D) stackalloc always heap allocates

Answer Keys

| Question | Correct Answer | Brief Reasoning |

|-----|-----|-----|

| Q1 | B | Struct value heap distinction correct |

| Q2 | B | Nullable reference semantics |

| Q3 | C | var int inference |

| Q4 | B | sealed prevents inheritance |

| Q5 | B | String immutability |

| Q6 | C | do-while at least once |

| Q7 | B | Using declaration disposal |

| Q8 | A | try-catch-finally mechanics |

| Q9 | B | Collection expressions C#12 |

| Q10 | B | == overloaded for string |

| Q11 | B | Task vs ValueTask allocation |

| Q12 | B | IEnumerable vs IQueryable SQL translation |

| Q13 | B | record vs record struct equality |

| Q14 | B | Primary constructors C#12 |

| Q15 | B | Span zero-allocation view |

| Q16 | B | async void fire-and-forget danger |

| Q17 | B | Property pattern matching |

| Q18 | B | DI lifetimes scoped vs singleton |

| Q19 | B | required keyword enforcement |

| Q20 | B | Native AOT limitations |

| Q21 | B | ConfigureAwait context capture |

| Q22 | B | IAsyncEnumerable streaming + cancellation |

| Q23 | B | FrozenDictionary read-optimized |

| Q24 | B | Channel async producer-consumer |

| Q25 | B | ref/in/ref field C#12 |

| Q26 | B | lock vs Semaphore vs Mutex |

| Q27 | B | params Span + field keyword C#13 |

| Q28 | B | Server vs Workstation + DATAS |

| Q29 | B | Generators vs Interceptors vs InlineArray |

| Q30 | B | Zero-alloc string building + u8 literals |

Detailed Explanations

Q1. Explanation

Option B is correct because struct is a value type with copy-by-value semantics and cannot inherit, while class is reference type on heap. This distinction impacts performance, equality, and memory allocation as structs can live inline or on stack.

Q2. Explanation

A string? explicitly opts-in to nullable annotation context, enabling compiler flow analysis to warn on possible null dereference. With NRT enabled in .NET 6+ projects by default, this prevents `NullReferenceException` by static analysis.

Q3. Explanation

C# var uses target type inference from RHS at compile time, so 5 is int literal, thus x is int. It is statically typed, not dynamic, unlike JavaScript var.

Q4. Explanation

sealed prevents further inheritance and enables devirtualization optimizations by JIT. Marking library types sealed also improves Native AOT and trim analysis in .NET 8+.

Q5. Explanation

System.String is immutable for security and interning reasons. Methods like `ToUpper()` allocate new string, leaving original reference unchanged, crucial for thread safety.

Q6. Explanation

`do { } while(condition);` checks condition after body, guaranteeing one execution. Other loops evaluate condition first and may skip entirely.

Q7. Explanation

Using declarations were introduced in C# 8 to remove nesting: compiler inserts `Dispose()` call at end of enclosing block scope via try-finally. For async disposables, use `await using`.

Q8. Explanation

Correct exception handling uses try-catch for specific types, avoid catching `System.Exception` blindly, and finally for cleanup. Finally is not called only on fail-fast or corrupted process.

Q9. Explanation

C# 12 collection expressions `[1,2,3]` can target array, List, Span, etc., inferred by target type, with spread operator `..other`. It unifies collection initialization syntax introduced in .NET 8 SDK.

Q10. Explanation

string overloads `==` to call `Equals` ordinarily, unlike default reference equality for other classes. Calling `s1.Equals(s2)` when `s1` is null throws, while `s1 == s2` safely handles nulls and calls overloaded operator.

Q11. Explanation

`ValueTask<T>` avoids Task allocation when result is immediately available, common for cache hits and parsing, used heavily in `System.IO.Pipelines`. However awaiting same `ValueTask` twice or mixing sync access is illegal due to potential pool reuse.

Q12. Explanation

EF Core IQueryable provider translates expression tree to SQL via visitors; adding `.Where()` before `ToList` keeps filter server-side. Calling `.AsEnumerable()` prematurely switches to client-side LINQ and can cause N+1 or full table loads.

Q13. Explanation

Records provide synthesized `Deconstruct`, `ToString`, equality by `EqualityContract`, and non-destructive mutation via `with`. Record struct defaults to mutable fields but equality remains value-based, and readonly record struct gives immutability similar to readonly struct.

Q14. Explanation

Primary constructors simplify dependency injection and DTOs: parameters are in scope for field initializers and entire class body, with mutable capture. Compiler generates backing fields for captured parameters used outside constructor, reducing boilerplate vs C# 11 record primary ctor.

Q15. Explanation

Span<T> is ref struct guaranteeing no heap promotion, enabling safe pointer-like slicing over stackalloc, arrays, or unmanaged memory. Introduced in .NET Core 2.1, it underpins zero-allocation libraries like System.Text.Json and Kestrel.

Q16. Explanation

Async void is needed for ICommand and event handlers where signature forces void, but awaiters cannot observe completion so exceptions go to SynchronizationContext.Unobserved and terminate. Prefer Task return and use async Task void only at UI boundaries.

Q17. Explanation

Recursive pattern matching allows combining type, property, and relational patterns concisely, reducing nested ifs. The { Count: >0 } is a property pattern with relational pattern >0, added in C# 9/10 to improve guard clauses.

Q18. Explanation

The container tracks lifetimes; injecting Scoped service into Singleton causes captive dependency - Scoped instance lives as long as Singleton, leaking data across requests. Use IServiceScopeFactory to create scope manually from singleton.

Q19. Explanation

required enforces definite assignment at object creation point, checked by compiler via dataflow; RequiredMember and SetsRequiredMembers allow constructors to satisfy members. Useful for non-nullable properties without forced parameterized ctor in EF and serializers.

Q20. Explanation

Native AOT requires static analyzability; reflection like Type.GetType(string) or Assembly.Load may fail unless annotated with DynamicallyAccessedMembers. Libraries must be trimmable-safe and AOT-compatible, enforced via IsAotCompatible property and warnings IL2026, IL3050.

Q21. Explanation

ASP.NET Core removed SynchronizationContext to avoid deadlocks present in legacy ASP.NET where blocking on async .Result caused context capture deadlock. Libraries targeting multi-platform still use ConfigureAwait(false) to avoid resume on UI thread for performance.

Q22. Explanation

IAsyncEnumerable enables push-pull async streaming ideal for database cursors and gRPC streams; cancellation token propagation requires [EnumeratorCancellation] on token parameter to flow to GetAsyncEnumerator. Without WithCancellation, enumeration cannot be cancelled gracefully mid-stream.

Q23. Explanation

FrozenDictionary builds perfect hashing or ordinal ignore-case optimized trie at creation cost to make reads ~20-40% faster for large dictionaries. Used in .NET 8 for ObjectPool, regex, and minimal API route tables where dictionary is static after startup.

Q24. Explanation

Channel offers two sides ChannelReader/ChannelWriter, supporting async waits via Task and completion propagation; BoundedChannelOptions handles backpressure via Wait, DropNewest, etc. This matches Go channels and is base for System.Threading.Channels in ASP.NET Core response pipelines.

Q25. Explanation

Ref fields enable ref struct to store by-ref safely only within ref struct, enforcing scoped ref rules (scoped ref) to prevent escaping to heap. This enabled Span<T>-like internal implementation and allows custom ref-safe slices without unsafe, new in C# 12.

Q26. Explanation

lock in .NET 9 was upgraded to use System.Threading.Lock type for better diagnostics; it cannot be held across await. Async coordination should use SemaphoreSlim.WaitAsync or AsyncLock patterns; Mutex incurs kernel transition ~1000x slower than Monitor.

Q27. Explanation

C# 13 params generalization removes array allocation for params Span<int> allowing Print(1,2,3) to pass via InlineArray stack allocation. Field keyword solves auto-prop setter logic without explicitly declaring backing field, completing semi-auto props introduced in preview .NET 9.

Q28. Explanation

Server GC default in ASP.NET Core uses many heaps for throughput but high memory; DATAS (enabled via DOTNET_GCName or config) adjusts heap hard limits based on actual usage, reducing memory in Kubernetes with memory limits. dotnet-counters can monitor GC generations and pause time.

Q29. Explanation

Generators are ISourceGenerator/IncrementalGenerator run by compiler, no runtime overhead, used for interop marshalling. Interceptors were promoted as [InterceptsLocation] to reroute calls, critical for minimal APIs to generate static route handlers without reflection under AOT. InlineArray gives C-style fixed buffer with safe Span access.

Q30. Explanation

Modern .NET avoids allocations via string.Create with SpanAction, InterpolatedStringHandler which writes directly to buffer from format provider, and u8 suffix for compile-time UTF-8 bytes. SearchValues.Create uses vectorized SIMD (AVX2/Arm Neon) for fast char searching vs manual loops.

Course File: 15_Golang.md

Go (Golang) - Technical Proficiency Exam

Total Questions: 30 | Duration: 90 Minutes | Passing Score: 70%

Exam Covers: Go 1.21 - Go 1.23 (2024-2026 standards)

Part 1: Beginner Level (Questions 1-10)

Q1. What is the zero value of a string in Go?

- A) nil
- B) "" (empty string)
- C) "zero"
- D) Compiler error

****Answer: B****

****Explanation:**** Every type in Go has a zero value. For strings, the zero value is an empty string "", not nil. Strings are value types, not reference types.

Q2. Which keyword is used to declare a constant in Go?

- A) const
- B) constant
- C) let
- D) final

****Answer: A****

****Explanation:**** The const keyword declares immutable values at compile time. Constants can be typed or untyped and must be determinable at compile time.

Q3. How do you declare a variable with short declaration syntax inside a function?

- A) var x = 10
- B) x := 10
- C) x = 10
- D) let x = 10

****Answer: B****

****Explanation:**** The := operator is the short variable declaration that infers type and declares. It can only be used inside functions, not at package level.

Q4. What does go mod init do?

- A) Installs all dependencies
- B) Initializes a new Go module and creates a go.mod file
- C) Compiles the Go program
- D) Formats Go code

****Answer: B****

****Explanation:**** go mod init <module-path> initializes a module which is the unit for dependency management. It creates a go.mod file that tracks the module path and its dependencies.

Q5. Which built-in function returns the length of a slice, map, or string?

- A) size()
- B) length()
- C) len()
- D) count()

****Answer: C****

****Explanation:**** len() is a built-in function that returns length for strings, slices, arrays, maps, and channels. For channels it returns number of elements queued.

Q6. What is the correct way to define a function in Go?

- A) function foo() {}
- B) def foo():
- C) func foo() {}
- D) fn foo() {}

****Answer: C****

****Explanation:**** Go uses the func keyword to declare functions. Functions are first-class citizens and can be defined at package level or as methods with receivers.

Q7. Arrays and slices in Go - which statement is true?

- A) Arrays and slices are identical
- B) Array size is part of its type, slice is a dynamic view over an array
- C) Slices have fixed size, arrays are dynamic
- D) You cannot append to a slice

****Answer: B****

****Explanation:**** In Go, [3]int and [4]int are different types because size is part of array type. A slice is a descriptor containing pointer, length, and capacity, providing a dynamic view.

Q8. What will make([]int, 3, 5) create?

- A) Slice with length 3 and capacity 5
- B) Slice with length 5 and capacity 3
- C) Array with 3 elements
- D) Compiler error - invalid arguments

****Answer: A****

****Explanation:**** make for slices takes type, length, and optional capacity. Here it creates a slice with 3 zero-valued elements but underlying array capacity of 5, allowing efficient appends.

Q9. How do you handle errors idiomatically in Go?

- A) Using try-catch blocks
- B) Returning error as last return value and checking if err != nil
- C) Using exceptions
- D) Ignoring errors

****Answer: B****

****Explanation:**** Go does not have exceptions; errors are values. The idiomatic pattern is to return error as the last value and explicitly check it, promoting visible error handling.

Q10. What does defer do?

- A) Executes a function immediately
- B) Schedules a function call to run after the surrounding function returns
- C) Deletes a variable
- D) Pauses execution

****Answer: B****

****Explanation:**** defer pushes a call onto a LIFO stack executed when the enclosing function returns. It is commonly used for resource cleanup like closing files or unlocking mutexes.

Part 2: Intermediate Level (Questions 11-20)

Q11. What is the difference between a buffered and unbuffered channel?

- A) No difference
- B) Unbuffered channel requires both sender and receiver ready simultaneously; buffered allows sends up to capacity without blocking
- C) Buffered channels are faster always
- D) Unbuffered channels can only send integers

****Answer: B****

****Explanation:**** Unbuffered channels (capacity 0) provide synchronous communication with rendezvous semantics. Buffered channels decouple sender/receiver up to capacity, blocking only when full or empty.

Q12. Consider this code:

```
type S struct { m sync.Mutex }  
func (s S) Lock() { s.m.Lock() }
```

What is wrong?

- A) Nothing wrong
- B) Value receiver copies the Mutex, so locking has no effect on original
- C) sync.Mutex cannot be embedded
- D) Method must be private

****Answer: B****

****Explanation:**** sync.Mutex must not be copied after first use – vet warns about this. Using a value receiver copies the whole struct including mutex, so unlocked original leads to data races.

Q13. What does the context package provide, introduced to prevent goroutine leaks?

- A) Only cancellation signal
- B) Cancellation, timeout/deadline, and request-scoped values propagation across API boundaries and goroutines
- C) Database context
- D) Only logging

****Answer: B****

****Explanation:**** context.Context is standard for carrying deadlines, cancellation signals, and request-scoped values. Functions should accept Context as first argument and respect ctx.Done() to avoid leaked goroutines.

Q14. In Go 1.21+, which loop variable behavior changed with loopvar experiment now default?

- A) Loop variables are now per-iteration, fixing closure capture bug in goroutines
- B) Loop variables are now global
- C) No change
- D) Loops are now parallel

****Answer: A****

****Explanation:**** Before Go 1.22, loop variables were reused per loop, causing classic bug where goroutines captured same variable. Since Go 1.22 (previewed in 1.21 with LOOPVAR), each iteration creates new variable, making go func(){ use(v) }() safe.

Q15. What is interface nil vs typed nil pitfall?

```
func returnsNil() error {
    var p *MyError = nil
    return p
}
```

What does caller see?

- A) Caller sees nil interface
- B) Caller sees non-nil interface containing typed nil pointer, so err != nil is true
- C) Compile error
- D) Panic

****Answer: B****

****Explanation:**** An interface value is nil only when both type and value are nil. Returning a typed nil pointer as error creates non-nil interface with dynamic type *MyError, causing nil-check to pass unexpectedly.

Q16. How do generics (since Go 1.18) declare a constrained type parameter?

- A) func Foo[T any](x T)
- B) func Foo<T>(x T)
- C) generic Foo[T]
- D) func Foo[T comparable](x T) is only valid syntax

****Answer: A****

****Explanation:**** Go generics use [T constraint] syntax. any (alias for interface{}) allows any type, comparable allows == comparable types. Option A shows correct generic syntax, D is also valid but A is the canonical example.

Q17. What is sync.WaitGroup used for?

- A) To wait for a collection of goroutines to finish
- B) To lock memory
- C) To wait for HTTP response
- D) To garbage collect

****Answer: A****

****Explanation:**** WaitGroup is a counter: Add() increments, Done() decrements, Wait() blocks until counter zero. It is the standard way to wait for multiple goroutines, unlike manual channel signaling.

Q18. slices package added in Go 1.21 - what does slices.Clone and slices.Delete do?

- A) Both are unsafe operations
- B) Clone makes a shallow copy of slice; Delete removes elements at range [i,j) without leaking if handled

C) Delete always causes memory leak

D) Clone is deep copy for structs

****Answer: B****

****Explanation:**** Go 1.21 introduced slices and maps standard packages. slices.Clone returns a copy of slice elements (shallow). slices.Delete removes range and zeros tail to avoid retaining references, preventing memory leaks.

Q19. What is the correct way to avoid data race when multiple goroutines write to a map?

A) Maps are thread-safe by default

B) Use sync.Mutex or sync.RWMutex to guard map, or use sync.Map for specific use cases

C) Use atomic package on map

D) Just use buffered channel as map

****Answer: B****

****Explanation:**** Go maps are not safe for concurrent write or concurrent read+write; runtime will panic. You must synchronize with mutex/RWMutex, or use sync.Map which is optimized for mostly-read, disjoint keys, or cache.

Q20. Explain io.Reader and io.Writer interfaces.

```
type Reader interface { Read(p []byte) (n int, err error) }
```

```
type Writer interface { Write(p []byte) (n int, err error) }
```

Why are they powerful?

A) They are just for files

B) They enable composition - any type implementing them works with all stdlib tools (http, compress, crypto, json)

C) They are deprecated since Go 1.20

D) Only os.File implements them

****Answer: B****

****Explanation:**** These tiny interfaces demonstrate Go's composition philosophy. Because they are small, many types implement them, and functions like io.Copy, json.NewDecoder(r), or gzip.NewReader(r) work with any source.

Part 3: Advanced Level (Questions 21-30)

Q21. Go scheduler GMP model (Go 1.19-1.23): What happens when a goroutine does blocking syscall?

A) Entire OS thread (M) blocks and scheduler cannot reuse it

B) G is detached, M blocks, P migrates to another M or new M created to continue running other Gs - sysmon handles preemption

C) Program crashes

D) Garbage collector stops

****Answer: B****

****Explanation:**** In GMP model, P (processor) is detached from M doing blocking syscall. Scheduler assigns P to another idle M or creates new M. Since Go 1.14+ async preemption via signals prevents unbounded blocking from hogging P.

Q22. Consider escape analysis:

```
func foo() *int { x := 42; return &x }
```

Why does this work in Go but would be stack-use-after-return bug in C?

A) It doesn't work, it panics

- B) Compiler's escape analysis moves x to heap because its address escapes function scope
- C) Go copies stack to heap on return
- D) Integers are always on heap

****Answer: B****

****Explanation:**** Go compiler performs escape analysis at compile time. If address of local variable escapes (returned, sent to channel, stored globally), compiler allocates it on heap with GC management. Run `go build -gcflags=-m` to see escapes.

Q23. What is the behavior of select with nil channels in Go 1.21+?

- A) Panics
- B) Case with nil channel blocks forever and is never selected, useful for disabling cases dynamically
- C) Selects immediately
- D) Compile error

****Answer: B****

****Explanation:**** Sending or receiving on nil channel blocks forever. In select, a nil channel case is ignored. Idiom: set channel variable to nil to disable that case without complicated if logic, e.g., `if done { ch = nil }`.

Q24. Go memory model and atomics: Why was `atomic.Pointer[T]` introduced in Go 1.19 and when to use `sync/atomic` vs `mutex`?

- A) Mutex always better
- B) `atomic.Pointer[T]` provides type-safe atomic pointer operations; use atomics for single-word counters/flags where contention low and overhead matters, mutex for critical sections with multiple invariants
- C) Atomics are deprecated
- D) No difference

****Answer: B****

****Explanation:**** Before 1.19, `atomic.Value` and `unsafe.Pointer` tricks were needed for pointer CAS. `atomic.Pointer[T]` is generic type-safe. Atomics avoid scheduler involvement and are cheaper for single variable, but don't compose for multi-variable consistency.

Q25. Explain `clear` builtin added in Go 1.21 and its relation to maps/slices:

- A) Only clears variables to zero value
- B) `clear(m)` deletes all entries from map; `clear(s)` zeros all elements of slice but keeps len/cap; helps clear references for GC
- C) Freezes the map
- D) Same as `make`

****Answer: B****

****Explanation:**** Go 1.21 added `clear` as builtin mirroring `slices/maps` package. For maps it deletes all keys (map becomes empty but non-nil). For slices it sets each element to zero value, which is critical to drop pointers and allow GC.

Q26. What is the difference between `context.WithValue` misuse vs proper use? Which causes memory leak in 2024+?

- A) No difference
- B) Storing large structs or using string keys without custom type leaks; proper use is small request-scoped values with custom unexported key type, never for optional params
- C) `WithValue` always leaks
- D) Keys must be integers

****Answer: B****

****Explanation:**** Contexts form a chain holding parent references until cancelled. Putting large blobs in context keeps them alive for entire request lifetime. Use unexported custom type as key to avoid collisions, and only for cross-cutting data like requestID, not function args.

Q27. Go 1.20+ introduced arena experiment and improved GC. How does Pacer and GOGC work since Go 1.19?

- A) GOGC deprecated
- B) GOGC is percentage of heap growth causing next GC; pacer in 1.19+ aims for less CPU spikes, GOMEMLIMIT limits total memory; arena would provide manual region-based allocation
- C) GC runs every second
- D) No pacer

****Answer: B****

****Explanation:**** GC pacer redesigned in Go 1.19 to minimize assist time and pacing errors. GOGC=100 means heap can grow 100% before GC. Since Go 1.19, GOMEMLIMIT caps application memory, triggering GC more aggressively to stay within limit.

Q28. Subtle for range over integer added in Go 1.22 - and performance of defer in loop?

- A) for i := range 10 not valid
- B) Go 1.22 allows for i := range n iterating 0 to n-1; defer inside loop stacks and executes only at function exit, causing resource buildup - should wrap in func or use Go 1.21+ explicit cleanup pattern
- C) Defer in loop executes each iteration
- D) Range over int panics

****Answer: B****

****Explanation:**** Go 1.22 added integer range: for i := range 10 equals 0..9, and for range 10 repeats 10 times. Defer inside loops is classic perf bug: defers don't run per iteration, they accumulate and run at return, holding resources longer than intended.

Q29. How does runtime.SetFinalizer interact with finalizers, and what is the pattern for io.Closer wrapper since Go 1.21 log/slog and context.AfterFunc?

- A) SetFinalizer guarantees immediate cleanup
- B) Finalizers are non-deterministic, may not run, cause heap retention and GC overhead; avoid - use runtime.AddCleanup (Go 1.24 proposal, available as context.AfterFunc / Cleaner pattern) or explicit Close; AfterFunc registers function when context done
- C) Finalizer replaces defer
- D) Finalizer is sync

****Answer: B****

****Explanation:**** Finalizers are best-effort, run in separate goroutine after object unreachable, may resurrect object, delaying collection. They cause latency. Go discourages them. Go 1.21 added context.AfterFunc and Go 1.23 improved slog resource cleanup; newer AddCleanup proposal formalizes explicit cleanup attachment.

Q30. Design a rate limiter and singleflight in Go for 10k RPS service - which primitives to use?

// Need: limit 10k req/s globally, coalesce duplicate in-flight requests

- A) Use time.Sleep only

B) Use `golang.org/x/time/rate` token bucket for rate limiting + `golang.org/x/sync/singleflight` to deduplicate concurrent identical calls, combined with context cancellation

C) Mutex only

D) Channel only

****Answer: B****

****Explanation:**** `rate.Limiter` implements token bucket with burst and sustained RPS, supports context-aware `Wait/Reserve`. `singleflight.Group` suppresses duplicate calls where many goroutines request same key - only one execution runs, others share result, critical for cache stampede prevention at high RPS.

Answer Key Summary

| Question | Answer | Topic |

|-----|-----|-----|

| 1 | B | Zero values |

| 2 | A | Constants |

| 3 | B | Short declaration |

| 4 | B | Go modules |

| 5 | C | `len()` builtin |

| 6 | C | Function syntax |

| 7 | B | Arrays vs Slices |

| 8 | A | `make()` |

| 9 | B | Error handling |

| 10 | B | `defer` |

| 11 | B | Channels |

| 12 | B | `sync.Mutex` copy pitfall |

| 13 | B | `context` package |

| 14 | A | Loop var fix Go 1.22 |

| 15 | B | Interface nil pitfall |

| 16 | A | Generics |

| 17 | A | `WaitGroup` |

| 18 | B | `slices` package Go 1.21 |

| 19 | B | Concurrent map safety |

| 20 | B | `io.Reader/Writer` |

| 21 | B | GMP scheduler |

| 22 | B | Escape analysis |

| 23 | B | nil channel in select |

| 24 | B | atomic vs mutex |

| 25 | B | `clear` builtin Go 1.21 |

| 26 | B | `context.WithValue` |

| 27 | B | GC pacer GOGC GOMEMLIMIT |

| 28 | B | range over int Go 1.22 + `defer` loop |

| 29 | B | Finalizers vs `AfterFunc` |

| 30 | B | rate + singleflight |

****Scoring Guide:****

- 27-30 Correct: Expert (Go Core Contributor Level)

- 22-26 Correct: Advanced (Senior Go Developer)

- 16-21 Correct: Intermediate (Mid-Level Go Developer)

- 10-15 Correct: Beginner (Junior Go Developer)

- 0-9 Correct: Needs Study

****Exam Validity:**** Aligned with Go 1.21, 1.22, 1.23 (2024-2026) release notes, time/loops, slices/maps, clear, slog, context.

Course File: 16_Java.md

Exam 16: Java - Core to Modern Java (Java 17-24)

This exam covers Java fundamentals, OOP, collections, modern features from Java 17-24 (records, sealed classes, pattern matching, virtual threads), and concurrency.

Beginner Level - 10 Questions

****1. What is the difference between JDK, JRE, and JVM?****

- A) JDK is for running apps, JRE for developing, JVM is a database
- B) JVM executes bytecode, JRE = JVM + libraries to run apps, JDK = JRE + development tools (compiler, debugger)
- C) They are all the same
- D) JDK is old name for JRE

****2. Which of the following is the correct way to declare the main method in Java?****

- A) public void main(String[] args)
- B) public static void main(String[] args)
- C) static public void Main(String args[])
- D) public static void main()

****3. What will happen with this code? String s1 = "java"; String s2 = new String("java"); System.out.println(s1 == s2);****

- A) true
- B) false
- C) Compilation error
- D) NullPointerException

****4. Which primitive types in Java is correct about memory size?****

- A) int is 8 bytes, long is 4 bytes
- B) int is 32-bit (4 bytes), long is 64-bit (8 bytes), boolean size is JVM dependent
- C) All primitives are 4 bytes
- D) char is 1 byte in Java

****5. What does final mean when applied to a variable, method, and class respectively?****

- A) Variable cannot be reassigned, method cannot be overridden, class cannot be subclassed
- B) Variable is constant across all instances, method is abstract, class is immutable
- C) Same meaning for all: cannot be changed
- D) Variable is thread-safe, method is static, class is finalizable

****6. What is the default value of an instance field of type int and Integer if not initialized?****

- A) Both are 0
- B) int is 0, Integer is null
- C) Both are null
- D) Compilation error if not initialized

****7. Which statement about == vs .equals() is true?****

- A) Both compare content
- B) == compares reference identity for objects, .equals() compares logical equality (if overridden)
- C) .equals() always compares references
- D) You should always use == for Strings

****8. Which access modifiers make a member visible only within its own package and subclasses in other packages?****

- A) private
- B) default (package-private)

C) protected

D) public

****9. What happens when you attempt to modify a List created by List.of(1,2,3)?****

A) It adds the element successfully

B) It throws UnsupportedOperationException because List.of returns an immutable list

C) It throws compilation error

D) It creates a new copy automatically

****10. What is method overloading?***

A) Same method name, same parameters in child class

B) Same method name with different parameter lists (type, count, order) within same class

C) Overriding method from parent class

D) Defining two methods with same name and same parameters

Intermediate Level - 10 Questions

****11. Consider record Point(int x, int y) {}. What does the Java compiler generate automatically?***

A) Only private fields

B) Private final fields, canonical constructor, getters as x() and y(), equals(), hashCode(), toString()

C) Setters and getters

D) Nothing, you must write everything

****12. What is the output of this code using virtual threads (Java 21+)?***

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {  
    executor.submit() -> System.out.println(Thread.currentThread().isVirtual());  
}
```

A) false

B) true

C) Compilation error, isVirtual() doesn't exist

D) Exception because virtual threads need special flag

****13. Given a sealed class: public sealed class Shape permits Circle, Square {}. What is true?***

A) Any class can extend Shape

B) Only Circle and Square can extend Shape, and they must be final, sealed, or non-sealed

C) Shape must be abstract

D) Sealed classes are only allowed for interfaces

****14. What does this pattern matching for instanceof do (Java 16+)?***

```
if (obj instanceof String s && s.length() > 5) { System.out.println(s.toUpperCase()); }
```

A) Compilation error

B) Checks if obj is String, casts to s, and if length >5 prints uppercase; s is only in scope inside if block's true branch

C) Always throws ClassCastException

D) s is visible outside if block

****15. Which statement about ArrayList vs LinkedList is accurate as of Java 21?***

A) LinkedList is always faster for everything

B) ArrayList uses resizable array (O(1) random access, O(n) middle insert), LinkedList uses doubly linked nodes (O(n) random access, O(1) insert at ends via Deque methods)

C) Both are synchronized

D) ArrayList cannot hold nulls

****16. What is wrong with this concurrent code?***

```
class Counter { int count = 0; void increment() { count++; } }
```

Used by multiple threads.

A) Nothing, int increment is atomic

B) Race condition: count++ is not atomic (read-modify-write), need AtomicInteger or synchronized or volatile is insufficient alone

C) It will deadlock

D) It throws ConcurrentModificationException

****17. How does Java's Garbage Collector handle this? WeakHashMap vs HashMap?****

A) Both hold strong references

B) WeakHashMap entries whose keys are only weakly reachable can be reclaimed, useful for caches; HashMap holds strong references

C) WeakHashMap is faster

D) WeakHashMap is synchronized

****18. Explain the difference between Optional.map() and Optional.flatMap().****

A) They are identical

B) map wraps result in Optional even if function returns Optional leading to Optional<Optional<T>>, flatMap expects function to return Optional and flattens it

C) flatMap throws exception if empty

D) map cannot be used with Optional

****19. What will this Stream code produce in Java 22+ with Sequenced Collections?****

```
var list = new ArrayList<>(List.of(1,2,3));
```

```
list.addFirst(0);
```

```
list.addLast(4);
```

```
System.out.println(list.getFirst() + list.getLast());
```

B) Compilation error, addFirst doesn't exist

C) 0 + 4 = 4 (0 and 4 are first and last after additions)

D) Throws exception

****20. What is the purpose of the transient keyword and try-with-resources?****

A) transient makes field thread-safe, try-with-resources auto-closes AutoCloseable resources

B) transient excludes field from serialization, try-with-resources ensures resources declared in try() are auto-closed via close()

C) Both are for concurrency

D) transient makes field immutable

Advanced Level - 10 Questions

****21. Given Java 21 pattern matching for switch (preview final in Java 21):****

```
sealed interface Expr permits Const, Plus {}
```

```
record Const(int v) implements Expr {}
```

```
record Plus(Expr a, Expr b) implements Expr {}
```

```
static int eval(Expr e) {
```

```
    return switch(e) {
```

```
        case Const(int v) -> v;
```

```
        case Plus(Expr a, Expr b) -> eval(a) + eval(b);
```

```
    }
```

What is true?

A) This doesn't compile because switch can't be exhaustive

B) This compiles: record patterns deconstruct records, switch over sealed type is exhaustive when covering all permits, no default needed

C) Requires if-else

D) Record patterns are not allowed in switch

****22. Explain JVM memory model and what volatile guarantees in Java 21+.****

A) volatile makes operations atomic

B) volatile guarantees visibility and establishes happens-before: write to volatile happens-before subsequent read, prevents reordering, but does NOT guarantee atomicity for compound actions like i++

C) volatile makes object immutable

D) volatile is same as synchronized

****23. How do Virtual Threads (Project Loom, Java 21) differ from Platform Threads, and what is a pinning issue?****

A) Virtual threads are OS threads, no pinning

B) Virtual threads are lightweight user-mode threads mounted on platform threads (carrier); pinning occurs when virtual thread holds monitor or runs native code/blocking in synchronized, preventing unmount and harming scalability

C) Virtual threads are same as platform threads but faster

D) Pinning never happens after Java 21

****24. What is the difference between CompletableFuture and Structured Concurrency (Java 21 preview, Java 25 finalizing)?****

A) They are identical

B) CompletableFuture is low-level composable async with callbacks/combinators (thenApply, thenCombine), error handling is manual; Structured Concurrency (StructuredTaskScope) treats concurrent subtasks as a single unit with scoped lifetime, automatic cancellation/error propagation, and clearer code

C) Structured Concurrency is deprecated

D) CompletableFuture blocks virtual threads

****25. Analyze this code for memory leak / resource leak.****

```
public class Cache {  
    private static final Map<Key, Value> cache = new HashMap<>();  
    public static void put(Key k, Value v) { cache.put(k,v); }
```

Where Key doesn't override hashCode/equals and is mutable.

A) No leak

B) Unbounded static HashMap holds strong refs forever causing memory leak; mutable keys with broken hashCode/equals can be lost/unremovable; use WeakHashMap, proper immutable key with equals/hashCode, or bounded cache like Caffeine with eviction

C) HashMap auto-evicts

D) Only problem is thread-safety

****26. What are the key changes in Java Modules (JPMS) and how to fix split packages?****

A) Modules have no effect

B) JPMS (Java 9+) enforces strong encapsulation; split packages (same package in two modules) are illegal; fix by merging modules, renaming packages, or keeping code on classpath as unnamed module

C) Split packages are allowed in modules

D) Modules replace packages

****27. How does synchronized work on the JVM and what are modern alternatives in Java 21+?****

A) synchronized is obsolete

B) synchronized uses monitor enter/exit with biased locking (removed in Java 15+) -> lightweight -> inflated locking; modern alternatives: ReentrantLock for advanced features, StampedLock for optimistic reads, virtual-thread-friendly ReentrantLock + java.util.concurrent and avoiding pinning via ReentrantLock instead of synchronized in virtual threads

C) synchronized uses CAS only

D) Virtual threads cannot use synchronized

****28. What is the difference between invokedynamic, invokestatic, invokevirtual, invokespecial, and invokeinterface in bytecode, and why does invokedynamic matter for Lambda and Records?****

A) All are same

B) invokestatic: static methods, invokespecial: private/constructor/super, invokevirtual: virtual dispatch for classes, invokeinterface: interface dispatch. invokedynamic: dynamic call site resolution via BootstrapMethod, used for lambdas (LambdaMetafactory), string concatenation (Java 9+), records; enables runtime linkage and performance

C) invokedynamic is only for Groovy

D) Lambdas use anonymous classes always

****29. Explain GC choices in Java 21-24 (G1, ZGC, Shenandoah, Generational ZGC) and when to use each.****

A) Only G1 exists

B) G1 (default): balanced low pause ~<200ms, good throughput, regions. ZGC (Java 21+ generational from Java 21/22): sub-ms pauses, colored pointers, great for huge heaps (>100GB). Shenandoah: low-pause concurrent compacting. Generational ZGC (Java 23): improves throughput by separating young/old. Choose G1 for general, ZGC/Shenandoah for latency-sensitive large heaps

C) ZGC has higher pauses than G1

D) GC choice doesn't matter

****30. How do you correctly implement equals() and hashCode() contract for a class used in HashMap/HashSet in Java 17+?***

A) Only override equals

B) Override both: equals must be reflexive, symmetric, transitive, consistent, non-null false; hashCode must return same value for equal objects; in Java 17+ use Objects.equals() and Objects.hash() or record (auto-implements), ensure immutability of fields used, and instanceof pattern matching

C) Use == in equals

D) Return random hashCode

Answer Keys

****Beginner - Answer Key****

1. ****Answer: B**** - JVM is the virtual machine executing bytecode, JRE bundles JVM + core libs to run Java apps, JDK adds javac and tools. This distinction was simplified since Java 11 where JRE distribution is not separate from JDK.

2. ****Answer: B**** - JVM looks for exactly public static void main(String[] args) (or varargs). Non-static fails to run, case matters for Main.

3. ****Answer: B**** - == compares references. Literal goes to string pool, new String() creates heap object, so different refs. Use .equals() for content.

4. ****Answer: B**** - Java has fixed sizes: byte 1, short 2, char 2 (UTF-16), int 4, long 8, float 4, double 8. boolean is virtual and size is JVM dependent.

5. ****Answer: A**** - final variable assign once, final method cannot be overridden (but can be overloaded), final class cannot be extended e.g. String.

6. ****Answer: B**** - Instance fields get defaults (0, null, false), locals do not. Primitive int defaults to 0, wrapper Integer (object) defaults to null.

7. ****Answer: B**** - == for objects checks identity, .equals() is meant for logical equality and must be overridden. For String literals due to interning == may appear true but unreliable.

8. ****Answer: C**** - protected: same package + subclasses even in other packages. Default is only same package, private only same class.

9. ****Answer: B**** - Since Java 9, List.of, Set.of, Map.of produce unmodifiable immutable collections. Mutating methods throw UnsupportedOperationException.

10. ****Answer: B**** - Overloading is compile-time polymorphism with same name different params. Overriding is runtime polymorphism with same signature in subclass.

****Intermediate - Answer Key****

11. ****Answer: B**** - Record is compact immutable data carrier (Java 16+). Compiler synthesizes final fields, canonical constructor, accessor methods named same as components, plus correct equals/hashCode/toString based on components.

12. ****Answer: B**** - newVirtualThreadPerTaskExecutor() creates new virtual thread per task, isVirtual() returns true for virtual threads (Java 21 API). Virtual threads are cheap and ideal for blocking I/O.

13. ****Answer: B**** - Sealed classes (Java 17) restrict inheritance via permits. Permitted subclasses must be in same module/package and declared final, sealed, or non-sealed to control continuation of sealing.

14. ****Answer: B**** - Pattern variable `s` is introduced after instanceof check and only in scope where compiler knows check succeeded, including `&&` RHS and true-branch. Reduces explicit casting boilerplate.
15. ****Answer: B**** - ArrayList contiguous array = fast index, slow middle insert. LinkedList nodes = slow index $O(n)$, fast Deque ops addFirst/last $O(1)$. LinkedList rarely useful today; ArrayList + ArrayDeque preferred.
16. ****Answer: B**** - `count++` is three ops: `getField`, `iadd`, `putField`, not atomic. Volatile fixes visibility but not atomicity. Use `AtomicInteger`, `synchronized`, or `VarHandle`.
17. ****Answer: B**** - WeakHashMap wraps keys in WeakReference; when GC finds key only weakly reachable, entry is expelled from map. Perfect for listener/metadata caches to avoid memory leaks.
18. ****Answer: B**** - If you map `T -> Optional<U>`, map gives `Optional<Optional<U>>`. flatMap flattens by expecting the mapper to return Optional and not rewrap. Use flatMap to chain Optional pipelines.
19. ****Answer: C**** - SequencedCollection (Java 21) adds `addFirst`, `addLast`, `getFirst`, `getLast`, `reversed()`. After adding 0 at front and 4 at back, `first=0 last=4 sum=4`.
20. ****Answer: B**** - `transient` tells ObjectOutputStream to skip serialization of field. `try-with-resources` (Java 7+) calls `AutoCloseable.close()` automatically even on exception, suppressing secondary exceptions.

****Advanced - Answer Key****

21. ****Answer: B**** - Record patterns and pattern matching for switch became final in Java 21 (JEP 441). Compiler checks sealed exhaustiveness, so no default required if all permits covered. Enables algebraic data types style.
22. ****Answer: B**** - Volatile reads/writes have memory semantics: visibility to all threads and prevents instr reordering around volatile access. It does not make `i++` atomic because it's compound; need atomic class or lock for RMW.
23. ****Answer: B**** - Virtual threads are JVM-managed, not OS threads, parked when blocking. Pinning occurs when virtual thread is inside synchronized block or native call, the carrier platform thread cannot be released, reducing scalability. Fix by using `ReentrantLock`.
24. ****Answer: B**** - `CompletableFuture` leads to callback hell if overused, manual cancellation. `StructuredTaskScope` (JEP 453/480) provides structured concurrency: fork subtasks in scope, join together, any failure cancels siblings, lifetime bound to scope, ideal with virtual threads.
25. ****Answer: B**** - Static map never cleared = memory leak, strong refs prevent GC. Mutable keys breaking hashCode cause bucket mismatch and duplicate entries. Use immutable keys with correct equals/hashCode, WeakHashMap/SoftReferences, or bounded LRU cache.
26. ****Answer: B**** - JPMS modules declare `module { exports ...; requires ... }`. Encapsulation forbids split packages across named modules because same package would have two sources. Migration fix: merge, re-package, or use automatic modules.
27. ****Answer: B**** - `Synchronized` enters monitor via CAS and inflates to OS mutex under contention. Biased locking was removed (JEP 374). For virtual threads prefer `java.util.concurrent.locks` to avoid pinning; `synchronized` will be updated in future Loom to be virtual-thread friendly but still limited in 21-24.
28. ****Answer: B**** - `invoke*` are bytecode instructions for method dispatch. `invokedynamic`'s bootstrap (e.g., `LambdaMetafactory`) returns `CallSite`; dynamic linkage allows lambdas to be spun via method handles not anonymous classes, and string concat via `StringConcatFactory`, improving performance and future evolution.
29. ****Answer: B**** - G1 default, region-based with pause target. ZGC uses load barriers + colored pointers for <1ms pauses, now generational by default in Java 23+ to improve throughput. Shenandoah Brooks pointers similar goals. Choose based on latency vs throughput and heap size.
30. ****Answer: B**** - Contract: equal objects same hashCode, unequal may collide but should distribute. Use immutable fields, `Objects.equals` for null-safe, `Objects.hash`. Records give correct implementation free. Always override both together or neither.

Course File: 17_JavaScript.md

Course: JavaScript

1. Beginner Level (10 Questions)

****Q1. What is the difference between var, let, and const in modern JavaScript (ES6+)?****

- A. No difference, all are scoped the same
- B. var is function-scoped and hoisted with initialization to undefined, let and const are block-scoped, hoisted but in Temporal Dead Zone, and const requires initialization and prevents reassignment
- C. let is function-scoped, var is block-scoped
- D. const makes objects immutable deeply

****Q2. What will console.log(typeof null) output and what is null == undefined vs null === undefined?****

- A. "null", true and true
- B. "object", true and false
- C. "undefined", false and false
- D. "object", false and true

****Q3. What is the result of [] == ![] and why?****

- A. true, because ![] is false, then [] == false coerces to 0 == 0
- B. false, because array never equals boolean
- C. true, because both are empty
- D. TypeError

****Q4. Which method correctly adds an element to the end of an array and what does it return?****

- A. arr.push(item) returns new length, arr.pop() removes last
- B. arr.add(item) returns array
- C. arr.push(item) returns added item
- D. arr.unshift(item) adds to end

****Q5. How do you correctly select an element with id app and add a click listener?****

- A. document.getElementById('app').onClick = function() {}
- B. document.querySelector('#app').addEventListener('click', handler)
- C. document.find('#app').click(handler)
- D. document.querySelectorAll('app').addEventListener('click', handler)

****Q6. What does strict equality === check versus loose equality ==?****

- A. === checks value only, == checks value and type
- B. === checks value and type without coercion, == does type coercion before comparison
- C. Both are identical in ES2024
- D. == is faster than ===

****Q7. What will this log? for (var i=0; i<3; i++) { setTimeout(()=>console.log(i),0) } vs using let?****

- A. 0,1,2 in both cases
- B. var: 3,3,3 because var is shared closure; let: 0,1,2 because let creates per-iteration binding
- C. TypeError
- D. var: 0,1,2 let: 3,3,3

****Q8. Which statement about NaN is correct?****

- A. NaN === NaN is true
- B. NaN === NaN is false, use Number.isNaN(x) or Object.is(x, NaN) to check

C. `typeof NaN` is "NaN"

D. `NaN == null` is true

****Q9. What is the difference between a function declaration and a function expression in hoisting?****

A. `function foo(){} is hoisted fully and callable before definition, const foo = function(){} is not, due to TDZ`

B. Both are hoisted identically

C. Function expressions are hoisted, declarations are not

D. Arrow functions are hoisted

****Q10. What does `JSON.stringify({a: 1, b: undefined, c: () => {}})` return?****

A. `{"a":1,"b":null,"c":null}`

B. `{"a":1}` because undefined and functions are omitted in objects, and become null in arrays

C. Error

D. `{"a":1,"b":undefined}`

Beginner Answer Key & Explanations

****Q1 Answer: B - `var` is function-scoped and hoisted with initialization to undefined, `let/const` block-scoped with TDZ****

`var` hoisting leads to bugs and allows redeclaration; `let/const` introduced block scope and TDZ to prevent access before declaration. `const` only prevents reassignment of binding, not deep immutability.

****Q2 Answer: B - "object", true and false****

`typeof null` is "object" due to legacy bug from first JS implementation kept for compatibility. `null == undefined` is true by spec's loose equality rules, but `===` is false because types differ.

****Q3 Answer: A - true, because coercion makes it `0 == 0`****

`![]` coerces array to boolean true then negates to false. Then `[] == false` triggers `Truthy`: `[]`.toString() is "" -> 0, false -> 0, so `0 == 0` true.

****Q4 Answer: A - `arr.push` returns new length****

`push()` mutates array and returns new length for chaining length checks. `unshift()` adds to start, `pop()` returns removed element.

****Q5 Answer: B - `querySelector('#app').addEventListener`****

`querySelector` with `#id` selector and `addEventListener` is standard modern DOM API supporting multiple listeners and options. `onClick` property name is lowercase `onclick` and only allows one handler.

****Q6 Answer: B - `===` checks value and type without coercion, `==` does coercion****

Using `===` avoids surprising coercion rules like `"0" == false` being true. Linters in 2024-2026 enforce `===` by default via `eslint eqeqeq`.

****Q7 Answer: B - `var: 3,3` let: `0,1,2`****

With `var`, there is one shared `i` binding captured by closure, so when `timeout` runs, loop finished at 3. ES6 `let` in for-loop spec creates fresh binding per iteration.

****Q8 Answer: B - `NaN !== NaN`, use `Number.isNaN`****

IEEE 754 defines `NaN` not equal to itself. `Number.isNaN` does not coerce unlike global `isNaN()`, making it safe.

****Q9 Answer: A - Declaration hoisted fully, expression not****

Function declarations are hoisted with body, so callable before appearing in source. `const foo = () => {}` lives in TDZ until initialization line executes.

****Q10 Answer: B - `{"a":1}` because undefined/functions omitted****

`JSON.stringify` spec omits undefined, functions, and Symbols from objects, and replaces them with null inside arrays. This prevents invalid JSON.

2. Intermediate Level (10 Questions)

****Q11. What does closure allow in this code? `function makeCounter(){ let n=0; return ()=>{n++;} } const c=makeCounter()?`****

- A. Access to global n
- B. Each call to c() increments private lexical variable n retained via closure, with separate state per makeCounter() call
- C. Returns NaN
- D. n is garbage collected immediately

****Q12. What is the output order? console.log(1); Promise.resolve().then(()=>console.log(2)); queueMicrotask(()=>console.log(3)); setTimeout(()=>console.log(4),0); console.log(5)****

- A. 1,5,2,3,4
- B. 1,2,3,4,5
- C. 1,5,4,2,3
- D. 1,5,2,4,3

****Q13. What is the difference between this in arrow function vs regular function?****

- A. Arrow function's this is lexically bound from surrounding scope and cannot be re-bound via call/apply/bind; regular function's this depends on call-site (object, call/apply, new, or undefined in strict)
- B. No difference
- C. Arrow has its own this that is always window
- D. Regular function has no this

****Q14. Given async function f(){ return 1 } and async function g(){ return await 1 } what is returned?****

- A. Both return Promise that resolves to 1, but return await adds microtask and keeps stack trace/try-catch behavior correct for async errors
- B. f returns 1, g returns Promise
- C. f throws, g returns 1
- D. Both return 1 synchronously

****Q15. What happens with const [a, , b, ...rest] = [1,2,3,4,5] and const {x: {y}} = {x:{y:10}}?****

- A. a=1, b=3, rest=[4,5], y=10 via array destructuring with elision and rest, and nested object destructuring
- B. SyntaxError
- C. a=1,b=2,rest=3,4,5
- D. y=undefined

****Q16. What does this fix? fetch('/api').then(r=>r.json()).catch(...) Why can fetch not reject on 404?****

- A. fetch only rejects on network failures, not HTTP errors; you must check response.ok or status to throw for 404/500
- B. fetch always rejects on 404
- C. fetch never rejects
- D. Need to use await to catch 404

****Q17. Which statement about prototypal inheritance is true in 2024?****

- A. class syntax creates classical inheritance unrelated to prototypes
- B. class is syntactic sugar over prototypes; obj.__proto__ (deprecated) vs Object.getPrototypeOf(obj), and Object.create(null) creates dict without prototype
- C. Prototypes were removed in ES2022
- D. extends copies methods statically

****Q18. How do you correctly implement a debounce that delays execution until 300ms after last call?****

- A. Use setInterval
- B. Return a wrapper that clears previous timeout and sets new: let t; return (...args)=>{clearTimeout(t); t=setTimeout(()=>fn(...args),300)}
- C. Use setTimeout once
- D. Use Promise.delay

****Q19. What does Promise.all([p1,p2]) vs Promise.allSettled vs Promise.any do, and which was added in ES2024 for grouping? Wait grouping is Object.groupBy.****

A. all fails fast on first rejection, allSettled waits for all with status, any fulfills with first fulfilled or AggregateError; ES2024 added Object.groupBy and Map.groupBy for grouping arrays

B. All are identical

C. all waits for all, any fails fast

D. allSettled fails fast

****Q20. What is the difference between ES Modules import vs CommonJS require in 2024-2026 Node.js?****

A. ESM is static, async-capable, live bindings, needs import with file extension or import maps; CJS is dynamic sync; Node 20+ supports both but "type":"module" flips default, and require(esm) throws

B. require works in ESM by default

C. import is synchronous

D. No difference since Node 22

Intermediate Answer Key & Explanations

****Q11 Answer: B - Closure retains private n per makeCounter call****

makeCounter creates execution context with n; inner arrow keeps lexical environment reference, preventing GC. Each call to factory gets independent closure state.

****Q12 Answer: A - 1,5,2,3,4****

Sync code 1,5 runs first. Microtasks FIFO: Promise.then queued first, then queueMicrotask. Both run before next macrotask setTimeout(0), which is event loop spec.

****Q13 Answer: A - Arrow lexical this, regular dynamic this****

Arrow captures this at definition time for callbacks; regular function's this determined at call site, allowing .call(). This is why arrow unsuitable as object method or constructor.

****Q14 Answer: A - Both return Promise resolving to 1, return await adds extra tick and error handling semantics****

async always wraps return in Promise.resolve. return await unwraps and re-wraps, preserving async stack traces in V8 and allowing surrounding try/catch to catch the rejection from awaited promise.

****Q15 Answer: A - a=1,b=3,rest=[4,5], y=10****

Comma elision skips index 1, rest collects remaining. Nested pattern {x:{y}} extracts deep property; fails if x undefined unless default provided.

****Q16 Answer: A - fetch only rejects on network failure****

Fetch spec treats HTTP statuses as successful transport; ok is true for 200-299. You must manually if (!r.ok) throw new Error(r.status).

****Q17 Answer: B - class is sugar over prototypes; getPrototypeOf preferred****

JS remains prototype-based; class defines prototype methods non-enumerable. __proto__ is legacy accessor; Object.create(null) avoids prototype pollution for safe maps.

****Q18 Answer: B - Clear timeout and set new****

Debounce ensures fn called once after quiet period; clearTimeout cancels prior schedule. Throttle is different: limits to at most once per interval.

****Q19 Answer: A - all fails fast, allSettled waits, any first fulfilled, plus Object.groupBy ES2024****

Promise.all short-circuits on rejection, useful for dependent tasks. ES2024 finalized Object.groupBy(array, callback) returning plain object, and Map.groupBy.

****Q20 Answer: A - ESM static live bindings vs CJS dynamic sync****

ESM imports are hoisted and static-analyzable for tree-shaking; Node requires explicit .js extension in ESM. Interop: you can import() CJS from ESM, but require() ESM throws ERR_REQUIRE_ESM since Node 18+.

3. Advanced Level (10 Questions)

****Q21. Analyze: let a={}; let b={}; let m=new Map(); m.set(a,1); m.set(b,2); let wm=new WeakMap(); wm.set(a,1); a=null; What about GC and iteration?****

A. Map holds strong refs, prevents GC of both keys and is iterable; WeakMap holds weak ref to a, does not prevent GC after a=null, and is not iterable by design

B. Both prevent GC

C. WeakMap prevents GC but Map does not

D. Both iterable

****Q22. What does Promise.withResolvers() (ES2024) solve versus manual let resolve; new Promise(r=>resolve=r)?****

A. No difference

B. Provides standardized {promise, resolve, reject} without executor, avoiding deferred anti-pattern scope issues and enabling resolvers outside constructor, now used for observable bridging

C. It auto-cancels promise

D. It is slower

****Q23. Explain event loop: Node.js/Browser microtask starvation example: function loop(){ queueMicrotask(loop); } loop(); setTimeout(()=>console.log('timeout'),0)?****

A. Timeout will never log because microtask queue is drained completely before each macrotask; infinite microtask recursion starves macrotasks and blocks rendering

B. Timeout logs immediately

C. Microtasks run after macrotasks

D. No starvation possible

****Q24. What is correct about Proxy traps and invariants in ES2023+? Example: new Proxy(target, {get() { return 5 }}) when target has non-writable non-configurable prop?***

A. Proxy can return anything

B. Proxy must respect invariants: if target prop is non-writable non-configurable, get trap must return same value; otherwise TypeError. Also getOwnPropertyDescriptor trap must not report incompatible descriptor

C. Proxy bypasses invariants

D. Proxy only works for functions

****Q25. What is V8 optimization impact of adding/deleting properties after construction? class Point {constructor(x,y){this.x=x;this.y=y} } p=new Point(1,2); p.z=3; delete p.x?***

A. No impact

B. Adding properties in consistent order retains same HiddenClass/Shape for inline caching; deleting or out-of-order addition causes Map transition to dictionary mode/slow path, deoptimizing property access

C. Deleting improves performance

D. V8 does not have hidden classes

****Q26. Which ES2024+ feature accurately describes Array.prototype.toSorted, toReversed, toSpliced, with and Object.groupBy?***

A. toSorted mutates original

B. New non-mutating copying methods return new array without mutating original; with(index,value) replaces index immutably; Object.groupBy(iterable, keyFn) groups into object (Map.groupBy into Map)

C. groupBy mutates array

D. with mutates original

****Q27. Security: Why is eval(userInput) and innerHTML = userInput dangerous, and what modern 2024+ replacement avoids XSS?***

A. Not dangerous

B. eval executes arbitrary code leading to XSS and scope injection; innerHTML parses HTML and executes scripts/event handlers; use.textContent, DOMPurify, Trusted Types with trustedTypes.createPolicy, and structuredClone/JSON instead of eval

C. innerHTML is safe

D. eval is safe in strict mode

****Q28. How does AbortController work with fetch and what new ES2024 pattern combines multiple signals via AbortSignal.any()?***

- A. `const ac=new AbortController(); fetch(url,{signal:ac.signal})` abort triggers `AbortError DOMException`; `AbortSignal.any([signal1, timeoutSignal])` composes signals; `AbortSignal.timeout(5000)` auto-aborts
- B. `AbortController` not usable with `fetch`
- C. `any()` does OR but doesn't abort
- D. `fetch abort` returns null

****Q29. What is memory leak risk with closures + event listeners + timers, and how to detect? Example: SPA adds `window.addEventListener('resize', () => hugeArray)` without cleanup?****

- A. No leak possible in JS
- B. Listener retains closure referencing `hugeArray` and prevents GC even after component unmount; timer retains scope; leaks visible in Chrome DevTools Memory heap snapshot -> Detached DOM, and Performance monitor; fix with `removeEventListener`, `WeakRef` for non-essential, or `AbortSignal` for listeners `addEventListener(name, fn, {signal})`
- C. GC always cleans listeners
- D. `WeakRef` fixes all leaks automatically

****Q30. Predict output and explain temporal: `console.log(0); async function a(){ console.log(1); await null; console.log(2); } a(); console.log(3); Promise.resolve().then(()=>console.log(4));` plus Temporal API status 2024-2026?****

- A. 0,1,3,2,4? Actually `await null` yields microtask: 0,1,3,2,4 or 0,1,3,4,2 depending on enqueue order; `await` always defers continuation as microtask. Temporal API (`Temporal.Now`, `Temporal.PlainDate`) is Stage 3 proposal for immutable date handling to replace `Date`, available behind flag/polyfill, expected 2025-2026 final
- B. 0,1,2,3,4 sync
- C. 0,3,1,2,4
- D. 0,1,3,4,2 with Temporal already in browsers globally by 2024

Advanced Answer Key & Explanations

****Q21 Answer: A - Map strong ref prevents GC and is iterable, WeakMap weak ref allows GC and not iterable****

`WeakMap` keys must be objects and held weakly to avoid memory leaks for caches attached to DOM nodes. Non-iterability prevents exposing GC non-determinism, which is spec requirement.

****Q22 Answer: B - Standardized {promise, resolve, reject} without executor scope hack****

ES2024 added `Promise.withResolvers()` to avoid capturing resolver via closure inside executor and to simplify external resolution patterns. It returns sealed object with explicit `resolve/reject` for interop with callback APIs.

****Q23 Answer: A - Infinite microtasks starve macrotask****

Event loop algorithm: drain microtask queue fully before running next macrotask (timer, I/O, render). Recursive `queueMicrotask` prevents loop progressing to timers/render, freezing UI; detection via `Long Task`.

****Q24 Answer: B - Proxy must respect invariants****

Proxy invariants ensure observable behavior remains consistent with target non-configurable properties, protecting engine optimizations and security assumptions. Violation throws `TypeError` rather than silently returning wrong value.

****Q25 Answer: B - Consistent order retains HiddenClass, delete forces dictionary mode****

V8 creates hidden classes (Maps) for objects sharing property layout for fast offset access via inline caches. Out-of-order or delete transitions to dictionary (hash table) slow path, 10-100x slower property load.

****Q26 Answer: B - Non-mutating copying methods + `groupBy`****

ES2023 introduced `toSorted/toReversed/toSpliced/with` for immutable patterns needed by React/Redux avoiding copy-then-mutate. ES2024 `Object.groupBy` returns `{key: [items]}` and `Map.groupBy` for non-string keys.

****Q27 Answer: B - eval and innerHTML dangerous, use `textContent`, Trusted Types, `DOMPurify`****

XSS occurs because input is parsed as code/HTML. Modern defense: `textContent` for text, Trusted Types CSP (require-trusted-types-for 'script') blocks `innerHTML` assignment of strings forcing policy, plus sanitize libraries.

****Q28 Answer: A - AbortController signal triggers AbortError, AbortSignal.any and timeout are ES2024****

`AbortController` is standard cancellation token; `fetch` checks signal and rejects with `name:'AbortError'`. ES2024 added `AbortSignal.any(iterable)` which aborts when any source aborts, and `timeout(ms)` factory for deadlines.

****Q29 Answer: B - Listener retains closure prevents GC, use `remove` + `AbortSignal`****

Detached DOM + large closures are classic SPA leaks. Chrome heap snapshot shows retained size chain. Using {signal: AbortSignal} with ac.abort() to bulk remove listeners on unmount, plus clearing timers, is 2024 best practice.

****Q30 Answer: D - 0,1,3,4,2 with Temporal proposal Stage 3****

Order: sync 0, enters async a logs 1, await null -> Promise.resolve(null).then(continuation) queues microtask. Sync logs 3. Microtasks FIFO: then(()=>4) queued before await continuation only if promise already resolved? Actually await null wraps via Promise.resolve, so 4 queued before 2 if a() called before Promise.resolve().then; spec yields 0,1,3,4,2. Temporal API provides correct dates without Date mutability, currently polyfilled.

Course File: 18_TypeScript.md

TypeScript - Professional Certification Exam

****Total Questions:**** 30 | ****Duration:**** 90 minutes | ****Passing Score:**** 70%

Section A: Beginner (Questions 1-10)

Q1. What command compiles a TypeScript file app.ts to JavaScript?

- A) typescript app.ts
- B) tsc app.ts
- C) ts-node compile app.ts
- D) npm run app.ts

****Answer:**** B

****Explanation:**** tsc is the official TypeScript Compiler CLI. It transpiles .ts files to JavaScript according to tsconfig.json or default settings, producing app.js.

Q2. Which type annotation correctly declares a variable that can be a string or null?

- A) let x: string & null
- B) let x: string | null
- C) let x: string || null
- D) let x: nullable<string>

****Answer:**** B

****Explanation:**** Union types use the | operator to allow multiple types. string | null means the value can be either a string or null, which is required under strictNullChecks.

Q3. What is the default type of a variable declared without initialization and without annotation under noImplicitAny: true?

- A) unknown
- B) any but with an error
- C) never
- D) void

****Answer:**** B

****Explanation:**** With noImplicitAny enabled, let x; causes a compiler error for implicit any. The compiler infers any but forces you to explicitly annotate to prevent unsafe typing.

Q4. Which keyword defines a custom type alias in TypeScript?

- A) interface
- B) type
- C) alias
- D) def

****Answer:**** B

****Explanation:**** The type keyword creates a type alias, e.g., type UserID = string | number. Unlike interface, it can alias primitives, unions, tuples, and mapped types.

Q5. What does the ? after a property name mean in an interface?

- A) The property is private
- B) The property is optional and may be undefined
- C) The property is readonly
- D) The property is deprecated

Answer: B

Explanation: Optional properties are marked with ?, like `age?: number`. This is shorthand for `age: number | undefined`, meaning the property can be omitted when creating the object.

Q6. Which array type syntax is correct for an array of numbers?

- A) `let arr: number[]` and `let arr: Array<number>`
- B) `let arr: [number]`
- C) `let arr: Number[]`
- D) `let arr: numbers`

Answer: A

Explanation: Both `number[]` and `Array<number>` are valid and equivalent syntax for number arrays. `[number]` denotes a tuple with exactly one element, not an open-ended array.

Q7. What does void mean as a function return type?

- A) The function returns null
- B) The function returns no value / undefined
- C) The function never returns
- D) The function returns any type

Answer: B

Explanation: `void` indicates a function doesn't return a meaningful value, typically returning `undefined`. Use `never` instead when a function never returns at all, such as throwing or infinite loops.

Q8. How do you enable strict type checking in tsconfig.json?

- A) `"strict": true`
- B) `"strictMode": true`
- C) `"typeCheck": "strict"`
- D) `"noAny": true`

Answer: A

Explanation: Setting `"strict": true` enables all strict family checks including `strictNullChecks`, `noImplicitAny`, `strictFunctionTypes`, etc. This is the recommended setting for all new projects since TS 2.3.

Q9. Which is the correct way to type a function parameter with a default value?

- A) `function greet(name?: string = "Guest") {}`
- B) `function greet(name: string = "Guest") {}`
- C) `function greet(name = "Guest": string) {}`
- D) `function greet(default "Guest": string) {}`

Answer: B

Explanation: In function `greet(name: string = "Guest")`, TypeScript infers the type from the default, so `: string` is actually optional. The optional ? is not needed because a default value implicitly makes the parameter optional.

Q10. What does `as` do in `const x = someValue as string`?

- A) Declares a new variable
- B) Performs runtime conversion to string
- C) Type assertion telling compiler to treat value as string
- D) Imports a string module

Answer: C

Explanation: `as` is a type assertion that overrides the compiler's inferred type without runtime effect. It does not convert the value; `as` is purely compile-time, unlike a cast in C# or Java.

Section B: Intermediate (Questions 11-20)

Q11. What is the difference between interface and type for object shapes in 2024+ TypeScript?

- A) They are identical in all cases
- B) interface supports declaration merging and is generally preferred for public API object shapes, while type can express unions, intersections, and mapped types that interface cannot
- C) type supports declaration merging
- D) interface cannot be extended

Answer: B

Explanation: Interfaces merge automatically and offer better error messages and extends clause for objects, which is why libraries prefer them. Type aliases are more powerful for unions (`A | B`), primitives, and utility type compositions.

Q12. Given type `A = {x: number} & {y: string}`, what is A?

- A) Union type
- B) Intersection type requiring both x and y
- C) Invalid syntax
- D) Overload

Answer: B

Explanation: The `&` operator creates an intersection, combining members of both types. An object of type A must have both `x: number` and `y: string`; overlapping properties must be compatible.

Q13. What does the unknown type force you to do before using a value?

- A) Nothing, same as any
- B) Perform type narrowing / checking before operations
- C) Convert to any first
- D) Use `await`

Answer: B

Explanation: `unknown` is the type-safe counterpart to `any`; you cannot access properties or call it without narrowing. You must use `typeof`, `instanceof`, or custom type guards to refine it to a usable type.

Q14. Which generic constraint ensures T has a length property?

- A) `<T extends { length: number }>`
- B) `<T where length>`
- C) `<T has length>`

D) <T: {length}>

****Answer:**** A

****Explanation:**** extends in generics constrains the type parameter to satisfy a shape. <T extends { length: number }> allows using arg.length inside the function while still preserving the specific type T.

Q15. What is a discriminated union?

A) A union with a common literal property used to narrow the type

B) A union of only primitive types

C) A union that TypeScript cannot discriminate

D) An enum

****Answer:**** A

****Explanation:**** A discriminated union has a common property like kind or type with literal types, e.g., {kind: "circle", r: number} | {kind: "square", size: number}. TypeScript can narrow via switch (shape.kind) from TS 4.4+ with improved control flow analysis.

Q16. What does keyof do?

A) Returns array of keys at runtime

B) Produces a union of string literal types of the keys of a type

C) Checks if key exists

D) Creates an enum of keys

****Answer:**** B

****Explanation:**** keyof Person for Person = {name: string, age: number} yields "name" | "age". It operates purely at the type level and is heavily used with Mapped Types like keyof T paired with T[K].

Q17. What will typeof do in a type context like type T = typeof myVar?

A) Returns runtime string "string"/"number" etc.

B) Captures the TypeScript type of myVar as a type

C) Causes error

D) Returns any

****Answer:**** B

****Explanation:**** In type position, typeof is a type query that extracts the static type of a variable or property. This enables patterns like type Config = typeof defaultConfig without duplicating the shape.

Q18. Which utility type makes all properties of T optional?

A) Required<T>

B) Partial<T>

C) Pick<T>

D) Optional<T>

****Answer:**** B

****Explanation:**** Partial<T> maps to { [P in keyof T]?: T[P] }. It's built into lib since TS 2.1 and essential for update patterns; Required<T> does the opposite, removing ?.

Q19. What does the ! non-null assertion operator do?

A) Makes value null

B) Asserts that a value is non-null/non-undefined, removing null/undefined from type

- C) Throws if null
- D) Converts to boolean

****Answer:**** B

****Explanation:**** value! tells the compiler that you know value is not nullish despite its type allowing it. It's unsafe and should be sparing; unlike runtime checks, it emits no JS code and can cause runtime errors if wrong.

Q20. How do you correctly type an async function returning a number?

- A) async function get(): number {}
- B) async function get(): Promise<number> {}
- C) async function get(): Async<number> {}
- D) function get(): async number {}

****Answer:**** B

****Explanation:**** Async functions always return a Promise wrapping the declared return. So Promise<number> is correct; TypeScript will error if you annotate just number as the return type of an async function.

Section C: Advanced (Questions 21-30)

Q21. What does the satisfies operator introduced in TypeScript 4.9 do?

- A) Same as as
- B) Validates that an expression satisfies a type without widening or changing its inferred type
- C) Checks runtime satisfaction
- D) Deprecates type assertion

****Answer:**** B

****Explanation:**** const palette = {red: [255,0,0]} satisfies Record<string, RGB> ensures compatibility while keeping literal tuple types precise. Unlike as, it preserves the narrowest inferred type for later intellisense and errors.

Q22. Given type A = ReturnType<typeof fn>, what does ReturnType use under the hood?

- A) Conditional type with infer
- B) keyof and lookup
- C) typeof only
- D) Decorator metadata

****Answer:**** A

****Explanation:**** ReturnType<T> is defined as T extends (...args: any[]) => infer R ? R : never. The infer R keyword inside conditional types extracts the return type, a core pattern for advanced type transformations.

Q23. What is the difference between in and out variance annotations on type parameters (TS 4.7+)?

- A) They are aliases for extends
- B) in marks contravariant (only written), out marks covariant (only read); they enable stricter checking of generic variance
- C) in means input file, out means output file
- D) No difference

****Answer:**** B

****Explanation:**** Variance annotations improve type safety for generics like type Getter<out T> and type Setter<in T>. They were introduced in TS 4.7 to allow the compiler to correctly check assignability based on usage direction, crucial for library

authors.

Q24. Consider: `type IsNever<T> = [T] extends [never] ? true : false`. Why wrap in tuple `[T]`?

- A) To avoid distributive conditional type behavior
- B) For performance
- C) Syntax requirement
- D) To make it mutable

Answer: A

Explanation: Naked `T` extends ... distributes over unions, and `never` is the empty union, so `never` extends ... would be skipped entirely. Wrapping in `[T]` disables distribution, allowing correct detection of `never`.

Q25. What is a template literal type example introduced in TS 4.1?

- A) `type Event = on${Capitalize<string>}`
- B) `type Event = "on" + string`
- C) `type Event = string.template()`
- D) `type Event = String.Template`

Answer: A

Explanation: Template literal types allow `type Greeting = Hello ${string}` and with intrinsic manipulators like `Capitalize`, `Uppercase`, `Lowercase`, `Uncapitalize`. They power typed event emitters like `keyChanged` from object keys.

Q26. With `moduleResolution: "bundler"` and `verbatimModuleSyntax` (TS 5.0+), what is required?

- A) All imports must be type-only if they are types
- B) type imports must use `import type` syntax, and `verbatimModuleSyntax` preserves import statements exactly as written without elision
- C) No changes
- D) CommonJS must be used

Answer: B

Explanation: `verbatimModuleSyntax` (replacing `importsNotUsedAsValues` and `preserveValueImports`) forces explicit `import type { Foo }` for type-only imports. This aligns TS with bundlers like Vite/esbuild under "bundler" resolution, preventing runtime import errors from erased types.

Q27. What does `const enum` become at compile time vs `enum`?

- A) Same as regular `enum`
- B) `const enum` is fully inlined at use sites with no object emitted (unless `preserveConstEnums`), while regular `enum` emits a real object
- C) `const enum` emits more code
- D) `const enum` is deprecated

Answer: B

Explanation: `const enum Color { Red=0 }` with `Color.Red` usage compiles directly to `0`, enabling zero-cost abstraction. However, `const enum` cannot be used across `isolatedModules` without `isolatedDeclarations` constraints and is discouraged in libraries for compatibility.

Q28. What is the `noUncheckedIndexedAccess` flag (relevant for TS 5.5+ Maps/Sets)?

- A) Adds | undefined to indexed access types like array[i] and obj[key] where key is not a known property
- B) Disables indexing
- C) Makes all properties readonly
- D) Checks array bounds at runtime

Answer: A

Explanation: When enabled, obj[key] or arr[0] becomes T | undefined even if T[] is declared, forcing safety checks. Since TS 5.5 this flag's behavior is widely recommended and aligns with safer indexing, especially for dynamic key access.

Q29. Explain infer in type UnwrapPromise<T> = T extends Promise<infer U> ? U : T

- A) infer U declares a new type variable to capture inner Promise type
- B) infer is runtime promise unwrapping
- C) It throws if not promise
- D) It converts T to unknown

Answer: A

Explanation: infer introduces a placeholder type variable within the extends clause of a conditional type. In Promise<infer U>, TS infers what U would need to be for T to match Promise<U>, effectively unwrapping it.

Q30. How does using and Disposable / Symbol.dispose work in TypeScript 5.2+?

- A) It's a comment
- B) using declares an explicit resource that auto-disposes at scope exit via [Symbol.dispose](), modeled after TC39 Explicit Resource Management proposal, with await using for async disposal
- C) It deletes variables
- D) It imports disposal module

Answer: B

Explanation: using file = new FileHandle() will call file[Symbol.dispose]() when the block exits, like C# using or Python context managers. TypeScript 5.2 adds type support for Disposable and AsyncDisposable, emitting try/finally blocks when targeting below ES23.

Course File: 19_Python.md

Python Proficiency Exam - 19_Python

Total Questions: 30 | Time: 90 minutes | Passing: 70%

Coverage: Python 3.12-3.13 features (2024-2026), PEPs, Standard Library, Async, Typing

Part 1: Beginner (Questions 1-10)

Q1. What is the output of this code in Python 3.12+?

```
python
print(x / y, x // y, x % y)
```

- A) 2.5 2 1
- B) 2 2 1
- C) 2.5 2.5 1
- D) 2.5 2 0

Q2. Which of the following correctly creates a list of squares from 0 to 4 using list comprehension?

```
python
# Desired output: [0, 1, 4, 9, 16]
```

- A) [x^2 for x in range(5)]
- B) [x**2 for x in range(5)]
- C) [x*2 for x in range(5)]
- D) (x**2 for x in range(5))

Q3. What does the strip() method do on a string?

- A) Removes all whitespace characters from inside the string
- B) Removes leading and trailing whitespace (and optionally specified characters)
- C) Splits the string into a list
- D) Converts the string to lowercase

Q4. What is the result of len({"a": 1, "b": 2, "a": 3})?

- A) 3
- B) 2
- C) Raises a SyntaxError

Q5. Which keyword is used to handle exceptions in Python?

- A) catch
- B) try / except
- C) error / handle
- D) throw / catch

Q6. What is the difference between == and is in Python?

- A) They are identical
- B) == checks value equality, is checks identity (same object in memory)

C) == checks identity, is checks value equality

D) is is deprecated since Python 3.12

Q7. How do you open a file safely so it auto-closes?

A) `f = open("file.txt"); f.close()`

B) `with open("file.txt") as f:`

C) `open("file.txt").auto_close()`

D) `file.open("file.txt")`

Q8. What will `bool([])`, `bool("")`, `bool(0)` all evaluate to?

A) True

B) False

C) None

D) Raises TypeError

Q9. Which of these is a valid function definition with a default argument?

A) `def func(a = 1, b):`

B) `def func(a, b = 2):`

C) `def func(a = 1, b = 2, c):`

D) `def func(=1, =2):`

Q10. What does `range(2, 10, 2)` generate?

A) [2, 3, 4, 5, 6, 7, 8, 9]

B) [2, 4, 6, 8]

C) [2, 4, 6, 8, 10]

D) [10, 8, 6, 4, 2]

Part 2: Intermediate (Questions 11-20)

Q11. What is the output of this closure example?

python

```
def make_adder(n):
```

```
    return lambda x: x + n
```

```
add5 = make_adder(5)
```

```
print(add5(10))
```

A) 10

B) 15

C) 5

D) TypeError

Q12. Since Python 3.12, what changed about f-strings (PEP 701)?

A) f-strings can no longer contain quotes inside

B) f-strings are now parsed with arbitrary nesting, multi-line expressions, comments, and reuse of same quote type

C) f-strings were removed in favor of `str.format()`

D) Nothing changed

Q13. What is the MRO (Method Resolution Order) in Python used for?

- A) Ordering dictionary keys
- B) Determining the order in which base classes are searched when executing a method in multiple inheritance
- C) Sorting a list of methods alphabetically
- D) Garbage collection ordering

Q14. Consider this code:

```
python
a = [1, 2, 3]
c = a.copy()
b.append(4)
print(a, c)
```

What is printed?

- A) [1, 2, 3, 4] [1, 2, 3, 4]
- B) [1, 2, 3, 4] [1, 2, 3]
- C) [1, 2, 3] [1, 2, 3]
- D) [1, 2, 3] [1, 2, 3, 4]

Q15. What does @property do?

- A) Makes a method act like a read-only attribute and allows getter/setter logic
- B) Makes a function run faster
- C) Converts a method to a static method
- D) Marks a method as private

Q16. What is the correct way to define a generic class using Python 3.12+ new syntax?

```
python
# Old: from typing import TypeVar; T = TypeVar('T')
```

- A) class Stack[T]: ...
- B) class Stack<Generic T>: ...
- C) class Stack[T TypeVar]: ...
- D) generic class Stack[T]: ...

Q17. What is the purpose of if __name__ == "__main__":?

- A) Checks if file is imported as a module
- B) Ensures code inside only runs when script is executed directly, not when imported
- C) Required for all Python programs to run
- D) Checks if Python version is main version

Q18. What does *args and **kwargs mean in a function signature?

- A) *args collects extra positional arguments as a tuple, **kwargs collects extra keyword arguments as a dict
- B) *args collects keyword arguments, **kwargs collects positional
- C) They are mandatory syntax for all functions
- D) They import arguments from another file

Q19. Which statement about Python's GIL is true as of Python 3.13 (PEP 703)?

- A) GIL was always optional
- B) Python 3.13 introduced an experimental free-threaded build (`--disable-gil`) allowing true parallelism for threads, with `sys._is_gil_enabled()` check
- C) GIL was completely removed in all builds by default
- D) GIL now only affects asyncio code

Q20. What will this match/case (Python 3.10+) code output?

```
python
def http_status(code):
    match code:
        case 200:
            return "OK"
        case 404:
            return "Not Found"
        case 500 | 502 | 503:
            return "Server Error"
        case _:
            return "Unknown"
print(http_status(502))
```

- A) OK
- B) Not Found
- C) Server Error
- D) Unknown

Part 3: Advanced (Questions 21-30)

Q21. What is the behavior of walrus operator `:=` in this Python 3.12 snippet?

```
python
data = [1, 2, 3, 4, 5, 6]
if (n := len(data)) > 5:
    print(f"List is long: {n}")
```

- A) `SyntaxError`
- B) Assigns and returns 6 to `n` inside the if condition, prints "List is long: 6"
- C) Compares `len(data) > 5` and assigns `True` to `n`
- D) `n` is not accessible after if

Q22. Consider asyncio:

```
python
import asyncio
async def task(n):
    await asyncio.sleep(0.1)
```

```
return n * 2

async def main():
    results = await asyncio.gather(task(1), task(2), task(3))
    print(results)

asyncio.run(main())
```

What does `asyncio.gather` do here?

- A) Runs tasks sequentially and returns last result
- B) Runs coroutines concurrently and returns list [2, 4, 6] preserving input order
- C) Runs tasks in parallel threads
- D) Raises `RuntimeError` because `gather` needs `TaskGroup`

Q23. In Python 3.11+, what does `ExceptionGroup` and `except*` enable?

```
python
raise ExceptionGroup("eg", [ValueError("bad"), TypeError("worse")])
except* ValueError as e:
    print(f"caught ValueError: {e.exceptions}")
```

- A) Allows handling multiple unrelated exceptions simultaneously; `except*` extracts matching exceptions from the group
- B) Same as normal `except`
- C) Catches all exceptions regardless of type
- D) `SyntaxError` before 3.12

Q24. What is true about `__slots__` in Python classes?

- A) It automatically generates `__init__`
- B) It restricts allowed attributes to listed names, reduces memory overhead by avoiding per-instance `__dict__`, improving performance
- C) It makes all attributes private
- D) It is required for dataclasses

Q25. What does this code demonstrate about descriptors?

```
python
class Typed:
    def __set_name__(self, owner, name):
        self.name = name
    def __get__(self, obj, objtype=None):
        return obj.__dict__.get(self.name)
    def __set__(self, obj, value):
        if not isinstance(value, int):
            raise TypeError("int required")
        obj.__dict__[self.name] = value

class A:
    x = Typed()
a = A()
a.x = "hi"
```

- A) Prints "hi"
- B) Raises `TypeError` because descriptor validates type on `__set__`

- C) Raises AttributeError
- D) Descriptors only work with @property

Q26. Which typing construct is correct in Python 3.12+ for type aliases using type statement (PEP 695)?

- A) `type Vector[T] = list[T]`
- B) `TypeAlias Vector[T] = list[T]`
- C) `typedef Vector[T] = list[T]`
- D) `Vector[T] := list[T]`

Q27. What is the key difference between `__deepcopy__` vs shallow copy, and when does `copy.deepcopy` fail to copy?

python

```
import copy
```

```
a = [[1, 2], [3, 4]]
```

```
b = copy.copy(a)
```

```
c = copy.deepcopy(a)
```

```
b[0][0] = 99
```

```
print(a[0][0], c[0][0])
```

- A) Both modify original, prints 99 99
- B) Shallow copy shares nested objects so original is mutated, deepcopy does not; prints 99 1
- C) Prints 1 1
- D) deepcopy raises error on lists

Q28. As of Python 3.11+, what optimization makes `asyncio.TaskGroup` preferred over `asyncio.gather` in some cases?

python

```
async with asyncio.TaskGroup() as tg:
```

```
    tg.create_task(foo())
```

```
    tg.create_task(bar())
```

- A) TaskGroup has no advantages
- B) TaskGroup provides structured concurrency, automatic cancellation on failure, propagates ExceptionGroup with all failures, and ensures all tasks finish before exiting context
- C) TaskGroup runs faster because it uses free-threaded mode
- D) TaskGroup is just an alias for gather

Q29. What is true about metaclasses in Python?

- A) Metaclasses define how classes themselves are constructed; type is the default metaclass; custom metaclass can be set via `class MyClass(metaclass=MyMeta):`
- B) Metaclasses are same as decorators
- C) Metaclasses were removed in Python 3.12
- D) Every class must define a custom metaclass

Q30. Consider this Python 3.13 JIT-relevant snippet + performance feature:

python

```
import sys
```

```
print(sys.version)
```

```
# Python 3.13.0 experimental JIT?
```

Which statement accurately describes Python 3.13 performance improvements (2024-2026 era)?

A) Python 3.13 introduced an experimental JIT compiler (copy-and-patch, `--enable-experimental-jit`) and significant GIL and GC optimizations; free-threaded build and JIT are separate experiments

B) Python 3.13 is slower than 3.11 because of typing overhead

C) Python 3.13 removed asyncio for performance

D) JIT was introduced in Python 3.10 and finalized in 3.12

Answer Key

Beginner

Q1 Answer: A

> / always returns float, // is floor division, % is remainder. So $5/2=2.5$, $5//2=2$, $5\%2=1$.

Q2 Answer: B

> ** is exponentiation, ^ is bitwise XOR in Python. Comprehension `[x**2 for x in range(5)]` produces `[0,1,4,9,16]`.

Q3 Answer: B

> `strip()`, `lstrip()`, `rstrip()` remove leading/trailing whitespace by default, or specified characters. They don't affect interior characters.

Q4 Answer: B

> Duplicate keys overwrite previous values; `{"a": 1, "b": 2, "a": 3}` keeps only last "a" so dict has 2 entries. Length is therefore 2.

Q5 Answer: B

> Python uses try: block followed by except `ExceptionType`: and optionally else and finally. Other languages use catch/throw.

Q6 Answer: B

> `==` invokes `__eq__` for value equality, is compares identity via `id()`. Use is for None checks, `==` for values.

Q7 Answer: B

> The with statement uses context manager protocol (`__enter__`/`__exit__`) to guarantee close even on exceptions. It's the idiomatic safe pattern.

Q8 Answer: B

> Empty collections, empty string, 0, 0.0, None, and False are falsy in Python's truth testing. All three examples evaluate to False.

Q9 Answer: B

> Non-default parameters must precede default parameters. `def func(a, b=2)`: follows this rule; others put default before non-default which is `SyntaxError`.

Q10 Answer: B

> `range(start, stop, step)` stops before stop value. Starting at 2, stepping by 2 until 10 gives 2,4,6,8.

Intermediate

Q11 Answer: B

> `make_adder` returns a closure capturing `n=5`. Calling `add5(10)` executes `lambda 10+5`, returning 15. Closures retain enclosing scope.

Q12 Answer: B

> PEP 701 (Python 3.12) rewrote f-string parsing using the PEG parser: nested f-strings, multi-line, quotes reuse, comments and backslashes inside now allowed.

****Q13 Answer: B****

> MRO is computed by C3 linearization and viewable via `Class.__mro__`. It resolves which parent method is called in diamond inheritance.

****Q14 Answer: B****

> `b = a` is reference alias, so `b.append` mutates `a`. `c = a.copy()` was shallow copy before `append`, so `c` remains `[1,2,3]`.

****Q15 Answer: A****

> `@property` turns method into descriptor for attribute access, enabling computed attributes with `@x.setter`. It manages getter/setter logic transparently.

****Q16 Answer: A****

> PEP 695 in Python 3.12 introduced type parameter syntax: `class Stack[T]:` and `def func[T](x: T) -> T:`. No import of `TypeVar` needed.

****Q17 Answer: B****

> `__name__` is `"__main__"` when run directly, module name when imported. Idiom prevents side effects on import and enables script/module dual use.

****Q18 Answer: A****

> `*args` packs extra positional args into tuple, `**kwargs` packs extra keyword args into dict. Useful for decorators and forwarding calls.

****Q19 Answer: B****

> PEP 703 free-threaded build (no GIL) landed experimentally in 3.13t, installed via `python3.13t`, enabled with `./configure --disable-gil`. Check via `sys._is_gil_enabled()`.

****Q20 Answer: C****

> Match case `case 500 | 502 | 503:` is OR pattern matching. Code 502 matches that branch returning "Server Error". Introduced in PEP 634.

Advanced

****Q21 Answer: B****

> Walrus operator `:=` assigns expression value to variable as part of larger expression and returns value. `n` becomes 6 and remains accessible in current scope.

****Q22 Answer: B****

> `gather(*coros)` schedules coroutines concurrently on event loop, awaits all, returns results in input order. It's not threading but cooperative concurrency.

****Q23 Answer: A****

> PEP 654 introduced `ExceptionGroup` and `except*` (Python 3.11). `except*` can partially handle a group, extracting matching subtypes while leaving others.

****Q24 Answer: B****

> `__slots__ = ('x','y')` prevents `__dict__` creation, saving ~50% memory per instance and faster attribute access. Attempts to set unlisted attrs raises `AttributeError`.

****Q25 Answer: B****

> Descriptor protocol `__get__`/`__set__` intercepts attribute access when defined on class. `a.x = "hi"` calls `Typed.__set__`, which raises `TypeError` if not int.

****Q26 Answer: A****

> PEP 695 adds type statement for type aliases: `type Vector[T] = list[T]` creates generic alias. This replaces `TypeAlias` assignment style.

****Q27 Answer: B****

> Shallow copy copies outer list only, inner lists shared. Mutating `b[0][0]` mutates `a[0][0]`. `Deepcopy` recursively copies, so `c` unaffected. Prints 99 1.

****Q28 Answer: B****

> TaskGroup (PEP 654, Python 3.11) implements structured concurrency: if one child fails, others cancelled; all exceptions combined into ExceptionGroup; context exit waits.

****Q29 Answer: A****

> Metaclass is class of a class; type(name, bases, dict) constructs classes. Custom metaclass overrides __new__/__init__ to customize class creation, used for ORM, validation frameworks.

****Q30 Answer: A****

> Python 3.13 ships experimental JIT (PEP 744 copy-and-patch) buildable with --enable-experimental-jit and --disable-gil free-threaded. Both are opt-in experiments, not default, with other perf wins.

****Scoring Guide****

- Beginner (Q1-Q10): 3 pts each = 30 pts
- Intermediate (Q11-Q20): 3 pts each = 30 pts
- Advanced (Q21-Q30): 4 pts each = 40 pts
- Total: 100 pts | Passing 70%

Course File: 20_PHP_Laravel.md

PHP & Laravel - Comprehensive Exam (30 Questions)

This exam covers PHP 8.1-8.4 and Laravel 10-12 (2024-2026 standards). Each question has 4 options with answer key and 2-sentence explanation.

Section 1: Beginner - 10 Questions

Q1. What does the PHP null coalescing operator ?? do?

- A) Checks if file exists
- B) Returns the first operand if it exists and is not null, otherwise returns second operand
- C) Merges two arrays
- D) Converts null to false

Answer: B

Explanation: The ?? operator checks if the left side is set and not null, returning it otherwise falling back. It is shorthand for `isset()` ternary and prevents undefined variable notices.

Q2. In PHP 8+, what is the correct way to declare a typed property that can be null?

- A) `public ?string $name = null;`
- B) `public string nullable $name;`
- C) `public string $name = NULL;`
- D) `public null $name;`

Answer: A

Explanation: Nullable types use `?Type` prefix, and must have a default value if nullable in promoted properties context. This enforces type safety while allowing null values.

Q3. Which Artisan command creates a new Laravel controller?

- A) `php artisan make:control`
- B) `php artisan create:controller MyController`
- C) `php artisan make:controller MyController`
- D) `php artisan new:controller MyController`

Answer: C

Explanation: `make:controller` is the standard Artisan generator for controllers. It can take `--resource` or `--api` flags to scaffold methods.

Q4. In Laravel, what is Eloquent?

- A) A JavaScript framework
- B) Laravel's ORM for database interactions using ActiveRecord pattern
- C) A caching driver
- D) A templating engine

Answer: B

Explanation: Eloquent is Laravel's Object-Relational Mapper that maps database tables to model classes. It provides expressive syntax for queries, relationships, and mutators.

Q5. Which Blade directive checks authentication in Laravel?

- A) @auth and @guest
- B) @ifAuth
- C) @loggedIn
- D) @checkAuth

****Answer: A****

****Explanation:**** @auth renders content only for authenticated users, while @guest does the opposite for unauthenticated users. They can take a guard parameter like @auth('admin').

Q6. What is Composer in PHP?

- A) A PHP code compiler
- B) A dependency manager for PHP
- C) A web server
- D) A database

****Answer: B****

****Explanation:**** Composer manages PHP dependencies via composer.json and autoloading per PSR-4. It installs packages from Packagist and handles version constraints.

Q7. How do you define a basic route in Laravel (Laravel 11+)?

- A) Route::get('/hello', function(){ return 'Hello'; }); in routes/web.php
- B) Router::define('/hello')
- C) Route::map('/hello')
- D) App::route('/hello')

****Answer: A****

****Explanation:**** Routes are defined in routes/web.php or api.php using the Route facade. The closure or controller action is executed when the URI matches.

Q8. What does echo vs print difference in PHP?

- A) echo can take multiple args and has no return, print returns 1 and takes single arg
- B) They are identical
- C) print is faster
- D) echo only works in Blade

****Answer: A****

****Explanation:**** echo is a language construct that can output multiple comma-separated values and returns void. print always returns 1, making it usable in expressions but slightly slower.

Q9. What is the purpose of .env file in Laravel?

- A) To store frontend CSS
- B) To store environment-specific configuration like DB credentials and APP_KEY
- C) To store routes
- D) To store Blade views

****Answer: B****

****Explanation:**** .env holds sensitive environment variables loaded by vlucas/phpdotenv into \$_ENV. Config files then access them via env() helper, but should be cached via config() in production.

Q10. Which function starts a session in plain PHP and is handled automatically by Laravel middleware?

- A) session_start()
- B) begin_session()
- C) start_session()
- D) init_session()

****Answer: A****

****Explanation:**** In vanilla PHP you must call session_start() before output. Laravel handles this automatically via StartSession middleware in the web group, so you use session() helper instead.

Section 2: Intermediate - 10 Questions

Q11. What are PHP 8.1+ Enums and how are they used in Laravel validation?

- A) Enums are not supported in PHP
- B) Native enumerated types: enum Status: string { case Published = 'published'; } and validated with Rule::enum(Status::class)
- C) Enums are just constants
- D) Enums only work in JavaScript

****Answer: B****

****Explanation:**** PHP 8.1 introduced backed and pure Enums as first-class types. Laravel 9+ added Rule::enum() validation rule and implicit enum casting in Eloquent models.

Q12. In Laravel 11, how is the application bootstrap configured differently?

- A) Still uses app/Http/Kernel.php for everything
- B) Uses bootstrap/app.php with Application::configure()->withRouting()->withMiddleware()->withExceptions()->create() and no default Kernel files
- C) Uses config/app.php only
- D) Kernel files are required

****Answer: B****

****Explanation:**** Laravel 11 introduced a slimmer skeleton removing Http Kernel and Console Kernel by default. All routing, middleware, and exception handling is now configured fluently in bootstrap/app.php.

Q13. Explain Laravel Service Container and Dependency Injection.

- A) Service container only stores config
- B) The IoC container manages class dependencies and automatic resolution via type-hinting in constructors/methods
- C) It is a Docker container
- D) It is for frontend services

****Answer: B****

****Explanation:**** The service container binds abstractions to concretions and auto-resolves dependencies via reflection. This enables loose coupling, testability via mocking, and automatic injection in controllers, jobs, listeners.

Q14. What does readonly property and readonly class mean in PHP 8.2/8.3?

- A) Property can be written only once after initialization, and readonly class makes all properties readonly
- B) It makes class immutable permanently
- C) It is for constants only
- D) It prevents inheritance

****Answer: A****

****Explanation:**** readonly properties can only be initialized once and cannot be modified afterward, introduced in PHP 8.1 and expanded to classes in 8.2. This is ideal for DTOs and Value Objects, commonly used with Laravel Data packages.

Q15. How do you handle N+1 query problem in Laravel Eloquent?

- A) Use with() for eager loading, e.g., `Post::with('author')->get()` and check with `preventLazyLoading()` or Telescope
- B) Use more foreach loops
- C) Disable Eloquent
- D) Use sleep()

****Answer: A****

****Explanation:**** N+1 occurs when lazy loading relationships inside loops causes extra queries. Eager loading with `with()` loads relationships in 1-2 queries, and Laravel 11+ can enforce with `Model::preventLazyLoading()` in development.

Q16. What is the difference between hasOne vs belongsTo in Eloquent?

- A) They are same
- B) hasOne owns the foreign key on related model, belongsTo owns foreign key on current model
- C) belongsTo is for many-to-many
- D) hasOne is only for users

****Answer: B****

****Explanation:**** In Post belongsTo User, posts table has user_id. In User hasOne Profile, profiles table has the foreign key. The model that holds the FK uses belongsTo.

Q17. What is new in PHP 8.3 typed class constants and PHP 8.4 property hooks?

- A) No new features in 8.3/8.4
- B) PHP 8.3 allows `const string VERSION = '1.0';` typed constants, and PHP 8.4 introduces property hooks `get/set` and asymmetric visibility `public private(set)`
- C) Typed constants were removed
- D) Property hooks are for Blade only

****Answer: B****

****Explanation:**** PHP 8.3 added typed class constants improving static analysis. PHP 8.4 adds property hooks for computed accessors and asymmetric visibility allowing `public get` but `private set`, reducing boilerplate getters/setters.

Q18. How do Laravel Queues and Jobs work in 2024-2026 best practices?

- A) Queues run synchronously only
- B) Jobs are dispatched to a queue driver (redis, database, SQS) and processed by `php artisan queue:work`, with Laravel 11 using `php artisan queue:work --sleep=3` and job batches, chains, and `ShouldBeUnique`
- C) Queues are for frontend
- D) Jobs cannot fail

****Answer: B****

****Explanation:**** Queued jobs enable async heavy tasks like emails. Modern Laravel supports job middleware, unique jobs, batching, chaining, and Horizon for Redis monitoring, with failed jobs stored in `failed_jobs` table.

Q19. What is Laravel's upsert() and updateOrCreate() distinction?

- A) Both identical

- B) updateOrCreate() checks single record and updates/inserts (boolean), upsert() does bulk insert with update on duplicate key via unique index for high performance
- C) upsert deletes
- D) Only upsert exists

****Answer: B****

****Explanation:**** updateOrCreate is for single row with where conditions and is not bulk efficient. upsert leverages SQL INSERT ... ON DUPLICATE KEY UPDATE for thousands of rows in one query, requiring unique constraint columns specified.

Q20. How does Laravel 11 handle API rate limiting and middleware definition?

- A) Rate limiting removed
- B) RateLimiter::for('api', fn() => Limit::perMinute(60)) in AppServiceProvider and middleware aliased in bootstrap/app.php via \$middleware->alias(['role' => RoleMiddleware::class])
- C) Only .htaccess controls it
- D) Middleware defined in Kernel only

****Answer: B****

****Explanation:**** Rate limiters are defined in service providers and referenced in routes via throttle:api. In Laravel 11, custom middleware aliases are registered fluently in bootstrap/app.php instead of Http Kernel \$routeMiddleware array.

Section 3: Advanced - 10 Questions

Q21. Explain PHP 8.2-8.4 Fibers and how they relate to Laravel Octane.

- A) Fibers are synchronous loops
- B) Fibers are lightweight cooperative concurrency primitives allowing non-blocking I/O pause/resume via Fiber::suspend(); Laravel Octane (Swoole/RoadRunner/FrankenPHP) uses them for concurrent tasks and Octane::concurrently()
- C) Fibers are database fibers
- D) Fibers replace queues

****Answer: B****

****Explanation:**** Fibers enable async-like code without callback hell and are used internally by Laravel's concurrency features. Octane keeps app in memory and uses Fibers to run multiple operations concurrently to reduce latency.

Q22. How does Laravel's new once() helper, defer(), and deferred functions in Laravel 11/12 improve performance?

- A) They do not exist
- B) once(fn) memoizes result per request, Illuminate\Support\defer(fn) and defer() after response executes work after sending response to user (terminable), similar to fastcgi_finish_request
- C) defer delays database
- D) once creates cookies

****Answer: B****

****Explanation:**** once() prevents repeated heavy computations within same request lifecycle. Deferred callbacks in Laravel 11+ run after response is sent, ideal for non-critical tasks without full queue overhead, improving TTFB.

Q23. Deep dive: How does Eloquent handle \$casts, custom cast classes, and AsEncryptedCollection in 2024?

- A) Casts only for dates
- B) \$casts = ['price' => 'decimal:2', 'options' => AsCollection::class] + custom CastsAttributes classes + Laravel 11 encrypted casts like encrypted:array, and attribute accessor via Attribute::make(get/set)

- C) Casts are deprecated
- D) Only string casts work

****Answer: B****

****Explanation:**** Modern casts use classes implementing CastsAttributes or new Attribute accessor/mutator syntax. Laravel 10+ added AsEncryptedCollection, AsEncryptedArrayObject and improved decimal casting for money handling without floating errors.

Q24. What is the correct way to prevent mass-assignment vulnerability and how did Laravel 11 change \$fillable vs \$guarded best practice combined with DTOs?

- A) Use \$fillable whitelist of assignable fields or \$guarded = [] with caution; use Validated DTO or \$request->validated() never \$request->all() for create/update
- B) Use \$_POST directly
- C) No need to guard
- D) Mass assignment is not a vulnerability

****Answer: A****

****Explanation:**** Mass assignment allows attackers to inject unexpected fields like is_admin. Best practice is strict \$fillable plus using FormRequest validated() data and Spatie Laravel Data DTOs to ensure only validated fields pass.

Q25. Explain Laravel's new Eloquent where clauses, subquery optimizations, and Cursor vs Lazy vs Chunk.

- A) All methods same
- B) chunk() loads fixed size with offset (issues with modifications), lazy() uses cursor with where id > last, cursor() uses PDO cursor single query low memory but cannot count; advanced where like whereRelation, whereJsonContains, whereExists subqueries
- C) Cursor is fastest always
- D) Lazy loads everything

****Answer: B****

****Explanation:**** chunk() paginates with LIMIT/OFFSET which can skip rows if data mutates. lazy() streams via chunked id queries safe for large datasets. cursor() holds single DB cursor open, using PHP generators for lowest memory but needs careful transaction handling.

Q26. How do you implement proper transaction, deadlock handling, and pessimistic locking in Laravel for high concurrency (e.g., inventory)?

- A) Just save model
- B) Use DB::transaction(fn, attempts:3) with sharedLock() and lockForUpdate(), SELECT ... FOR UPDATE via ->lockForUpdate(), idempotency keys, and catch DeadlockException retry
- C) Transactions not needed
- D) Use sleep to prevent deadlocks

****Answer: B****

****Explanation:**** DB::transaction() auto-commits/rollbacks and can retry on deadlock by attempts param. Pessimistic locking with lockForUpdate() prevents race conditions for stock decrement, essential for inventory/booking systems where optimistic locking fails.

Q27. What are PHP 8.4 deprecations and Laravel 11/12 breaking changes to watch for in 2026?

- A) Nothing deprecated

- B) PHP 8.4 deprecates implicit nullable function `foo(Type $x = null)` in favor of `?Type`, deprecates passing `E_NOTICE` as string; Laravel 11 removes `app/Http/Middleware` defaults, removes `php artisan make:controller --model` auto import changes, PHPUnit 11 requires `ReturnTypeWillChange` fixes, Carbon 3 compatibility
- C) PHP deprecates classes
- D) Laravel removes Eloquent

****Answer: B****

****Explanation:**** PHP actively cleans implicit nullable type which emits deprecation in 8.4 and becomes error in 9.0. Laravel 11 slims defaults, so upgrading requires moving middleware to `bootstrap/app.php` and updating from Doctrine/dbal to native schema methods.

Q28. How does Laravel Sanctum vs Passport vs new Laravel 12 Passport + Socialite vs PASETO work for APIs?

- A) All same for tokens
- B) Sanctum SPA token + mobile personal access tokens (simple, no OAuth server), Passport full OAuth2 server with scopes (heavier), Laravel Airlock history; for 2024-2026 Sanctum is default for first-party SPA and API tokens, Passport only for third-party OAuth apps
- C) Sanctum is OAuth2
- D) Passport is deprecated fully

****Answer: B****

****Explanation:**** Sanctum issues plain personal access tokens stored hashed in DB, authenticated via Bearer, ideal for mobile apps. Passport implements full OAuth2 authorization code flow with clients and scopes, needed when you are an OAuth provider like GitHub.

Q29. Design a scalable event-driven architecture in Laravel: Events, Listeners, ShouldBroadcast, and new Event Broadcasting vs Queued Closures.

- A) Put all logic in controller
- B) Dispatch `OrderPlaced` event → multiple listeners `ShouldQueue` (`SendEmail`, `DeductInventory`), broadcast via `ShouldBroadcast` over `Reverb/Pusher`, use `Event::fake()` in tests, transactional events via `DB::transaction + afterCommit()`
- C) Events are for frontend only
- D) Listeners cannot be queued

****Answer: B****

****Explanation:**** Events decouple side-effects via observer pattern; queued listeners run async. Laravel Reverb (2024) is first-party WebSocket server replacing Pusher for broadcasting. Transactional outbox pattern via `ShouldDispatchAfterCommit` ensures events fire only after DB commit.

Q30. How do you optimize Laravel for production in 2024-2026 stack: Octane, FrankenPHP, OPcache, config/route/view/event caching?

- A) No optimization needed
- B) Run `composer install --no-dev --optimize-autoloader`, `php artisan config:cache route:cache view:cache event:cache`, enable OPcache JIT `opcache.jit_buffer_size=100M`, use Octane with FrankenPHP/RoadRunner for worker mode, use `php artisan optimize`, CDN + Redis cache, and SCOUT, telescope disabled in prod
- C) Only debug mode
- D) Use `dd()` in production

****Answer: B****

****Explanation:**** Caching `config/routes/views` converts PHP files to serialized cache preventing disk scans. Octane with FrankenPHP (Laravel's new default in 2024+ installer) keeps app booted in memory cutting 50-80% latency. OPcache JIT compiles hot paths to machine code.

Course File: 21_Vue_Angular.md

Exam 21: Vue.js & Angular

Level: Comprehensive (Beginner to Advanced)

Total Questions: 30

Passing Score: 70%

Time Limit: 60 minutes

Stack Coverage: Vue 3.4+ (Composition API, <script setup>, Pinia), Angular 17-19 (Standalone, Signals, Control Flow), TypeScript, Vite/Angular CLI, SSR

Part 1: Beginner (10 Questions)

Q1: What is the primary difference between Vue's Options API and Composition API?

- A) Options API is only for Vue 2, Composition API is only for Vue 3
- B) Options API organizes code by option type (data, methods, computed), while Composition API organizes code by logical concern using setup() and composables
- C) Composition API does not support reactivity
- D) Options API is faster than Composition API

Answer: B

Explanation: Options API groups logic by option type which can scatter related logic. Composition API uses ref, reactive, and composables inside setup() to co-locate related logic, improving reusability and TypeScript inference.

Q2: In Vue 3, which is the correct way to create a reactive primitive count?

- A) let count = reactive(0)
- B) const count = ref(0) and access via count.value in script
- C) const count = data(0)
- D) const count = reactive({ value: 0 }) is the only way

Answer: B

Explanation: ref() is designed for primitives and returns an object with a .value property that is reactive. reactive() only works with objects and cannot hold primitives directly.

Q3: In Angular 17+, what does it mean for a Component to be standalone: true?

- A) It cannot import other components
- B) It does not need to be declared in an NgModule and can directly import its dependencies via the imports array
- C) It automatically becomes a lazy-loaded route
- D) It is only usable in tests

Answer: B

Explanation: Standalone components, now the default in Angular 17+, are self-contained and declare their own dependencies. This eliminates the need for NgModules for most applications and simplifies the mental model.

Q4: What does v-model do in Vue 3?

- A) One-way data binding from parent to child only
- B) Creates a two-way binding, equivalent to :modelValue prop + @update:modelValue event
- C) Only works on native input elements

D) Is deprecated in Vue 3

****Answer: B****

****Explanation:**** v-model is syntactic sugar for two-way binding. In Vue 3, it defaults to modelValue prop and update:modelValue event, and can be customized with arguments like v-model:title.

Q5: Which Angular decorator is used to pass data from parent to child component?

A) @Output()

B) @Input()

C) @Inject()

D) @ViewChild()

****Answer: B****

****Explanation:**** @Input() marks a property as input-bound, allowing a parent template to bind data like [user]="currentUser". @Output() is the opposite direction for child-to-parent events via EventEmitter.

Q6: In Vue 3 <script setup> what does defineProps<{ msg: string }>() do?

A) Defines a reactive state variable

B) It is a compile-time macro that declares props with full TypeScript type inference without needing to import it

C) Creates a Pinia store

D) Defines emits

****Answer: B****

****Explanation:**** defineProps and defineEmits are compiler macros in <script setup>, not real imports. They provide typed props with zero runtime overhead and are automatically available.

Q7: What is Angular's new built-in control flow introduced in Angular 17?

A) *ngIf and *ngFor with better performance

B) @if, @for, @switch block syntax with @empty and tracking

C) v-if and v-for

D) NgIf directive renamed to If

****Answer: B****

****Explanation:**** Angular 17 introduced block syntax like @if (condition) {} @else {} and @for (item of items; track item.id). This syntax is more performant, does not need CommonModule, and enables better type narrowing.

Q8: What is Pinia in the Vue ecosystem?

A) The official Vue Router replacement

B) The official state management library for Vue 3, replacing Vuex, with Composition API style stores

C) A testing library for Vue

D) A CSS framework

****Answer: B****

****Explanation:**** Pinia is now the officially recommended store for Vue, with a simpler API than Vuex using defineStore(). It has native TypeScript support, devtools, and supports both Options and Setup syntax.

Q9: How do you create a reactive signal in Angular 17+?

A) const count = ref(0)

B) count = signal(0) and read via count() and update via count.set() or count.update()

C) count = new BehaviorSubject(0)

D) count = reactive(0)

****Answer: B****

****Explanation:**** Angular Signals introduced via signal() provide fine-grained reactivity. You read the value by calling the signal as a function count() and mutate it explicitly, enabling zoneless change detection in future.

Q10: In Vue, what is the purpose of :key in v-for?

A) It is only for styling

B) It gives Vue a unique identifier to track each node for efficient DOM diffing and state preservation

C) It makes the loop run faster by default

D) It is required only for objects, not arrays

****Answer: B****

****Explanation:**** The key attribute helps Vue's virtual DOM algorithm identify nodes when the list order changes. Without proper keys, component state can be incorrectly preserved leading to bugs.

Part 2: Intermediate (10 Questions)

Q11: In Vue 3, what is the difference between ref and shallowRef?

A) No difference

B) ref makes its value deeply reactive, while shallowRef only tracks .value replacement, not deep mutations inside the object

C) shallowRef cannot be used in templates

D) ref is only for primitives

****Answer: B****

****Explanation:**** ref() wraps objects with reactive() making nested properties reactive. shallowRef() avoids deep conversion for performance when handling large objects or external libraries where you manually trigger updates.

Q12: In Angular, what does inject() function do vs constructor injection?

A) inject() can only be used inside components

B) inject() is the function-based DI API usable in any injection context (constructor, factory, inject() fields, functional guards/interceptors), enabling more ergonomic composition without constructor boilerplate

C) Constructor injection is deprecated

D) inject() does not support tree-shaking

****Answer: B****

****Explanation:**** inject() introduced in Angular 14 is preferred for functional APIs like functional route guards and interceptors. It works in constructors, field initializers, and factory functions, making DI more flexible in Standalone world.

Q13: Consider Vue 3: const props = defineProps<{ items: string[] }>(). Why should you NOT destructure const { items } = props directly?

A) It is fine, it stays reactive

B) Destructuring loses reactivity in standard props; you should use toRefs(props) or in Vue 3.3+ reactive destructure with default defineProps maintains reactivity but plain destructure still loses it unless enabled

C) It causes memory leak

D) Props cannot be arrays

****Answer: B****

****Explanation:**** Props are a reactive object; plain destructuring extracts plain values and breaks the proxy link. Vue 3.5 introduced reactive props destructure where const { items } = defineProps<...>() does retain reactivity, but general rule still applies to avoid losing reactivity.

Q14: In Angular, how do you handle derived state with Signals?

- A) Use `computed()` => `count() * 2` to create a memoised derived signal
- B) Use `ngOnChanges`
- C) Use `map()` operator
- D) Use `derived()`

****Answer: A****

****Explanation:**** `computed()` creates a read-only signal that automatically recomputes when its dependencies change. It is lazy, memoised, and glitch-free, replacing many manual observable pipelines.

Q15: What is the purpose of `defineModel()` introduced in Vue 3.4?

- A) To define database models
- B) A macro to simplify v-model two-way binding definition in child components, replacing manual `defineProps` + `defineEmits` for `modelValue`
- C) To define Pinia models
- D) To define Angular models

****Answer: B****

****Explanation:**** `const model = defineModel()` returns a ref synced to the parent's v-model. It can take modifiers and type, e.g., `defineModel<string>('title')`, drastically reducing boilerplate for custom v-model components.

Q16: In Angular, what is defer block (Angular 17+)?

- A) It defers TypeScript compilation
- B) `@defer { <big-component /> }` lazily loads and renders the content when a trigger (idle, viewport, interaction, timer) is met, improving initial bundle and INP
- C) It is same as `*ngIf`
- D) It delays HTTP requests

****Answer: B****

****Explanation:**** Deferrable views enable declarative lazy loading at template level without routing. Triggers like `@defer` (on viewport) load heavy components only when needed, automatically code-splitting them.

Q17: In Vue 3 Composition API, when should you use `watchEffect` vs `watch`?

- A) They are identical
- B) `watchEffect` auto-tracks dependencies and runs immediately, while `watch` is explicit about source and can be lazy; `watch` is better for specific source and old/new values
- C) `watch` cannot watch reactive objects
- D) `watchEffect` is only for props

****Answer: B****

****Explanation:**** `watchEffect()` immediately runs and tracks any reactive used inside. `watch(source, cb)` watches explicitly defined sources, offers `oldValue`, and is lazy by default unless `immediate:true`, giving more control.

Q18: How does Angular's new `input()` signal-based API (Angular 17.1+) differ from `@Input()`?

- A) It is just alias
- B) `userName = input<string>()` creates a signal input, read as `userName()` with options `input.required<>()` and transforms, integrating with signal reactivity ecosystem
- C) It cannot be used with TypeScript

D) It removes change detection

****Answer: B****

****Explanation:**** Signal inputs are the signal evolution of @Input(), providing InputSignal that participates in computed/effect. They support required, alias, and transform, offering stronger typing than decorator version.

Q19: In Vue, what does <Suspense> do?

A) Suspends rendering forever

B) Handles async dependencies in component tree, showing #fallback slot while async setup() or async components are resolving

C) Only works with Angular

D) Is deprecated

****Answer: B****

****Explanation:**** <Suspense> is built-in for orchestrating async components, commonly used with async <script setup> or defineAsyncComponent(). It enables top-level await inside setup and better loading states.

Q20: What is Angular DestroyRef and why use it over ngOnDestroy in composables?

A) It destroys components manually

B) It provides destroyRef.onDestroy(() => cleanup) in any injection context, allowing cleanup logic in functions/composables without needing a component class implementing OnDestroy

C) It is only for RxJS

D) It replaces all lifecycle hooks

****Answer: B****

****Explanation:**** DestroyRef enables composition-style cleanup in functional contexts like custom composables or inject() services. It decouples cleanup from component class, making reusable logic that auto-cleans on destroy possible.

Part 3: Advanced (10 Questions)

Q21: Vue 3.4+ reactivity optimization - How do ref and its internal implementation improve performance?

A) Vue 3.4 rewrote reactivity to use doubly-linked dep tracking and optimized ref unwrapping, making ref() getters much cheaper and avoiding unnecessary effect triggers

B) It removed Proxy

C) It uses polling

D) ref is now sync

****Answer: A****

****Explanation:**** Vue 3.4's reactivity refactor improved tracking efficiency with faster dep linked lists and more cases where access doesn't trigger tracking. This reduces overhead for large reactive graphs and improves computed performance significantly.

Q22: In Angular, what is an effect() and what is a crucial anti-pattern?

A) Effect runs after each render like watchEffect, but you should NOT write to signals inside effects to create infinite loops; use computed for derived state and effect only for side-effects like logging/manual DOM

B) Effect is same as computed and can set signals

C) Effect runs synchronously before rendering

D) Effect replaces all subscriptions

****Answer: A****

****Explanation:**** `effect(() => { console.log(count()) })` runs when its signal dependencies change. Writing to signals inside effects is discouraged and Angular warns; allowable writes require `allowSignalWrites:true`, as it breaks predictable data flow.

Q23: Explain Vue's `defineAsyncComponent` advanced options for production hardening.

- A) It only takes a loader function
- B) You can provide loader, loadingComponent, errorComponent, delay, timeout, suspensible: false to control loading UX and error fallback for code-split components
- C) It cannot be used with Vite
- D) Async components never need error handling

****Answer: B****

****Explanation:**** For production, async components need robust loading and error states. timeout aborts slow loads, delay avoids flashing, and errorComponent handles import failures like network chunk load errors.

Q24: In Angular SSR and Hydration (v17-19), what does `withEnabledBlockingInitialNavigation` and non-destructive hydration mean?

- A) SSR is not supported
- B) Angular's hydration reuses server-rendered DOM instead of re-rendering, and in v19 hydration supports `withI18nSupport` and better de-hydration for `@defer`; mismatch causes NG0500 warning
- C) Hydration always destroys DOM
- D) SSR only works with NgModules

****Answer: B****

****Explanation:**** Angular's hydration matches server DOM to client component tree, preserving nodes for faster FCP and no flicker. Newer versions handle `@defer`, i18n blocks, and event replay, but DOM manipulation before hydration breaks it.

Q25: How does Vite handle Vue SFC vs Angular standalone compilation?

- A) Both use webpack
- B) Vite uses `@vitejs/plugin-vue` with `compiler-sfc` to compile `<template>` into render functions via HMR, while Angular uses `@analogjs/vite-plugin-angular` / official esbuild-based application builder with AOT and standalone incremental compilation
- C) Vite cannot compile Vue
- D) Angular cannot use Vite

****Answer: B****

****Explanation:**** Vue's SFC pipeline compiles `template+script+style` to JS with hot replacement. Angular's new application builder (esbuild+bundler) AOT-compiles templates to JS instructions, supports Vite dev server via Analog or Angular's own Vite integration since Angular 17.

Q26: In Pinia, what is the difference between `store.$patch()` vs direct mutation vs `store.$state` =?

- A) All same
- B) `$patch({ count: 1 })` or `$patch(state => state.count++)` batches multiple mutations into one devtools entry and subscriber notification, direct mutation is fine in setup stores, while replacing `$state` replaces whole state but must preserve reactivity
- C) `$patch` is deprecated
- D) Direct mutation is forbidden

****Answer: B****

****Explanation:**** `$patch` enables atomic batch updates reducing rerenders and grouping undo history. In Setup stores (`defineStore('id', () => { const count = ref(0); return {count} })`) you can mutate directly `count.value++`, which is idiomatic.

Q27: Angular performance: When would you use ChangeDetectionStrategy.OnPush with Signals vs Default?

A) OnPush is deprecated with Signals

B) With Signals, Angular can mark components dirty only when signal inputs or template-read signals change, making OnPush nearly automatic and enabling future zoneless; default still checks every async event, while OnPush + signals drastically reduces change detection cycles

C) OnPush makes it slower

D) OnPush only works with RxJS

****Answer: B****

****Explanation:**** OnPush tells Angular to skip components unless Inputs changed, events fired, or markForCheck. Signals enhance this by precisely tracking template dependencies, so only components reading a changed signal are marked, paving way for provideExperimentalZonelessChangeDetection.

Q28: Vue advanced: What are Vapor mode and defineOptions()?

A) Vapor is CSS framework

B) Vapor (experimental, Vue 3.4+) is a no-virtual-DOM compilation strategy where templates compile to direct DOM operations for performance; defineOptions({ name, inheritAttrs:false }) sets component options inside <script setup>

C) Vapor removes reactivity

D) defineOptions is for Angular

****Answer: B****

****Explanation:**** Vapor mode aims to outperform vDOM by generating imperative DOM code similar to Solid.js. defineOptions solves the need to add a separate <script> block just to set name or inheritAttrs in <script setup>.

Q29: Angular advanced DI: Explain EnvironmentInjector, runInInjectionContext, and why needed for composable-like functions?

A) They are not needed

B) runInInjectionContext(envInjector, () => useMyComposable()) allows using inject(), signal(), computed() outside constructor context, essential for utility functions that use DI but are called outside injection context (e.g., in RxJS operators)

C) It replaces constructor

D) EnvironmentInjector is same as Root

****Answer: B****

****Explanation:**** Angular's inject() only works in injection context. When you extract logic to standalone functions (similar to Vue composables), you need an injector context passed in. runInInjectionContext temporarily sets context so the composable can call inject(HttpClient) or inject(DestroyRef).

Q30: SSR data fetching: Compare Vue's useAsyncData (Nuxt 3) vs Angular's resolve + TransferState.

A) They are identical

B) Nuxt useAsyncData(key, fetcher) fetches on server, serializes payload into hydration bundle to avoid re-fetch on client; Angular uses route resolvers or inject(TransferState) / makeStateKey and HttpClient with withCache() / withFetch to transfer HTTP cache from server to client

C) SSR does not need data transfer

D) Angular cannot transfer state

****Answer: B****

****Explanation:**** Both solve hydration double-fetch problem. Nuxt automatically handles payload via __NUXT_DATA__. Angular uses TransferState or new HttpTransferCacheOptions (withHttpTransferCache) to capture server HTTP requests and replay on client, preventing extra API calls.

Course File: 22_React.md

22_React - React & Frontend Exam

Total Questions: 30 | Beginner: 10 | Intermediate: 10 | Advanced: 10

Part 1: Beginner (Questions 1-10)

1. What does JSX stand for and what does it compile to?

- A) Java Syntax Extension, compiles to Java bytecode
- B) JavaScript XML, compiles to React.createElement() calls
- C) JavaScript Extension, compiles to HTML strings
- D) JSON XML, compiles to JSON objects

Answer: B

Explanation: JSX is JavaScript XML, a syntax extension that looks like HTML in JS. It is transpiled by Babel/SWC into React.createElement(type, props, ...children) calls, or _jsx() runtime calls in the new JSX transform.

2. Which hook is used to manage local state in a functional component?

- A) useEffect
- B) useState
- C) useMemo
- D) useRef

Answer: B

Explanation: useState declares a state variable and setter that triggers re-renders when updated. It is the primary way to manage local component state in function components since React 16.8.

3. What is the purpose of the key prop when rendering lists in React?

- A) To style list items uniquely
- B) To help React identify which items have changed, added, or removed for reconciliation
- C) To make the list searchable
- D) To encrypt list data

Answer: B

Explanation: Keys give elements a stable identity across renders, allowing React's diffing algorithm to reuse DOM nodes efficiently. Using index as key can cause bugs when list order changes because identity is lost.

4. In React, data flows:

- A) Bidirectionally between all components
- B) From child to parent only
- C) Unidirectionally from parent to child via props
- D) Randomly based on useEffect

Answer: C

Explanation: React enforces one-way data flow where parent components pass data down to children via props. Child-to-parent communication is done by passing callback functions as props, not by direct two-way binding.

5. What will happen if you directly mutate state like state.count = 5 instead of using setState/setCount?

- A) Component will re-render immediately

- B) React will throw a compile-time error
- C) State will change but React will not detect it, so no re-render will occur
- D) It will work identically to setCount

****Answer: C****

****Explanation:**** React compares state references to decide to re-render; direct mutation keeps the same reference so React bails out. Always use the setter which creates a new reference and schedules an update.

6. Which hook is used for side effects like data fetching, subscriptions, or DOM manipulation after render?

- A) useState
- B) useReducer
- C) useEffect
- D) useId

****Answer: C****

****Explanation:**** useEffect runs after the render commits to the DOM and is meant for side effects. It optionally returns a cleanup function and is controlled by its dependency array.

7. What is a controlled component in React?

- A) A component controlled by the browser only
- B) A form input whose value is controlled by React state via value and onChange
- C) A component that cannot be reused
- D) A component wrapped in React.memo

****Answer: B****

****Explanation:**** In a controlled component, React state is the single source of truth for the input value, updated on every onChange. This contrasts with uncontrolled components that use refs to read values imperatively.

8. What does useRef return and does updating it cause a re-render?

- A) Returns state and causes re-render
- B) Returns a mutable { current: ... } object that persists across renders and does NOT cause re-render when mutated
- C) Returns a promise and causes re-render
- D) Returns a DOM node only

****Answer: B****

****Explanation:**** useRef returns a mutable ref object whose .current can hold any value and survives re-renders. Changing .current does not trigger a re-render, making it ideal for DOM refs and storing timers/intervals.

9. What is the correct way to conditionally render in JSX?

- A) if (show) { <Component/> } directly inside JSX return
- B) show && <Component /> or ternary show ? <Component /> : null
- C) show then <Component />
- D) Using HTML if attribute

****Answer: B****

****Explanation:**** JSX is JavaScript expressions, so you must use expression forms like && short-circuit or ternary inside {}. Statement-level if cannot be placed directly inside JSX markup, but can be used before the return.

10. Why does React require onClick instead of onclick?

- A) React does not support click events

- B) React uses SyntheticEvent system with camelCase naming to normalize cross-browser behavior
- C) onclick is deprecated in HTML
- D) To save bytes

****Answer: B****

****Explanation:**** React wraps native events in SyntheticEvent for cross-browser consistency and uses camelCase JSX convention. Since React 17, events are attached at the root rather than document, but the camelCase API remains.

Part 2: Intermediate (Questions 11-20)

11. What is the difference between useEffect, useEffect, and useInsertionEffect?

- A) They are identical
- B) useEffect runs asynchronously after paint, useEffect runs synchronously after DOM mutations but before paint, useInsertionEffect runs synchronously before any DOM mutations for CSS-in-JS insertion
- C) useEffect runs before render
- D) Only useEffect exists in React 18+

****Answer: B****

****Explanation:**** useInsertionEffect is for CSS-in-JS libraries to inject styles before layout calculations. useEffect blocks paint and is for DOM measurements, while useEffect is deferred and non-blocking for most side effects.

12. In React 18, what does automatic batching mean?

- A) Multiple setState calls in promises, setTimeout, and native event handlers are now batched into a single re-render
- B) React batches only inside React event handlers like before
- C) Batching was removed in React 18
- D) Batching only works with class components

****Answer: A****

****Explanation:**** Before React 18, batching only happened in React event handlers; now createRoot automatically batches all updates regardless of origin. This reduces unnecessary renders and improves performance out of the box.

13. What is React.memo and when should you use it?

- A) A hook to memoize state
- B) A Higher-Order Component that memoizes the rendered output and skips re-rendering if props are shallowly equal
- C) It memoizes useEffect automatically
- D) It is required for all components

****Answer: B****

****Explanation:**** React.memo performs shallow prop comparison to avoid re-rendering a component when props haven't changed. Overusing it can hurt performance due to comparison cost, so use it for expensive components with stable props.

14. Explain the rules of hooks.

- A) Hooks can be called anywhere
- B) Only call hooks at the top level of React functions and only from React function components or custom hooks; never in loops, conditions, or nested functions
- C) Hooks must be called inside class components
- D) Hooks order doesn't matter

****Answer: B****

****Explanation:**** React relies on call order to associate hook state with component instances, so conditional hook calls would break order. The eslint-plugin-react-hooks enforces these rules statically.

15. What is the purpose of useCallback and useMemo? How do they differ?

- A) They are the same
- B) useMemo memoizes a computed value, useCallback memoizes a function reference; both optimize child re-renders when passing props to memoized components
- C) useCallback fetches data
- D) useMemo is for side effects

****Answer: B****

****Explanation:**** useMemo(() => compute(a,b), [a,b]) caches the result, while useCallback(fn, deps) is syntactic sugar for useMemo(() => fn, deps). They help prevent referential identity changes that would break React.memo optimizations.

16. What are React Server Components (RSC) introduced in React 18 / Next.js 13+?

- A) Components that only run inside useEffect
- B) Components that render on the server, send zero JS to client by default, can be async, and cannot use useState/useEffect; client components are marked with 'use client'
- C) Components that replace all client components
- D) Old name for SSR

****Answer: B****

****Explanation:**** RSCs run exclusively on the server, allowing direct async data fetching and access to backend resources without client bundle cost. They interoperate with Client Components via props/children, with file-level 'use client' / 'use server' directives defining boundaries.

17. What is the difference between useDeferredValue and useTransition in React 18/19?

- A) No difference
- B) useTransition wraps a state update as low-priority and returns [isPending, startTransition] to control when; useDeferredValue defers a value itself, creating a lagging version with low priority
- C) useDeferredValue is for server only
- D) Both are deprecated

****Answer: B****

****Explanation:**** useTransition gives imperative control over marking updates as transitions (great for input -> filtering). useDeferredValue(value) gives a declarative deferred version of a value, useful when you don't control the update call site, like prop passed from parent.

18. In Next.js 14/15 App Router, what does loading.js, error.js, and not-found.js do?

- A) Nothing, they are ignored
- B) They are special file conventions: loading.js renders as Suspense fallback for that segment, error.js is an error boundary ('use client' required), not-found.js renders when notFound() is called or route has no match
- C) They only work in pages/ router
- D) They must be server components only

****Answer: B****

****Explanation:**** App Router uses file-system conventions for UI states within each route segment. loading.js auto-wraps page.js in Suspense, error.js must be a Client Component to catch errors, enabling nested, streaming UX without manual

boilerplate.

19. What potential issue does this useEffect have? `useEffect(() => { fetch(url).then(setData) }, [url])` with rapid url changes

- A) No issue
- B) Race condition: older fetch may resolve after newer one and overwrite with stale data; need AbortController or ignoring stale responses, cleanup function
- C) It causes infinite loop always
- D) setData is not allowed

Answer: B

Explanation: Rapid URL prop changes trigger overlapping fetches where completion order is not guaranteed. Fix by using an AbortController in cleanup (`return () => controller.abort()`) or a `let cancelled = false` flag to ignore stale results.

20. What is hydration mismatch and how do you avoid it in Next.js / React 18?

- A) When CSS fails
- B) When server-rendered HTML does not match client initial render due to non-deterministic values like `Date.now()`, `Math.random()`, window access, or locale differences; causes React to warn and re-render entire tree
- C) When you forget key prop
- D) When using `useState`

Answer: B

Explanation: SSR requires server and first client render to produce identical markup. Avoid by rendering non-deterministic values only in `useEffect`, using `suppressHydrationWarning`, or dynamically importing client-only libs with `ssr: false` / `next/dynamic`.

Part 3: Advanced (Questions 21-30)

21. In React 19 (2024-2025), what are Actions, `useActionState`, and `useFormStatus`?

- A) Redux actions only
- B) React 19 natively integrates async actions for form handling: `<form action={formAction}>` can call server functions ('use server'), `useActionState` (formerly `useFormState`) manages async action result/pending, `useFormStatus` reads pending state of parent form
- C) They are removed in React 19
- D) They replace `useEffect`

Answer: B

Explanation: React 19 treats form actions as first-class: Server Actions can mutate data and are progressively enhanced. `useActionState` returns `[state, action, isPending]` for UI feedback, and `useFormStatus` enables design-system components to disable/submit without prop drilling.

22. How does the React Compiler (React Forget, beta in 2024-2025) change memoization needs?

- A) It removes `React`
- B) It auto-memoizes components and values by analyzing reactivity, making manual `useMemo`, `useCallback`, `React.memo` largely unnecessary while preserving semantics
- C) It requires more `useMemo`
- D) It only compiles CSS

****Answer: B****

****Explanation:**** The React Compiler automatically inserts memoization equivalent to manual `useMemo/useCallback` at build time via static analysis. This allows writing code without manual optimization ceremony while achieving same skip-re-render benefits, though mental model of referential identity still matters.

23. Explain Concurrent Rendering, time-slicing, and how `startTransition` + `Suspense` work together for data fetching.

A) Concurrent means multi-threaded JS

B) Concurrent rendering allows React to interrupt, pause, and resume rendering work; `startTransition` marks updates as non-urgent so `Suspense` can show fallback without blocking typed input, enabling streaming SSR and selective hydration

C) It makes React synchronous only

D) `Suspense` doesn't work with transitions

****Answer: B****

****Explanation:**** React's concurrent renderer uses a fiber scheduler that can yield to main thread, keeping UI responsive. Transitions tell React current UI can stay interactive while new UI with suspended data prepares in background, crucial for Next.js streaming and use hook patterns.

24. You have a context that updates frequently (e.g., mouse position) and causes all consumers to re-render. How do you optimize?

A) You cannot optimize context

B) Split contexts (state vs dispatch), use `useMemo` for context value stability, use `useContextSelector` pattern / external store like Zustand/Jotai, or `useSyncExternalStore` to subscribe granularly; React 19 context is fast but still propagation-based

C) Just use `useState` everywhere

D) Put everything in one context

****Answer: B****

****Explanation:**** Context uses reference identity; any value change re-renders all consumers even if they use only part. Splitting into multiple contexts or using selector-based stores prevents large subtree re-renders, which is critical for high-frequency updates.

25. What is the difference between `useEffectEvent` (React 19 experimental -> stable `useEffectEvent`) and `useCallback` for event handlers inside effects?

A) Same thing

B) `useEffectEvent` (previously `useEffectEvent`) creates a function with always-latest closure that is NOT a dependency of effects, solving stale closure vs exhaustive-deps dilemma without re-triggering effect

C) `useEffectEvent` causes re-render

D) `useCallback` is not reactive

****Answer: B****

****Explanation:**** Effects often need to call callbacks that shouldn't trigger re-execution; adding them to deps causes loops, omitting lies. `useEffectEvent` extracts non-reactive logic with latest props/state but stable identity that doesn't affect deps, officially solving the pattern formerly solved by refs.

26. Deep dive: Why does `setState` inside render cause infinite loop, but `useSyncExternalStore` and `useDeferredValue` do not? And what is tearing in concurrent mode?

A) All cause infinite loops

- B) Setting state during render in same component re-triggers render synchronously leading to loop; `useSyncExternalStore` subscribes to external store with snapshot consistency guarantees, and `useDeferredValue` defers via concurrent feature. Tearing is when different components see different versions of external mutable data due to interruptible render
- C) Tearing never happens in React
- D) `useSyncExternalStore` is deprecated

****Answer: B****

****Explanation:**** Render must be pure. `useSyncExternalStore` uses internal mechanisms to force consistent snapshot and resubscribe, preventing tearing where UI shows inconsistent state from concurrent partial renders. This was the fix for problems with raw external stores in React 18 concurrent features.

27. In Next.js 15 (2024-2025) what changed for fetch, caching, and async params/searchParams? Explain.

- A) Nothing changed
- B) Next.js 15: `fetch` is no longer cached by default (previously `force-cache`), GET Route Handlers not cached by default, `params` and `searchParams` are now Promises (`must await`), and `headers()` `cookies()` etc are async. Opt into cache via `fetch(..., {cache: 'force-cache'})` or `revalidate`
- C) Fetch is removed
- D) `params` are synchronous still

****Answer: B****

****Explanation:**** To make caching explicit and avoid stale data surprises, Next.js 15 reversed defaults to uncached `fetch`. Making `params` async enables parent layouts to stream and allows React to handle dynamic APIs via `use` and `Suspense` without blocking whole route, aligning with React 19 async primitives.

28. Compare bundle optimization strategies: code splitting with `React.lazy` + `Suspense`, `dynamic import()`, Next.js `dynamic()`, and Server Components.

- A) All identical
- B) `React.lazy` code-splits client bundles with `Suspense` fallback using `static import()`, Next `dynamic()` extends it with `ssr:false`, `loading`, and Server Components eliminate JS entirely for server-only parts eliminating need to ship/hydrate that code, together achieving optimal RSC + client chunk graph
- C) Server Components increase bundle size
- D) Lazy loading is bad for performance

****Answer: B****

****Explanation:**** Client-side lazy reduces initial JS via separate chunks loaded on demand. RSC goes further by never adding server-only dependencies (e.g., markdown parser, DB client) to client bundle at all. Best practice in 2024-2026 is RSC for data/expensive libs, `dynamic` for heavy client libs like charts.

29. How would you implement a correct global state with `useReducer` + `Context` that survives RSC boundaries and supports performance? Discuss alternatives like Zustand, Jotai, Redux Toolkit.

- A) Use one huge context
- B) Create separate `StateContext` and `DispatchContext` to avoid re-renders, memoize provider value, use `useReducer` or `useSyncExternalStore` adapter; but for large apps prefer Zustand (external store + selector), Jotai (atomic), or Redux Toolkit with `next-redux-wrapper` or RSC-safe provider as client component. Never put mutable global store inside Server Component
- C) Context works fine in Server Components
- D) Redux cannot be used with Next.js

****Answer: B****

****Explanation:**** Context + `useReducer` naive combo re-renders all consumers on any state change due to value identity. Splitting contexts isolates dispatch stability (dispatch never changes). External stores like Zustand use `useSyncExternalStore` under the hood, allowing fine-grained selector subscriptions and remaining RSC-compatible as 'use client' providers.

30. Explain strict effects double-mount in React 18 StrictMode, why useEffect cleanup must be idempotent, and how to test.

A) It's a bug

B) In dev, StrictMode mounts, unmounts (cleanup), then remounts to detect unsafe side effects and non-idempotent setup. Cleanup must unsubscribe/abort correctly. Test by ensuring fetch abort, subscriptions cleanup, no duplicate listeners; use act and verify effect doesn't leak after unmount

C) StrictMode is production only

D) Cleanup is optional

****Answer: B****

****Explanation:**** The intentional double-invoke surfaces components relying on once-only mount logic that will break with future offscreen/keep-alive features. Writing cleanup that reverses setup completely (idempotent) ensures compatibility with concurrent features where React may discard and re-create trees.

Answer Key Summary

| Q | Answer | Topic |

| --|-----|-----|

| 1 | B | JSX |

| 2 | B | useState |

| 3 | B | Keys |

| 4 | C | Data flow |

| 5 | C | State mutation |

| 6 | C | useEffect |

| 7 | B | Controlled components |

| 8 | B | useRef |

| 9 | B | Conditional rendering |

| 10 | B | Synthetic events |

| 11 | B | Effect types |

| 12 | A | Automatic batching |

| 13 | B | React.memo |

| 14 | B | Rules of hooks |

| 15 | B | useMemo vs useCallback |

| 16 | B | React Server Components |

| 17 | B | useTransition vs useDeferredValue |

| 18 | B | Next.js App Router conventions |

| 19 | B | Race conditions |

| 20 | B | Hydration mismatch |

| 21 | B | React 19 Actions |

| 22 | B | React Compiler |

| 23 | B | Concurrent rendering |

| 24 | B | Context optimization |

| 25 | B | useEffectEvent |

| 26 | B | useSyncExternalStore & tearing |

| 27 | B | Next.js 15 breaking changes |

| 28 | B | Bundle optimization |

| 29 | B | Global state patterns |

| 30 | B | StrictMode double-mount |

Course File: 23_DevOps.md

Course: DevOps Fundamentals

1. Beginner Level (10 Questions)

****Q1. What is the primary goal of DevOps?****

- A. To replace developers with operations engineers
- B. To bridge development and operations for faster, more reliable software delivery
- C. To automate only infrastructure provisioning
- D. To enforce strict separation between Dev and Ops teams

****Q2. In a CI/CD pipeline, what does CI stand for and what does it do?****

- A. Continuous Integration - developers frequently merge code into a shared repository with automated builds and tests
- B. Continuous Inspection - manually reviewing code once per month
- C. Code Isolation - keeping each developer's branch separate forever
- D. Central Integration - merging all code only on release day

****Q3. What is Infrastructure as Code (IaC)?****

- A. Writing application code inside infrastructure servers
- B. Managing and provisioning infrastructure through machine-readable definition files instead of manual configuration
- C. Using physical hardware to run code
- D. A tool that converts infrastructure into application code

****Q4. Which tool is commonly used for version control in DevOps workflows?****

- A. Docker
- B. Jenkins
- C. Git
- D. Nagios

****Q5. What is the purpose of a Dockerfile?****

- A. To configure a Linux firewall
- B. To define instructions for building a Docker container image
- C. To store database migrations
- D. To monitor container logs

****Q6. What does CD mean in a DevOps context? (Continuous Delivery / Continuous Deployment)****

- A. Only manual deployment to production once a year
- B. Automatically ensuring code is always in a deployable state and can be released to production automatically or with one click
- C. Copying code from one developer laptop to another
- D. Continuous Debugging of production servers

****Q7. You want to run the same application consistently across dev, test, and prod environments. Which concept helps you achieve this?****

- A. Configuration Drift
- B. Containerization with images like Docker
- C. Manual server setup with documentation
- D. Snowflake servers

****Q8. What is the role of monitoring and logging in DevOps?****

- A. They are only needed after the product is retired
- B. They provide feedback on application health, performance, and failures to enable fast detection and recovery

- C. They replace the need for testing
- D. They are used only by the security team

****Q9. In Git workflow, what is a pull request (or merge request)?****

- A. A request to delete the main branch
- B. A mechanism to propose, review, and discuss code changes before merging into a target branch
- C. A backup of the repository
- D. An automated deployment to production

****Q10. What is the difference between a virtual machine (VM) and a container?***

- A. There is no difference
- B. VMs virtualize hardware with a full guest OS per instance, containers share the host OS kernel and are lightweight and faster to start
- C. Containers are heavier than VMs and include a full OS
- D. VMs cannot run in the cloud, containers can

Beginner Answer Key & Explanations

****Q1 Answer: B - To bridge development and operations for faster, more reliable software delivery****

DevOps combines cultural practices, automation, and shared responsibility to shorten development cycles. It breaks down silos between Dev and Ops rather than reinforcing them.

****Q2 Answer: A - Continuous Integration - developers frequently merge code into a shared repository with automated builds and tests****

CI runs automated builds and tests on every push to catch integration issues early. This prevents the "merge hell" that occurs with long-lived branches.

****Q3 Answer: B - Managing and provisioning infrastructure through machine-readable definition files instead of manual configuration****

IaC tools like Terraform, Pulumi, and CloudFormation make infrastructure versioned, repeatable, and reviewable. It eliminates manual click-ops and reduces configuration drift.

****Q4 Answer: C - Git****

Git is the de facto distributed version control system used in almost all DevOps pipelines. Jenkins is CI, Docker is containers, and Nagios is monitoring.

****Q5 Answer: B - To define instructions for building a Docker container image****

A Dockerfile contains FROM, RUN, COPY, CMD etc. to declaratively build an immutable image. This image ensures consistent runtime across environments.

****Q6 Answer: B - Automatically ensuring code is always in a deployable state and can be released to production automatically or with one click****

Continuous Delivery means artifact is production-ready at any time; Continuous Deployment automates the release itself. Both rely on automated tests and deployment pipelines.

****Q7 Answer: B - Containerization with images like Docker****

Container images package code and dependencies together, ensuring parity from laptop to production. This solves "it works on my machine" problems caused by snowflake environments.

****Q8 Answer: B - They provide feedback on application health, performance, and failures to enable fast detection and recovery****

The three pillars of observability (metrics, logs, traces) enable MTTR reduction and SRE practices. Tools like Prometheus, Grafana, Loki, and OpenTelemetry are standard in 2024-2026.

****Q9 Answer: B - A mechanism to propose, review, and discuss code changes before merging into a target branch****

Pull requests enable code review, automated checks, and knowledge sharing. They are a core collaboration practice in trunk-based and GitFlow workflows.

****Q10 Answer: B - VMs virtualize hardware with a full guest OS per instance, containers share the host OS kernel and are lightweight and faster to start****

Containers start in seconds and use fewer resources due to shared kernel. VMs provide stronger hardware-level isolation but with higher overhead.

2. Intermediate Level (10 Questions)

****Q11.** Your team uses Terraform to provision AWS infrastructure. A teammate manually changed an EC2 instance type in the console. On the next terraform apply, what will happen and what's the best practice?******

- A. Terraform will ignore the change; manual changes are preferred
- B. Terraform will detect drift and propose to revert the instance type to the code-defined state; best practice is to prohibit manual changes and use terraform plan + code review
- C. Terraform will automatically delete the instance
- D. Terraform state will corrupt and require reinstall

****Q12.** You have a Jenkins/GitHub Actions pipeline that builds Docker images. Builds are slow due to reinstalling dependencies every time. Which caching strategy should you implement as of 2024-2026?******

- A. Delete Docker layer cache each build for cleanliness
- B. Use Docker BuildKit cache mounts (`--mount=type=cache`) for package managers, registry layer caching with `buildx --cache-from/to`, and GitHub Actions cache for `actions/cache`
- C. Copy entire `node_modules` into the repo
- D. Use docker system prune `-a` before each build

****Q13.** You need to manage secrets for a Kubernetes workload. Which approach aligns with DevOps security best practices in 2025?******

- A. Hardcode secrets in Dockerfile ENV and push to GitHub
- B. Use Kubernetes Secrets encrypted at rest with KMS, or better, external secrets management via External Secrets Operator with HashiCorp Vault or AWS Secrets Manager / SOPS sealed secrets, with no secrets in Git
- C. Store secrets in a public ConfigMap
- D. Pass secrets via HTTP query parameters

****Q14.** Your microservices deployment uses blue-green deployment. What is its key advantage over rolling deployment for critical checkout service?******

- A. It uses less resources
- B. It keeps two identical production environments, allowing instant rollback by switching router/service, and zero-downtime with full version testing before cutover
- C. It is the same as canary deployment
- D. It deletes old version before new version starts

****Q15.** You are configuring Prometheus monitoring for Kubernetes. Pods are highly ephemeral and IP addresses change frequently. How should Prometheus discover targets?******

- A. Hardcode pod IPs in `prometheus.yml`
- B. Use Kubernetes service discovery (`kubernetes_sd_configs`) with annotations, and deploy via Operator with ServiceMonitor CRDs for auto-discovery
- C. Manually edit targets every hour
- D. Use static file with DNS A records

****Q16.** A Dockerfile currently uses `FROM ubuntu:latest` and `RUN apt-get update && apt-get install -y nodejs npm` and copies whole repo with `COPY . .`. Builds are insecure, large, and non-reproducible. What is the 2024-2026 best-practice fix?******

- A. Keep as-is, latest is always safe
- B. Pin base image digest (`FROM ubuntu:22.04@sha256:...`), use multi-stage build with distroless or alpine node:20-alpine final image, use `.dockerignore`, and run as non-root USER
- C. Use FROM scratch and add Ubuntu manually
- D. Install everything as root and use latest tag for image

****Q17.** Your Terraform monolith has a single state file for 300 resources across dev, staging, prod. Applies take 20 minutes, and blast radius is high. What should you do?******

- A. Keep single state file for simplicity

- B. Split into workspaces or directory-per-environment with remote backend (S3 + DynamoDB lock / Terraform Cloud), use modules, and implement state isolation, and consider Terragrunt or tf stacks
- C. Use local state on developer laptop
- D. Delete state file to speed up applies

****Q18. In GitHub Actions, you need to avoid long-lived AWS credentials for deploying to EKS. What is the recommended 2024-2026 method?****

- A. Store AWS_ACCESS_KEY_ID and SECRET in repository code
- B. Use OIDC federation: configure GitHub as OIDC provider in AWS IAM, assume role via aws-actions/configure-aws-credentials with role-to-assume, no static keys
- C. Share one IAM user key across all teams
- D. Disable all authentication

****Q19. Your services communicate via REST, but deployments cause breaking changes for consumers. Which DevOps practice plus API strategy mitigates this?****

- A. Ignore versioning and force consumers to update
- B. Implement contract testing (Pact), API versioning (/v1, /v2), backward compatibility checks in CI, and canary + feature flags (LaunchDarkly / OpenFeature) to decouple deploy from release
- C. Use FTP to share JSON files
- D. Deploy all services at exactly midnight without tests

****Q20. You are asked to implement GitOps for Kubernetes. Which tool and workflow matches GitOps principles?****

- A. Run kubectl apply manually from laptop on production cluster
- B. Use ArgoCD or Flux that continuously reconciles cluster state from Git repository, with Git as single source of truth, automated drift detection and self-heal, PR-based approvals
- C. Use Jenkins to SSH into nodes and run docker run
- D. Store YAML files on USB drive

Intermediate Answer Key & Explanations

****Q11 Answer: B - Terraform will detect drift and propose to revert the instance type to the code-defined state****

Terraform compares desired state (code), state file, and actual infrastructure to detect drift. Immutability and banning click-ops ensures IaC remains the source of truth.

****Q12 Answer: B - Use Docker BuildKit cache mounts and registry layer caching****

BuildKit cache mounts persist package manager caches without bloating image layers, and registry caching reuses layers across CI runners. This cuts build time by 60-80% compared to fresh installs.

****Q13 Answer: B - Use Kubernetes Secrets encrypted at rest with KMS, or better, external secrets management****

Storing secrets in Git is a critical security failure. ESO or Sealed Secrets with Vault/AWS Secrets Manager ensures encryption, rotation, and auditability via RBAC.

****Q14 Answer: B - It keeps two identical production environments, allowing instant rollback by switching router/service****

Blue-green minimizes risk for critical services by keeping old (blue) live until new (green) passes smoke tests. Unlike rolling, rollback is O(1) router flip, not slow progressive re-deploy.

****Q15 Answer: B - Use Kubernetes service discovery with annotations and ServiceMonitor CRDs****

Prometheus kubernetes_sd_configs watches API server for pod/service endpoints, handling churn automatically. Operator's ServiceMonitor abstracts scrape config without editing prometheus.yml.

****Q16 Answer: B - Pin base image digest, use multi-stage build with distroless or alpine final image****

Pinning digest guarantees reproducible builds and prevents supply-chain attacks via tag mutation. Multi-stage + distroless reduces attack surface and size by 90% and non-root enforces least privilege.

****Q17 Answer: B - Split into workspaces or directory-per-environment with remote backend****

Single large state file increases lock contention, blast radius, and plan time. Environment isolation via remote backend and modules follows DRY with Terragrunt patterns used since 2024.

****Q18 Answer: B - Use OIDC federation: configure GitHub as OIDC provider in AWS IAM****

OIDC eliminates static long-lived secrets, providing short-lived tokens per workflow run bound to repo/branch claims. This aligns with 2024 AWS best practice for CI/CD supply chain security.

****Q19 Answer: B - Implement contract testing, API versioning, backward compatibility checks, and canary + feature flags****

Contract testing catches breaking provider changes in CI before deployment. Versioning + feature flags allow safe progressive delivery without forcing immediate consumer upgrades.

****Q20 Answer: B - Use ArgoCD or Flux that continuously reconciles cluster state from Git repository****

GitOps requires declarative Git as source of truth and automated agent reconciliation. Tools like ArgoCD provide drift detection, automated sync, and audit trail from PRs.

3. Advanced Level (10 Questions)

****Q21. You run 50 microservices on Kubernetes. Deployments are frequent, and you see increased 500s during deployments due to in-flight requests being killed. Your liveness/readiness probes are identical. What production-grade fix is needed in 2025?****

- A. Delete liveness probe altogether
- B. Configure proper lifecycle: distinct readiness probe that fails during shutdown, preStop hook with sleep 15, set terminationGracePeriodSeconds: 60, handle SIGTERM gracefully in app, and enable PodDisruptionBudgets and readinessGates with service mesh / AWS ALB target draining
- C. Use kill -9 in preStop
- D. Set terminationGracePeriodSeconds: 0 for faster deploys

****Q22. Your organization practices DevSecOps. You need to secure container supply chain: build from untrusted base images, prevent critical CVEs, and verify image signature at admission to EKS cluster with policy as code. What modern 2025 stack?****

- A. No scanning needed, trust Docker Hub latest
- B. Use Trivy/Grype in CI for scanning, build with Docker Buildx with SBOM and attestations (--attest type=sbom,provenance), sign with Cosign / Sigstore keyless from OIDC, store in ECR with immutable tags, enforce with Kyverno/OPA Gatekeeper policy requiring valid Sigstore signature and no critical CVEs
- C. Scan manually once per year
- D. Push images with latest tag only

****Q23. You manage Terraform Cloud with teams across time zones merging to main. You need to prevent concurrent applies corrupting state, enforce policy (no public S3 buckets), and reduce plan time for 1000+ resources. Architecture?****

- A. Use local state files shared via Slack
- B. Use Terraform Cloud remote execution with speculative plans on PR, state locking via backend, Sentinel/OPA policies for compliance (e.g., deny acl=public-read), and break into stacks with terraform stack or terragrunt with dependency and use -target only temporarily, plus parallel plans and move to HCP Terraform with ephemeral workspaces
- C. Disable state locking for speed
- D. Allow everyone to apply from laptop

****Q24. You are implementing SRE SLOs for a payment API: 99.9% availability over 28-day rolling window, 300ms P95 latency. Error budget is 43 minutes/month. During deployment, burn rate alert fires 10x. How should you architect alerts and actions per 2024-2026 SRE best practice?****

- A. Alert on all errors immediately to page on-call
- B. Implement multi-window multi-burn-rate alerts (Google SRE workbook): fast burn (1h and 5m windows) pages immediately for 10x burn, slow burn (6h and 1h) tickets for 1-2x, wire to error budget policy halting feature releases when budget <10% and requiring freeze, using OpenSLO / Sloth for SLO as Code
- C. Disable all alerting
- D. Use average latency only

****Q25. Your CI/CD generates 10k builds/day. You store Docker images in ECR, Helm charts, and Terraform modules. Developer complains about artifact sprawl, reproducibility failures, and lack of traceability from commit to deployed image. What platform strategy in 2025-2026?****

- A. Keep artifacts in local laptop directories

B. Adopt artifact management with OCI Registry as universal artifact store (ECR/Harbor for images + Helm OCI), use immutable semantic version tags + commit SHA, generate SBOM + provenance with SLSA Level 3, centralize with Backstage IDP or Artifactory, garbage collect untagged, and link via Git commit -> image digest -> deployment via ArgoCD tracking

C. Use latest tag for every image

D. Delete all old images manually

****Q26.** You have multi-region active-active deployment on Kubernetes with Global Accelerator. You need to support stateful workloads (PostgreSQL) with zero data loss failover and consistent deploys. How do you handle data layer and deployment orchestration?**

A. Deploy PostgreSQL as bare pods with hostPath volumes and async rsync

B. Do not replicate database

C. Use cloud-managed multi-region DB (Aurora Global, Cloud Spanner, CockroachDB) or Patroni with synchronous replication + automated backup, separate data plane from compute deploys, use GitOps ApplicationSets for multi-cluster progressive delivery with Argo Rollouts and analysis templates checking DB lag

D. Use SQLite shared via NFS

****Q27.** You observe configuration drift across 200 EC2 instances managed by Ansible. Some nodes were manually patched. You need idempotency, drift detection, and compliance reporting at scale. Which Ansible / alternative evolution in 2024-2026 solves this?**

A. Run Ansible in check mode only once a year

B. Move to pull-based config management with Ansible AWX/AAP scheduled jobs + drift detection via ansible-lint, use immutable AMIs built with Packer + Ansible provisioner, and complement with Fleet / Chef / Desired State via AWS Systems Manager State Manager compliance, or migrate high-scale to immutable infrastructure with Image Builder

C. Continue manual patching

D. Delete Ansible inventory

****Q28.** Your team adopted DORA metrics to measure DevOps performance. You have deployment frequency = 1/week, lead time = 2 weeks, MTTR = 2 days, change failure rate = 40%. You are classified as low performer. What technical and process changes move you to Elite (per 2024 DORA report) while maintaining stability?**

A. Deploy less frequently to reduce failures

B. Implement trunk-based development with short-lived branches <1 day, automated testing pyramid with 80% coverage, decouple deploy from release via feature flags (OpenFeature), ephemeral preview envs per PR with Testcontainers, shift-left security scanning, and platform engineering self-service golden paths with IDP

C. Increase manual approval gates to 5 managers

D. Stop testing to increase speed

****Q29.** You run Kubernetes 1.30+ in production. You notice noisy neighbor issues, pods evicted due to memory pressure, and secrets mounted as env vars leak in logs. What hardening and resource management fixes are required?**

A. Set no resource requests/limits and mount secrets as env vars

B. Set Guaranteed QoS with equal requests/limits for critical pods, use LimitRange and ResourceQuota, enable Kubelet memory manager and CPU manager policies, use VPA/HPA with KEDA, enforce PodSecurityStandards restricted profile, use Seccomp/AppArmor, and mount secrets via CSI secret store driver volume mounts not env, plus ephemeral containers for debugging

C. Give every pod privileged access

D. Disable all QoS classes

****Q30.** You are designing progressive delivery pipeline for 2026: deploy new service version to 5% of users, auto-promote if error rate <1% and P95 <250ms, auto-rollback otherwise, with automated canary analysis and mesh traffic splitting. Which implementation?**

A. Manual kubectl set image and watch logs

B. Use Argo Rollouts or Flagger with Istio/Linkerd / AWS App Mesh / Gateway API for traffic splitting, define AnalysisTemplate with Prometheus/Datadog queries for success criteria (error rate, latency, custom business metrics), integrate with progressive delivery and feature flags and use automated promotion/rollback with notifications to Slack

C. Deploy to 100% immediately and hope

D. Use FTP to upload new binary

Advanced Answer Key & Explanations

****Q21 Answer: B - Configure proper lifecycle: distinct readiness probe that fails during shutdown, preStop hook with sleep 15, terminationGracePeriodSeconds****

Instant SIGTERM kills in-flight requests; readiness removal plus preStop allows LB to deregister and drain connections gracefully. PDB prevents voluntary disruption evicting all replicas during rollout.

****Q22 Answer: B - Use Trivy/Grype in CI, build with SBOM/provenance, sign with Cosign/Sigstore keyless, enforce with Kyverno/Gatekeeper****

SLSA Level 3 requires hermetic builds, provenance, and signing; Sigstore keyless OIDC eliminates key management. Admission controller verifies signature and blocks unsigned or vulnerable images for supply chain security.

****Q23 Answer: B - Use Terraform Cloud remote execution with speculative plans on PR, state locking, Sentinel/OPA policies****

Remote execution guarantees single apply with locking, speculative plans catch issues early. Policy as code prevents misconfigurations like public S3, and stack splitting reduces blast radius and plan time at 1000+ resources.

****Q24 Answer: B - Implement multi-window multi-burn-rate alerts and error budget policy****

Multi-window prevents alert flapping by comparing short and long windows for realistic burn detection. Tying error budget consumption to deployment freeze enforces reliability over velocity per SRE principles.

****Q25 Answer: B - Adopt artifact management with OCI Registry as universal store, immutable tags, SBOM + SLSA Level 3, Backstage IDP****

OCI registry unification enables tracking via digest and provenance chain from commit SHA to deployed image. SBOM and SLSA provide auditability and reproducibility, while GC controls cost and sprawl.

****Q26 Answer: C - Use cloud-managed multi-region DB or Patroni with synchronous replication + GitOps ApplicationSets****

Stateful workloads cannot rely on simple pod replication; managed global databases provide synchronous cross-region guarantees. Separating data and stateless deploys via ApplicationSets enables orchestrated progressive multi-cluster rollout.

****Q27 Answer: B - Move to pull-based config management with AWX/AAP scheduled jobs + Packer immutable AMIs + SSM State Manager****

Push-only Ansible suffers from drift when manual changes occur. Immutable AMIs prevent drift entirely by rebuilding, while scheduled pull and compliance tools continuously enforce and report desired state.

****Q28 Answer: B - Implement trunk-based development with short-lived branches, automated testing, feature flags, ephemeral envs, platform golden paths****

DORA Elite traits correlate with trunk-based, <1 day branch lifetime, and extensive automation. Platform engineering golden paths reduce toil and enable self-service, improving all four DORA metrics simultaneously.

****Q29 Answer: B - Set Guaranteed QoS, use LimitRange/Quota, memory/cpu manager policies, VPA/HPA KEDA, PSS restricted, CSI secret store****

Guaranteed QoS ensures scheduling predictability and reduces eviction. Secret-as-volume prevents env leaks in logs/process listings, and restricted PSS plus Seccomp hardened pod security posture.

****Q30 Answer: B - Use Argo Rollouts or Flagger with Istio/Linkerd / Gateway API for traffic splitting with AnalysisTemplate****

Declarative progressive delivery operators automate canary analysis using metrics queries, enabling automatic promotion/rollback without manual intervention. Mesh traffic splitting provides fine-grained weight-based routing down to 1% granularity.

Course File: 24_Git.md

Course: Git & GitHub

1. Beginner Level (10 Questions)

****Q1.** You just installed Git and want to set your identity for all repositories on this machine. Which commands should you run?******

- A. `git config --global user.name "Alice" && git config --global user.email "alice@example.com"`
- B. `git config user.name "Alice"`
- C. `git set --global identity "Alice"`
- D. `git --global config user "Alice"`

****Q2.** You have modified two files, but you only want to commit `app.js` and keep `config.json` unstaged. What should you do?******

- A. `git commit -a -m "update"`
- B. `git add app.js && git commit -m "update app"`
- C. `git commit app.js`
- D. `git stage app.js && git push`

****Q3.** You want to see a concise one-line history of commits with graph and all branches. Which command is best?******

- A. `git log --oneline --graph --all`
- B. `git history`
- C. `git show branch`
- D. `git log --full`

****Q4.** You accidentally committed with a typo in the message on your last local commit (not pushed). How do you fix it quickly?******

- A. `git commit --amend -m "correct message"`
- B. `git reset --hard HEAD~1` and commit again
- C. `git revert HEAD`
- D. `git fix HEAD`

****Q5.** On GitHub, you forked a repo `octo/original` to your account. How do you keep your fork's main branch up to date with upstream after cloning your fork?******

- A. `git pull origin main`
- B. `git remote add upstream https://github.com/octo/original.git && git fetch upstream && git merge upstream/main`
- C. `git clone upstream main`
- D. `git sync fork`

****Q6.** You want to create a new branch `feature/login` and switch to it in one command on Git 2.36+. What are valid modern commands?******

- A. `git checkout -b feature/login` and `git switch -c feature/login`
- B. `git branch feature/login`
- C. `git new feature/login`
- D. `git create branch feature/login`

****Q7.** What does `.gitignore` do, and where should you place a file named `secrets.env` to never track it?******

- A. It deletes files from remote; add `secrets.env` to `.git`
- B. It prevents Git from tracking matching untracked files; add `secrets.env` or `.env` pattern to `.gitignore` in repo root
- C. It encrypts files; add to `.gitencrypt`
- D. It commits files faster; add to `.gitinclude`

****Q8.** You cloned a repo, made a commit, and `git push` fails with `! [rejected] main -> main (fetch first)`. Why and what to do?******

- A. GitHub is down; wait
- B. Remote has commits you don't have locally; run `git pull --rebase origin main` (or `fetch + merge/rebase`) then push
- C. You need to force push always
- D. You forgot `git add`

****Q9. On GitHub, what is the difference between a Pull Request and an Issue?****

- A. They are same
- B. Issue tracks bug/task/discussion; Pull Request proposes code changes to be reviewed and merged into a branch
- C. Pull Request is for private repos only
- D. Issue automatically merges code

****Q10. How do you discard all local modifications to a tracked file `index.html` and restore it to last committed state?****

- A. `git restore index.html` or `git checkout -- index.html`
- B. `git delete index.html`
- C. `git ignore index.html`
- D. `git remove index.html`

Beginner Answer Key & Explanations

****Q1 Answer: A - `git config --global user.name` and `user.email`****

These commands set the commit author identity globally in `~/.gitconfig` for all repos. From late 2023 GitHub also requires email verification but commit identity is still locally configured.

****Q2 Answer: B - `git add app.js` & `git commit -m "update app"`****

Git only commits what is staged in the index. Staging selectively allows you to commit `app.js` while `config.json` remains modified but unstaged.

****Q3 Answer: A - `git log --oneline --graph --all`****

This combination shows compact history with branch topology across all refs. It's the standard alias many teams use for understanding branch structure quickly.

****Q4 Answer: A - `git commit --amend -m "correct message"`****

`--amend` replaces the last commit with a new one containing fixed message and/or staged changes. It should only be used if the commit hasn't been pushed to a shared branch to avoid rewriting public history.

****Q5 Answer: B - `add upstream remote`, `fetch`, `merge`****

A fork has origin pointing to your copy. Adding an upstream remote pointing to original repo lets you fetch upstream/main and integrate updates, or use GitHub's Sync fork button in UI since 2021.

****Q6 Answer: A - `git checkout -b` and `git switch -c`****

Both create and switch in one step. `git switch -c` is the newer, safer command introduced in Git 2.23 to split checkout's many responsibilities.

****Q7 Answer: B - `.gitignore` prevents tracking untracked files****

`.gitignore` patterns tell Git to ignore unmatched files, so they won't show in `git status` or be added accidentally. It does not affect already tracked files unless you untrack them with `git rm --cached`.

****Q8 Answer: B - Remote ahead, need to pull/rebase first****

The error means remote main diverged; pushing would overwrite. Fetching then rebasing keeps linear history, the default strategy for protected main branches on GitHub in 2024-2026.

****Q9 Answer: B - Issue tracks task, PR proposes code changes****

Issues are for discussion/bugs, while Pull Requests link commits, trigger CI checks, code review, and merge workflows. GitHub can auto-close issues with keywords like `Fixes #123` in PR.

****Q10 Answer: A - `git restore index.html`****

`git restore` (since Git 2.23) restores working tree file from index/HEAD, discarding local edits. Older `git checkout -- <file>` does same but `restore` is clearer and less error-prone.

2. Intermediate Level (10 Questions)

****Q11.** You need to temporarily save your current changes without committing, then switch branches to hotfix, then return. Which sequence works and preserves untracked files too?

- A. `git stash push -m "wip" --include-untracked`, `git switch hotfix`, ... `git switch back`, `git stash pop`
- B. `git commit -m temp` && `git checkout hotfix`
- C. `git save stash`
- D. `git branch --stash`

****Q12.** What is the difference between `git merge` and `git rebase` when integrating feature into main, and when should you avoid rebase?

- A. Merge creates a merge commit preserving history; rebase rewrites feature commits on top of main for linear history. Avoid rebase on public/shared branches already pushed to others.
- B. They are identical
- C. Rebase deletes commits
- D. Merge rewrites history

****Q13.** You want to undo the last commit that is already pushed to origin/main, without rewriting history for teammates. What should you do?

- A. `git reset --hard HEAD~1` && `git push -f`
- B. `git revert HEAD` && `git push origin main`
- C. `git checkout HEAD~1`
- D. `git delete commit`

****Q14.** Your repo has a large `video.mp4` (400MB) accidentally committed. GitHub push fails. How do you remove it from history and prevent future large files for 2024-2026 best practice?

- A. `git rm video.mp4` and push
- B. Use `git filter-repo` or `git filter-branch` / BFG to rewrite history removing the file, add it to `.gitignore`, and use Git LFS for large files going forward, then force push with team coordination
- C. Just add to `.gitignore`
- D. Use `git large-file`

****Q15.** Explain `git cherry-pick` and give a scenario where it's useful.

- A. It copies a specific commit from one branch to current branch. Useful to backport a hotfix from main to release branch without merging entire branch.
- B. It deletes a cherry
- C. It merges all branches
- D. It picks random commits

****Q16.** You have main protected requiring PR reviews and status checks. How do you configure a GitHub branch protection rule and what happens if you try direct push to main?

- A. Repo -> Settings -> Branches -> Add rule for main, check Require pull request before merging and Require status checks, disallow bypass. Direct push is rejected with remote error.
- B. Enable via `git push protection`
- C. Can't protect main
- D. Use `.gitprotect` file

****Q17.** In a GitHub Actions workflow `.github/workflows/ci.yml`, you want to run tests on push to any branch and on PRs to main, using Node 20. What trigger and setup is correct for 2024-2026?

push:

pull_request:

branches: [main]

test:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- uses: actions/setup-node@v4

with: { node-version: 20 }

- run: npm ci && npm test

B. on: [manual only]

C. Use Jenkinsfile

D. on: push: branches: none

****Q18. What does git reset --soft HEAD~1, --mixed (default), and --hard do differently?****

A. --soft keeps changes staged in index, --mixed (default) keeps changes unstaged in working tree but unstages index, --hard discards all changes in index and working tree

B. All delete commits permanently

C. They are same as revert

D. --soft deletes working tree

****Q19. How do you resolve a merge conflict in app.js during git merge feature, and how does Git mark conflicts?****

A. Git marks with <<<<<< HEAD, =====, >>>>>> feature sections, edit file to choose/combine, then git add app.js and git commit to finish merge

B. Git auto deletes conflicted files

C. Run git conflict --resolve

D. Delete app.js

****Q20. What is GitHub's Pull Request workflow with forks and what does gh CLI do to streamline it?****

A. Fork -> clone fork -> branch -> commit -> push to fork -> open PR via GitHub UI or gh pr create, then review, CI, and gh pr merge or squash merge. gh authenticates and automates PR creation from terminal.

B. Push directly to upstream always

C. Email patches

D. Use FTP

Intermediate Answer Key & Explanations

****Q11 Answer: A - git stash push --include-untracked****

git stash saves dirty working directory and index to a stack, allowing clean switch. --include-untracked also stashes new files, and pop restores and drops the stash entry.

****Q12 Answer: A - Merge preserves history vs rebase rewrites linear****

Merge is non-destructive and keeps true topology, good for shared branches. Rebase creates cleaner linear log for feature branches but rewrites SHA, so avoid on public branches to prevent force-push disruption.

****Q13 Answer: B - git revert HEAD && git push****

revert creates a new commit that undoes the changes of a previous commit without deleting history. Reset + force push would rewrite public history and break collaborators.

****Q14 Answer: B - filter-repo + .gitignore + Git LFS****

git rm only removes from next commit, file remains in history and still blocks push >100MB GitHub limit. History rewriting via git filter-repo (recommended replacement for BFG/filter-branch since 2023) fully purges it, LFS stores large binaries outside Git.

****Q15 Answer: A - cherry-pick copies specific commit****

Cherry-pick creates new commit with same diff but new SHA on current branch. It's ideal for selective backports without pulling whole feature branch, though it can cause duplicate commits if later merged.

****Q16 Answer: A - Settings -> Branches -> protection rule****

Branch protection enforces review, status checks, signed commits, and blocks force push via API. Direct push returns GH006: Protected branch update failed because GitHub server rejects it.

****Q17 Answer: A - on push + pull_request with checkout@v4 and setup-node@v4****

This is canonical 2024-2026 Actions pattern. actions/checkout@v4 and setup-node@v4 are current stable versions with Node20 runtime, replacing deprecated Node16 actions.

****Q18 Answer: A - --soft vs --mixed vs --hard scope****

Understanding reset's three trees (HEAD, index, working tree) is key to undoing. Soft is safe for recommitting, mixed for unstaging, hard is destructive and permanently discards local work.

****Q19 Answer: A - Conflict markers <<<<<< ===== >>>>>>****

Git inserts markers around conflicting hunks from both sides. Developer must manually resolve, stage resolved file, and commit; tools like VS Code or git mergetool help parse markers.

****Q20 Answer: A - Fork-based PR + gh CLI****

Fork workflow maintains security for open-source. gh CLI (GitHub official) lets you gh auth login, gh repo fork, gh pr create --fill, gh pr view, gh pr checks without browser, adopted widely by 2024.

3. Advanced Level (10 Questions)

****Q21. Your team uses Conventional Commits and wants to auto-generate changelog and semantic versioning. A feature branch has 30 messy commits that must be cleaned into 3 logical commits before PR merge to main. Detail exact Git + GitHub strategy using interactive rebase, squash merge options, and protection to avoid losing authorship.****

A. Use git rebase -i main to squash/fixup 30 into 3, keep meaningful commit messages like feat: and fix:, force push to feature branch with --force-with-lease, open PR, then use Squash or Rebase merge strategy according to repo policy, with protected main requiring linear history

B. Use git merge --no-ff always

C. Delete commits and recommit

D. Use git reset --hard

****Q22. You need to debug when a bug was introduced between v2.0 and HEAD across 500 commits. What Git bisect strategy with automation and how does it work internally?****

A. git bisect start; git bisect bad HEAD; git bisect good v2.0; git bisect run npm test (or script returning 0/1), Git does binary search checking out middle commits, finds first bad commit in $-\log_2(500) \sim 9$ steps

B. Manually checkout each commit

C. Use git blame only

D. Use git log --grep

****Q23. Explain Git internal objects: blob, tree, commit, tag, and how HEAD, index, reflog, and .git/refs work. How would you recover a commit deleted by git reset --hard 2 days ago?****

A. Blob stores file content, tree maps names to blobs/trees, commit points to tree + parents + metadata, annotated tag is object. HEAD is ref to current branch, index is staging area. Recover via git reflog which logs HEAD moves, find SHA, then git branch recovery <SHA> or git checkout

B. Git stores files directly

C. Can't recover after hard reset

D. Use GitHub recycle bin

****Q24. You manage a monorepo with 10 teams. You need CODEOWNERS, required review from owning team, and prevent secrets leak. Configure GitHub's code ownership + push protection + secret scanning for 2024-2026.****

A. Add .github/CODEOWNERS with patterns /apps/payment/ @org/payment-team, enable Branch protection -> Require review from CODEOWNERS, enable Settings -> Security -> Secret scanning + Push protection, enable Private vulnerability reporting, use secret-scanning.yml and bypass list

B. Use .gitowners file

C. Can't enforce ownership

D. Use manual Slack messages

****Q25. A GitHub Actions workflow needs to deploy to AWS without storing long-lived AWS keys in GitHub Secrets. Also needs OIDC, environment approvals, and reusable workflows in 2025. Design it.****

A. Configure AWS IAM Identity Provider for GitHub OIDC (token.actions.githubusercontent.com), create IAM role with StringLike: repo:org/repo:* condition, use aws-actions/configure-aws-credentials@v4 with role-to-assume, use environment:

production with required reviewers, and call reusable workflow via uses: `./github/workflows/deploy.yml` or `org/reusable/./github/workflows/deploy.yml@v1`

B. Store `AWS_ACCESS_KEY` in Secrets always

C. Use root keys

D. Use personal access token

****Q26.** You have a large repo (5GB history). `git clone` is slow for CI. How do you optimize with partial clone, shallow clone, sparse-checkout, and GitHub's new features in 2024-2026?

A. Use `git clone --depth 1 --filter=blob:none --sparse` + `git sparse-checkout set --no-cone packages/app`, enable GitHub's sparse index and use `actions/checkout@v4` with `fetch-depth: 1` and `filter: blob:none`; for very large use Git LFS or split monorepo with submodules / subtree or use `scalar/git sparse-clone`

B. Always full clone

C. Use zip download

D. Use FTP

****Q27.** Explain `git worktree` vs `git submodule` vs `git subtree` vs `git subrepo` pattern. When would you use worktrees for parallel development in 2024?

A. `git worktree add ../hotfix hotfix-branch` creates linked working tree sharing same `.git` for parallel checkouts without switching; submodule links another repo at fixed SHA, requires separate commits; subtree merges external repo history into subdir. Worktrees are ideal for reviewing PR while continuing feature without stash.

B. All same feature

C. Worktree deletes repo

D. Submodule auto updates

****Q28.** You accidentally pushed secrets (AWS key `AKIA...`) to public GitHub repo main history 5 commits ago. GitHub push protection didn't block because bypass. Detail immediate mitigation steps including history rewrite, credential revocation, and GitHub security workflow 2024-2026.

A. Immediately revoke/rotate leaked key in AWS IAM, enable GitHub Secret Scanning alert, use `git filter-repo` or BFG to purge file from all history, force push with `--force-with-lease` after team warning, notify stakeholders, open GitHub Security advisory, add `trufflehog/gitleaks` pre-commit hook, enable push protection that blocks even admins

B. Just delete file and push

C. Make repo private temporarily and forget

D. Use `git rm` only

****Q29.** How does Git's 3-way merge algorithm work, what are `merge.ours`, `theirs`, `union`, and `recursive` vs `ort` strategy? Give config to auto-resolve lockfiles.

A. 3-way uses common ancestor base + `ours` + `theirs` to compute changes. `ort` (Ostensibly Recursive's Twin) is default since Git 2.33, faster than `recursive`. Drivers: `-X ours` keeps our side, `-X theirs` keeps theirs. For lockfiles you can set `echo 'package-lock.json merge=binary' >> .gitattributes` or custom driver, but better set `merge=ours` or `union` driver; e.g., `git config merge.unity.driver -` is anti-pattern, use `.gitattributes: npnm-lock.yaml merge=binary`

B. Git uses 2-way

C. Can't configure drivers

D. `ort` is deprecated

****Q30.** You need to enforce Signed Commits, verified by GitHub, and explain GPG vs SSH vs S/MIME signatures in 2024-2026. How do you set up commit signing for team?

A. Generate SSH key `ssh-keygen -t ed25519`, add signing key as allowed via GitHub Settings -> SSH and GPG keys -> Signing key, config `git config --global gpg.format ssh`, `git config --global user.signingkey ~/.ssh/id_ed25519.pub` and `git config --global commit.gpgsign true`; or GPG `gpg --gen-key` then `git config gpg.format openpgp`. GitHub verifies with Verified badge if email matches. Require via Branch protection -> Require signed commits and via Rulesets in 2024.

B. Signing not supported

C. Use PGP only since 2010

D. Signing encrypts commit

Advanced Answer Key & Explanations

****Q21 Answer: A - Interactive rebase to reword/squash/fixup****

Interactive rebase lets you collapse messy WIP commits into logical atoms preserving authorship with Co-authored-by trailer. Force-with-lease is safer than -f as it fails if remote changed, preventing accidental overwrites.

****Q22 Answer: A - git bisect with run script binary search****

Bisect automates binary search between good and bad to find regression in $O(\log n)$. git bisect run uses exit status to mark good/bad automatically, essential for large histories and CI reproducible bugs.

****Q23 Answer: A - Objects blob/tree/commit/tag + HEAD/index/reflog****

Git is content-addressable SHA-1/SHA-256 DAG. Reflog keeps HEAD history for ~90 days locally even if branch refs deleted, allowing recovery after reset unless GC pruned. Understanding objects explains why rewrites change SHA.

****Q24 Answer: A - CODEOWNERS + CODEOWNERS review + secret push protection****

CODEOWNERS enforces team ownership and review routing. GitHub's push protection (GA 2023) blocks secret pushes at client side and server, secret scanning alerts catch historical leaks, mandatory for enterprise compliance in 2024-2026.

****Q25 Answer: A - GitHub OIDC federation with AWS IAM role, environments, reusable workflows****

OIDC eliminates long-lived secrets via short-lived tokens with aud/sub claims. Environments add manual approvals and secrets scoping, reusable workflows deduplicate deployment logic following DRY and 2025 best practices.

****Q26 Answer: A - --filter=blob:none, --depth 1, sparse-checkout, scalar****

Partial clone fetches only needed objects, reducing 5GB to <200MB. Sparse-checkout limits working tree. GitHub Actions checkout supports filter and sparse. Scalar (Microsoft) optimizes huge monorepos with background prefetch and FSMonitor.

****Q27 Answer: A - worktree for parallel working trees vs submodule/subtree for nested repos****

Worktrees share object store and allow multiple branches checked out simultaneously without cloning twice. Submodules pin external repo at SHA (detached), subtree merges code, each has different update/merge semantics for dependency management.

****Q28 Answer: A - Revoke immediately, then filter-repo purge, then harden****

Rotation is step 1 because history remains public even after delete. Purge requires rewriting all branches/tags, force push, and notifying contributors to rebase. GitHub's push protection + pre-commit hooks with gitleaks prevent recurrence.

****Q29 Answer: A - 3-way merge with ort strategy and merge drivers****

Base+ours+theirs enables conflict detection; ort solves performance/memory of recursive for large merges. Merge drivers per .gitattributes allow custom handling, e.g., keeping generated lockfiles via binary or union to reduce noise.

****Q30 Answer: A - SSH signing (new in 2022) vs GPG vs S/MIME, enforced via Rulesets****

SSH signing is simpler as devs already have SSH keys, supported by GitHub since 2022, verification via key in account. GPG is legacy but still common. New Rulesets (2024 GA) replace branch protection and enforce signing organization-wide.

Course File: 25_RabbitMQ.md

RabbitMQ & Messaging - Certification Exam

****Course Code:**** 25_RabbitMQ

****Total Questions:**** 30 (10 Beginner, 10 Intermediate, 10 Advanced)

****Pass Mark:**** 70% (21/30)

****Time Limit:**** 90 minutes

****Version:**** 2024-2026 (RabbitMQ 3.12.x / 3.13.x, AMQP 0-9-1, AMQP 1.0 preview, Quorum Queues default)

Part A: Beginner Level (Questions 1-10)

Q1. What is the primary protocol used by RabbitMQ for messaging?

- A) HTTP/2
- B) AMQP 0-9-1
- C) MQTT 5.0 only
- D) gRPC

Q2. In RabbitMQ, what is the role of an Exchange?

- A) Stores messages persistently on disk
- B) Receives messages from producers and routes them to queues based on rules
- C) Consumes messages and processes them
- D) Authenticates client connections

Q3. Which exchange type routes messages to all bound queues regardless of routing key?

- A) Direct exchange
- B) Topic exchange
- C) Fanout exchange
- D) Headers exchange

Q4. What is a vhost in RabbitMQ?

- A) A virtual machine hosting RabbitMQ
- B) A logical grouping of exchanges, queues, and permissions providing multi-tenancy
- C) A backup of queue data
- D) A type of consumer

Q5. Which of the following is the default port for AMQP client connections to RabbitMQ?

- A) 5432
- B) 6379
- C) 5672
- D) 15672

Q6. What happens to a non-persistent message when RabbitMQ restarts?

- A) It is recovered from disk

- B) It is lost
- C) It is moved to a dead-letter exchange
- D) It is automatically re-queued with higher priority

Q7. Which RabbitMQ management plugin port serves the HTTP management UI and REST API by default?

- A) 5672
- B) 25672
- C) 15672
- D) 1883

Q8. In RabbitMQ, what does basic.ack do?

- A) Publishes a new message
- B) Acknowledges that a message was successfully processed by a consumer
- C) Creates a new queue
- D) Binds a queue to an exchange

Q9. What is the purpose of a binding in RabbitMQ?

- A) To encrypt messages
- B) To link an exchange to a queue with an optional routing key / pattern
- C) To bind a consumer to a vhost
- D) To create a user permission

Q10. Which client library is the official RabbitMQ-maintained Java client?

- A) rabbitmq-java-client (com.rabbitmq:amqp-client)
- B) spring-amqp
- C) qpuid-jms-client
- D) pika

Answer Key - Beginner Level

| Question | Correct Answer | Explanation |

|-----|-----|-----|

| Q1 | B | AMQP 0-9-1 is the core protocol. RabbitMQ also supports MQTT, STOMP, AMQP 1.0 via plugins, but primary is AMQP 0-9-1. |

| Q2 | B | Exchanges are routers. Queues store messages. Producers never write directly to queues in AMQP model. |

| Q3 | C | Fanout ignores routing key and broadcasts to all bound queues. |

| Q4 | B | Vhosts isolate resources and permissions, similar to virtual hosts / namespaces. |

| Q5 | C | 5672 for AMQP, 5671 for AMQP TLS. 15672 is management UI, 25672 clustering, 1883 MQTT. |

| Q6 | B | Non-persistent messages exist only in RAM. Persistent requires both persistent delivery_mode=2 AND durable queue. |

| Q7 | C | 15672 is management UI. Enabled via rabbitmq-plugins enable rabbitmq_management. |

| Q8 | B | Consumer acknowledgments prevent message loss. Unacked messages are requeued on consumer failure. |

| Q9 | B | Binding defines routing relationship. For direct: exact key match. Topic: pattern match (* and #). |

| Q10 | A | com.rabbitmq:amqp-client is official. Pika is Python, Spring AMQP builds on top of official client. |

Part B: Intermediate Level (Questions 11-20)

Q11. As of RabbitMQ 3.12+, which queue type is the default and recommended replacement for classic mirrored queues?

- A) Classic queues with ha-mode=all
- B) Quorum Queues (Raft-based replicated FIFO)
- C) Stream queues only
- D) Transient in-memory queues

Q12. What does the x-message-ttl argument do when applied to a queue?

- A) Sets how long a queue can be unused before auto-delete
- B) Sets per-message expiration for all messages in that queue if no per-message TTL is set
- C) Sets consumer timeout
- D) Sets clustering heartbeat timeout

Q13. How does RabbitMQ handle a message that is nacked with `requeue=false` and has a Dead Letter Exchange (DLX) configured?

- A) Discards it silently
- B) Routes it to the configured DLX
- C) Returns it to the original producer
- D) Blocks the channel

Q14. What is the purpose of prefetch count (QoS) setting `channel.basicQos(10)`?

- A) Limits total messages in queue to 10
- B) Limits unacknowledged messages a consumer can have to 10, enabling fair dispatch and backpressure
- C) Sets max publishers to 10
- D) Configures cluster size

Q15. In a Topic Exchange, what do wildcards `*` and `#` mean?

- A) `*` = exactly one word, `#` = zero or more words separated by dots
- B) `*` = zero or more words, `#` = exactly one word
- C) Both mean same: any string
- D) `*` = any character, `#` = numeric only

Q16. What is a quorum queue's behavior when a minority of nodes are partitioned?

- A) Continues accepting writes on all sides (AP)
- B) Remains available only if majority of replicas are reachable, sacrificing availability for consistency
- C) Converts to classic queue automatically
- D) Deletes all messages

Q17. Which feature was introduced to replace classic mirrored queue deprecation and provides a non-destructive replayable log for large

fanouts / event sourcing?

- A) Exclusive queues
- B) Streams (RabbitMQ Streams)
- C) Lazy queues
- D) Federation plugin

Q18. What is Publisher Confirms (confirm.select) used for?

- A) To confirm consumer received message
- B) To get asynchronous acknowledgment from broker that message was taken responsibility for (written to queue / replicated)
- C) To confirm TCP connection
- D) To authenticate publisher

Q19. How does mandatory flag in basic.publish work?

- A) Forces message to be persistent
- B) If true, broker returns unroutable messages via basic.return instead of silently dropping them
- C) Forces TLS
- D) Makes queue durable automatically

Q20. What is the function of the rabbitmq_shovel plugin?

- A) Monitors queue depth
- B) Reliably moves/copies messages from a queue/exchange in one broker to another broker/exchange (including cross-DC)
- C) Deletes dead letters
- D) Encrypts messages at rest

Answer Key - Intermediate Level

| Question | Correct Answer | Explanation |

|-----|-----|-----|

| Q11 | B | Since 3.8, Quorum Queues using Raft are recommended. Since 3.12, classic queue mirroring (ha-mode) was removed. Quorum queues are default for new queues in some client wrappers from 2024+. |

| Q12 | B | x-message-ttl is queue-level default TTL. Can be overridden by per-message expiration. Triggers dead-lettering or discard. |

| Q13 | B | DLX routing occurs on reject/nack requeue=false, TTL expiry, max-length overflow. Requires x-dead-letter-exchange arg. |

| Q14 | B | QoS prefetch is critical for work queues. Without it one slow consumer could hoard all messages. Recommended 10-50 for throughput. |

| Q15 | A | Topic spec: * substitutes exactly one dot-separated word, # substitutes zero or more words. Example: logs.# matches logs.error.db |

| Q16 | B | Quorum queues are CP (consistent) via Raft. Need (N/2)+1 replicas. Minority partition cannot elect leader / accept writes. |

| Q17 | B | Streams (since 3.9) provide append-only log, high throughput, non-destructive consuming with offsets. Ideal for event sourcing. |

| Q18 | B | Publisher confirms guarantee broker responsibility. Required for reliable publishing. waitForConfirms or async ConfirmListener. |

| Q19 | B | Mandatory + alternate exchange are two ways to handle unroutable messages. Without either, unroutable messages are dropped. |

| Q20 | B | Shovel is core for inter-broker replication, much more reliable than federation for point-to-point WAN replication. Dynamic shovels since 3.12. |

Part C: Advanced Level (Questions 21-30)

Q21. In RabbitMQ 3.13, what is the default `quorum_commands.soft_limit` change and how does it improve memory management for large quorum queue deployments?

- A) It was removed; quorum queues now use unbounded memory
- B) It limits number of enqueued Raft commands before applying backpressure to publishers, preventing unbounded heap growth (new default `32*queue_count`)
- C) It limits number of consumers per quorum queue
- D) It forces all messages to be transient

Q22. Consider a super-stream (partitioned stream) with 6 partitions. A publisher uses routing key based hashing. Consumer uses Single Active Consumer (SAC) per partition. What guarantees does this provide and what does RabbitMQ 3.12+ recommend for scaling?

- A) Global total order across all partitions
- B) Per-partition order, SAC ensuring only one consumer processes a partition at a time, enabling horizontal scaling; use super streams to increase throughput while preserving key-based ordering
- C) No ordering at all
- D) Requires classic mirrored queues for ordering

Q23. In a production cluster with 3 nodes, you see frequent resource alarm: free disk space alarm triggering flow control. Quorum queues pause accepting publishes. What is the correct 2024-2026 approach to mitigate?

- A) Disable disk monitoring via `disk_free_limit: disabled`
- B) Set `disk_free_limit.relative = 1.0` and add disk capacity, configure quorum queue to use separate data dir on fast SSD, set retention policies, enable lazy/stream for large backlogs; never disable alarm in prod
- C) Delete `mnesia` dir
- D) Increase `vm_memory_high_watermark` to 1.5

Q24. What is the impact of setting `queue_master_locator = min-masters` vs `client-local` in classic queue context, and what is the equivalent concern for quorum queues in 3.12+?

- A) No impact; both obsolete. For quorum queues, load balancing is based on Raft leader distribution and can be rebalanced with `rabbitmq-queues rebalance all`
- B) `min-masters` still required for quorum queues via config
- C) `client-local` is mandatory for streams
- D) Master locator controls TLS ciphers

Q25. How does AMQP 1.0 protocol support (added as native in RabbitMQ 3.12 via `rabbitmq_amqp1_0` disabled by default / 4.x enabled) differ from AMQP 0-9-1 handling of flow control and addressing?

- A) AMQP 1.0 uses credit-based flow control (link credits) and address-centric model (queue addresses as first-class), vs 0-9-1 channel-level prefetch and exchange-binding model

- B) AMQP 1.0 is identical to 0-9-1
- C) AMQP 1.0 only supports fanout
- D) AMQP 1.0 removes queue concept entirely

Q26. You have a latency-sensitive workload: 100k msg/s, 1KB messages, at-least-once delivery required. Which combination delivers lowest latency while ensuring durability in RabbitMQ 3.13?

- A) Classic durable queues with publisher confirms sync, auto-ack, no prefetch
- B) Quorum queues with 3 replicas, async publisher confirms (correlation ID), manual ack, prefetch 50-200, x-queue-type=quorum, persistent messages, consumer with single active consumer off=false, use fast SSD, increase raft log compaction
- C) Transient queues with no ack
- D) Use management HTTP API for publishing

Q27. What is Poison Message handling best practice in quorum queues vs classic queues (2024 pattern)?

- A) Infinite requeue loop is recommended
- B) Use x-delivery-limit (quorum queue native argument, e.g., 20) to dead-letter after N redeliveries, combined with DLX for quarantine queue and x-single-active-consumer inspection; classic queues need manual tracking via header or retry exchange topology
- C) Poison messages auto-delete in quorum queues
- D) Disable ack to drop poison messages

Q28. In RabbitMQ Streams, what does OffsetTracking with autoCommitInterval and consumerUpdate listener enable for Single Active Consumer failover?

- A) Nothing, SAC not supported for streams
- B) Enables stateful offset tracking and ability for new active consumer to resume from last committed offset + custom logic via update listener to decide starting offset (e.g., replay last 10 minutes on failover)
- C) Forces consumer to start from first message always
- D) Only enables message deletion

Q29. A financial system requires exactly-once processing semantics. How can RabbitMQ be used to achieve *effectively-once* end-to-end given AMQP limitations?

- A) RabbitMQ alone guarantees exactly-once natively with exactly-once=true queue arg
- B) Achieve effectively-once by combining idempotent consumers (idempotency key stored in idempotent DB/table), transactional outbox pattern for producers, publisher confirms + quorum queue/stream persistence, deduplication plugin rabbitmq_deduplication (or client-side idempotency), and consumer manual transaction commit after processing
- C) Set delivery_mode=2 achieves exactly-once
- D) Use auto-ack=true for exactly-once

Q30. How does Khepri metadata store (introduced experimentally in RabbitMQ 3.13, default in 4.0+) improve over Mnesia for cluster metadata?

- A) Khepri uses Raft-based replicated state machine for schema/metadata, removing Mnesia's split-brain and table load issues, improving boot time and stability for large clusters (1000s queues) and enabling faster recovery; backward

incompatible migration path managed via feature flags

B) Khepri is slower but uses less memory

C) Khepri removes need for queues

D) Khepri only used for MQTT sessions

Answer Key - Advanced Level

| Question | Correct Answer | Explanation |

|-----|-----|-----|

| Q21 | B | 3.13 introduced `soft_limit` for quorum command batching to control memory. Default formula prevents OOM when publishers outpace Raft log apply. Tunable via `rabbitmq.conf`. |

| Q22 | B | Super streams partition by key hash; order guaranteed per partition. SAC per partition avoids competing consumers. 3.12+ super stream client support matured (Java/Go/Python). |

| Q23 | B | Disabling disk alarm risks data loss/corruption. Proper fix: expand disk, monitor, use separate data dirs, set retention. Memory watermark 0.4-0.6 typical, not >1. |

| Q24 | A | Master locator removed with mirroring removal. For quorum queues: leader balanced via `rabbitmq-queues` `rebalance`. Can also use `rabbitmq-queues --vhost ... rebalance`. |

| Q25 | A | Core AMQP 1.0 difference: credit flow, link-based. RabbitMQ maps AMQP 1.0 addresses to queues/exchanges. Important for interoperability in 2024-2026. |

| Q26 | B | Low latency + durability requires async confirms, quorum for HA, prefetch for steady flow, SSD, Raft tuning. Sync confirms kill throughput. |

| Q27 | B | `x-delivery-limit` is native poison handling for quorum queues since 3.8.11+. Classic needs custom retry topology (retry -> wait -> DLX -> main). |

| Q28 | B | `OffsetTracking` + `Consumer Update` is 3.12+ feature allowing smart SAC failover, e.g., resume with offset spec or handle promotions. |

| Q29 | B | True exactly-once impossible in distributed messaging alone. Effectively-once requires idempotency + outbox + deduplication. This is industry standard pattern for 2024-2026. |

| Q30 | A | Khepri replaces Mnesia. Uses Raft. Solves long-standing Mnesia limitations for large metadata sets. Feature flag `khepri_db` introduced 3.13, GA 4.0. |

****End of Exam****

****Author Notes:**** Questions validated against RabbitMQ docs: <https://www.rabbitmq.com/docs>, Release notes 3.12 (2023), 3.13 (2024), 4.0 GA (2024 H2). Quorum queues mandatory HA model, classic mirroring removed. Streams & Super Streams production-ready. Khepri feature flag awareness required for advanced level.

Course File: 26_ELK.md

Elasticsearch & ELK Stack - Certification Exam

****Course Code:**** 26_ELK

****Total Questions:**** 30 (10 Beginner, 10 Intermediate, 10 Advanced)

****Passing Score:**** 70%

****Time Limit:**** 90 minutes

****Version:**** 2024-2026 (Elasticsearch 8.x / 8.13+)

Part A: Beginner Level (Questions 1-10)

Q1. What does ELK stand for?

- A) Elasticsearch, Logstash, Kibana
- B) Elastic, Log, Kubernetes
- C) Event, Log, Knowledge
- D) Extract, Load, Kibana

Q2. What is the default port for Elasticsearch HTTP API?

- A) 5601
- B) 9200
- C) 9300
- D) 5044

Q3. What is the default port for Kibana?

- A) 9200
- B) 5601
- C) 9300
- D) 8080

Q4. What type of database is Elasticsearch classified as?

- A) Relational database
- B) Document-oriented NoSQL / Search engine
- C) Graph database
- D) Columnar database

Q5. In Elasticsearch, what is an index analogous to in relational terms?

- A) A row
- B) A database / table
- C) A column
- D) A foreign key

Q6. Which component of ELK is primarily responsible for data ingestion and transformation?

- A) Kibana
- B) Elasticsearch

- C) Logstash
- D) Beats

Q7. What is a document in Elasticsearch?

- A) A PDF file
- B) A JSON object that is the basic unit of information stored
- C) A shard replica
- D) A configuration file

Q8. Which query language is used in Kibana's Discover and for filtering?

- A) SQL only
- B) KQL (Kibana Query Language) and Lucene query syntax / ESQL in 8.11+
- C) PromQL
- D) GraphQL

Q9. What are Beats in the Elastic Stack?

- A) Heavy ETL servers
- B) Lightweight data shippers installed on edge servers
- C) Visualization plugins
- D) Machine learning jobs

Q10. Which REST method is used to index a new document with auto-generated ID?

- A) GET
- B) DELETE
- C) POST to /index/_doc
- D) PUT to /_cluster/settings

Answer Key - Beginner

| Q | Answer | Explanation |

|---|-----|-----|

| 1 | A | ELK = Elasticsearch, Logstash, Kibana. Now called Elastic Stack with addition of Beats. |

| 2 | B | 9200 is HTTP REST API, 9300 is transport/inter-node communication. |

| 3 | B | Kibana default is 5601. |

| 4 | B | Elasticsearch is a distributed document-oriented search engine built on Lucene. |

| 5 | B | Index ~ database/table, document ~ row, field ~ column (approx analogy). |

| 6 | C | Logstash processes logs/events: inputs -> filters -> outputs. |

| 7 | B | Documents are JSON objects containing fields. |

| 8 | B | Kibana supports KQL (default since 7.x) and Lucene, plus ES|QL introduced in 8.11 as new piped query language. |

| 9 | B | Beats (Filebeat, Metricbeat, etc.) are lightweight shippers. |

| 10 | C | POST /<index>/_doc creates document with auto ID; PUT /<index>/_doc/<id> with specified ID. |

Part B: Intermediate Level (Questions 11-20)

Q11. In Elasticsearch 8.x, what happened to mapping types?

- A) You can have unlimited types per index
- B) Mapping types were removed; only `_doc` is implicit (single type per index)
- C) Types are mandatory
- D) Types were replaced by SQL tables

Q12. What is the purpose of a Logstash grok filter?

- A) To encrypt data
- B) To parse unstructured log data into structured fields using regex patterns
- C) To compress indices
- D) To visualize data

Q13. Which Elasticsearch feature ensures high availability by copying shards?

- A) Aliases
- B) Replica shards
- C) Curator
- D) Ingest pipelines

Q14. What is an inverted index in Elasticsearch/Lucene?

- A) A reverse proxy
- B) Data structure mapping terms to the documents that contain them for fast full-text search
- C) An index sorted in descending order
- D) A backup index

Q15. Which API is used for bulk indexing/updating large volumes efficiently?

- A) `_search` API
- B) `_bulk` API with NDJSON format
- C) `_cat` API
- D) `_mget` API

Q16. What is ILM in Elasticsearch?

- A) Index Lifecycle Management - automates rollover, shrink, freeze, delete phases
- B) Internal Log Manager
- C) Ingest Log Mapper
- D) Integrated License Manager

Q17. As of Elasticsearch 8.13+, what is the primary vector search capability?

- A) No vector search support
- B) `dense_vector` field type with kNN search and HNSW algorithm, supporting cosine, dot_product, l2_norm
- C) Only through plugin
- D) Using Logstash only

Q18. In Kibana, what does a Data View (formerly Index Pattern) do?

- A) Deletes data
- B) Defines which Elasticsearch indices to query and the time field for visualizations
- C) Backs up Elasticsearch
- D) Encrypts indices

Q19. Which Filebeat module would you use to collect and parse Nginx logs?

- A) system module
- B) nginx module with access and error filesets
- C) elasticsearch module
- D) You must write custom Logstash config only

Q20. What does the Elasticsearch `_source` field contain?

- A) Only metadata
- B) The original JSON document body as indexed
- C) Shard allocation
- D) User credentials

Answer Key - Intermediate

| Q | Answer | Explanation |

|---|-----|-----|

| 11 | B | Mapping types deprecated in 6.x and removed in 7.0+. In 8.x, single type, typeless APIs. |

| 12 | B | Grok uses `%{PATTERN:field}` to extract structured data. Dissect is faster alternative since 7.x. |

| 13 | B | Replica shards are copies of primary shards placed on different nodes for HA and read scale. |

| 14 | B | Inverted index is core to Lucene: term -> doc IDs, enables O(1) term lookups. |

| 15 | B | `_bulk` API uses NDJSON: action line + optional source line. Far more efficient than individual requests. |

| 16 | A | ILM manages hot -> warm -> cold -> frozen -> delete phases. Used with data streams and rollover. |

| 17 | B | `dense_vector` added kNN since 8.0, GA improved in 8.4+. HNSW graph, int8 quantization in 8.7+, BBF in 8.13+. Essential for RAG/gen AI. |

| 18 | B | Data View ties Kibana visualizations to one or more indices (e.g., `logs-*`) and time field. |

| 19 | B | Filebeat modules pre-configure inputs, parsers, pipelines, dashboards: filebeat modules enable nginx. |

| 20 | B | `_source` is stored original JSON. Can be disabled or filtered but usually retained for reindex, highlighting, fetch. |

Part C: Advanced Level (Questions 21-30)

Q21. You have a 100GB daily log volume with 30-day retention. Which 2024-2026 best-practice architecture should you use?

- A) One large index per month with 5 primary shards
- B) Data stream with ILM rollover (e.g., 50GB or age) + hot-warm-cold tiers and searchable snapshots on S3/GCS
- C) Single index with 100 shards
- D) Daily indices without ILM or aliases

Q22. A cluster suffers from mapping explosion (10,000+ fields). How to mitigate in Elasticsearch 8.x?

- A) Increase `indices.query.bool.max_clause_count`
- B) Use flattened, object with `enabled:false`, `ignore_above`, `dynamic: strict/runtime`, and re-evaluate dynamic templates; consider ECS
- C) Add more heap
- D) Disable replicas

Q23. What is the difference between text and keyword field types?

- A) No difference
- B) text is analyzed/tokenized for full-text search, keyword is not analyzed and used for exact match, aggregations, sorting via `doc_values`
- C) keyword is for full-text, text is for sorting
- D) Both deprecated

Q24. In Elasticsearch 8.x security is enabled by default. What ships by default since 8.0?

- A) No security
- B) TLS enabled for transport, security auto-configured with enrollment token, elastic user password generated at bootstrap
- C) Only HTTPS required
- D) Only API keys

Q25. How does cross-cluster search (CCS) differ from cross-cluster replication (CCR)?

- A) Same thing
- B) CCS allows searching remote clusters from coordinating cluster (query fan-out), CCR replicates indices shard-by-shard for DR / locality (requires Platinum/Enterprise license)
- C) CCS replicates data
- D) CCR is free, CCS is paid

Q26. You need P95 latency aggregations over 1 billion events in Kibana Lens. Which aggregation is most accurate and performant in 8.x?

- A) avg aggregation
- B) HDR histogram backed `tdigest` / percentile aggregation, or `ES|QL STATS` with `PERCENTILE` in 8.11+
- C) terms aggregation
- D) scripted field

Q27. What is the role of a coordinating-only node (`node.roles: []`) and a dedicated ingest node?

- A) Both hold data
- B) Coordinating node routes requests, aggregates results, reduces data-node overhead; ingest node runs ingest pipelines for preprocessing
- C) Master election only
- D) Required for Kibana

Q28. For 2024-2026 observability, which Elastic Stack approach is recommended to replace legacy Logstash-heavy ingestion?

- A) Only Logstash is supported
- B) Elastic Agent with Fleet-managed integrations (replaces multiple Beats) with edge processing, plus Logstash only where advanced transformation needed
- C) Disable ingestion
- D) Use Kafka alone

Q29. A query using wildcard "*"error*" at start is slow. How to optimize?

- A) Keep as-is
- B) Avoid leading wildcard; use edge n-gram/autocomplete, search_as_you_type, keyword with wildcard field type (2021+) which uses ngrams for efficient wildcard/regex, or use ESQL LIKE optimizations
- C) Increase shards
- D) Use SQL

Q30. You are designing ES|QL (Elasticsearch Query Language, GA in 8.15) query for security analytics. Which ES|QL statement is valid?

- A) `SELECT * WHERE log.level = "error"`
- B) `FROM logs-* | WHERE @timestamp > NOW() - 1 hour | STATS count=COUNT(*) BY host.name | SORT count DESC | LIMIT 10`
- C) `GET logs/_search {match_all}`
- D) `db.logs.find({level: "error"})`

Answer Key - Advanced

| Q | Answer | Explanation |

|---|-----|-----|

| 21 | B | Data streams are recommended for time-series in 8.x. Rollover prevents oversized shards (ideal 10-50GB). Searchable snapshots reduce storage cost; hot-warm-cold ILM introduced in 7.x, refined 8.x. |

| 22 | B | Mapping explosion causes heap/memory issues. Mitigate via flattened for dynamic key-value, strict mapping control, runtime fields (8.x) to avoid indexing unused, ECS normalization. |

| 23 | B | Core concept: text -> inverted index tokens, keyword -> doc_values for sort/agg/exact. Use multi-fields: "field" text + "field.keyword". |

| 24 | B | In 8.0 security enabled by default. Bootstrap creates certs, enrollment token for Kibana/nodes, enables TLS. Previously opt-in. |

| 25 | B | CCS = search across clusters (platinum?). CCR = replication (active-passive/follow). Important distinction for global clusters. CCS is available on all tiers, CCR needs enterprise/platinum. |

| 26 | B | Percentile via tdigest (default) or HDR (more accurate but memory). ES|QL 8.11+ introduces PERCENTILE. For high-cardinality use sampling or rollups. |

| 27 | B | Architecture design: dedicate master, data hot/warm, coordinating (load balancer), ingest, ML nodes. Coordinating node smart routing. |

| 28 | B | 2024 strategy: Elastic Agent unified agent (single binary replaces Beats), Fleet central management, prebuilt integrations (500+). Logstash still valid for heavy transformation. |

| 29 | B | Leading wildcard cannot use index and triggers full scan. wildcard field type uses binary doc values + ngram index to accelerate contains/regex. Edge n-grams better for autocomplete. |

| 30 | B | ES|QL is piped language: FROM source | WHERE | STATS | SORT | LIMIT. Introduced tech preview 8.11, GA 8.13-8.15. Becoming default in Discover. Correct piped syntax is option B. |

Exam Summary

****Topics Covered:****

- ELK components, ports, beats
- Indices, documents, shards, replicas
- Logstash grok, Filebeat modules
- Mapping types removal, text vs keyword, _source
- _bulk, inverted index, ILM, Data Views
- Security on by default (8.x), TLS, enrollment
- Data streams, searchable snapshots, hot-warm-cold
- Vector search / kNN / HNSW / RAG use cases 2024-2026
- Elastic Agent / Fleet, ES|QL, wildcard field optimization
- CCS vs CCR, cluster roles, percentile aggregation, mapping explosion mitigation

****Elasticsearch Versions Referenced:**** 8.0 - 8.15+ (2024-2026 relevant)

****License Considerations:**** Basic vs Platinum/Enterprise for CCR, ML, searchable snapshots

Course File: 27_Prometheus.md

Course Title: Prometheus & Grafana - Monitoring and Observability Deep Dive

Section 1: Beginner (Questions 1-10)

****1. What is the primary data model used by Prometheus for storing time-series data?****

- A) Documents with JSON fields
- B) Multi-dimensional time series identified by metric name and key/value labels
- C) Relational tables with timestamp columns
- D) Graph nodes and edges

****2. By default, on which port does the Prometheus server listen?****

- A) 3000
- B) 9090
- C) 8080
- D) 9093

****3. In Prometheus, what is an Exporter?****

- A) A Grafana plugin to export dashboards to PDF
- B) A piece of software that exposes existing metrics from third-party systems in Prometheus format
- C) A component that exports data from Prometheus to long-term storage
- D) A tool to convert PromQL to SQL

****4. Which HTTP endpoint do Prometheus exporters typically use to expose metrics?****

- A) /api/v1/metrics
- B) /metrics
- C) /prom/metrics
- D) /stats

****5. What is Grafana's primary role in a Prometheus stack?****

- A) To scrape and store metrics
- B) To trigger alerts via Alertmanager
- C) To visualize metrics, build dashboards, and explore data from multiple data sources
- D) To manage service discovery

****6. What are the four official Prometheus metric types?****

- A) Count, Sum, Average, Percentile
- B) Counter, Gauge, Histogram, Summary
- C) Integer, Float, String, Boolean
- D) Rate, Increase, Delta, Irate

****7. Which PromQL function calculates the per-second average rate of increase for a Counter?****

- A) increase()
- B) delta()
- C) rate()
- D) avg_over_time()

****8. In Grafana, what is a Data Source?****

- A) A folder where dashboards are stored
- B) A connection configuration to a backend system like Prometheus, Loki, or MySQL where Grafana queries data
- C) A user role for viewing panels
- D) An alert notification channel

****9. What file does Prometheus use for its main configuration?****

- A) prometheus.yaml
- B) prometheus.yml
- C) prometheus.json
- D) prom.conf

****10. What is the purpose of Alertmanager in the Prometheus ecosystem?****

- A) To visualize firing alerts on Grafana dashboards
- B) To handle alerts sent by Prometheus server, including deduplicating, grouping, silencing, and routing them to receivers like Slack, PagerDuty, or email
- C) To automatically fix the underlying issue that caused the alert
- D) To scrape alerting rules from targets

Answer Key - Beginner

| Question | Correct Answer | Explanation |

|---|---|---|

- | 1 | B | Prometheus stores data as time series identified by metric name + label set. |
- | 2 | B | Prometheus server runs on 9090 by default; Grafana on 3000; Alertmanager on 9093. |
- | 3 | B | Exporters (e.g., node_exporter, blackbox_exporter) translate system metrics to Prometheus exposition format. |
- | 4 | B | /metrics is the standard exposition endpoint. |
- | 5 | C | Grafana is the visualization and dashboarding layer, not storage or scraping. |
- | 6 | B | The four core types are Counter, Gauge, Histogram, and Summary. |
- | 7 | C | rate() calculates per-second rate; increase() gives total increase over range. |
- | 8 | B | Data Source defines where Grafana gets data from. |
- | 9 | B | Main config file is prometheus.yml. |
- | 10 | B | Alertmanager manages the alert lifecycle after Prometheus evaluates rules. |

Section 2: Intermediate (Questions 11-20)

****11. Given a Counter metric http_requests_total, which PromQL query correctly shows the 5-minute per-second request rate aggregated by status code?****

- A) http_requests_total[5m] by (code)
- B) rate(http_requests_total[5m])
- C) sum by (code) (rate(http_requests_total[5m]))
- D) avg(rate(http_requests_total[5m])) by (code)

****12. In Prometheus service discovery, what does relabeling allow you to do?****

- A) Encrypt labels for security
- B) Modify label sets for discovered targets before scraping - filter targets, change __address__, set custom labels
- C) Rename metrics in Grafana
- D) Change the retention period of time series

****13. What is the difference between a Prometheus Histogram and Summary?****

- A) No difference, they are aliases
- B) Histogram exposes bucket counts and is calculated client-side for aggregatable quantiles; Summary exposes pre-calculated quantiles on client-side and cannot be aggregated across instances
- C) Summary is for counters, Histogram is for gauges
- D) Histogram requires Grafana Enterprise; Summary is open-source

****14. As of Prometheus 2.50+ / 3.x (2024-2026), which feature improves downsampling and query performance for looking back at old data?****

- A) Agent Mode only

B) Native Histograms and improved OpenTelemetry compatibility, plus expanded scrape with UTF-8 metric names support and Prometheus 3.x's new UI and performance improvements

C) Removal of TSDB

D) SQL query engine

****15. In Grafana, what is the purpose of \$__rate_interval variable in Prometheus dashboards?****

A) A fixed 5m interval

B) A dynamic variable that calculates a suitable rate interval based on scrape interval + panel time range, to avoid extrapolation and missing data artifacts

C) The alert evaluation interval

D) The dashboard refresh rate

****16. You have high cardinality issue: metric user_session_info{user_id="...">1M values...}. What is the best practice?****

A) Keep it, Prometheus handles unlimited cardinality

B) Drop the high-cardinality label via metric_relabel_configs or redesign metric to not use user_id as label; use logs/tracing for user-specific data

C) Increase Prometheus RAM to 512GB

D) Move label to metric name

****17. What does promtool provide?****

A) A GUI for Prometheus config

B) CLI tool to check config validity (promtool check config), check rules, debug PromQL, and perform TSDB operations like block analysis

C) A Grafana dashboard importer

D) A replacement for Alertmanager

****18. In Grafana Alerting (Grafana 9+ unified alerting, 2024-2026), where are alerts evaluated?****

A) Only in Prometheus

B) Only in Alertmanager

C) In Grafana itself using its own alert engine that can query multiple data sources (Prometheus, Loki, etc.), then optionally route to Grafana-managed or external Alertmanager

D) In the browser

****19. Which PromQL selector returns time series that have NOT been stable for 10 minutes?****

A) ALERTS{alertstate="firing"}

B) changes(metric[10m]) > 0

C) metric and (metric offset 10m)

D) absent(metric[10m])

****20. What is Recording Rule in Prometheus?****

A) A rule to record raw logs to disk

B) A precomputed frequently needed PromQL expression stored as a new time series for faster dashboard queries and to reduce load

C) A rule to record alerts as audio files

D) A Grafana feature to record dashboard usage

Answer Key - Intermediate

| Question | Correct Answer | Explanation |

|---|---|---|

| 11 | C | sum by (code) (rate(...)) correctly calculates per-second rate then aggregates by code label. A is invalid syntax, B misses aggregation, D uses avg incorrectly. |

| 12 | B | Relabeling (relabel_configs and metric_relabel_configs) mutates target labels to control scraping. |

| 13 | B | Histogram: server-side quantile via `histogram_quantile()` and aggregatable. Summary: client-side quantiles, not aggregatable. Also Native Histograms (SPARSE histograms) stable as of 2024+. |

| 14 | B | Prometheus 2.50+ added stable Native Histograms, OTEL UTF-8 names, and Prometheus 3.0 (Nov 2024) introduced new UI, improved query performance, and better compression. |

| 15 | B | $\$_{rate_interval} = \max(\$_{interval} + scrape_interval, 4 * scrape_interval)$ prevents aliasing. Recommended since Grafana 9. |

| 16 | B | High cardinality kills TSDB. Never use unbounded IDs as labels. Drop or refactor to logs/traces. |

| 17 | B | promtool is essential for validation and TSDB debugging. |

| 18 | C | Unified Alerting evaluates inside Grafana, supports many data sources, routes via contact points or external AM. |

| 19 | B | `changes()` counts value changes in range vector; alternative: `last_over_time` comparison. |

| 20 | B | Recording rules create new series like `job:http_requests:rate5m` for efficiency. |

Section 3: Advanced (Questions 21-30)

****21.** In Prometheus TSDB, what is the purpose of the Head block vs. persisted blocks, and how does WAL work?

- A) Head block is for alerts, persisted blocks are for metrics; WAL is Write-Ahead Log for Grafana
- B) Head block is mutable in-memory block for recent samples (2h default); WAL ensures durability on crash by logging samples; every 2h Head is compacted to immutable persisted block on disk with compaction and retention
- C) Head block stores config, persisted blocks store logs
- D) WAL means Web Alert Logging

****22.** You are querying `histogram_quantile(0.99, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))` and getting high 99th percentile. With Prometheus Native Histograms (2024+), what is the modern equivalent?

- A) No equivalent exists
- B) `histogram_quantile(0.99, sum by (le) (...))` still required
- C) `histogram_quantile(0.99, sum(rate(http_request_duration_seconds[5m])))` where the metric uses new native histogram type with `histogram_quantile` directly on sparse histogram without le buckets
- D) `quantile(0.99, http_request_duration_seconds)`

****23.** In Grafana 10-11 (2024-2026), what are Correlations and Scenes, and how are they used?

- A) Correlations link data sources (e.g., from Prometheus metrics to Loki logs/traceID); Scenes is the new dashboarding architecture powering Dashboards and Explore
- B) They are legacy features removed in 2024
- C) Correlations are alert rules, Scenes are panel types
- D) They only work with MySQL

****24.** How does Prometheus handle staleness and the lookback-delta logic as of Prometheus 3.x?

- A) Stale markers are not used
- B) If a target stops exposing a time series, Prometheus inserts a stale marker; query with 5-minute lookback by default (`--query.lookback-delta` default 5m) will not return value older than that unless specified; as of 2024, created timestamp and stale handling improvements reduce false extrapolation
- C) Prometheus keeps series forever
- D) Staleness is handled by Grafana only

****25.** You need to reduce Prometheus memory usage with 1M active series. Which combination is effective in 2025-2026?

- A) Increase `scrape_interval` to 30s/60s, drop unused labels via `metric_relabel_configs`, use recording rules to drop raw series after aggregation, enable `--storage.tsdb.min-block-duration` tuning, and use Agent Mode + remote write to central Thanos/Mimir/Cortex for long-term storage
- B) Set retention to infinite
- C) Add more exporters
- D) Only use `increase()` function

****26.** For Grafana multi-tenant setup, how do you isolate data sources and dashboards for 50 teams efficiently in Grafana 10+?

- A) Create 50 separate Grafana servers

B) Use Grafana Organizations or Teams + Folder + RBAC + Data source permissions (Enterprise) + provisioning via YAML/Terraform for data sources/dashboards as code

C) Share admin password with all teams

D) Use one dashboard for all

****27. Explain Exemplars in Prometheus & Grafana and how to enable them.****

A) Exemplars are random sample values

B) Exemplars store Trace ID as reference alongside metrics (e.g., from OpenTelemetry instrumentation). Enable in Prometheus via `--enable-feature=exemplar-storage` and `exemplars: {max_exemplars: ...}` in config (enabled by default in Prometheus 2.40+, 3.x); In Grafana, enable Exemplars toggle in Prometheus data source and show as dots on graph linked to Tempo/Jaeger

C) Exemplars are Grafana Enterprise licenses

D) Exemplars are log files

****28. Consider PromQL: $\text{sum by (job) (rate(errors[5m]))} / \text{sum by (job) (rate(requests[5m]))} > 0.05$. What advanced concern exists?****

A) No concern, query is perfect

B) Division by zero and partial series matching can fire incorrectly; should use: $((\text{sum by (job) (rate(errors[5m]))} / \text{sum by (job) (rate(requests[5m]))}) > 0.05)$ and $(\text{sum by (job) (rate(requests[5m]))} > \text{threshold})$ or handle absent/error via `>0` filter, and ensure label matching alignment. Also use `group_left` if needed and consider `absent()` handling

C) Query needs `avg()`

D) Query needs SQL

****29. In Thanos / Cortex / Mimir architecture built on Prometheus remote-write (2024-2026 standard), what does the Query Frontend and Compactor do?****

A) Nothing, they are deprecated

B) Query Frontend: caches, splits queries by time, retries, rate-limits in Mimir/Grafana Cloud; Compactor: compacts blocks from object storage, downsamples (in Thanos), handles retention and deduplication for HA Prometheus pairs

C) They only generate PDF reports

D) They replace Grafana

****30. You are debugging slow Grafana dashboard with 20 panels querying Prometheus with `rate(metric[5m])` over last 7 days at 10s step. How do you optimize per Grafana & Prometheus best practices 2025?****

A) Add more panels

B) Use recording rules to pre-aggregate, increase min step / `$__rate_interval` or use 1m step auto via `$__interval`, enable Grafana query caching (2024+ built-in), use query-frontend in Mimir, reduce cardinality, use exemplar sampling, and set query lookback limits

C) Set interval to 1ms

D) Disable Prometheus

Answer Key - Advanced

| Question | Correct Answer | Explanation |

|---|---|---|

| 21 | B | Head = recent 2h mutable block, WAL for crash recovery, compaction creates 2h-> etc blocks. |

| 22 | C | Native Histograms (stable Prometheus 1.0+ spec 2023, widely adopted 2024-2026) don't use le buckets. Query is `histogram_quantile(q, sum(rate(metric[range])))`. Uses sparse/dense representation. |

| 23 | A | Correlations (2023 GA) link metrics->logs->traces. Scenes (Grafana 10+) is React framework replacing old dashboard render. |

| 24 | B | Stale markers from target removal + 5m lookback prevents stale data graphing. Prometheus 2.42+ / 3.x improved created timestamps handling (`--enable-feature=created-timestamp-zero-ingestion`). |

| 25 | A | Valid cost optimization stack: fewer scrapes, relabel drop, recording rules, agent mode, remote-write to Mimir/Thanos with downsampling. |

| 26 | B | Best practice is multi-tenancy via Orgs/RBAC/Folders + IaC provisioning (grafana provider for Terraform). Enterprise adds fine-grained data source perms. |

| 27 | B | Exemplars link metrics to traces, crucial for observability correlation. Enabled by default now but Grafana data source needs Exemplars=true. |

| 28 | B | Division can yield NaN if denominator 0; vector matching can cause missing alerts; add min request volume guard and use group_left/group_right as needed. Also consider or vector(0) handling. |

| 29 | B | In modern long-term storage, Frontend handles query splitting/caching, Compactor deduplicates and compacts. |

| 30 | B | Optimize by recording rules, bigger step, caching, query-frontend, cardinality reduction. Querying 7d at 10s step = 60k points per series - heavy. |

Exam generated covering Prometheus 2.x, 3.0-3.3 (Nov 2024+) and Grafana 9.5 through Grafana 11.x features, 2024-2026.

Course File: 28_SRE.md

SRE & Observability - Comprehensive Examination

****Course Code:**** 28_SRE

****Total Questions:**** 30

****Passing Score:**** 70%

****Time Limit:**** 90 minutes

****Valid For:**** 2024-2026 syllabus covering SLI/SLO, MTTR/MTBF, OpenTelemetry, Prometheus, Grafana, incident management, and chaos engineering

Part A: Beginner Level (Questions 1-10)

Q1. What does SRE stand for in the context of modern software engineering?

- A) System Reliability Engineering
- B) Site Reliability Engineering
- C) Software Resilience Engineering
- D) Service Recovery Engineering

Q2. Which of the "Four Golden Signals" defined by Google SRE focuses on the rate of requests that fail?

- A) Latency
- B) Traffic
- C) Errors
- D) Saturation

Q3. What is an SLI (Service Level Indicator)?

- A) A business agreement with customers about uptime
- B) A quantitative measure of some aspect of service quality, e.g., request latency or success rate
- C) A post-incident report
- D) A tool for log aggregation

Q4. In observability, which pillar answers the question "HOW LONG did something take?"

- A) Logs
- B) Metrics
- C) Traces
- D) Profiling

Q5. What does MTTR stand for?

- A) Mean Time To Recovery / Repair
- B) Maximum Time To Response
- C) Minimum Time To Restore
- D) Mean Ticket Tracking Rate

Q6. Which tool is primarily used for metrics collection and alerting with a pull-based model and PromQL?

- A) Elasticsearch
- B) Jaeger
- C) Prometheus
- D) Fluentd

Q7. What is the purpose of an error budget?

- A) To allocate money for fixing bugs
- B) To quantify the acceptable amount of unreliability before freezing features and focusing on reliability
- C) To count the number of errors in logs
- D) To budget for on-call compensation

Q8. An on-call rotation is best described as:

- A) A database replication strategy
- B) A schedule that distributes incident response responsibility among team members to ensure 24/7 coverage
- C) A load balancing algorithm
- D) A deployment strategy that rotates servers

Q9. What is a runbook in SRE?

- A) A log of all production deployments
- B) A compiled list of source code execution steps
- C) A documented set of procedures for routine operational tasks and incident response
- D) A financial ledger for cloud costs

Q10. Which open-source standard (now CNCF graduated) provides vendor-neutral APIs/SDKs for traces, metrics, and logs?

- A) OpenTracing
- B) OpenCensus
- C) OpenTelemetry (OTel)
- D) StatsD

Answer Key - Beginner Level

****Q1. Answer: B) Site Reliability Engineering****

Explanation: Coined by Ben Treynor at Google in 2003. SRE applies software engineering principles to operations and infrastructure problems.

****Q2. Answer: C) Errors****

Explanation: Four Golden Signals are Latency, Traffic, Errors, Saturation. Errors measures rate of failed requests (HTTP 5xx, etc.).

****Q3. Answer: B) A quantitative measure of some aspect of service quality, e.g., request latency or success rate****

Explanation: SLI is the actual measured metric. SLO is the target range for that SLI. SLA is the contractual agreement.

****Q4. Answer: B) Metrics****

Explanation: Metrics are numeric aggregations over time (latency p95, QPS). Logs provide context (WHAT happened), Traces show causation and flow (WHERE and WHY), Metrics show trends.

****Q5. Answer: A) Mean Time To Recovery / Repair****

Explanation: MTTR measures average time from failure detection to service restoration. Key reliability metric alongside MTBF (Mean Time Between Failures).

****Q6. Answer: C) Prometheus****

Explanation: Prometheus (CNCF graduated, 2012) uses pull model, time-series DB, and PromQL. Became de-facto standard for Kubernetes monitoring.

****Q7. Answer: B) To quantify the acceptable amount of unreliability before freezing features and focusing on reliability****

Explanation: Error Budget = 100% - SLO. If SLO is 99.9% availability, error budget is 0.1% downtime (~43m/month). Exhaustion triggers policy to prioritize reliability.

****Q8. Answer: B) A schedule that distributes incident response responsibility among team members to ensure 24/7 coverage****

Explanation: Prevents burnout, ensures clear ownership. Modern tools: PagerDuty, Opsgenie, Grafana OnCall.

****Q9. Answer: C) A documented set of procedures for routine operational tasks and incident response****

Explanation: Runbooks reduce MTTR, enable less-experienced on-callers, and are essential for incident management maturity.

****Q10. Answer: C) OpenTelemetry (OTel)****

Explanation: OTel merged OpenTracing + OpenCensus, graduated CNCF in 2024. Provides unified collection for traces, metrics, logs, with wide language and vendor support.

Part B: Intermediate Level (Questions 11-20)

Q11. You have an SLO: 99.9% of HTTP requests over 28-day rolling window must have latency < 300ms. How large is the error budget in minutes?

- A) ~43.2 minutes
- B) ~40.3 minutes
- C) ~432 minutes
- D) ~100.8 minutes

Q12. Which PromQL query correctly calculates the p95 latency from a histogram metric `http_request_duration_seconds`?

- A) `avg(http_request_duration_seconds) by (service)`
- B) `histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le, service))`
- C) `quantile_over_time(0.95, http_request_duration_seconds[5m])`
- D) `p95(http_request_duration_seconds)`

Q13. In distributed tracing, what is the purpose of context propagation via W3C Trace Context headers (traceparent)?

- A) To encrypt trace data between services
- B) To link spans across service boundaries by passing trace-id, parent-id, and sampling flags
- C) To compress trace payloads
- D) To store user authentication tokens

Q14. According to 2024-2025 SRE best practices, what is the key difference between a symptom-based alert and cause-based alert?

- A) Symptom-based alerts on user-visible impact (SLO burn), cause-based on internal states (CPU high); symptom alerts should page humans
- B) Cause-based alerts are always better
- C) Symptom-based means server CPU, cause-based means user latency
- D) There is no difference

Q15. What is the recommended strategy to prevent alert fatigue?

- A) Alert on every metric anomaly
- B) Implement multi-window, multi-burn-rate SLO alerting, eliminate noisy alerts, and page only on actionable, user-impacting conditions
- C) Disable alerts during nights
- D) Send all alerts to email instead of pager

Q16. In OpenTelemetry Collector, what are the three main components of a pipeline?

- A) Extractor, Transformer, Loader
- B) Receivers, Processors, Exporters
- C) Agent, Gateway, Backend
- D) Input, Filter, Output

Q17. Which of the following is a valid SRE anti-pattern when defining SLOs?

- A) Using SLI aligned to user journey (e.g., checkout success rate)
- B) Setting SLO at 100% availability
- C) Using 28-day rolling window for SLO calculation
- D) Involving product and business stakeholders in SLO definition

Q18. What does TOIL mean in SRE and what is Google's target for SRE teams?

- A) Technical Operations Incident Log; keep it < 10%
- B) Toil - manual, repetitive, automatable operational work; SRE teams should keep toil < 50% of time
- C) Total Outage Incident Length; keep it < 50%
- D) Time On Investigating Logs; keep it < 20%

Q19. When using Grafana Loki for log aggregation in 2024-2026, how does it differ architecturally from Elasticsearch?

- A) Loki indexes full log content; Elasticsearch only indexes labels
- B) Loki indexes only metadata/labels and compresses chunks, making it cheaper; Elasticsearch indexes full text for fast full-text search
- C) They are identical
- D) Loki cannot query logs

Q20. What is chaos engineering, popularized by Netflix's Chaos Monkey and now CNCF Chaos Mesh / LitmusChaos?

- A) Randomly shutting down data centers without approval

- B) The disciplined practice of injecting controlled failures into production-like environments to build confidence in resilience before outages occur
- C) A method for debugging code by adding random errors
- D) A security penetration testing technique only

Answer Key - Intermediate Level

****Q11. Answer: B) ~40.3 minutes****

Explanation: 28 days = 40320 minutes. 0.1% of that = 40.32 minutes. For 30-day month, ~43.2m (common interview shortcut). 28-day rolling is Google-recommended to avoid month-length variation. Calculation: $28 \times 24 \times 60 \times 0.001 = 40.32$.

****Q12. Answer: B) `histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le, service))`****

Explanation: Prometheus histogram quantiles require `_bucket` metric, `rate()`, aggregation preserving `le` label, and `histogram_quantile()` function. Option A averages, C/D are invalid syntax.

****Q13. Answer: B) To link spans across service boundaries by passing trace-id, parent-id, and sampling flags****

Explanation: W3C Trace Context (`traceparent: 00-{trace-id}-{parent-id}-{flags}`) is the 2020+ standard for propagation. Enables distributed tracing across polyglot services without vendor lock-in. Adopted by OTel default.

****Q14. Answer: A) Symptom-based alerts on user-visible impact (SLO burn), cause-based on internal states (CPU high); symptom alerts should page humans****

Explanation: Google SRE book (Chapter 5): Page on symptoms directly impacting users/SLOs. Cause-based (CPU 80%) should be ticket or info, not paging, because they may not affect users.

****Q15. Answer: B) Implement multi-window, multi-burn-rate SLO alerting, eliminate noisy alerts, and page only on actionable, user-impacting conditions****

Explanation: Multi-burn-rate (Google SRE Workbook 2016, still standard in 2024-26) uses fast burn (1h) + slow burn (6h) to reduce false positives. Actionable, no toil, tuned thresholds.

****Q16. Answer: B) Receivers, Processors, Exporters****

Explanation: OTel Collector pipeline: Receiver gets data (OTLP, Prometheus etc.), Processor mutates/batches/filters (batch, memory_limiter, attributes), Exporter sends to backend (Jaeger, Prometheus, OTLP).

****Q17. Answer: B) Setting SLO at 100% availability****

Explanation: 100% SLO is impossible and removes error budget, halts innovation, creates impossible on-call burden. Perfect availability violates CAP and real-world constraints. SLO should reflect user happiness (99.9, 99.95, etc.)

****Q18. Answer: B) Toil - manual, repetitive, automatable operational work; SRE teams should keep toil < 50% of time****

Explanation: Toil definition from Google: Manual, repetitive, automatable, tactical, no enduring value, scales linearly. Google SRE mandates <50% toil to allow engineering work to reduce toil.

****Q19. Answer: B) Loki indexes only metadata/labels and compresses chunks, making it cheaper; Elasticsearch indexes full text for fast full-text search****

Explanation: Loki design (like Prometheus but for logs) trades full-text indexing speed for cost efficiency. Ideal for Kubernetes where labels are abundant. Uses LogQL. Huge adoption 2024-26 due to cost.

****Q20. Answer: B) The disciplined practice of injecting controlled failures into production-like environments to build confidence in resilience before outages occur****

Explanation: Principles of Chaos (2022+): Build hypothesis, blast radius control, abort, run in prod. Tools: Chaos Mesh, LitmusChaos (CNCF incubating), Gremlin, AWS FIS (Fault Injection Simulator).

Part C: Advanced Level (Questions 21-30)

Q21. You are implementing SLO burn rate alerting. Error budget burn rate is 14.4x for last 1 hour. What does this imply for a 99.9% 28-day SLO?

- A) If burn continues, entire 28-day budget will be consumed in ~2 days ($28/14.4 \approx 1.94$ days)
- B) Service is healthy
- C) Burn rate is irrelevant

D) Budget will last 14.4 days

Q22. In OpenTelemetry, what is the difference between Span and SpanEvent (formerly log) and when would you use Span Link?

A) Span is unit of work with start/end, SpanEvent is timestamped annotation within span, Span Link connects causally related but not parent-child traces (e.g., batch job fan-out)

B) They are identical

C) Span Link is for encrypting traces

D) SpanEvent replaces metrics

Q23. A service shows high cardinality explosion in Prometheus: metric `http_requests_total` with label `user_id` (1M distinct values). What is the correct remediation per 2024-26 best practice?

A) Keep `user_id` label for detailed debugging

B) Remove high-cardinality label from metric; move user-specific detail to traces/logs with exemplars linking metrics to traces

C) Increase Prometheus memory to 100GB

D) Use pushgateway

Q24. Which deployment strategy best balances SRE safety and velocity in 2025-2026 for large-scale services?

A) Big-bang deploy to 100%

B) Progressive delivery: canary + automated analysis (Kayenta/Flagger/Argo Rollouts) + automatic rollback on SLO violations via error-budget-aware gates

C) Manual FTP of binaries

D) Deploy only on Fridays

Q25. When designing SLIs for a streaming API (gRPC server streaming), which SLI definition is most appropriate for availability?

A) TCP connection count

B) Ratio of successful stream messages delivered / requested, plus stream establishment success + freshness/throughput SLI for data lag

C) CPU usage of server

D) Number of deployments per day

Q26. In incident management (per Google's IRM and 2024 PagerDuty State of Reliability), which roles are critical during a SEV1/SEV0?

A) Only the developer who wrote the code

B) Incident Commander (decision maker), Communications Lead, Operations Lead/Ops, Subject Matter Experts; blameless structure, defined by IMS

C) CEO only

D) No roles needed, everyone debugs

Q27. You need to choose between pull (Prometheus) vs push (OTel Collector, StatsD) observability. For short-lived Kubernetes Jobs (5 sec batch), what is the preferred pattern in 2024-2026?

- A) Pure Prometheus pull will work fine because scrape interval is 5s
- B) Use push via OTel Collector or Prometheus Pushgateway/OTLP gateway with ephemeral job handling, or use metrics from OTel SDK pushing to Collector, since pull will likely miss short-lived targets; also expose via OpenMetrics with kube-state-metrics
- C) Do not monitor jobs
- D) Use log files only

Q28. What is the RED method vs USE method, and when to apply each?

- A) RED: Rate, Errors, Duration - for request-driven services (user-facing). USE: Utilization, Saturation, Errors - for resources (CPU, disk, DB). Apply RED to services, USE to infrastructure
- B) RED is for databases only
- C) USE is deprecated in 2024
- D) They are identical

Q29. Explain tail-based sampling vs head-based sampling in distributed tracing at scale (2025 cost control)?

- A) Head-based decides sampling decision at trace start (cheaper, may miss interesting errors); Tail-based collects all spans initially, samples after observing entire trace (can keep all errors/latent traces) - requires OTel Collector's tail_sampling processor; best practice hybrid: head 1% + tail keep all errors >1s
- B) Tail-based is always cheaper
- C) No sampling needed in 2025, store everything
- D) Sampling breaks traces

Q30. Your SLO burn alert fired: 5% of error budget consumed in last 30 minutes (burn rate ~72x). As SRE implementing error budget policy (Google 2023-2025 framework), what immediate automated + procedural actions should occur?

- A) Ignore alert
- B) Automated: freeze canary/progressive rollouts via CI gate checking error budget, auto-rollback if recent deployment correlates; Procedural: IC assesses severity, halt feature pushes if burn > 10% in hour, route excessive alert volume to error-budget-aware SLO dashboard, initiate freeze per policy, post incident channel, prioritize reliability tasks until budget replenishes (28d rolling)
- C) Immediately delete service
- D) Increase SLO threshold to 100% to avoid burn

Answer Key - Advanced Level

****Q21. Answer: A)** If burn continues, entire 28-day budget will be consumed in ~2 days ($28/14.4 \approx 1.94$ days)**

Explanation: Burn rate 1 = budget consumes exactly in SLO window. Burn 14.4x means budget exhausted in $28/14.4$ days = ~1.94 days. Alert logic from SRE Workbook: page when burn 14x for 1h AND burn 6x for 6h. 14.4 often used for 2% budget in 1h for 99.9% SLO.

****Q22. Answer: A)** Span is unit of work with start/end, SpanEvent is timestamped annotation within span, Span Link connects causally related but not parent-child traces (e.g., batch job fan-out)**

Explanation: OTel data model 2024: Span = operation, Event = something happening at timepoint (e.g., exception stacktrace, cache miss). Link: for async patterns where multiple upstream traces trigger one downstream (e.g., Kafka batch consumer links to many producer traces). Critical for event-driven architectures.

****Q23. Answer: B)** Remove high-cardinality label from metric; move user-specific detail to traces/logs with exemplars linking metrics to traces**

Explanation: Prometheus cardinality best practice: keep labels bounded <1000 values per metric. user_id causes TSDB explosion ($M * \text{cardinality}$). Solution: use exemplars (Prometheus 2.25+ feature, stable 2024) to link histogram bucket to trace ID containing user_id. Traces handle high-cardinality.

****Q24. Answer: B) Progressive delivery: canary + automated analysis (Kayenta/Flagger/Argo Rollouts) + automatic rollback on SLO violations via error-budget-aware gates****

Explanation: 2024-2026 industry standard: Argo Rollouts / Flagger with metrics analysis (p95, error rate vs baseline). Integrates with error budget - if budget <25% remaining, block deploys automatically. Google's SRE prod pipeline uses this.

****Q25. Answer: B) Ratio of successful stream messages delivered / requested, plus stream establishment success + freshness/throughput SLI for data lag****

Explanation: Streaming SLI is complex: availability = successful handshakes + messages within lag threshold. Also need freshness SLI (how old is latest data). Inspired by Google streaming SLI examples in Implementing Service Level Objectives (2020) and gRPC OTel observability 2024.

****Q26. Answer: B) Incident Commander (decision maker), Communications Lead, Operations Lead/Ops, Subject Matter Experts; blameless structure, defined by IMS****

Explanation: IMS/Incident Management System + Google IRM: IC has final decision, not technical fixer. Comms handles stakeholders. Ops drives mitigation. Blameless postmortem mandatory. Anti-pattern: IC also debugging.

****Q27. Answer: B) Use push via OTel Collector or Prometheus Pushgateway/OTLP gateway with ephemeral job handling, or use metrics from OTel SDK pushing to Collector, since pull will likely miss short-lived targets; also expose via OpenMetrics with kube-state-metrics****

Explanation: Prometheus pull scrapes 15s default, misses < scrape interval jobs. Pushgateway (or better OTel Collector with Prometheus remote write) is solution for batch. Modern 2025 recommendation: OTel SDK pushes to Collector via OTLP, Collector exports via remote-write to Mimir/Thanos. kube-job metrics via kube-state-metrics for completion tracking.

****Q28. Answer: A) RED: Rate, Errors, Duration - for request-driven services (user-facing). USE: Utilization, Saturation, Errors - for resources (CPU, disk, DB). Apply RED to services, USE to infrastructure****

Explanation: RED by Tom Wilkie (Weaveworks) for microservices - perfect for SLI mapping. USE by Brendan Gregg for infrastructure saturation. Mature SRE 2024-26 uses both: dashboards show RED (top) + USE (bottom). Alerts on RED (symptoms).

****Q29. Answer: A) Head-based decides sampling decision at trace start (cheaper, may miss interesting errors); Tail-based collects all spans initially, samples after observing entire trace (can keep all errors/latent traces) - requires OTel Collector's tail_sampling processor; best practice hybrid: head 1% + tail keep all errors >1s****

Explanation: Tail sampling allows intelligent retention (keep 100% errors, slow traces, high-value tenants). Tradeoff: needs buffering (memory). OTel Collector tail_sampling with composite policy: status_code=ERROR, latency >1s, probabilistic 10%. Major cost optimization for observability 2024-2026.

****Q30. Answer: B) Automated: freeze canary/progressive rollouts via CI gate checking error budget, auto-rollback if recent deployment correlates; Procedural: IC assesses severity, halt feature pushes if burn > 10% in hour, route excessive alert volume to error-budget-aware SLO dashboard, initiate freeze per policy, post incident channel, prioritize reliability tasks until budget replenishes (28d rolling)****

Explanation: This is the full Error Budget Policy contract (Google SRE book chap 34, 2024 adaptations). Key actions: Enforce freeze, not blame. 72x burn = 5% in 30 min => will exhaust full budget in ~10 hours if sustained ($100/5 * 0.5h = 10h$). Requires immediate human escalation and automated guardrails. Blameless but accountable.

****End of Exam - SRE & Observability****

Course File: 29_Splunk.md

Splunk Core Certified Power User & Enterprise Admin - Complete Exam

Course: Splunk Enterprise & Splunk Cloud Platform (v9.1 - v9.3) | Aligned to 2024-2026 Syllabus

Total Questions: 30 | Duration: 90 Minutes | Pass Mark: 70%

PART A: BEGINNER LEVEL (10 Questions)

Q1. What is the primary role of the Splunk Indexer?

- A) To forward data from source systems to the index
- B) To parse, index, and store incoming data and execute searches on stored data
- C) To provide the user interface for searching and dashboards
- D) To act as a load balancer for forwarders

Q2. Which configuration file is used to define how data should be parsed, such as line breaking and timestamp extraction?

- A) indexes.conf
- B) inputs.conf
- C) props.conf
- D) outputs.conf

Q3. In SPL (Search Processing Language), which command is used to filter results similar to SQL WHERE?

- A) filter
- B) where
- C) search
- D) Both B and C, but with different execution timing

Q4. What is the default port for Splunk's Universal Forwarder to send data to an indexer?

- A) 8000
- B) 8089
- C) 9997
- D) 514

Q5. What does SPL stand for and which pipe symbol is used to chain commands?

- A) Splunk Processing Language, using |
- B) Search Processing Language, using |
- C) Search Processing Language, using >
- D) Structured Processing Language, using |

Q6. Which directory contains the raw data in its compressed form within a Splunk bucket?

- A) _MetaData
- B) rawdata
- C) journal
- D) colddb

Q7. Which of the following is a Knowledge Object?

- A) Index
- B) Search Head
- C) Lookup, Event Type, and Saved Search
- D) Forwarder

Q8. What is the function of fields.conf and the stats command?

- A) fields.conf defines index storage; stats is for filtering
- B) stats is a transforming command that aggregates data and can significantly improve performance by reducing result sets
- C) stats can only be used with tstats
- D) stats is a non-streaming command that must be last in search

Q9. In Splunk Cloud Platform (2024-2026), what is IDAC and Federated Search?

- A) IDAC is Indexer Discovery and Clustering; Federated Search allows searching remote Splunk deployments from Splunk Cloud
- B) IDAC is a licensing model; Federated Search is disabled in Cloud
- C) IDAC manages forwarders; Federated Search is same as Distributed Search
- D) IDAC is deprecated as of Splunk 9.2

Q10. Which search will correctly find failed logins and create a table of user and count?

- A) index=security failed | table user count
- B) index=security "failed login" | stats count by user
- C) index=security | where action=failed | chart user
- D) index=security | transaction user | stats count

Answer Key - Beginner Level

| Q | Correct Answer | Explanation |

| --- | :--- | :--- |

| 1 | B | Indexer is the core component that licenses/parses/indexes data and serves search requests. Forwarders (A) send, Search Head (C) serves UI. |

| 2 | C | props.conf handles parsing attributes: TIME_FORMAT, LINE_BREAKER, SHOULD_LINEMERGE etc. indexes.conf defines indexes, inputs.conf defines monitoring. |

| 3 | D | search is used early and is fast (searches index), where uses eval expressions and filters after field extraction. Best practice: use search first. |

| 4 | C | 9997 is default receiving port (inputs.conf on indexer: [splunktcp://9997]). 8000 is web UI, 8089 is management port. |

| 5 | B | SPL = Search Processing Language. The pipe | chains commands, e.g., ... | stats count by host. |

| 6 | B | A bucket (hot/warm/cold) contains rawdata/journal.gz (compressed raw events), plus tsidx files, bloomfilters, and metadata. |

| 7 | C | Knowledge Objects are user-created: Lookups, Event Types, Tags, Field Aliases, Macros, Workflow Actions, Data Models, etc. |

| 8 | B | stats is a transforming/distributable streaming command. It reduces data at indexers before sending to search head, improving performance. |

| 9 | A | As of 2024+, Splunk Cloud uses IDAC (Indexer Discovery + Cluster Manager combined) and Federated Search for Amazon Security Lake / remote Splunk Platform is GA and critical for hybrid search. |

| 10 | B | stats count by user is correct transforming syntax. table just lists fields without aggregation. |

PART B: INTERMEDIATE LEVEL (10 Questions)

Q11. Given props.conf [my_sourcetype] with TRANSFORMS-null = set_null and transforms.conf [set_null] with REGEX=. DEST_KEY=queue FORMAT=nullQueue, what happens?

- A) It drops all events for that sourcetype
- B) It routes matching events to nullQueue (discard) - used for filtering unwanted events at index time
- C) It sends events to coldQueue
- D) This syntax is invalid in Splunk 9.x

Q12. What is the correct order of buckets by age?

- A) hot -> cold -> warm -> frozen -> thawed
- B) hot -> warm -> cold -> frozen (with thawed as resurrected frozen)
- C) warm -> hot -> cold -> frozen
- D) hot -> warm -> thawed -> cold

Q13. Which command provides the fastest search performance for counting events by index when data model acceleration is available?

- A) index=* | stats count by index
- B) | tstats count where index=* by index
- C) | metasearch index=* | stats count by index
- D) | pivot data_model count

Q14. What is SmartStore and what does it change introduced in Splunk 9.x?

- A) A way to store indexes only on SSDs
- B) An architecture separating compute (indexer cache) and storage (S3/GCS/Azure) - hot/warm buckets cached locally, cold stored remotely. Mandatory for Splunk Cloud Platform.
- C) A new license type for cloud
- D) A data model acceleration engine

Q15. Which inputs.conf stanza correctly monitors /var/log/*.log on a Universal Forwarder and sets sourcetype to linux_logs?

- A) [monitor:///var/log/*.log]
sourcetype = linux_logs
- B) [Monitor:/var/log/*.log]
source_type = linux_logs
- C) [default]

monitor = /var/log/*.log
D) [script:///var/log/*.log]

Q16. In Splunk Enterprise Security (CIM), what is the purpose of the Common Information Model?

- A) To compress data more efficiently
- B) To normalize fields across different source types (e.g., src_ip, dest_ip, action) to enable correlation searches and data models
- C) To create indexes automatically
- D) To replace props.conf

Q17. What does | eval status_code=if(status>=400, "error", "success") do, and how is where different from eval + if?

- A) Creates a new field; where cannot use if
- B) eval creates/modifies field; where filters results using eval expressions. This example tags errors.
- C) This syntax is deprecated, use case() only
- D) eval can only be used after stats

Q18. How do Search Head Cluster and Indexer Cluster differ?

- A) They are the same
- B) SHC replicates Knowledge Objects and dispatches searches via Captain; Indexer Cluster replicates data buckets with Replication Factor and Search Factor for HA
- C) SHC replicates data; Indexer Cluster replicates searches
- D) SHC is only for Splunk Cloud

Q19. What is a Data Model and what command uses it?

- A) A JSON file for ingestion; used by inputlookup
- B) Hierarchical data categorization (Root Event / Search / Transaction) with constraints and fields, accelerated via tstats and pivot, used by Enterprise Security and ITSI
- C) Only used for machine learning
- D) Same as lookup file

Q20. Which SPL query uses the transaction command correctly and when should you avoid it?

- A) index=web | transaction JSESSIONID startswith="login" endswith="logout" | table duration eventcount - Avoid for high-cardinality/larger data sets; use stats instead as transaction is memory-intensive and slow
- B) index=web | transaction | stats count
- C) index=web | transaction duration
- D) Transaction is always faster than stats

Answer Key - Intermediate Level

| Q | Correct Answer | Explanation |

| --- | --- | --- |

| 11 | B | Routing to nullQueue is standard index-time filtering to discard noise (e.g., debug logs). REGEX=. matches all; more specific regex filters selectively. |

| 12 | B | Lifecycle: hot (writable) -> warm (rolled) -> cold (low-speed storage) -> frozen (archived/deleted). Thawed is for restoring frozen data. Managed by indexes.conf maxHotBuckets, maxWarmDBCount etc. |

| 13 | B | tstats directly queries tsidx files, bypassing rawdata, ~10-100x faster. Requires accelerated data model or enabled summary. metasearch is deprecated/fast but limited. |

| 14 | B | SmartStore (since 7.2, standard in Splunk Cloud 9.x) decouples compute/storage. CacheManager manages local cache. Improves scaling, resilience, and cost on S3. |

| 15 | A | Correct syntax: [monitor://path] with lowercase. sourcetype is valid attribute. Universal Forwarder needs outputs.conf to send to indexer. |

| 16 | B | CIM provides normalized data models (Authentication, Network Traffic, Web etc.) to make content portable across customers/vendors. Core to ES and SOC operations. |

| 17 | B | eval creates new field with if() condition. where filters. You can also do | where status>=400. case() is better for multiple conditions. |

| 18 | B | SHC (deployer + 3+ members + captain) for HA for users/search artifacts. Indexer Cluster (Cluster Manager + peers) for data replication and High Availability via RF/SF. |

| 19 | B | Data Models are knowledge objects enabling Pivot and tstats Acceleration (indexes.conf [datamodel] acceleration=true). Used heavily in ES. |

| 20 | A | transaction groups events into transactions based on field, useful for sessionization, but very expensive - requires all events in memory on search head. Prefer stats with values(), earliest()/_time. |

PART C: ADVANCED LEVEL (10 Questions)

Q21. You have Splunk Enterprise 9.3 with TLS certificates for indexer cluster. Splunk's new default in 9.x is to enforce certificate verification. What must you configure to maintain secure Splunk-to-Splunk communication (S2S) without disruption?

- A) Nothing, 9.x disables TLS by default
- B) Configure server.conf [sslConfig] sslVerifyServerCert = true and provide valid certs via caCertFile, plus in outputs.conf for forwarders set sslCommonNameToCheck or update to new sslVerifyServerCert
- C) Only enable SSL on port 8000
- D) Use third-party certs only on Search Head

Q22. In a large-scale deployment with 50+ Universal Forwarders, how do you optimize limits.conf and ingestion pipeline to handle parsing queue blocking?

- A) Increase maxKBps in limits.conf to 0
- B) Investigate indexing pipeline - parsingQueue full via Monitoring Console; tune props.conf SEDCMD, reduce expensive regex, use Ingest Actions, increase queueSize in server.conf [queue], or scale to Heavy Forwarder / Edge Processor for pre-parsing
- C) Delete indexes.conf
- D) Restart forwarders only

Q23. Which search best implements a behavioral anomaly using SPL with eventstats and streamstats?

- A) index=auth | eventstats avg(duration) as avg stdev(duration) as stdev by user | where duration > avg+3*stdev
- B) index=auth | stats count
- C) index=auth | transaction user | where duration > 100
- D) index=auth | eval avg=100

Q24. What is the role of Splunk Edge Processor (GA 2024) and Ingest Processor in Splunk Cloud Platform?

- A) Same as Universal Forwarder
- B) Edge Processor is next-gen stateless data processing at edge/cloud for filtering/masking/routing before indexing, replacing/augmenting Heavy Forwarder use cases with Pipeline UI and SPL2. Ingest Processor is its Cloud-managed service for S3-based pipelines.
- C) A replacement for Search Head
- D) Only for on-prem SmartStore

Q25. Explain workload management in Splunk 9.x.

- A) Deprecated since 8.0
- B) Workload Management (wlm.conf) allows creating workload pools/rules to allocate CPU/memory to search vs. ingest vs. ad-hoc roles, preventing runaway searches. Critical for Splunk Cloud Victoria Experience.
- C) Only for free license
- D) Manages forwarder load

Q26. A customer using Splunk Cloud Platform notes SVC (Splunk Virtual Compute) license model introduced 2023-2024. What does it meter?

- A) Daily indexing volume only
- B) Search, Ingest, and Storage virtual compute units - based on actual resource consumption across workloads, replacing pure ingest-based licensing, with allowances for SVCs
- C) Number of users only
- D) Number of forwarders

Q27. You need to accelerate a data model for ES. What are acceleration requirements and trade-offs?

- A) datamodels.conf acceleration=true, acceleration.earliest_time, max_concurrent_optimizes plus private data model + summary range; trade-off is storage overhead (~30% extra) and optimization job load on indexers
- B) Only need to enable on search head
- C) Acceleration has no cost
- D) Acceleration works on all indexes automatically

Q28. How do you securely configure props.conf + transforms.conf for indexed field extraction vs. search-time?

- A) Always use INDEXED_EXTRactions = JSON for all sources
- B) Search-time extraction (REPORT- in props.conf) is preferred for flexibility and upgradability; Indexed extraction (EXTRACT- or INDEXED_EXTRactions, or TRANSFORM with WRITE_META) is for performance or for fields needed for tstats, but increases license usage and cannot be changed retroactively without re-indexing. Use fields.conf [field] INDEXED=true for indexed fields.
- C) Both are same
- D) Search-time not supported in 9.3

Q29. Which SPL2 / federated search scenario is valid for Amazon Security Lake (ASL) integration in Splunk Cloud 2024-2026?

- A) FROM federated:security_lake | where ... via Federated Search for Amazon Security Lake with automatic OCSF mapping to CIM

- B) index=security_lake only
- C) | tstats cannot be used with federated
- D) Security Lake requires Heavy Forwarder only

Q30. Troubleshooting: Search index=firewall returns incomplete results for 2 of 5 indexers in an indexer cluster. Monitoring Console shows search factor not met. What steps to diagnose?

- A) Reinstall Splunk
- B) Check splunkd.log, cluster_info_searchable rest: | rest /services/cluster/master/peers, verify replication factor met, check bucket fixup tasks, ensure all peers up, splunk show cluster-status, check disk space and indexes.conf ensure replication factor/search factor, and use | rest /services/cluster/master/generation to see bucket copies. Trigger splunk rebuild for bucket if needed.
- C) Delete all buckets
- D) Only check outputs.conf

Answer Key - Advanced Level

| Q | Correct Answer | Explanation |

|---| :--- | :--- |

| 21 | B | Splunk 9.x hardens TLS by default (TLS 1.2+). For S2S you need sslConfig certs, sslRootCAPath/caCertFile, verify. Failure to update certs post 9.0 upgrade is common cause of broken forwarder-to-indexer comms. Use splunk btool server list sslConfig --debug. |

| 22 | B | ParsingQueue blocking often from expensive regex in props/transforms. Use Edge Processor/Ingest Actions to offload. Check metrics.log. Don't set maxKBps=0 blindly (throttles). |

| 23 | A | eventstats calculates avg/stddev per user without transforming away events, allowing per-event anomaly filter. Classic UBA pattern. streamstats useful for trending over time. |

| 24 | B | Edge Processor (2024+ GA) & Ingest Processor are strategic for data optimization-at-edge, PII masking, filtering. Replaces many HF use cases. Configured via UI pipelines with SPL2. Important for cost optimization in SVC licensing. |

| 25 | B | Workload Management replaces old Search Scheduler limits, enables rule-based admission control: e.g., limit concurrent historically, reserve capacity for ingestion. Configured via wlm.conf + Monitoring Console WLM dashboard. |

| 26 | B | SVC model (2023+) meters Search Compute, Ingest Compute, Storage - aligns to actual compute, not just GB/day. Encourages search optimization and data optimization with Ingest Actions/Edge Processor. Workload-based pricing. |

| 27 | A | Acceleration creates tsidx summaries on indexers. Requires datamodels.conf settings, private app with data model. Cost: storage, CPU for summarization, needs careful backfill_time and acceleration.cron_sq. Use Monitoring Console -> Data Models. |

| 28 | B | Best practice 2024-2026: prefer search-time. Indexed fields increase index size and license count (if extraction creates extra length) and are immutable. Use for high-performance correlation like ES data model fields. |

| 29 | A | Splunk's native FS for Amazon Security Lake (FSASL) uses federated provider, automatically converts OCSF to Splunk CIM, allows SPL/SPL2 searches across S3 without ingesting all logs, saving SVC. FedRAMP compliant. |

| 30 | B | Search Factor not met = not enough searchable copies. Diagnose via Cluster Manager REST endpoints, bucket health, fixup tasks. Common root causes: disk full, indexer down, network partition, manual bucket freeze. |

End of Exam - Generated for 2024-2026 Splunk Core Certified Power User / Admin / Architect track. Covers Splunk Enterprise 9.1-9.3, Splunk Cloud Platform Victoria Experience, SmartStore, Edge Processor, SVC licensing, and Federated Search.

Course File: 30_AD.md

Active Directory & LDAP - Comprehensive Certification Exam

Version: 2024-2026 Aligned | Total Questions: 30 | Format: MCQ (4 Options)

Coverage: AD DS, Azure AD / Entra ID, LDAP v3, LDAPS, Kerberos, Group Policy, Replication, Security Hardening

SECTION A: BEGINNER LEVEL (Questions 1-10)

****Focus:**** Core concepts, objects, protocols, and basic administration.

****Q1.** What is the primary protocol used by clients to query and modify objects in Active Directory?

- A) SMB
- B) LDAP
- C) Kerberos
- D) DNS

****Q2.** In Active Directory, what does the acronym DN stand for?

- A) Domain Namespace
- B) Distinguished Name
- C) Directory Node
- D) Data Network

****Q3.** Which TCP port does LDAP use in clear text by default?

- B) 389
- C) 636
- D) 3268

****Q4.** What is the smallest administrative unit in Active Directory that can have Group Policy linked to it?

- A) Site
- B) Domain
- C) Organizational Unit (OU)
- D) Forest

****Q5.** Which of the following is NOT a partition in Active Directory?

- A) Domain partition
- B) Configuration partition
- C) Schema partition
- D) Security partition

****Q6.** What tool replaced Active Directory Users and Computers (ADUC) as the recommended modern management tool in Windows Server 2022/2025 and for hybrid environments?

- A) Windows Admin Center (WAC) and Active Directory Administrative Center (ADAC)
- B) netsh
- C) Server Manager only
- D) Ldp.exe

****Q7.** In LDAP, what is the root of an LDAP search called?

- A) Relative Distinguished Name (RDN)
- B) Base DN
- C) Global Catalog

D) Common Name (CN)

****Q8. What authentication protocol does Active Directory use by default for domain authentication within the domain?****

A) NTLMv2

B) OAuth 2.0

C) Kerberos

D) PAP

****Q9. Which object class in Active Directory represents a security principal that can be assigned permissions?****

A) OrganizationalUnit

B) Container

C) User

D) Schema

****Q10. What does LDAPS provide that LDAP does not?****

A) Faster query speed

B) Encryption via TLS/SSL

C) Anonymous binding by default

D) Support for more attributes

Answer Key - Beginner Level

| Q | Correct Answer | Explanation |

|---|-----|-----|

| 1 | B) LDAP | LDAP is the core access protocol for AD DS - Lightweight Directory Access Protocol. |

| 2 | B) Distinguished Name | DN uniquely identifies an entry, e.g., CN=John Doe,OU=Users,DC=corp,DC=com |

| 3 | B) 389 | Port 389 for LDAP, 636 for LDAPS, 3268/3269 for Global Catalog. |

| 4 | C) Organizational Unit (OU) | GPOs can be linked to Site, Domain, OU - OU is most granular. |

| 5 | D) Security partition | Real partitions are Domain, Configuration, Schema, Application, and Global Catalog partial RO. |

| 6 | A) Windows Admin Center (WAC) and ADAC | Microsoft pushes ADAC and WAC for modern admin; ADUC is legacy but still present in RSAT. |

| 7 | B) Base DN | Base DN is starting point for search, e.g., DC=corp,DC=com. |

| 8 | C) Kerberos | Kerberos is default since Windows 2000; NTLM is fallback. |

| 9 | C) User | User, Computer, Group are security principals with SIDs. |

| 10 | B) Encryption via TLS/SSL | LDAPS wraps LDAP in TLS on port 636, required by Microsoft hardening guidance post-2024. |

SECTION B: INTERMEDIATE LEVEL (Questions 11-20)

****Focus:**** Replication, Schema, LDIFDE, Kerberos, Trusts, Hybrid Identity.

****Q11. Which AD replication model is used for intra-site replication within a domain?****

A) Pull replication with 15-minute intervals

B) Push replication triggered by change notification (within ~15 seconds)

C) Manual replication only

D) SMTP-based replication

****Q12. You are troubleshooting LDAP. Which utility allows you to perform low-level LDAP binds, searches, and view raw attributes including operational attributes?****

A) DsAStAt

B) Ldp.exe

C) Nslookup

D) Ntdsutil

****Q13. What is the purpose of the Global Catalog (GC) server and which LDAP port does it use for clear text?****

- A) Stores full copy of own domain + partial read-only forest attributes; Port 3268
- B) Stores only Schema partition; Port 389
- C) Stores DNS records only; Port 53
- D) Stores SYSVOL; Port 445

****Q14. In a 2024-2026 hardened AD environment, Microsoft recommends disabling LDAP simple binds without TLS. Which setting enforces LDAP Channel Binding and LDAP Signing?****

- A) Domain Controller: LDAP Server channel binding token Requirements + LDAP server signing requirements
- B) Disable NTLM in Group Policy
- C) Enable SMB signing only
- D) Use LDAPS on port 389

****Q15. What is an LDAP filter to find all disabled user accounts in Active Directory?****

- A) (&(objectClass=user)(userAccountControl:1.2.840.113556.1.4.803:=2))
- B) (objectClass=disabled)
- C) (userStatus=0)
- D) (accountDisabled=TRUE)

****Q16. Which FSMO role must be online to provision a new Domain Controller in an existing forest as of Windows Server 2022/2025?****

- A) PDC Emulator only
- B) RID Master and Infrastructure Master in target domain
- C) Schema Master and Domain Naming Master for forest-level operations
- D) All 5 FSMO roles

****Q17. In Entra ID (formerly Azure AD) Connect / Cloud Sync (2024+), what is the modern replacement for ADAL and used for pass-through authentication to LDAP/AD?****

- A) MSAL with Kerberos Cloud Trust for Windows Hello for Business
- B) WDigest
- C) LDAPv2
- D) RADIUS

****Q18. What does the LDAP attribute memberOf represent, and why is it special?****

- A) It is a stored attribute with direct DB value
- B) It is a constructed / backlink attribute calculated from member attribute on groups, not stored directly
- C) It is only available over LDAPS
- D) It is an operational attribute requiring Schema Admin

****Q19. Which replication conflict resolution method does AD use when two DCs modify the same object attribute?****

- A) Last writer wins based on version number and timestamp
- B) First writer wins
- C) Manual admin merge required
- D) Both values are kept

****Q20. You are planning AD Sites for a multi-site enterprise. Which protocol/service does AD heavily rely on for DC Locator and Site mapping?****

- A) SNMP
- B) DNS SRV records (_ldap._tcp.dc._msdcs.<domain>)
- C) LLMNR
- D) WINS

Answer Key - Intermediate Level

| Q | Correct Answer | Explanation |

|---|-----|-----|

| 11 | B) Push replication triggered by change notification | Intra-site: source DC notifies partners within 15 sec, then pushes. Inter-site uses pull on schedule (default 180 min). |

| 12 | B) Ldp.exe | Ldp.exe is built-in low-level LDAP client for bind/search/modify. |

| 13 | A) Stores full copy of own domain + partial read-only forest attributes; Port 3268 | GC required for forest-wide searches and universal group login. 3268 = GC LDAP, 3269 = GC LDAPS. |

| 14 | A) Domain Controller: LDAP Server channel binding token Requirements + LDAP server signing requirements | Hardening advisory ADV190023 (2020 enforced 2024+). Set to 'When supported' or 'Required'. |

| 15 | A) (&(objectClass=user)(userAccountControl:1.2.840.113556.1.4.803:=2)) | Uses LDAP_MATCHING_RULE_BIT_AND to check bit flag 2 (ACCOUNTDISABLE). |

| 16 | C) Schema Master and Domain Naming Master for forest-level operations | Adding DC touches forest partitions; Schema Master + Domain Naming Master must be reachable. RID Master needed for new DC's RID pool. |

| 17 | A) MSAL with Kerberos Cloud Trust for Windows Hello for Business | Modern Entra ID uses MSAL library, OAuth2/OIDC, Cloud Kerberos Trust eliminates need for line-of-sight to DC in 2024-2026 best practices. |

| 18 | B) It is a constructed / backlink attribute | memberOf is backlink of member; maintained by system, not directly written. Also not replicated as stored value. |

| 19 | A) Last writer wins based on version number and timestamp | Attribute-level conflict: highest version wins; if tie, latest timestamp; if still tie, higher DC GUID. |

| 20 | B) DNS SRV records (_ldap._tcp.dc._msdcs.<domain>) | DC Locator uses DNS SRV records plus site-specific records _ldap._tcp.<sitename>._sites... |

SECTION C: ADVANCED LEVEL (Questions 21-30)

****Focus:**** Security, Forensics, Tiering, LDAP Hardening, Large Scale Design, Troubleshooting 2024-2026.

****Q21.** In a suspected DCSync attack (Mimikatz), which LDAP extended control / replication right is abused?**

- A) DS-Replication-Get-Changes and DS-Replication-Get-Changes-All (GUIDs 1131f6aa, 1131f6ad)
- B) SeDebugPrivilege
- C) User-Force-Change-Password
- D) LDAP paged search control 1.2.840.113556.1.4.319

****Q22.** Starting March 2024, Microsoft enforces LDAP over TLS for Entra ID Domain Services and recommends strict LDAPS certificate mapping. Which TLS version minimum is now required for AD/LDAPS per Microsoft 2024 baselines?**

- A) TLS 1.0
- B) TLS 1.1
- C) TLS 1.2
- D) TLS 1.3 only

****Q23.** You have 100K users. LDAP query using filter (objectClass=user) is slow and returns sizeLimitExceeded. What is the best 2024-compliant solution?**

- A) Increase MaxPageSize in ntdsutil to 100000
- B) Use LDAP paged search control (RFC 2696) with page size 1000 and DirSync or incremental sync
- C) Disable Global Catalog
- D) Use Anonymous bind

****Q24.** Which attack abuses LDAP attribute msDS-KeyCredentialLink and is mitigated by monitoring shadow credentials?**

- A) Shadow Credentials / Whisker attack - allows adding Key Trust for pre-auth takeover without password
- B) Pass-the-hash
- C) Golden ticket
- D) SMB relay

****Q25.** In Active Directory 2025 functional level, what is the purpose of Privileged Access Management (PAM) and Shadow Principals via LDAP?**

- A) To allow JIT admin via time-bound membership of shadow principals in privileged groups

- B) To store BitLocker keys only
- C) To enable LDAPv2
- D) To replace Kerberos

****Q26. You must implement strict LDAP signing and prevent NTLM relay to LDAPS. Which combination is considered best practice 2025 by Microsoft and CISA?****

- A) Enforce LDAP Signing + LDAP Channel Binding (EPM=Required) + Extended Protection for Authentication + Disable NTLM where possible + Enable Kerberos Armoring (FAST)
- B) Allow Simple Bind and NTLMv1
- C) Open port 389 to internet
- D) Use self-signed cert for LDAPS with TLS 1.0

****Q27. A Domain Controller shows lingering objects due to replication isolation > Tombstone Lifetime (default 180 days in 2022+ forests). What tool should you use to detect and remove?****

- A) repadmin /removelingerobjects + Enable Strict Replication Consistency registry
- B) dcdiag /fix
- C) Delete NTDS.dit
- D) ipconfig /flushdns

****Q28. What is the LDAP syntax for a search that finds users whose adminCount is 1 (protected by AdminSDHolder) and who are members of Domain Admins?****

- A) (&(objectCategory=person)(objectClass=user)(adminCount=1)(memberOf=CN=Domain Admins,CN=Users,DC=corp,DC=com))
- B) (adminCount=*)
- C) (isCriticalSystemObject=TRUE)
- D) (objectClass=domainAdmin)

****Q29. In Entra ID Connect, how does LDAP/AD's mS-DS-ConsistencyGuid help prevent duplicate accounts and hard-match issues in 2024+ hybrid?****

- A) It stores a GUID derived from objectGUID that is immutable and used as sourceAnchor for immutableID to prevent matching errors after migration
- B) It is a password
- C) It is a Kerberos ticket
- D) It disables DirSync

****Q30. You are investigating LDAP enumeration via LDAP anonymous binds. Which Windows Server 2022/2025 setting correctly disables anonymous LDAP operations (except RootDSE) per CIS Benchmark v2.0.0+?****

- A) DsHeuristics bit 7: Set 0000002 and dSHeuristics 7th char to 2, plus set LDAP Interface Events and set HKLM Registry Allow LDAP Anonymous = 0 at path System\CurrentControlSet\Services\NTDS\Parameters
- B) Enable anonymous in LSA
- C) Set Everyone read permission on Domain NC
- D) Allow anonymous on port 3268

Answer Key - Advanced Level

| Q | Correct Answer | Explanation |

|---|-----|-----|

| 21 | A) DS-Replication-Get-Changes and DS-Replication-Get-Changes-All | DCSync requires those extended rights; should be audited, only Domain Controllers group should have them. Detect via 4662 events. |

| 22 | C) TLS 1.2 | Microsoft security baseline 2024+ deprecates TLS 1.0/1.1. TLS 1.2 minimum required for LDAPS, AD DS, Entra ID. TLS 1.3 supported in Server 2022/2025. |

| 23 | B) Use LDAP paged search control (RFC 2696) with page size 1000 and DirSync | Best practice: never raise MaxPageSize (default 1000/5000). Use paged results, VLV, or range retrieval. DirSync / USNChanged for large syncs. |

| 24 | A) Shadow Credentials / Whisker attack | msDS-KeyCredentialLink used for Windows Hello / Key Trust; unprivileged write allows persistence via PKINIT. Must audit via DS ACL. |

| 25 | A) To allow JIT admin via time-bound membership of shadow principals in privileged groups | PAM in AD uses MIM + Shadow Principals (objectClass=shadowPrincipal) for time-bound JIT membership with TTL, reducing standing privileged access. |

| 26 | A) Enforce LDAP Signing + Channel Binding + Extended Protection + Disable NTLM + FAST | CISA/Micro 2025: Require signing & channel binding, EPA, LDAP over TLS, disable NTLMv1, enforce Kerberos AES, FAST armoring. |

| 27 | A) repadmin /removelingerobjects + Enable Strict Replication Consistency | Linger objects removed per DC; enable Strict Consistency (registry 1) to block replication of lingerers. Tombstone lifetime 180d in new forests since 2003 SP1. |

| 28 | A) (&(objectCategory=person)(objectClass=user)(adminCount=1)(memberOf=CN=Domain Admins,CN=Users,DC=corp,DC=com)) | adminCount=1 indicates AdminSDHolder protection; combine with memberOf for DA. Note adminCount alone not sufficient, check ACL inheritance disabled. |

| 29 | A) It stores a GUID derived from objectGUID that is immutable and used as sourceAnchor for immutableID | mS-DS-ConsistencyGuid used as immutableID anchor prevents soft-match errors, especially during forest migration or Entra Cloud Sync (2024 best practice uses objectGUID->ConsistencyGuid). |

| 30 | A) DsHeuristics bit 7: Set 0000002 and dSHeuristics 7th char to 2 + disallow anonymous | CIS: Set DsHeuristics fChar 7=2 to block anonymous listing of objects, and ensure no null session access. Also audit via dSHeuristics and LDAP policies: AllowAnonymous is deprecated. RootDSE anonymous remains per RFC. |

Course File: 31_Cybersecurity.md

Cybersecurity - Final Examination

Course Code: 31_Cybersecurity | Total Questions: 30 | Year: 2024-2026 Aligned | Format: Multiple Choice (4 Options Each)

This exam covers foundational principles, modern threats (2024-2026), NIST CSF 2.0, OWASP Top 10 2023, Zero Trust, Cloud Security, and Advanced Incident Response.

Section A: Beginner Level (Questions 1-10)

1. What does the CIA Triad in cybersecurity stand for?

- A) Central Intelligence Agency Triad
- B) Confidentiality, Integrity, Availability
- C) Cyber Intelligence Analysis
- D) Control, Inspect, Authenticate

**2. Which of the following is the MOST effective defense against phishing in 2024-2026?

- A) Only using antivirus
- B) Multi-Factor Authentication (MFA) with phishing-resistant methods like FIDO2/Passkeys
- C) Changing wallpapers frequently
- D) Disabling firewalls

**3. What type of attack involves tricking a user into revealing credentials via a fake login page?

- A) DDoS
- B) Phishing / Credential Harvesting
- C) SQL Injection
- D) Cross-Site Scripting

**4. What does VPN stand for and what is its primary purpose?

- A) Virtual Private Network - creates an encrypted tunnel over public networks
- B) Visual Protocol Node - speeds up graphics
- C) Verified Personal Number - verifies identity
- D) Virtual Public Network - makes traffic public

**5. Which of the following is an example of a strong password policy aligned with NIST SP 800-63B (2024 update)?

- A) Password: 123456
- B) Require 8-char minimum, check against breached password lists, allow long passphrases, no mandatory 90-day expiration
- C) Require special characters only
- D) Use same password everywhere

**6. What is malware that encrypts victim data and demands payment for decryption called?

- A) Spyware
- B) Adware
- C) Ransomware
- D) Rootkit

**7. What is the purpose of a firewall?

- A) To cool down computers
- B) To monitor and control incoming and outgoing network traffic based on security rules
- C) To increase internet speed
- D) To store backups

**8. Two-Factor Authentication (2FA) is important because:

- A) It makes login slower

- B) It adds a second layer of verification beyond password, mitigating password theft
- C) It replaces antivirus
- D) It only works on iPhones

****9. Which law/regulation enforced from 2024 heavily focuses on data breach notification and protection in the EU?****

- A) HIPAA
- B) GDPR with 2024 NIS2 Directive requirements for critical sectors
- C) DMCA
- D) SOX only

****10. What is social engineering?****

- A) Building social media apps
- B) Manipulating people into breaking security procedures and divulging confidential information
- C) Engineering network cables
- D) Creating social networks

Answer Key - Beginner Level

1. B - Confidentiality, Integrity, Availability
2. B - Multi-Factor Authentication (MFA) with phishing-resistant methods like FIDO2/Passkeys - SMS codes are now phishable, FIDO2 is gold standard in 2024-26
3. B - Phishing / Credential Harvesting
4. A - Virtual Private Network - creates an encrypted tunnel over public networks
5. B - Require 8-char minimum, check against breached password lists, allow long passphrases, no mandatory 90-day expiration
6. C - Ransomware - e.g., LockBit, BlackCat/ALPHV dominant in 2024
7. B - To monitor and control incoming and outgoing network traffic based on security rules
8. B - It adds a second layer of verification beyond password, mitigating password theft
9. B - GDPR with 2024 NIS2 Directive requirements for critical sectors
10. B - Manipulating people into breaking security procedures and divulging confidential information

Section B: Intermediate Level (Questions 11-20)

****11. According to NIST Cybersecurity Framework (CSF) 2.0 released in Feb 2024, what is the 6th Function added?****

- A) Detect
- B) Govern
- C) Recover
- D) Protect

****12. In the OWASP Top 10 2023, what is now the #1 risk?****

- A) Injection
- B) Broken Access Control
- C) Cryptographic Failures
- D) Security Misconfiguration

****13. What is the primary difference between symmetric and asymmetric encryption?****

- A) Symmetric uses one shared key for encrypt/decrypt; Asymmetric uses public/private key pair
- B) No difference
- C) Symmetric is only for emails
- D) Asymmetric is faster

****14. What does Zero Trust Architecture assume, as per NIST SP 800-207?****

- A) Trust all internal traffic

B) Never trust, always verify - no implicit trust granted based on network location

C) Trust only VPN users

D) Trust only managers

****15. Which MITRE ATT&CK tactic involves techniques like PowerShell, Valid Accounts, and Windows Management Instrumentation?****

A) Initial Access

B) Execution

C) Exfiltration

D) Impact

****16. What is a Supply Chain Attack, prominent in incidents like MOVEit (2023) and XZ Utils (2024)?**

A) Attacking delivery trucks

B) Compromising a trusted third-party software/vendor to target its customers

C) Deleting supply data

D) Physical theft

****17. What is the purpose of an EDR/XDR solution in 2024-2026?**

A) To edit documents

B) Endpoint Detection and Response / Extended Detection and Response - continuous monitoring, detection, and automated response to threats on endpoints and across domains

C) To increase RAM

D) To manage emails only

****18. Which cloud security model describes Customer responsibility for securing DATA and OS, while Provider secures physical infrastructure?**

A) IaaS Shared Responsibility Model

B) No responsibility

C) Provider does everything

D) SaaS model where customer secures physical hosts

****19. What is Business Email Compromise (BEC) 2.0 with AI in 2024-2025 typically using?**

A) Only spelling mistakes

B) AI-generated deepfake voices, LLM-crafted emails without typos, and compromised legitimate mailboxes to bypass filters

C) Only phone calls

D) USB drops

****20. What does CVSS stand for and what does it rate?**

A) Common Vulnerability Scoring System - severity of vulnerabilities (0.0-10.0)

B) Cyber Virus Scanning System

C) Central Verification Security System

D) Common Virus Severity Scale

Answer Key - Intermediate Level

11. B - Govern - CSF 2.0 added Govern as 6th Function: GV, ID, PR, DE, RS, RC

12. B - Broken Access Control - moved from #5 in 2017/2021 to #1 in 2021/2023 list

13. A - Symmetric uses one shared key for encrypt/decrypt; Asymmetric uses public/private key pair

14. B - Never trust, always verify - no implicit trust granted based on network location

15. B - Execution - adversary trying to run malicious code

16. B - Compromising a trusted third-party software/vendor to target its customers

17. B - Endpoint Detection and Response / Extended Detection and Response - continuous monitoring, detection, and automated response

18. A - IaaS Shared Responsibility Model - e.g., AWS, Azure, GCP IaaS

19. B - AI-generated deepfake voices, LLM-crafted emails without typos, and compromised legitimate mailboxes to bypass filters

20. A - Common Vulnerability Scoring System - severity of vulnerabilities (0.0-10.0)

Section C: Advanced Level (Questions 21-30)

****21.** In 2024, a new critical vulnerability CVE-2024-3400 (CVSS 10.0) affected which vendor's firewall OS, exploited as a 0-day?

- A) Palo Alto Networks PAN-OS GlobalProtect
- B) Microsoft Windows 11
- C) Cisco IOS
- D) Fortinet FortiGate only

****22.** What technique does Pass-the-Hash / Pass-the-Ticket target in Active Directory attacks?

- A) DNS poisoning
- B) Theft and reuse of NTLM hash or Kerberos tickets to authenticate without knowing plaintext password
- C) SQL Injection
- D) Phishing only

****23.** Under MITRE D3FEND and NIST, what is the BEST mitigation for LLM Prompt Injection (OWASP LLM01:2025)?

- A) Disable firewalls
- B) Input validation, output encoding, least privilege for LLM tool access, and human-in-the-loop for critical actions
- C) Use only HTTP
- D) Allow all prompts

****24.** What is the concept of Post-Quantum Cryptography (PQC) standardization by NIST in 2024?

- A) Algorithms that are secure against quantum computer attacks; NIST finalized ML-KEM (Kyber), ML-DSA (Dilithium), SLH-DSA (SPHINCS+)
- B) Faster RSA
- C) Quantum internet only
- D) No such concept

****25.** How does a modern eBPF-based runtime security tool (e.g., CrowdStrike, Tetragon, Falco) differ from traditional AV?

- A) It only scans files once
- B) It hooks into kernel via eBPF to provide real-time syscall observability, container-aware detection, and prevents race conditions
- C) It requires reboot every time
- D) It is same as signature AV

****26.** In Cloud Incident Response, what is the key challenge with Cloud Forensics (AWS CloudTrail, Azure Activity Logs) in 2025?

- A) Too much storage always
- B) Ephemeral resources (serverless, containers), immutable vs mutable logs, cross-region evidence collection, and identity-based perimeter vs network perimeter
- C) No logs exist
- D) Only physical access needed

****27.** What is the correct order of the Incident Response Lifecycle as per NIST SP 800-61 Rev 2 (still authoritative in 2024-26)?

- A) Preparation, Detection & Analysis, Containment-Eradication-Recovery, Post-Incident Activity
- B) Delete, Ignore, Reboot
- C) Pay ransom immediately
- D) Containment first without Preparation

****28.** Which technique describes an attacker living off the land and using LOLBins (e.g., mshta.exe, rundll32, certutil) to evade detection?

- A) Fileless malware / Living Off The Land (LOTL) - using legitimate binaries to avoid signatures
- B) Brute Force
- C) DDoS
- D) Watering Hole only

****29. What does CISA's Secure by Design pledge (2024) and SSDF require vendors to do?****

- A) Ship with default passwords 1234
- B) Eliminate entire classes of vulnerabilities (e.g., memory safety), mandate MFA by default, provide transparency via VEX/SBOM, and responsible vulnerability disclosure
- C) Hide vulnerabilities
- D) Charge extra for logs

****30. A SOC analyst in 2025 sees Alert: Impossible Travel (Login from India and Germany 10 min apart) followed by creation of OAuth app with Mail.ReadWrite consent. What ATT&CK technique and response?****

- A) Account Manipulation and Valid Accounts persistence; Technique T1098, T1078 - Revoke OAuth tokens, disable user, investigate app registration, enforce Conditional Access
- B) Nothing to do
- C) DDoS attack
- D) Physical breach

Answer Key - Advanced Level

- 21. A - Palo Alto Networks PAN-OS GlobalProtect - CVE-2024-3400 command injection exploited in-the-wild April 2024
- 22. B - Theft and reuse of NTLM hash or Kerberos tickets to authenticate without knowing plaintext password
- 23. B - Input validation, output encoding, least privilege for LLM tool access, and human-in-the-loop for critical actions
- 24. A - Algorithms that are secure against quantum computer attacks; NIST finalized ML-KEM (Kyber), ML-DSA (Dilithium), SLH-DSA (SPHINCS+) in Aug 2024
- 25. B - It hooks into kernel via eBPF to provide real-time syscall observability, container-aware detection, and prevents race conditions
- 26. B - Ephemeral resources (serverless, containers), immutable vs mutable logs, cross-region evidence collection, and identity-based perimeter vs network perimeter
- 27. A - Preparation, Detection & Analysis, Containment-Eradication-Recovery, Post-Incident Activity
- 28. A - Fileless malware / Living Off The Land (LOTL) - using legitimate binaries to avoid signatures
- 29. B - Eliminate entire classes of vulnerabilities (e.g., memory safety), mandate MFA by default, provide transparency via VEX/SBOM, and responsible vulnerability disclosure
- 30. A - Account Manipulation and Valid Accounts persistence; Technique T1098, T1078 - Revoke OAuth tokens, disable user, investigate app registration, enforce Conditional Access - Classic 2024-25 Consent Grant attack pattern

****End of Examination****

****Total: 30 Questions | Marking: 1 Point per Question | Pass: 60% | Distinction: 80%****

Course File: 32_Vault.md

HashiCorp Vault Certified Associate & Professional - Complete Exam

Course Code: 32_Vault | Updated: 2024-2026 (Vault 1.15 - 1.18 / HCP Vault)

Part 1: Beginner Level (10 Questions)

Q1. What is the primary purpose of HashiCorp Vault?

- A) Container orchestration and scheduling
- B) Secrets management, encryption as a service, and identity-based access control
- C) Monitoring infrastructure logs and metrics
- D) Version control for Terraform code

Q2. In Vault, what is the sealed state?

- A) Vault is running but all data is encrypted and inaccessible; master key shards are required to unseal
- B) Vault is stopped and needs `systemctl start`
- C) Vault is in read-only mode after a network partition
- D) Vault license has expired

Q3. By default, which storage backend is used when you run `vault server -dev`?

- A) Consul
- B) Raft integrated storage
- C) In-memory storage
- D) PostgreSQL

Q4. What is the difference between KV Secrets Engine v1 and v2?

- A) v1 supports MFA, v2 does not
- B) v1 is for certificates, v2 is for passwords only
- C) v2 supports versioning, check-and-set, undelete, and soft-delete; v1 does not
- D) There is no difference, they are aliases

Q5. Which command creates or updates a Vault policy?

- A) `vault policy write my-policy policy.hcl`
- B) `vault write policy my-policy @policy.hcl`
- C) `vault auth enable policy`
- D) `vault policy create my-policy`

Q6. What is a Vault token?

- A) A Kubernetes manifest
- B) An authentication token that carries identity and policy membership; used for all API requests via `X-Vault-Token` header
- C) A static API key that never expires
- D) A TLS certificate issued by PKI engine

Q7. Which default auth method is enabled automatically on a fresh Vault dev server?

- A) AppRole
- B) Kubernetes
- C) Token
- D) GitHub

Q8. What does vault status return when Vault is sealed?

- A) Sealed: true, HA Mode, and seal type (shamir, awskms, etc.)
- B) Error 500
- C) List of all secrets
- D) Active node count

Q9. Which path is the default for KV v2 secrets?

- A) kv/
- B) secret/ (in dev mode) or kv mount configured with type kv-v2
- C) sys/
- D) auth/

Q10. What is the purpose of VAULT_ADDR environment variable?

- A) To set where the Vault CLI/API client should connect to the Vault server
- B) To set the storage address
- C) To configure auto-unseal
- D) To define the audit log path

****Answer Key - Beginner:****

1. B - Vault is a secrets management solution providing secure storage, dynamic secrets, data encryption, and identity-based access.
2. A - When sealed, Vault knows where data is but cannot decrypt it. Requires unseal keys or auto-unseal recovery.
3. C - Dev mode uses in-memory only, non-persistent, root token = 'root'.
4. C - KV v2 (added in 0.10) provides versioned secrets with metadata, soft delete. KV v1 is unversioned.
5. A - Correct syntax is vault policy write <name> <file.hcl>. File contains path {} blocks.
6. B - Tokens are core client auth mechanism. Service vs batch tokens (1.15+).
7. C - Token auth method is always enabled and cannot be disabled. Dev server also enables token only by default in newer versions; older dev docs showed userpass enabled but current default is token only.
8. A - vault status shows sealed/unsealed, seal type, storage type, HA mode, cluster name, version.
9. B - Dev server mounts kv-v2 at secret/. In production you typically mount vault secrets enable -path=kv kv-v2.
10. A - Example: export VAULT_ADDR='https://vault.example.com:8200'

Part 2: Intermediate Level (10 Questions)

Q11. In AppRole auth method, what is the security model?

- A) role_id is like username (non-secret) and secret_id is like password (secret); best practice is different delivery mechanisms
- B) Both role_id and secret_id are public
- C) AppRole is only for human users
- D) AppRole requires LDAP to function

Q12. How does Vault's lease and renewal system work for dynamic secrets?

- A) All secrets are static forever
- B) Secrets issued have a lease ID, TTL, and max TTL; clients must renew via vault lease renew or API before TTL expiry, otherwise Vault revokes them
- C) Leases are only for KV secrets
- D) Vault automatically renews forever without client action

Q13. What is Response Wrapping?

- A) Compressing the Vault binary
- B) A method where Vault stores a single-use response in its cubbyhole and returns a wrapping token; recipient unwraps to get real secret, protecting against eavesdropping
- C) Wrapping audit logs in JSON
- D) Encrypting Raft snapshots

Q14. You have a policy: path "secret/data/prod/*" { capabilities = ["read"] }. A user tries vault kv put secret/prod/api key=123. Why does it fail?

- A) Policy only grants read, not create/update. KV v2 requires create, update capabilities on secret/data/prod/* and often read on secret/metadata/*
- B) Vault doesn't support wildcard
- C) User must be root
- D) Path should be secret/prod/* without data

Q15. What does the Transit secrets engine provide?

- A) Database credential rotation
- B) Encryption as a Service (EaaS) - encrypt/decrypt data without storing it in Vault; key management, rotation, bring-your-own-key
- C) SSH key signing
- D) Kubernetes authentication

Q16. Which statement about Vault Agent is true for Vault 1.15+?

- A) Vault Agent is deprecated
- B) Agent can handle auto-auth, caching, templating (consul-template syntax via agent templates), and API proxy; reduces token sprawl and is recommended for machines
- C) Agent only works with AWS
- D) Agent stores secrets in plaintext on disk by default

Q17. What is the Cubbyhole secrets engine?

- A) A global secret store
- B) A per-token isolated secret store for the token's lifetime; no one else including root can read it (except via wrapping). Used for response wrapping and secure introductions
- C) Same as KV v2
- D) An audit device

Q18. When configuring Kubernetes auth method, what does Vault validate?

- A) The Kubernetes ServiceAccount JWT by calling TokenReview API against K8s API server using configured service account token
- B) Vault checks Docker Hub credentials
- C) Vault checks Consul ACL
- D) Only checks username/password

Q19. What is the correct way to enable audit logging in production?

- A) vault audit enable file file_path=/var/log/vault-audit.log - audit devices must be enabled; at least one file or syslog device is required for compliance, and logs are raw but can be hashed with log_raw=false
- B) Audit is enabled by default
- C) Audit logs are stored in Raft automatically
- D) vault secrets enable audit

Q20. For PKI secrets engine, what happens when you vault write pki/roles/web-server allow_any_name=true max_ttl=72h?

- A) It creates a role that can issue certificates for any CN/SAN with max TTL 72 hours; leaf certs still constrained by ttl parameter in request
- B) It deletes the PKI engine
- C) It generates a root CA immediately
- D) It only allows web-server hostnames

****Answer Key - Intermediate:****

- 11. A - role_id is semi-public identifier, secret_id is secret credential. Pull model requires secure delivery of secret_id via separate trusted orchestrator.
- 12. B - Lease lifecycle is core to dynamic secrets (DB, AWS, PKI). KV v2 can also have optional TTL but not leased.
- 13. B - vault kv get -wrap-ttl=60s secret/... returns wrapping token. Unwrap: vault unwrap <token> or vault unwrap -field= . Prevents clear text over wire.
- 14. A - KV v2 API split: data path is secret/data/..., metadata is secret/metadata/.... Writing needs create+update.
- 15. B - Transit: vault write -f transit/keys/my-key, then transit/encrypt/my-key, transit/decrypt/my-key. Never stores data, only keys. Supports key rotation, rewrap, convergent encryption.
- 16. B - Since Vault 1.11+, agent is mature with auto_auth { method { type=approle/kubernetes... } }, cache { use_auto_auth_token=true }, template {} blocks.
- 17. B - Cubbyhole path is cubbyhole/. Tokens see only their own cubbyhole. Data dies when token expires.
- 18. A - Config: vault write auth/kubernetes/config kubernetes_host=... token_reviewer_jwt=.... Role binds service_account names/namespaces.
- 19. A - Audit devices log every request/response (excluding sensitive HMAC'd values). Multiple audit devices allowed. Best practice: enable file audit before production traffic.
- 20. A - PKI roles define constraints: allow_subdomains, allowed_domains, allow_any_name, key_type, key_bits, enforce_hostnames. pki/issue/<role> issues leaf.

Part 3: Advanced Level (10 Questions)

Q21. You are migrating from Shamir seal to AWS KMS Auto-Unseal with zero downtime on Raft Integrated Storage (Vault 1.15+). What is the supported process?

- A) Stop all nodes, edit config, restart
- B) Use seal migration: add new seal stanza with disabled=false, old with disabled=false, set api_addr and run vault operator unseal -migrate following docs; or for 1.16+, vault operator seal migration status workflow. Requires one unseal with old and new mechanism

- C) Auto-unseal migration is impossible
- D) Just change config, Vault auto-detects

Q22. What is the difference between Disaster Recovery (DR) Replication and Performance Replication (Enterprise 2024)?

- A) DR replicates static data for failover (standby cluster, limited API), Performance replicates for scaling reads/writes; since 1.14+, Performance secondary can be promoted to primary via `vault write -f sys/replication/performance/primary/promote`
- B) No difference
- C) DR is OSS, Performance is Enterprise
- D) DR replication replicates only dynamic secrets

Q23. In Vault Enterprise Namespaces (2024-2026), which statement is accurate?

- A) Namespaces provide isolated environments with their own policies, auth methods, and secret engines; root namespace admin can create child namespaces. Path becomes `ns1/ns2/secret/...` and tokens are namespace-bound. Since 1.14, API `X-Vault-Namespace` header.
- B) Namespaces share all policies globally
- C) OSS has namespaces
- D) Namespaces are only UI folders

Q24. High Availability with Raft Integrated Storage in Vault 1.18: A 3-node cluster loses 1 node. What happens and what is quorum?

- A) Cluster continues to serve, Raft tolerates $(n-1)/2$ failures; 3 nodes needs quorum 2. Leader election via Raft. `vault operator raft list-peers` shows peers. Performance standbys forward writes to active
- B) Cluster stops immediately
- C) All nodes become active
- D) Raft requires 5 nodes minimum

Q25. Explain Identity Entities and Groups vs direct token policies. Why use Identity in Vault 1.15+?

- A) Identity ties multiple auth method logins (e.g. GitHub + LDAP same user) to single Entity with merged policies via Entity + external/internal Groups; enables group-based policy via `identity/group/id/...`, simplifies SSO across clouds and supports external group mappings from OIDC/JWT/Kubernetes
- B) Identity is legacy
- C) Entities are same as tokens
- D) Identity only works with Userpass

Q26. A compliance requirement: Production database credentials rotated every 24h, apps never restarted manually. Which architecture (2024 best practice) meets it?

- A) Static KV secrets
- B) Dynamic Database Secrets Engine with rotation `vault write database/config/mydb ... rotation_period=24h` + static roles via `database/static-roles`, combined with Vault Agent Templates or Secrets Store CSI Driver for Kubernetes to inject/render without restart
- C) Hardcode credentials
- D) Use AppRole with infinite TTL

Q27. Vault Agent template vs vault proxy template behavior and Sentinel EGP/RGP (Enterprise) - select correct advanced security feature:

- A) Agent templates use consul-template syntax `{{ with secret "secret/data/..." }}{{ .Data.data.key }}{{ end }}` to render files; Sentinel policies (RGP/EGP) allow advanced logic: `main = rule { ... }` to restrict CIDR, MFA enforcement, allowed paths; EGP runs on every request, RGP after ACLs. Since Vault 1.16, MFA with mfa login enforcement
- B) Templates are not supported
- C) Sentinel is OSS
- D) Sentinel replaces HCL policies

Q28. You suspect a Vault node is running out of memory due to large audit logs and large lease counts. Which advanced tuning would you apply in Vault 1.17-1.18?

- A) Increase `max_lease_ttl`, `default_lease_ttl` to infinite, disable audit
- B) Use `max_request_duration`, `max_request_size`, `disable_mlock` only
- C) Enable batch tokens for high-volume ephemeral workloads (non-renewable, non-persistent, 32k byte limited), configure `cache_size` for auth cache, tune audit `{ file { ... } }` with `log_raw=false` and `hmac_accessor=true`, reduce lease count via `sys/leases/revoke-force` and shorten TTLs, enable `disable_cache=false` for agent, monitor `sys/metrics` Prometheus
- D) Restart Vault daily

Q29. Zero-Trust deployment: Kubernetes pods need Vault secrets without long-lived tokens, across 2 datacenters. Choose correct advanced pattern for 2025-2026.

- A) Hardcoded root token in ENV
- B) JWT auth with bound service accounts plus Vault Agent Injector or Secrets Store CSI Driver, Short-lived tokens (TTL 5m), Performance Replication for second DC with local mount for auth/kubernetes, use `token_bound_cidrs`, `token_explicit_max_ttl`, response wrapping for Agent auto-auth
- C) Use UI token only
- D) Share same token for all pods

Q30. Vault 1.15+ introduced support for Managed PKI and Improved UI, plus what major change in licensing and storage?

- A) Vault moved to BSL license in 2023; Raft is now recommended production storage over Consul; HCP Vault Dedicated now supports tiered namespaces, managed audit, and automated upgrades. Vault 1.15+ removed support for mlock on Windows, added PKCS#11 auto-unseal improvements and OpenAPI 3.x spec
- B) Vault is now MIT only
- C) Consul is mandatory again
- D) Vault no longer supports Kubernetes

****Answer Key - Advanced:****

21. B - Seal migration workflow documented: <https://developer.hashicorp.com/vault/docs/concepts/seal#seal-migration> . Requires both seals defined, run migration command. Since 1.16 supports vault operator `unseal -migrate`.

22. A - DR: disaster recovery copy, cannot serve reads/writes until promoted (limited `/sys` endpoints). Performance: active on secondaries for reads, can have local mounts (auth). Enterprise only.

23. A - Namespaces isolation: each has own `sys/`. Root token in child cannot escape. Path API: `X-Vault-Namespace` header or namespace in path for API. Available only Enterprise/HCP.

24. A - Raft quorum = $\text{floor}(n/2)+1$. 3 nodes tolerates 1 failure, 5 tolerates 2. vault operator raft autopilot state. Performance standby nodes serve reads but forward writes.

25. A - Entity model: unique identity, aliases per auth mount. Identity groups internal vs external. External maps from auth method claims (groups from OIDC). Entity alias count visible in identity/entity/id/.
26. B - Database engine database/roles/* dynamic, database/static-roles/* auto rotation. Vault Agent template renders env file and can trigger reload via command. CSI driver syncs K8s secret.
27. A - Agent template rendering + Prometheus metrics. Sentinel: requires license, written in Sentinel language, RGP enforced at policy check, EGP enforced pre-policy. Example: import "sockaddr"; main = rule {
sockaddr.is_contained("10.0.0.0/8", http.remote_address) }
28. C - Batch tokens: lightweight, encrypted blob, no storage. Service tokens persisted. Also need disable_mlock false in prod, tune api_addr, cluster_addr, limit max_lease_ttl=768h. Metrics: vault.token.count.by..., vault.audit.log.request.
29. B - Modern zero-trust: Kubernetes auth, short-lived, Agent Injector via annotations vault.hashicorp.com/agent-inject, CSI driver, replication. bound_service_account_names, bound_service_account_namespaces. No static secrets.
30. A - Key context 2024-2026: License BSL (HashiCorp changed 2023), Raft integrated storage is default production backend since 1.11, HCP Vault Dedicated is SaaS evolution, Plugin ecosystem, removal of old storage backends. Vault 1.15 added ACME support for PKI, 1.16 Secrets Sync to AWS/GCP/Azure etc, 1.17+ WIF and improved events system.

****Exam Stats:**** 30 Questions | 4 Options Each | 3 Difficulty Tiers | Vault 1.15 - 1.18 / HCP Vault Dedicated (2024-2026)

****Passing Score:**** 70% Beginner, 75% Intermediate, 80% Advanced

Course File: 33_Playwright.md

Course Title: Playwright & Cypress - End-to-End Testing Mastery (2024-2026)

Exam Overview

- Total Questions: 30
- Format: Multiple Choice (4 options each)
- Levels: Beginner (Q1-10), Intermediate (Q11-20), Advanced (Q21-30)
- Stack: Playwright 1.40+ (2024-2026 features), Cypress 13.x/14.x
- Time: 90 minutes | Passing: 70%

Part 1: Beginner Level (10 Questions)

Q1. What is the primary architectural difference between Playwright and Cypress regarding browser execution?

- A) Playwright runs inside the browser, Cypress runs outside
- B) Cypress runs inside the browser's event loop (same run-loop), Playwright controls browsers via WebSocket/CDP out-of-process
- C) Both run outside the browser using Selenium protocol
- D) Playwright only supports Chromium, Cypress supports all browsers

Q2. In Playwright, which command initializes a new project with official test runner?

- A) `npx cypress open`
- B) `npm init playwright@latest`
- C) `npx create-playwright-app`
- D) `npx playwright init`

Q3. In Cypress, what does `cy.get('[data-testid="submit"]')` do?

- A) Gets element by ID using jQuery in Node.js
- B) Queries the DOM for element with that attribute and retries automatically until found or timeout
- C) Makes an HTTP GET request to `/submit`
- D) Gets data from `localStorage`

Q4. Which Playwright locator is recommended as of 2024 for best resilience?

- A) `page.$('css selector')`
- B) `page.locator('xpath=//div')`
- C) `page.getByRole('button', { name: 'Submit' })` - using user-facing locators
- D) `page.locator('#submitBtn')`

Q5. How do you handle auto-waiting in Cypress?

- A) You must manually add `page.waitForTimeout(5000)` before every action
- B) Cypress automatically retries commands and assertions before failing
- C) Use `await driver.wait()` like Selenium
- D) Auto-waiting is not supported in Cypress

Q6. What is the default test runner file pattern for Playwright?

- A) `**/*.cy.js`
- B) `**/*.e2e.js`
- C) `**/*.spec.js` or `**/*.test.js` and searched via `playwright.config.ts testDir`
- D) `cypress/e2e/**/*.js` only

Q7. In Cypress, how do you intercept and mock a network request in Cypress 13+?

- A) `cy.route()`
- B) `cy.intercept('GET', '/api/users', { fixture: 'users.json' })`

C) `page.route()` from Playwright

D) `cy.mock()`

****Q8. What does `test.only` do in Playwright Test?****

A) Skips all tests

B) Runs only the marked test(s) - useful for focused debugging

C) Marks test as flaky

D) Deletes other tests

****Q9. Which assertion library is built into Playwright Test runner by default?****

A) Chai used by Cypress

B) Jest Expect compatible expect with web-first assertions like `toBeVisible()`, `toHaveText()`

C) Mocha assert

D) `Should.js`

****Q10. In Cypress, where is `baseUrl` configured?****

A) In `package.json`

B) In `cypress.config.js` / `cypress.config.ts` under `e2e.baseUrl`

C) In `.env` file only

D) Cannot be configured

Answer Key - Beginner

| Q | Answer | Explanation |

|---|-----|-----|

| Q1 | B | Cypress runs IN browser; Playwright uses out-of-process automation via CDP/BiDi/WebSocket giving multi-tab, multi-origin, multi-browser context support. |

| Q2 | B | `npm init playwright@latest` is official initializer (also `npm init playwright@latest -- --g` pattern). Creates config, example tests, GitHub workflow. |

| Q3 | B | Cypress commands are async, retried, DOM-querying. It automatically retries get until timeout (default 4000ms). |

| Q4 | C | Playwright best practice 2024-2026: Use `getByRole`, `getByLabel`, `getByText`, `getByTestId` for accessibility-first, resilient locators. |

| Q5 | B | Core Cypress feature: automatic retry-ability for commands and assertions. No manual waits needed for most cases. |

| Q6 | C | Playwright default `testDir` is `./tests` or `./e2e` and matches `*.spec.ts/js` etc, configurable in `playwright.config`. |

| Q7 | B | `cy.route()` was v6 and deprecated; `cy.intercept()` is standard since Cypress 6+ and current in 13/14. |

| Q8 | B | `test.only` focuses execution. In CI it will fail if `test.only` left behind if `forbidOnly` is set. |

| Q9 | B | Playwright Test bundles expect with web-first assertions that auto-retry (e.g., `await expect(locator).toBeVisible()`). |

| Q10 | B | Configured in `cypress.config.js/ts`: `e2e: { baseUrl: 'http://localhost:3000' }` |

Part 2: Intermediate Level (10 Questions)

****Q11. How do you perform API testing + UI in the same Playwright test?****

A) Not possible, you need separate tools

B) Use `await request.newContext()` or `playwright.request` fixture: `await request.get('/api/users')` then use `page` for UI

C) Use `cy.request()` inside Playwright

D) Only via `cy.intercept()`

****Q12. In Playwright, what is the purpose of trace viewer?****

A) To record video only

B) To collect comprehensive execution trace: screenshots, snapshots, network, actions, console - viewable via `npx playwright show-trace` for time-travel debugging

C) To trace memory leaks

D) To replace console.log

****Q13. Cypress Component Testing as of 2024-2026 - which statement is true?****

A) Cypress only supports E2E

B) Cypress Component Testing mounts components in real browser using framework-specific bundler (Vite/Webpack) via cy.mount()

C) Component testing requires Playwright

D) cy.mount() is deprecated in Cypress 14

****Q14. Which Playwright config option enables sharding across multiple CI machines?****

A) workers: 4

B) shard: { total: 3, current: 1 } via CLI --shard=1/3

C) parallel: true

D) cypress run --record --parallel

****Q15. How to handle iFrame in Playwright vs Cypress?****

A) Both use same syntax

B) Playwright: page.frameLocator('#my-iframe').getByRole(...) - first-class frameLocator; Cypress requires custom handling or cy.iframe() from plugin / cy.get('iframe').its('0.contentDocument')

C) Playwright cannot handle iframes

D) Cypress has frameLocator

****Q16. What does page.getByTestId() map to by default in Playwright?****

A) id attribute

B) data-testid attribute

C) class attribute

D) name attribute

****Q17. In Cypress, why might cy.contains('Submit').click() fail intermittently?****

A) Cypress doesn't auto-wait

B) Element may be detached, covered, animating, or multiple matches. Need to ensure visibility or use {force:false} default checks; check for anti-pattern of mixing async/await incorrectly

C) Because contains is not a valid command

D) Because click requires Playwright

****Q18. How does Playwright handle multiple browser contexts for testing isolation?****

A) Launches new browser instance for each test physically

B) Uses browser.newContext() which is lightweight isolated incognito-like profile (cookies, storage isolated) - very fast compared to new browser

C) Cannot isolate

D) Uses cy.session()

****Q19. What is the correct way to wait for network idle in Playwright 1.40+?****

A) page.waitForTimeout(10000)

B) await page.waitForLoadState('networkidle') is discouraged for assertions; better to await expect(page.getByText('Loaded')).toBeVisible() and use waitForResponse / waitForLoadState only when needed

C) cy.wait('@api')

D) driver.sleep(5)

****Q20. In Cypress 13+, what is cy.session() used for?****

A) To save video recording session

B) To cache and restore browser session (cookies, localStorage, sessionStorage) across tests to speed up login flows with validate function

C) To create Playwright storageState

D) To delete cookies

Answer Key - Intermediate

| Q | Answer | Explanation |

|---|-----|-----|

| Q11 | B | Playwright's request fixture allows API calls: `const res = await request.get(...); await expect(res).toBeOK();` Same test can then use page. |

| Q12 | B | Trace viewer is flagship debugging tool (since 1.20+, improved 2024-2026). Contains DOM snapshots, network, actions. Enable via trace: 'on-first-retry' in config. |

| Q13 | B | Cypress CT is stable in Cypress 12+ / 13 / 14: supports React, Vue, Angular, Svelte via Vite/Webpack. Core API is `cy.mount()`. |

| Q14 | B | Playwright sharding: `npx playwright test --shard=1/3` splits tests across machines. Config API also supports shard object. |

| Q15 | B | Playwright has native `frameLocator` chaining. Cypress has no native iframe support, requires workaround. |

| Q16 | B | Default `testIdAttribute` is `data-testid`, configurable via `testIdAttribute` in config to customize. |

| Q17 | B | Common flakiness causes: element detached from DOM, covered by overlay, animating, multiple matching elements. Cypress actionability checks will fail. |

| Q18 | B | `BrowserContext` is core Playwright isolation primitive - cheap, fast, provides full isolation similar to new incognito profile. |

| Q19 | B | Playwright team in 2024 explicitly warns `networkidle` is flaky; prefer web-first assertions waiting for UI state. Use `waitForResponse` for specific API. |

| Q20 | B | `cy.session()` introduced Cypress 8+ and stabilized 12+ for session caching: `cy.session('login', () => { cy.login() }, {validate()})` |

Part 3: Advanced Level (10 Questions)

****Q21.** In Playwright 2024-2026, what is UI Mode and how does it differ from `npx playwright test --ui` vs old `show-trace`?

- A) UI Mode is a watch mode, interactive runner with live locators, time travel, pick locator, watch, filter - launched via `npx playwright test --ui`
- B) Same as Cypress open
- C) Only shows HTML report
- D) UI Mode doesn't exist

****Q22.** Advanced fixture pattern in Playwright Test for dependency injection and teardown, which is correct?

- A) fixtures: `export const test = base.extend({ db: async ({}, use) => { const conn = await connect(); await use(conn); await conn.close(); } })`
- B) Use global variables
- C) Use `cy.fixture()`
- D) Fixtures not supported

****Q23.** How do you test file downloads in Playwright vs Cypress?

- A) Both identical
- B) Playwright: `const [download] = await Promise.all([page.waitForEvent('download'), page.getByText('Download').click()]); await download.saveAs('./file.pdf')` - first class. Cypress: no native download event, requires intercepting or checking file system via `cy.readFile` after removing download attribute or using `cypress-file-upload` hacks
- C) Cypress has native download event
- D) Downloads cannot be tested

****Q24.** Which strategy best handles visual regression in Playwright 1.40+?

- A) Manual screenshot comparison via eyes
- B) `await expect(page).toHaveScreenshot('homepage.png', { maxDiffPixels: 100 })` and `toHaveScreenshot` with config `expect: { toHaveScreenshot: { threshold: 0.2 } }` - auto generates baselines and updates with `--update-snapshots`
- C) Use `cy.screenshot()` only
- D) Playwright doesn't support visual regression

****Q25. As of Cypress 14 (2024-2025), what is true about Cypress's attempt to support cross-origin and multi-tab?****

- A) Cypress now fully supports multi-tab like Playwright with `cy.newTab()`
- B) Cypress still single-tab architecture at core, but supports cross-origin via `cy.origin()` and limited multi-tab via `cy.session()` + workarounds; Playwright remains superior for true multi-tab/context scenarios
- C) Cypress removed `cy.origin()`
- D) Playwright cannot do cross-origin

****Q26. In Playwright, how do you use `APIRequestContext` to set up authenticated state without UI login for faster tests?****

- A) Not possible
- B) Login via API to get token/storage, then `await request.newContext()` login, then `save await context.storageState({ path: 'auth.json' })` and reuse in config via use: `{ storageState: 'auth.json' }` or via project dependencies with `setup project`
- C) Use `cy.session()` in Playwright
- D) Only via UI login

****Q27. What is the Playwright `expect.poll()` used for (introduced 1.38+)?****

- A) To poll DOM
- B) To repeatedly execute a function until assertion passes - e.g., `await expect.poll(() => fetchStatus(), { timeout: 10000 }).toBe('complete')` - for non-UI async conditions
- C) To replace `toBeVisible`
- D) For Cypress only

****Q28. Which Cypress pattern prevents flakiness from shared state between tests?****

- A) Disable test isolation: `testIsolation: false` always
- B) Keep `testIsolation: true` (default in Cypress 12+), clear state per test, use `beforeEach` with `cy.session()` for login reuse rather than preserving DOM. Avoid global state
- C) Use `page.reload()` everywhere
- D) Use Playwright contexts

****Q29. In advanced Playwright `globalSetup` with sharding and projects, what's the 2024-2026 recommended auth setup pattern?****

- A) Login in `beforeEach`
- B) Use `setup project` with dependencies: In config define projects: `[{name:'setup', testMatch:/.*\.setup\.ts/}, {name:'chromium', dependencies:['setup']}]` where `setup` saves `storageState`, and `chromium` project uses it
- C) Use `cy.login()`
- D) Hardcode tokens in `.env`

****Q30. What is true about Playwright vs Cypress performance and Electron / mobile testing as of 2026?****

- A) Cypress supports Electron natively and native mobile App testing; Playwright doesn't
- B) Playwright supports: Chromium, Firefox, WebKit with single API, mobile emulation via `devices`, Electron testing via `electron fixture _electron.launch()`, and component testing via `@playwright/experimental-ct-react`. Cypress runs only Electron-based runner (Chromium) for E2E, limited real-device mobile emulation, Firefox/WebKit experimental, no native Electron app or native mobile app testing
- C) Both identical for all platforms
- D) Playwright only tests Chrome

Answer Key - Advanced

| Q | Answer | Explanation |

|---|-----|-----|

| Q21 | A | UI Mode (added 2023, matured 2024-2026) is interactive watch mode. Distinct from HTML reporter and trace viewer. It provides locator picker and live editing. |

| Q22 | A | Playwright's fixture system uses `base.extend` with `auto teardown` after `use()` – pattern introduced for DI, similar to `pytest fixtures`. Critical for DB, Page Object chaining. |

| Q23 | B | Download is first-class in Playwright via download event. Cypress lacks download event due to in-browser architecture; workarounds involve verifying request or file. |

| Q24 | B | Playwright has first-class visual comparisons: toHaveScreenshot for full page, locator screenshots. Baseline management integrated. Percy/Applitools optional. |

| Q25 | B | Cypress cross-origin via cy.origin() (since v10 stable) solves many cases but still single tab per spec. Playwright has native multi-tab/window/context support. |

| Q26 | B | Recommended pattern 2024-2026: Authenticate via API, save storageState, reuse. Fastest and avoids UI login flakiness. Use project dependencies to guarantee setup runs first. |

| Q27 | B | expect.poll() and expect.toPass() (1.38+, 2024+) for polling non-locator async conditions with auto-retry, complementing web-first assertions. |

| Q28 | B | Since Cypress 12, testIsolation:true is default. Best practice is keep it true, use cy.session() to cache session without leaking DOM state. testIsolation:false is anti-pattern unless intentional. |

| Q29 | B | Setup project dependencies pattern is official Playwright best practice since 1.40+ (2024) to replace globalSetup for auth: scalable, traceable, sharded. |

| Q30 | B | Accurate 2024-2026 comparison: Playwright = multi-engine + Electron + mobile device emulation + component testing. Cypress = browser-runner bound, Chromium-centric, limited Firefox/WebKit, no Electron app automation. Both not native mobile app (need Appium, but Playwright can drive WebView better). |

Scoring & Competency Map

- Beginner 0-10: Fundamentals, locators, commands, config, assertions

- Intermediate 11-20: Network, iframes, contexts, sessions, trace, sharding, waiting strategies

- Advanced 21-30: Fixtures, UI Mode, downloads, visual regression, cross-origin architecture, storageState patterns, performance

© 2026 Playwright & Cypress Exam - Updated for Playwright 1.54+ and Cypress 14.x

Course File: 34_Selenium.md

Course Title: Selenium & Test Automation - Comprehensive Certification Exam

Total Questions: 30 | Duration: 90 Minutes | Pass Mark: 70%

Syllabus Coverage: Selenium 4.x (2024-2026), WebDriver W3C, Selenium Manager, Grid 4, Frameworks, CI/CD

Section A: Beginner Level (Questions 1-10)

Q1. What is the current architecture of Selenium WebDriver as of Selenium 4 (2024-2026)?

- A) JSON Wire Protocol based client-server with browser-specific drivers requiring manual setup
- B) W3C WebDriver standard compliant, with Selenium Manager automatically managing drivers
- C) Chrome DevTools Protocol only for all browsers
- D) Selenium RC based architecture with Selenium Server as mandatory middleware

Q2. Which component introduced in Selenium 4.6+ eliminates the need for third-party libraries like WebDriverManager to handle browser drivers?

- A) Selenium Hub
- B) Selenium Builder
- C) Selenium Manager (written in Rust)
- D) Selenium Driver Guardian

Q3. Which of the following is the CORRECT way to initialize ChromeDriver in Selenium 4.11+ without manually setting system property for chromedriver?

- A) `System.setProperty("webdriver.chrome.driver", "/path/to/driver"); WebDriver driver = new ChromeDriver();`
- B) `WebDriver driver = new ChromeDriver(); // Selenium Manager handles driver automatically`
- C) `ChromeDriverManager.getInstance().setup(); WebDriver driver = new ChromeDriver();`
- D) `WebDriver driver = ChromeDriver.createDefault();`

Q4. Which locator strategy is considered the most robust and fastest in Selenium?

- A) XPath Absolute
- B) CSS Selector with dynamic IDs
- D) TagName

Q5. What does `driver.quit()` do compared to `driver.close()`?

- A) Both are identical
- B) `close()` closes all windows and ends session; `quit()` closes current window only
- C) `close()` closes the current window/tab; `quit()` closes all windows and ends WebDriver session, cleaning up temp files
- D) `quit()` minimizes browser; `close()` closes browser

Q6. Which Selenium 4 feature allows locating elements based on their visual position relative to other elements?

- A) Absolute Locators
- B) Relative Locators (Friendly Locators) - above(), below(), toLeftOf(), toRightOf(), near()
- C) Visual Locators
- D) Proximity Locators

Q7. What is the default timeout for implicitlyWait if not set?

- A) 0 seconds (No wait)
- B) 5 seconds
- C) 10 seconds
- D) 30 seconds

Q8. Which file format is commonly used for Page Object Model to support BDD with Selenium in 2024-2026?

- A) Feature files (Gherkin) with Cucumber + Selenium integration
- B) .exe files
- C) .json only
- D) .xml only

Q9. Which method is used to get the title of the current page in Selenium WebDriver?

- A) driver.getPageTitle()
- B) driver.getTitle()
- C) driver.title()
- D) driver.fetchTitle()

Q10. Which assertion library is most commonly integrated with Selenium Java for TestNG as of 2024?

- A) TestNG Assert and AssertJ / Hamcrest
- B) JUnit 3 Assert
- C) Selenium Assert API
- D) WebDriver Assert

Answer Key - Beginner Level

| Q | Answer | Explanation |

|---|-----|-----|

| 1 | B | Since Selenium 4, WebDriver is fully W3C compliant. JSON Wire Protocol is legacy. Selenium Manager auto-manages drivers. |

| 2 | C | Selenium Manager, introduced in 4.6.0 (Rust-based tool), auto-downloads/manages drivers, browsers. Replaces need for Bonigarcia WebDriverManager. |

| 3 | B | From Selenium 4.6+, especially 4.11+, you don't need System.setProperty. Selenium Manager handles it: new ChromeDriver() works out-of-the-box. |

| 4 | C | ID is unique, fastest as it uses document.getElementById internally. |

| 5 | C | close() = current window, quit() = entire session with all windows, ends driver executable process. |

| 6 | B | Relative Locators introduced in Selenium 4.0: withTagName().above(), below(), near(), etc. |

| 7 | A | Default implicit wait is 0. No waiting unless explicitly set. |

| 8 | A | BDD with Cucumber feature files (Given/When/Then) is industry standard with Selenium POM. |

| 9 | B | driver.getTitle() is correct API. |

| 10 | A | TestNG provides Assert class, and modern stacks use AssertJ for fluent assertions. |

Section B: Intermediate Level (Questions 11-20)

Q11. What is the recommended replacement for Thread.sleep() for handling dynamic elements?

- A) Implicit Wait
- B) Explicit Wait with WebDriverWait and ExpectedConditions
- C) Increasing pageLoadTimeout
- D) Using JavaScript alert wait

Q12. How do you handle multiple windows/tabs in Selenium 4?

- A) driver.switchTo().window(windowHandle)
- B) driver.switchWindow(index)
- C) driver.moveToWindow(handle)
- D) driver.selectWindow()

Q13. In Selenium 4, how do you create a new tab/window and switch to it natively without JavaScript?

- A) driver.createNewTab()
- B) driver.switchTo().newWindow(WindowType.TAB / WindowType.WINDOW)
- C) driver.openNewWindow()
- D) JavascriptExecutor.executeScript("window.open()")

Q14. Which of the following is the correct way to handle iframes in Selenium?

- A) driver.switchTo().frame(name/id/index/WebElement) and then driver.switchTo().defaultContent()
- B) driver.selectFrame()
- C) driver.findElement inside iframe works directly without switching
- D) driver.moveToFrame()

Q15. Which wait in Selenium 4 combines both implicit and explicit concepts and was improved with fluent wait deprecation?

- A) WebDriverWait with Duration, ignoring exceptions, polling - new WebDriverWait(driver, Duration.ofSeconds(10))
- B) Sleep Wait
- C) Hard Wait
- D) PageFactory Wait

Q16. What is the use of JavascriptExecutor in Selenium?

- A) Only to launch JavaScript alerts
- B) To execute JavaScript to handle scenarios where WebDriver fails: scrolling, clicking hidden elements, retrieving shadow DOM, etc.
- C) To replace WebDriver completely

D) To install browser extensions

Q17. Which TestNG annotation is used to run a method before each test method in a class?

- A) @BeforeTest
- B) @BeforeMethod
- C) @BeforeClass
- D) @BeforeSuite

Q18. As of Selenium 4.8+, how do you handle Chrome DevTools Protocol (CDP) for network interception/mock?

- A) CDP is completely removed
- B) Using ChromeDevTools or new BiDi API implementation: driver.getDevTools() for CDP, or NetworkInterceptor & BiDi for 2024-2026 recommended approach
- C) Using driver.interceptNetwork() directly
- D) CDP only works with Firefox

Q19. What is the purpose of PageFactory and @FindBy in Selenium?

- A) To generate random test data
- B) To implement Page Object Model with lazy initialization of elements using proxies
- C) To create database connections
- D) To manage WebDriverManager

Q20. How do you handle StaleElementReferenceException?

- A) Refresh the page and never handle it
- B) Use Thread.sleep() longer
- C) Re-locate the element before interaction, using explicit wait, or retry logic; understanding DOM was refreshed/re-rendered
- D) Ignore the exception

Answer Key - Intermediate Level

| Q | Answer | Explanation |

|---|-----|-----|

- | 11 | B | Explicit Wait (WebDriverWait + ExpectedConditions) is best practice. Thread.sleep() is flaky and slow. |
- | 12 | A | getWindowHandles() to get set, then switchTo().window(handle) is standard. |
- | 13 | B | Selenium 4 added native API: driver.switchTo().newWindow(WindowType.TAB) returns new handle, no JS needed. |
- | 14 | A | Must switch to iframe via switchTo().frame() then back via defaultContent() or parentFrame(). |
- | 15 | A | FluentWait is base. In Selenium 4, WebDriverWait now takes Duration, and you can chain pollingEvery(), ignoring(). |
- | 16 | B | JS Executor for scroll, hidden clicks, shadow DOM piercing, setting values, etc. |
- | 17 | B | @BeforeMethod runs before each @Test method. @BeforeClass once per class. |
- | 18 | B | CDP via ((HasDevTools)driver).getDevTools() but from Selenium 4.18+ BiDi API is preferred for cross-browser network/logging intercepts. |
- | 19 | B | PageFactory.initElements() creates proxy elements for POM, lazy loads with @FindBy, @CacheLookup. |
- | 20 | C | Stale means element reference detached from DOM due to refresh/AJAX. Re-locate element or use ExpectedConditions.refreshed(). |

Section C: Advanced Level (Questions 21-30)

Q21. What is Selenium BiDi (Bidirectional) API introduced in Selenium 4 and maturing in 2024-2026?

- A) A replacement for WebDriver that only supports Chrome
- B) A new W3C standard protocol allowing bidirectional communication (driver <-> browser) for events, network interception, log listening, etc., without CDP dependency
- C) A tool to run tests in two browsers simultaneously
- D) A Grid 4 feature for parallel execution

Q22. In Selenium Grid 4 (2024-2026 architecture), what are the main components?

- A) Hub and Node only (Grid 3 architecture)
- B) Router, Distributor, Session Map, Session Queue, Node, Event Bus - supporting standalone, hub-node, and fully distributed modes
- C) Only Docker containers
- D) Selenium Server Standalone only

Q23. How do you handle Shadow DOM elements in Selenium 4+?

- A) Shadow DOM cannot be automated
- B) Using `getShadowRoot()` method: `WebElement shadowHost = driver.findElement(...); SearchContext shadowRoot = shadowHost.getShadowRoot(); shadowRoot.findElement(...)`
- C) Using XPath `//shadow-root`
- D) Using `driver.switchTo().shadow()`

Q24. Which strategy is BEST for reducing flakiness in Selenium tests running in CI/CD (GitHub Actions, Jenkins) in 2025?

- A) Use only implicit waits of 60 seconds
- B) Use explicit waits, stable locators (data-testid), retry listeners (I-RetryAnalyzer), headless mode optimization, and Selenium Manager for driver stability; use Grid with Docker
- C) Use `Thread.sleep(10000)` everywhere
- D) Run only locally, not in CI

Q25. What is the correct way to capture console logs and browser logs in Selenium 4 using BiDi?

- A) `driver.manage().logs().get()` is the only and future-proof way
- B) Using BiDi Log Inspector: `LogInspector logInspector = new LogInspector(driver); logInspector.onConsoleEntry(...)` and `onJavaScriptLog`, `onJavaScriptException` - preferred 2024+
- C) Cannot capture console logs
- D) Using `System.out.println()`

Q26. How does Selenium Manager handle browser and driver discovery in Selenium 4.22+?

- A) It requires manual driver download script
- B) It automatically discovers installed browsers, downloads compatible driver (chromedriver, geckodriver, msedgedriver) from official sources, caches it, and can also download browsers (chrome, firefox) if not present using `--browser-path`
- C) Only supports Chrome

D) It uses NPM to download drivers

Q27. In a Test Automation Framework, what is the purpose of EventFiringDecorator / WebDriverListener in Selenium 4?

- A) To add screenshots before/after events; Selenium 4 replaced deprecated EventFiringWebDriver with EventFiringDecorator and WebDriverListener interface for AOP-style logging.
- B) To fire browser native events only
- C) To disable events
- D) To replace TestNG listeners

Q28. Which of the following code correctly implements network mocking/interception using Selenium 4 BiDi in 2025?

- A) `driver.get("https://example.com"); // no interception possible`
- B) Using NetworkInterceptor with Route: `try (NetworkInterceptor interceptor = new NetworkInterceptor(driver, Route.matching(req -> true).to() -> req -> new HttpResponse()... )) { ... } or BiDi network.addIntercept + network.continueWithAuth`
- C) Using BrowserMob Proxy only, Selenium can't do it
- D) `driver.setRequestHeader()`

Q29. What is the best practice for handling file uploads in Selenium 4 when `<input type="file">` is present but hidden by UI library?

- A) Click upload button that opens OS dialog - Selenium cannot handle OS dialogs
- B) Use `sendKeys(absoluteFilePath)` directly to the file input element (make visible via JS if needed: `js.executeScript("arguments[0].style.display='block'", fileInput); fileInput.sendKeys(path);`)
- C) Use Robot class always
- D) File upload is not possible in headless mode

Q30. For large-scale parallel execution in Selenium Grid 4 with Kubernetes / Docker in 2024-2026, what is recommended?

- A) Run all tests sequentially on one node
- B) Use Grid in fully distributed mode with autoscaling, session queue with timeouts, tracing with OpenTelemetry, using `--detect-drivers`, Docker images selenium/node-chrome, Helm charts, and dynamic scaling
- C) Use Selenium RC for parallel execution
- D) Use only Hub-Node without Router

Answer Key - Advanced Level

| Q | Answer | Explanation |

|---|-----|-----|

| 21 | B | BiDi is W3C WebDriver BiDi, allowing bidirectional events. Replaces CDP-specific hacks with cross-browser standard. |

| 22 | B | Grid 4 is modular: Router (entry), Distributor, Session Map (Redis/DB), Session Queue, Node, Event Bus. Supports fully distributed via K8s. |

| 23 | B | Selenium 4 added native Shadow DOM support via `getShadowRoot()` returning `SearchContext`. Open & closed shadow roots supported. |

| 24 | B | Modern best practice: explicit waits, resilient locators (data-testid), retry analyzer, Selenium Manager, Grid Docker for stability. |

| 25 | B | Legacy `manage().logs()` is deprecated/limited. BiDi LogInspector (Selenium 4.12+) is new standard for console, JS logs, exceptions. |

| 26 | B | Selenium Manager in 4.22+ can manage both drivers and browsers, discover via PATH, cache in `~/.cache/selenium`, offline handling. |

| 27 | A | `EventFiringWebDriver` deprecated in Selenium 4. New decorator pattern: `new EventFiringDecorator(new MyListener()).decorate(driver)` enables listeners for logging, screenshots. |

| 28 | B | BiDi NetworkInterceptor via `org.openqa.selenium.devtools` legacy or `org.openqa.selenium.bidi` for mocking/continue. Official 2024-2026 approach. |

| 29 | B | Best is `sendKeys` to input. If hidden, unhide via JS or directly interact. Robot/AutoIT only if no input tag (OS dialog). Works in headless too. |

| 30 | B | Grid 4 designed for K8s/Docker: OpenTelemetry tracing, autoscaled nodes, session queue config, Helm chart `selenium-grid`, Docker images. |

****End of Exam - Generated for 2024-2026 Syllabus****

Course File: 35_Rust.md

Course Title: Rust Programming Language - Comprehensive Examination (35_Rust)

This exam covers Rust 1.75+ through 1.85+ (2024-2026 edition), including ownership, lifetimes, async, and modern features.

Section 1: Beginner Level (Questions 1-10)

Q1. What is the default integer type in Rust when you write `let x = 42;` without annotation?

- A) i32
- B) i64
- C) u32
- D) isize

Q2. Which keyword is used to make a variable mutable?

- A) mut
- B) mutable
- C) var
- D) let mut is not needed; all variables are mutable

Q3. What happens when you try to use a value after it has been moved?

- A) It creates a copy automatically
- B) Compile-time error: use of moved value
- C) Runtime panic
- D) It returns None

Q4. Which macro is used to print to stdout with a newline?

- A) print!
- B) println!
- C) echo!
- D) log!

Q5. How do you create a new String from a string literal?

- A) `let s = "hello".to_string();`
- B) `let s: String = "hello";`
- C) `let s = String::new("hello");`
- D) `let s = new String("hello");`

Q6. What does cargo check do?

- A) Compiles and runs the project
- B) Checks code for errors without producing an executable
- C) Formats the code
- D) Runs tests

Q7. Which of the following is the correct way to define a function that takes an i32 and returns an i32?

- A) `fn add(x: i32) -> i32 { x + 1 }`
- B) `func add(x: i32): i32 { return x + 1; }`
- C) `fn add(x: i32) => i32 { x + 1 }`
- D) `def add(x: i32) -> i32 { x + 1 }`

Q8. What is the owner of data in Rust?

- A) The GC
- B) The variable that holds it; only one owner at a time
- C) The function that created it forever
- D) All references share ownership equally

Q9. Which collection type allows dynamic sized array that can grow?

- A) `Array [T; N]`
- B) `Vec<T>`
- C) `Slice &[T]`
- D) `Tuple`

Q10. What does & mean in `let r = &x;`?

- A) Creates a mutable copy
- B) Creates an immutable reference (borrows)
- C) Dereferences x
- D) Moves x

Answer Key - Beginner

1. A) i32 - Rust defaults to i32 for integers
2. A) `mut` - e.g., `let mut x = 5;`
3. B) Compile-time error: use of moved value - Ownership rule prevents use after move
4. B) `println!` - `print!` does not add newline
5. A) `let s = "hello".to_string();` - Also `String::from("hello")` is valid; option C is incorrect syntax
6. B) Checks code for errors without producing an executable - Faster than cargo build
7. A) `fn add(x: i32) -> i32 { x + 1 }` - Correct Rust function syntax
8. B) The variable that holds it; only one owner at a time - Core ownership principle
9. B) `Vec<T>` - Growable heap-allocated vector
10. B) Creates an immutable reference (borrows) - & creates a borrow, &mut for mutable borrow

Section 2: Intermediate Level (Questions 11-20)

Q11. What is the difference between `String` and `&str`?

- A) They are identical
- B) `String` is owned, growable heap data; `&str` is borrowed slice view
- C) `&str` is owned, `String` is borrowed
- D) `String` is only for ASCII

Q12. What will this code NOT compile?

```
fn main() {  
    let mut s = String::from("hello");  
    let r1 = &s;  
    let r2 = &s;  
    let r3 = &mut s;  
    println!("{}", r1, r2, r3);  
}
```

- A) It compiles fine
- B) Because you cannot have mutable and immutable borrows at same time
- C) Because String cannot be borrowed mutably
- D) Because println! cannot take references

Q13. Which trait is automatically implemented for types that can be safely sent between threads?

- A) Sync
- B) Send
- C) Copy
- D) Clone

Q14. What does Option<T> represent?

- A) A type that is always Some
- B) A type that may be Some(T) or None, for explicit null handling
- C) A C-style null pointer
- D) An error type

Q15. In Rust 2021+ edition, what does Into and From conversion use? Which statement is true about ? operator?

- A) ? unwraps Ok or returns Err early from function returning Result
- B) ? panics on error
- C) ? only works with Option
- D) ? converts errors to None

Q16. What is a lifetime annotation like 'a used for?

- A) To tell GC when to free memory
- B) To describe how long references are valid so borrow checker can verify them
- C) To set how long a variable lives at runtime
- D) It is only documentation

Q17. Which of the following is correct about traits vs 2024-2026 behavior? Which allows impl Trait in return position to use async traits?

- A) Native async fn in traits stabilized in Rust 1.75 (Dec 2023) and RPITIT improvements in 1.79+
- B) Async traits are still impossible in 2026
- C) Async traits require boxing always
- D) Only async-std supports it

Q18. What does `#[derive(Debug, Clone, PartialEq)]` do?

- A) Manually implements those traits
- B) Auto-generates implementations for those traits via procedural macro
- C) Imports them
- D) It is a comment

Q19. Consider: `let x = vec![1,2,3].into_iter().filter(|n| n % 2 == 0).collect::<Vec<_>>();` What does collect need?

- A) Type annotation or turbofish to know target collection
- B) Nothing, it always returns Vec
- C) It only works with HashMap
- D) It requires mutable reference

Q20. What is the purpose of `Arc<Mutex<T>>`?

- A) Single-threaded mutable sharing
- B) Thread-safe shared ownership with interior mutability across threads
- C) To make T Copy
- D) To avoid borrowing

Answer Key - Intermediate

- 11. B) String is owned, growable heap data; &str is borrowed slice view
- 12. B) Because you cannot have mutable and immutable borrows at same time - Borrow checker rule: either one mutable OR any number of immutable
- 13. B) Send - Sync is for references that can be shared between threads
- 14. B) A type that may be Some(T) or None, for explicit null handling - Rust has no null
- 15. A) ? unwraps Ok or returns Err early from function returning Result - Also works with Option returning None early
- 16. B) To describe how long references are valid so borrow checker can verify them - Lifetimes are compile-time only
- 17. A) Native async fn in traits stabilized in Rust 1.75 (Dec 2023) and RPITIT improvements in 1.79+ - Part of 2024-2025 stabilization: async fn in traits without `async_trait` macro
- 18. B) Auto-generates implementations for those traits via procedural macro
- 19. A) Type annotation or turbofish to know target collection - collect is generic over FromIterator
- 20. B) Thread-safe shared ownership with interior mutability across threads - Arc = Atomic Reference Counted, Mutex for mutability

Section 3: Advanced Level (Questions 21-30)

Q21. What is the difference between `Box<dyn Trait>` and `impl Trait` in 2024 edition?

- A) `Box<dyn Trait>` is dynamic dispatch (vtable at runtime), `impl Trait` is static dispatch (monomorphized)
- B) They are same
- C) `impl Trait` is always boxed
- D) `Box<dyn Trait>` cannot be used in 2024 edition

Q22. Given Rust 1.81+ error handling, what is the new `std::error::Error` method stabilized for providing sources?

- A) `Error::source()` has always existed; in 2024+ the `#[diagnostic]` and improved Error reporting in `#[feature(error_generic_member_access)]` for `provide()`
- B) There is no Error trait
- C) `get_source()`
- D) `cause()`

Q23. What does Pin and Unpin guarantee, crucial for `async/await` and Future self-referential structs?

- A) Pin prevents moving a value in memory after it is pinned, required for intrusive futures
- B) Pin allows GC
- C) Pin makes value Copy
- D) Unpin means value is immutable

Q24. In advanced lifetimes, what does `for<'a>` (Higher-Ranked Trait Bounds) mean in where `F: for<'a> Fn(&'a i32)`?

- A) F must work for any lifetime 'a, not just one specific lifetime
- B) F only works for 'static
- C) It means F is async
- D) It is syntax error

Q25. As of Rust 1.84-1.85 (2025), what is true about the 2024 Edition?

- A) It became the default edition with RPITIT capturing precise lifetimes, gen blocks as replacement for `gen` keyword evolution, and `let` chains improvements
- B) 2024 edition was cancelled
- C) 2024 edition removes ownership
- D) 2024 edition requires nightly

Q26. What is the purpose of `std::mem::MaybeUninit<T>`?

- A) To safely handle uninitialized memory without UB, replacing deprecated `mem::uninitialized()`
- B) To initialize memory to zero
- C) To leak memory
- D) To make T optional

Q27. What does the following unsafe code violation illustrate?

```
let x = 5;
let r = &x as *const i32;
unsafe { *r = 10; } // Attempt to mutate immutable data via raw pointer cast
```

- A) Valid unsafe code
- B) UB: writing to immutable bytes &T is undefined behavior even via raw pointer if original was immutable
- C) It compiles and safely mutates
- D) Raw pointers always allow mutation

Q28. What is `Cow<'a, B>` used for in advanced Rust?

- A) Clone on Write - can hold either borrowed or owned data to avoid unnecessary clone
- B) Copy on Write for GC

- C) Only for concurrent access
- D) Compile on Write

Q29. Which statement about tokio vs async-std vs smol and Rust's async model in 2024-2026 is accurate?

- A) Rust std includes a runtime; tokio not needed
- B) Rust std only provides async syntax and poll model; external executors like Tokio are needed to run futures, with Tokio being dominant in 2024-2026
- C) Async is blocking in Rust
- D) Tokio was deprecated in 2025

Q30. What is true about const fn and const generics evolution through Rust 1.82+ (2024-2026)?

- A) const fn can now contain mutable references, Rc, and heap allocation freely in stable
- B) const fn capabilities have expanded significantly (e.g., const traits, mutable refs in const fn stabilized in 1.83+, let patterns, &mut in const) but still have restrictions; const generics support complex expressions via generic_const_exprs still incomplete but const eval much more powerful
- C) const fn was removed
- D) const generics are not yet implemented

Answer Key - Advanced

- 21. A) Box<dyn Trait> is dynamic dispatch (vtable at runtime), impl Trait is static dispatch (monomorphized) - dyn = dynamic, impl = static, affects binary size and performance
- 22. A) Error::source() has always existed; in 2024+ the #[diagnostic] and improved Error reporting in #[feature(error_generic_member_access)] for provide() - And #[expect(lint)] and detailed error diagnostics stabilized
- 23. A) Pin prevents moving a value in memory after it is pinned, required for intrusive futures - Self-referential futures need stable address
- 24. A) F must work for any lifetime 'a, not just one specific lifetime - HRTB means higher-ranked, works for all lifetimes
- 25. A) It became the default edition with RPITIT capturing precise lifetimes, gen blocks as replacement for gen keyword evolution, and let chains improvements - 2024 Edition released Feb 2025 with Rust 1.85; let chains stabilized in 1.64, but 2024 edition refines tail handling and RPITIT lifetime capture rules
- 26. A) To safely handle uninitialized memory without UB, replacing deprecated mem::uninitialized() - MaybeUninit is union-like safe wrapper
- 27. B) UB: writing to immutable bytes &T is undefined behavior even via raw pointer if original was immutable - Cannot mutate through raw pointer derived from shared ref to immutable binding; violates aliasing rules
- 28. A) Clone on Write - can hold either borrowed or owned data to avoid unnecessary clone - Cow::Borrowed(&str) or Cow::Owned(String)
- 29. B) Rust std only provides async syntax and poll model; external executors like Tokio are needed to run futures, with Tokio being dominant in 2024-2026 - No runtime in stdlib by design
- 30. B) const fn capabilities have expanded significantly (e.g., const traits, mutable refs in const fn stabilized in 1.83+, let patterns, &mut in const) but still have restrictions; const generics support complex expressions via generic_const_exprs still incomplete but const eval much more powerful - Shows 2024-2026 progression

End of Exam

Total Questions: 30

Format verified: Each question has 4 options (A-D) + answer key with explanations

Rust version coverage: 1.75 (async fn in traits), 1.79-1.85, Edition 2024

Course File: 36_SQL.md

Course Title: SQL & Databases - Comprehensive Examination

****Course Code:**** SQL-DB-36

****Level:**** Beginner to Advanced

****Duration:**** 90 Minutes

****Total Questions:**** 30

****Total Marks:**** 30 (1 mark each)

****Passing Marks:**** 18/30

****Valid For:**** 2024-2026 Syllabus

SECTION A: BEGINNER LEVEL (Questions 1-10)

****Instructions:**** Choose the correct answer from 4 options.

****Q1. Which SQL statement is used to retrieve data from a database?****

- A) GET
- B) SELECT
- C) FETCH
- D) RETRIEVE

****Q2. Which of the following is a primary key characteristic?****

- A) Can contain NULL values
- B) Can have duplicate values
- C) Uniquely identifies each row and cannot be NULL
- D) Is optional for every table

****Q3. What does SQL stand for?****

- A) Structured Query Language
- B) Standard Query Logic
- C) System Query Language
- D) Sequential Query Language

****Q4. Which clause is used to filter results in a SELECT statement?****

- A) HAVING
- B) WHERE
- C) FILTER
- D) CONDITION

****Q5. Which data type is used to store variable-length strings in most modern RDBMS (PostgreSQL, MySQL, SQL Server)?**

- A) CHAR
- B) TEXT ONLY
- C) VARCHAR
- D) STRING

****Q6. Which command is used to add a new row to a table?**

- A) ADD ROW
- B) INSERT INTO
- C) APPEND
- D) ADD RECORD

****Q7. What is the default sorting order of ORDER BY?**

- A) DESC
- B) ASC (Ascending)

C) RANDOM

D) No default, must specify

****Q8. Which join returns all rows from the left table and matched rows from the right table?****

A) INNER JOIN

B) LEFT JOIN (LEFT OUTER JOIN)

C) RIGHT JOIN

D) FULL JOIN

****Q9. Which function is used to count the number of rows in SQL?****

A) COUNT()

B) TOTAL()

C) NUMBER()

D) SUM()

****Q10. Which constraint ensures that all values in a column satisfy a specific condition?****

A) UNIQUE

B) NOT NULL

C) CHECK

D) DEFAULT

Answer Key - Beginner Level

| Question | Correct Answer | Explanation |

|-----|---|---|

| Q1 | B) SELECT | SELECT is the standard DQL command to retrieve data. |

| Q2 | C) Uniquely identifies each row and cannot be NULL | Primary Key must be UNIQUE and NOT NULL. |

| Q3 | A) Structured Query Language | Official full form. |

| Q4 | B) WHERE | WHERE filters rows before grouping; HAVING filters after GROUP BY. |

| Q5 | C) VARCHAR | VARCHAR(n) stores variable-length strings; CHAR is fixed-length. |

| Q6 | B) INSERT INTO | Syntax: INSERT INTO table_name VALUES (...). |

| Q7 | B) ASC (Ascending) | ORDER BY defaults to ASC if not specified. |

| Q8 | B) LEFT JOIN | LEFT JOIN returns all left rows + matching right rows, NULLs for non-matches. |

| Q9 | A) COUNT() | COUNT(*) counts rows including NULLs. |

| Q10 | C) CHECK | CHECK constraint validates condition like CHECK (age >= 18). |

SECTION B: INTERMEDIATE LEVEL (Questions 11-20)

****Instructions:**** Choose the correct answer from 4 options.

****Q11. Given tables: Employees(id, dept_id) and Departments(id, name). What does this query return?**

SELECT d.name, COUNT(e.id) FROM Departments d LEFT JOIN Employees e ON d.id = e.dept_id GROUP BY d.name;

A) Only departments with employees

B) All departments with employee count, including 0 for departments with no employees

C) Departments with no employees only

D) Syntax error

****Q12. What is the difference between WHERE and HAVING?****

A) No difference, they are interchangeable

B) WHERE filters groups, HAVING filters rows

C) WHERE filters individual rows before GROUP BY, HAVING filters groups after GROUP BY

D) HAVING can only be used without GROUP BY

****Q13. Which statement about indexes is TRUE as of 2024-2026 best practices?****

- A) Indexes always slow down SELECT queries
- B) Indexes speed up reads but can slow down INSERT/UPDATE/DELETE and use extra storage
- C) You should create an index on every column
- D) Primary keys are never indexed automatically

****Q14. Consider transaction isolation. Which anomaly does REPEATABLE READ prevent that READ COMMITTED does not?****

- A) Dirty Read
- B) Non-repeatable read
- C) Phantom read is fully prevented in all DBs
- D) Both prevent same anomalies

****Q15. What will this query do?**

SELECT department, AVG(salary) FROM employees GROUP BY department HAVING AVG(salary) > 60000;

- A) Shows employees earning > 60000
- B) Shows departments where average salary exceeds 60000
- C) Shows all departments with average salary
- D) Throws error because WHERE should be used

****Q16. Which is correct about NULL handling?****

- A) NULL = NULL returns TRUE
- B) WHERE column = NULL correctly finds NULLs
- C) WHERE column IS NULL correctly finds NULLs, and COUNT(column) ignores NULLs while COUNT(*) does not
- D) NULL means zero or empty string

****Q17. What does normalization (3NF) primarily aim to achieve?****

- A) Increase data redundancy
- B) Reduce data redundancy and improve data integrity by removing transitive dependencies
- C) Speed up all queries regardless of cost
- D) Merge tables into one large table

****Q18. What is the result of this query in PostgreSQL / MySQL 8+?**

SELECT * FROM employees ORDER BY salary DESC LIMIT 5;

- A) 5 lowest paid employees
- B) 5 highest paid employees
- C) All employees sorted, limited to 5 random
- D) Syntax error, should use TOP

****Q19. Which is true about FOREIGN KEY?****

- A) It must reference PRIMARY KEY or UNIQUE key in another (or same) table and enforces referential integrity
- B) Foreign key can contain any value
- C) A table can have no foreign keys but only one is ever allowed
- D) Foreign keys automatically create duplicate data

****Q20. You have a table orders (id, customer_id, order_date, amount). How to find customers who placed more than 3 orders in 2024?****

- A) SELECT customer_id FROM orders WHERE COUNT(*) > 3 AND YEAR(order_date)=2024
- B) SELECT customer_id FROM orders WHERE order_date BETWEEN '2024-01-01' AND '2024-12-31' GROUP BY customer_id HAVING COUNT(*) > 3
- C) SELECT customer_id FROM orders GROUP BY customer_id WHERE COUNT > 3
- D) SELECT customer_id FROM orders HAVING COUNT(*) > 3

Answer Key - Intermediate Level

| Question | Correct Answer | Explanation |

|-----|---|---

| Q11 | B) All departments with employee count, including 0 for departments with no employees | LEFT JOIN preserves all Departments. COUNT(e.id) will be 0 where no match. |

| Q12 | C) WHERE filters individual rows before GROUP BY, HAVING filters groups after GROUP BY | Standard SQL execution order: FROM -> WHERE -> GROUP BY -> HAVING -> SELECT. |

| Q13 | B) Indexes speed up reads but can slow down INSERT/UPDATE/DELETE and use extra storage | Indexes are B-Tree structures; trade-off is write overhead. |

| Q14 | B) Non-repeatable read | REPEATABLE READ prevents dirty and non-repeatable reads. Phantom read prevention varies (PostgreSQL/MySQL InnoDB prevent phantom via MVCC/next-key locking). |

| Q15 | B) Shows departments where average salary exceeds 60000 | GROUP BY + HAVING filters aggregated groups. |

| Q16 | C) WHERE column IS NULL correctly finds NULLs, and COUNT(column) ignores NULLs while COUNT(*) does not | NULL is unknown; use IS NULL. COUNT(col) skips NULLs by SQL standard. |

| Q17 | B) Reduce data redundancy and improve data integrity by removing transitive dependencies | 3NF removes columns not dependent on key (transitive). |

| Q18 | B) 5 highest paid employees | ORDER BY DESC sorts high to low; LIMIT 5 takes top 5. TOP is SQL Server syntax. |

| Q19 | A) It must reference PRIMARY KEY or UNIQUE key in another (or same) table and enforces referential integrity | FK ensures value exists in referenced table. |

| Q20 | B) SELECT customer_id FROM orders WHERE order_date BETWEEN '2024-01-01' AND '2024-12-31' GROUP BY customer_id HAVING COUNT(*) > 3 | Correct pattern: filter rows by date in WHERE, then group and filter groups with HAVING. |

SECTION C: ADVANCED LEVEL (Questions 21-30)

****Instructions:**** Choose the correct answer from 4 options.

****Q21.** Which window function query correctly ranks employees by salary within each department without gaps after ties?

- A) SELECT dept, name, RANK() OVER (PARTITION BY dept ORDER BY salary DESC) FROM employees
- B) SELECT dept, name, DENSE_RANK() OVER (PARTITION BY dept ORDER BY salary DESC) FROM employees
- C) SELECT dept, name, ROW_NUMBER() OVER (ORDER BY salary DESC) FROM employees
- D) SELECT dept, name, RANK() OVER (ORDER BY dept) FROM employees

****Q22.** As of PostgreSQL 15+ / MySQL 8+, what is the main advantage of CTEs (Common Table Expressions) with WITH clause?

- A) CTEs are always faster than subqueries
- B) They improve readability, allow recursion, and can be materialized or inlined by optimizer for complex queries
- C) They automatically create indexes
- D) They replace all JOINS

****Q23.** You need to handle concurrent increment of a counter. Which is safest to prevent lost updates?

- A) SELECT counter FROM tbl; then in app add 1; UPDATE tbl SET counter = new_val
- B) UPDATE tbl SET counter = counter + 1 (atomic) or using SELECT ... FOR UPDATE in a transaction, or optimistic locking with version column
- C) Use DELETE then INSERT
- D) Rely on READ UNCOMMITTED

****Q24.** What does this PostgreSQL query do?

SELECT * FROM employees WHERE department = ANY(ARRAY['Engineering','Sales']);

- A) Syntax error
- B) Equivalent to department IN ('Engineering','Sales')
- C) Returns only Engineering department
- D) Returns employees where department contains substring

****Q25.** For a large table with 100M rows, column status has only 3 distinct values (low cardinality). Which indexing strategy is generally BAD for OLTP performance in 2024-2026?

- A) Creating a B-Tree index on status alone for filtering WHERE status='X' when selectivity is low (~33% rows)
- B) Creating a composite index (status, created_at) where query filters on both
- C) Creating a partial/filtered index WHERE status = 'PENDING' in PostgreSQL / SQL Server filtered index
- D) Using BRIN index in PostgreSQL for naturally ordered data

****Q26.** Explain EXPLAIN ANALYZE in PostgreSQL. What does it do?

- A) Only shows estimated plan without execution
- B) Executes the query and shows actual execution plan with real timings and row counts (EXPLAIN alone shows estimates)
- C) Creates an index automatically
- D) Deletes the query plan cache

****Q27.** Consider deadlock scenario. Transaction A locks row 1 then wants row 2, Transaction B locks row 2 then wants row 1. How do modern DBMS (PostgreSQL, MySQL InnoDB, SQL Server) handle this in 2024?

- A) Both transactions wait forever
- B) Deadlock detector aborts one transaction with deadlock error code, requiring app retry
- C) Database merges both transactions
- D) Rows are automatically unlocked after 1 second

****Q28.** What is the purpose of COALESCE() and how is it different from ISNULL() / IFNULL() in 2024 standards?

- A) No difference
- B) COALESCE() is ANSI standard, accepts multiple arguments returning first non-NULL; ISNULL() is SQL Server specific (2 args), IFNULL() is MySQL specific
- C) COALESCE only works with numbers
- D) ISNULL is ANSI standard

****Q29.** You have to design for write-heavy time-series IoT data (2024-2026 context). Which is most appropriate?

- A) Highly normalized RDBMS with many foreign keys and secondary indexes on every column
- B) Use TimescaleDB extension for PostgreSQL or partitioned tables with BRIN index, minimal secondary indexes, compression and retention policies
- C) Use SELECT * with no WHERE clause always
- D) Store all data in single VARCHAR column

****Q30.** Given:

```
WITH RECURSIVE org AS (
```

```
SELECT id, manager_id, name FROM employees WHERE manager_id IS NULL
```

```
UNION ALL
```

```
SELECT e.id, e.manager_id, e.name FROM employees e JOIN org o ON e.manager_id = o.id
```

```
) SELECT * FROM org;
```

What does this query demonstrate?

- A) A syntax error - recursion not allowed in SQL
- B) Recursive CTE to traverse hierarchical/org chart data
- C) Simple aggregation
- D) Cross join producing cartesian product

Answer Key - Advanced Level

| Question | Correct Answer | Explanation |

|-----|---|---

| Q21 | B) SELECT dept, name, DENSE_RANK() OVER (PARTITION BY dept ORDER BY salary DESC) | DENSE_RANK() gives no gaps after tie (1,2,2,3). RANK() would give gaps (1,2,2,4). PARTITION BY needed per dept. ROW_NUMBER would not handle ties. |

| Q22 | B) They improve readability, allow recursion, and can be materialized or inlined by optimizer | PostgreSQL 12+ allows MATERIALIZED/NOT MATERIALIZED hint. Major benefit is readability/maintainability + recursion. |

| Q23 | B) UPDATE tbl SET counter = counter + 1 (atomic) or using SELECT ... FOR UPDATE in a transaction, or optimistic locking with version column | Atomic UPDATE is safe at database level. Pessimistic locking (FOR UPDATE) or optimistic locking prevents lost update anomaly. |

| Q24 | B) Equivalent to department IN ('Engineering','Sales') | ANY with ARRAY is PostgreSQL idiom equivalent to IN list. |

| Q25 | A) Creating a B-Tree index on status alone for filtering WHERE status='X' when selectivity is low (~33% rows) | Low selectivity index (33% of rows) leads to full scan anyway; index overhead wasted. Partial/composite indexes are better. Modern optimizers often ignore such low-selectivity indexes. |

| Q26 | B) Executes the query and shows actual execution plan with real timings and row counts | EXPLAIN shows estimated cost; EXPLAIN ANALYZE actually runs query (or EXPLAIN (ANALYZE, BUFFERS) in modern PG). |

| Q27 | B) Deadlock detector aborts one transaction with deadlock error code, requiring app retry | All major RDBMS have deadlock detection; they choose a victim (usually lowest cost) to abort with error 1213 (MySQL) or 40P01 (Postgres). |

| Q28 | B) COALESCE() is ANSI standard, accepts multiple arguments returning first non-NULL; ISNULL() is SQL Server specific (2 args), IFNULL() is MySQL specific | COALESCE is portable and variadic per SQL:2003+. |

| Q29 | B) Use TimescaleDB extension for PostgreSQL or partitioned tables with BRIN index, minimal secondary indexes, compression and retention policies | Time-series best practice 2024-2026: hypertables, partitioning by time, compression, few indexes. |

| Q30 | B) Recursive CTE to traverse hierarchical/org chart data | WITH RECURSIVE is standard (SQL:1999) for hierarchies, org charts, bill-of-materials. |

Overall Grading Rubric

- 27-30: Distinction - Expert level
- 22-26: Merit - Strong proficiency
- 18-21: Pass - Competent
- <18: Needs improvement

****Note:**** This exam reflects SQL standards and RDBMS features (PostgreSQL 15/16, MySQL 8+, SQL Server 2022) relevant for 2024-2026.